

Studentu pazymiu sistema

v2.0

Generated by Doxygen 1.17.0

Chapter 1

Directory Hierarchy

1.1 Directories

include	??
student.h	??
student_io_deque.h	??
student_io_list.h	??
student_io_vector.h	??
timer.h	??
utils.h	??
zmogus.h	??
src	??
io_deque.cpp	??
io_list.cpp	??
io_vector.cpp	??
main_deque.cpp	??
main_list.cpp	??
main_vector.cpp	??
student.cpp	??
utils.cpp	??
tests	??
doctest.h	??
test_studentas.cpp	??

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all namespaces with brief descriptions:

doctest	??
doctest::assertType	??
doctest::Color	??
doctest::detail	??
doctest::detail::binaryAssertComparison	??
doctest::detail::types	??
doctest::TestCaseFailureReason	??
doctest_detail_test_suite_ns	??
std	??

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

doctest::Approx	??
doctest::AssertData	??
doctest::detail::ResultBuilder	??
std::basic_istream< charT, traits >	??
std::basic_ostream< charT, traits >	??
std::char_traits< charT >	??
doctest::Contains	??
doctest::Context	??
doctest::ContextOptions	??
doctest::CurrentTestCaseStats	??
doctest::detail::types::enable_if< COND, T >	??
doctest::detail::types::enable_if< true, T >	??
doctest::detail::Expression_lhs< L >	??
doctest::detail::ExpressionDecomposer	??
doctest::detail::types::false_type	??
doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >(), declval< const T & >()), void())>	??
doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())>>	??
doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)>>	??
doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype(declval< const T & >().c_str()), const char * >value &&types::is_same< decltype(declval< const T & >().size()), size_t >value >type >	??
doctest::detail::types::is_array< T[SIZE]>	??
doctest::detail::types::is_pointer< T * >	??
doctest::detail::types::is_rvalue_reference< T && >	??
doctest::detail::types::is_same< T, T >	??
doctest::detail::deferred_false< T >	??
doctest::detail::has_insertion_operator< T, typename >	??
doctest::detail::is_container< T, typename >	??
doctest::detail::is_pair< T, typename >	??
doctest::detail::is_std_string< T, Enable >	??
doctest::detail::types::is_array< T >	??
doctest::detail::types::is_pointer< T >	??

doctest::detail::types::is_rvalue_reference< T >	??
doctest::detail::types::is_same< T, U >	??
doctest::detail::filldata< T, Enable >	??
doctest::detail::filldata< const char[N]>	??
doctest::detail::filldata< const void * >	??
doctest::detail::filldata< const volatile void * >	??
doctest::detail::filldata< T * >	??
doctest::detail::filldata< T, typename detail::types::enable_if< !detailhas_insertion_operator< T >value &&detail::is_container< T >value >type >	??
doctest::detail::filldata< T, typename detail::types::enable_if< !detailhas_insertion_operator< T >value &&detail::is_pair< T >value >type >	??
doctest::detail::filldata< T[N]>	??
doctest::IContextScope	??
doctest::detail::ContextScopeBase	??
doctest::detail::ContextScope< L >	??
doctest::detail::IExceptionTranslator	??
doctest::detail::ExceptionTranslator< T >	??
doctest::IReporter	??
doctest::detail::types::is_enum< T >	??
doctest::IsNaN< F >	??
doctest::MessageData	??
doctest::detail::MessageBuilder	??
doctest::QueryData	??
doctest::detail::RelationalComparator< int, L, R >	??
doctest::detail::types::remove_const< T >	??
doctest::detail::types::remove_const< const T >	??
doctest::detail::types::remove_reference< T >	??
doctest::detail::types::remove_reference< T & >	??
doctest::detail::types::remove_reference< T && >	??
doctest::detail::Result	??
doctest::detail::should_stringify_as_underlying_type< T >	??
doctest::String	??
doctest::AssertData::StringContains	??
doctest::detail::StringMakerBase< C >	??
doctest::detail::StringMakerBase< detail::has_insertion_operator< T >value detailtypes::is_pointer< T >value detailtypes::is_array< T >value detailis_pair< T >value detailis_container< T >value >	??
doctest::StringMaker< T >	??
doctest::detail::StringMakerBase< true >	??
doctest::detail::Subcase	??
doctest::SubcaseSignature	??
doctest::TestCaseData	??
doctest::detail::TestCase	??
doctest::TestCaseException	??
doctest::detail::TestFailureException	??
doctest::TestRunStats	??
doctest::detail::TestSuite	??
Timer	??
doctest::detail::types::true_type	??
doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >()), declval< const T & >()), void())>	??
doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())> >	??
doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)> >	??


```

doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype(declval<
    const T & >().c_str()), const char * >value &&types::is_same< decltype(declval< const T
    & >().size()), size_t >value >type > . . . . . ??
doctest::detail::types::is_array< T[SIZE]> . . . . . ??
doctest::detail::types::is_pointer< T * > . . . . . ??
doctest::detail::types::is_rvalue_reference< T && > . . . . . ??
doctest::detail::types::is_same< T, T > . . . . . ??
std::tuple< Types > . . . . . ??
doctest::detail::types::underlying_type< T > . . . . . ??
Zmogus . . . . . ??
    Studentas . . . . . ??

```


Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

doctest::Approx	??
doctest::AssertData	??
std::basic_istream< charT, traits >	??
std::basic_ostream< charT, traits >	??
std::char_traits< charT >	??
doctest::Contains	??
doctest::Context	??
doctest::ContextOptions	??
doctest::detail::ContextScope< L >	??
doctest::detail::ContextScopeBase	??
doctest::CurrentTestCaseStats	??
doctest::detail::deferred_false< T >	??
doctest::detail::types::enable_if< COND, T >	??
doctest::detail::types::enable_if< true, T >	??
doctest::detail::ExceptionTranslator< T >	??
doctest::detail::Expression_lhs< L >	??
doctest::detail::ExpressionDecomposer	??
doctest::detail::types::false_type	??
doctest::detail::filldata< T, Enable >	??
doctest::detail::filldata< const char[N]>	??
doctest::detail::filldata< const void * >	??
doctest::detail::filldata< const volatile void * >	??
doctest::detail::filldata< T * >	??
doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value && detail::is_container< T >::value >::type >	??
doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value && detail::is_pair< T >::value >::type >	??
doctest::detail::filldata< T[N]>	??
doctest::detail::has_insertion_operator< T, typename >	??
doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >()), declval< const T & >()), void())>	??
doctest::IContextScope	??
doctest::detail::IExceptionTranslator	??
doctest::IReporter	??
doctest::detail::types::is_array< T >	??

doctest::detail::types::is_array< T[SIZE]>	??
doctest::detail::is_container< T, typename >	??
doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())> >	??
doctest::detail::types::is_enum< T >	??
doctest::detail::is_pair< T, typename >	??
doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)> >	??
doctest::detail::types::is_pointer< T >	??
doctest::detail::types::is_pointer< T * >	??
doctest::detail::types::is_rvalue_reference< T >	??
doctest::detail::types::is_rvalue_reference< T && >	??
doctest::detail::types::is_same< T, U >	??
doctest::detail::types::is_same< T, T >	??
doctest::detail::is_std_string< T, Enable >	??
doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype(declval< const T & >().c_str()), const char * >::value &&types::is_same< decltype(declval< const T & >().c_str(), size_t >::value >::type >	??
doctest::IsNaN< F >	??
doctest::detail::MessageBuilder	??
doctest::MessageData	??
doctest::QueryData	??
doctest::detail::RelationalComparator< int, L, R >	??
doctest::detail::types::remove_const< T >	??
doctest::detail::types::remove_const< const T >	??
doctest::detail::types::remove_reference< T >	??
doctest::detail::types::remove_reference< T & >	??
doctest::detail::types::remove_reference< T && >	??
doctest::detail::Result	??
doctest::detail::ResultBuilder	??
doctest::detail::should_stringify_as_underlying_type< T >	??
doctest::String	??
doctest::AssertData::StringContains	??
doctest::StringMaker< T >	??
doctest::detail::StringMakerBase< C >	??
doctest::detail::StringMakerBase< true >	??
Studentas	
Klase, skirta saugoti studento duomenis, paveldi Zmogus klase	??
doctest::detail::Subcase	??
doctest::SubcaseSignature	??
doctest::detail::TestCase	??
doctest::TestCaseData	??
doctest::TestCaseException	??
doctest::detail::TestFailureException	??
doctest::TestRunStats	??
doctest::detail::TestSuite	??
Timer	
Klase, skirta programos vykdymo laikui matuoti	??
doctest::detail::types::true_type	??
std::tuple< Types >	??
doctest::detail::types::underlying_type< T >	??
Zmogus	
Abstrakti bazine klase, sauganti bendrus zmogaus duomenis	??

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

include/student.h	Studentas	klases aprasymas	??
include/student_io_deque.h	Funkcijos darbui su Studentas	objektais naudojant deque konteineri	??
include/student_io_list.h			??
include/student_io_vector.h			??
include/timer.h	Laiko matavimo klases aprasymas		??
include/utills.h	Pagalbiniu funkciju ir duomenu aprasymas		??
include/zmogus.h	Zmogus	abstrakcios bazines klases aprasymas	??
src/io_deque.cpp			??
src/io_list.cpp			??
src/io_vector.cpp			??
src/main_deque.cpp			??
src/main_list.cpp			??
src/main_vector.cpp			??
src/student.cpp			??
src/utills.cpp			??
tests/doctest.h			??
tests/test_studentas.cpp			??

Chapter 6

Directory Documentation

6.1 include Directory Reference

Files

- file [student.h](#)
Studentas klases aprasymas.
- file [student_io_deque.h](#)
Funkcijos darbui su Studentas objektais naudojant deque konteineri.
- file [student_io_list.h](#)
- file [student_io_vector.h](#)
- file [timer.h](#)
Laiko matavimo klases aprasymas.
- file [utils.h](#)
Pagalbiniu funkciju ir duomenu aprasymas.
- file [zmogus.h](#)
Zmogus abstrakcios bazines klases aprasymas.

6.2 src Directory Reference

Files

- file [io_deque.cpp](#)
- file [io_list.cpp](#)
- file [io_vector.cpp](#)
- file [main_deque.cpp](#)
- file [main_list.cpp](#)
- file [main_vector.cpp](#)
- file [student.cpp](#)
- file [utils.cpp](#)

6.3 tests Directory Reference

Files

- file [doctest.h](#)
- file [test_studentas.cpp](#)

Chapter 7

Namespace Documentation

7.1 doctest Namespace Reference

Namespaces

- namespace [detail](#)
- namespace [assertType](#)
- namespace [Color](#)
- namespace [TestCaseFailureReason](#)

Classes

- class [String](#)
- struct [StringMaker](#)
- class [Contains](#)
- struct [Approx](#)
- struct [IsNaN](#)
- struct [ContextOptions](#)
- struct [AssertData](#)
- struct [SubcaseSignature](#)
- struct [TestCaseData](#)
- struct [IContextScope](#)
- struct [MessageData](#)
- class [Context](#)
- struct [CurrentTestCaseStats](#)
- struct [TestCaseException](#)
- struct [TestRunStats](#)
- struct [QueryData](#)
- struct [IReporter](#)

Typedefs

- typedef decltype(sizeof(void*)) [size_t](#)

Functions

- [DOCTEST_INTERFACE](#) [String](#) [operator+](#) (const [String](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator==](#) (const [String](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator!=](#) (const [String](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator<](#) (const [String](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator>](#) (const [String](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator<=](#) (const [String](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator>=](#) (const [String](#) &lhs, const [String](#) &rhs)
- [template](#)<typename T>
[String](#) [toString](#) ()
- [template](#)<typename T, typename [detail::types::enable_if](#)<![detail::should_stringify_as_underlying_type](#)< T >::value, bool >::type = true>
[String](#) [toString](#) (const [DOCTEST_REF_WRAP](#)(T) value)
- [String](#) && [toString](#) ([String](#) &&in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (const [String](#) &in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) ([std::nullptr_t](#))
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (bool in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (float in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (double in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (double long in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (char in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (char signed in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (char unsigned in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (short in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (short unsigned in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (signed in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (unsigned in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (long in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (long unsigned in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (long long in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (long long unsigned in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (const [Contains](#) &in)
- [DOCTEST_INTERFACE](#) bool [operator==](#) (const [String](#) &lhs, const [Contains](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator==](#) (const [Contains](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator!=](#) (const [String](#) &lhs, const [Contains](#) &rhs)
- [DOCTEST_INTERFACE](#) bool [operator!=](#) (const [Contains](#) &lhs, const [String](#) &rhs)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) (const [Approx](#) &in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) ([IsNaN](#)< float > in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) ([IsNaN](#)< double > in)
- [DOCTEST_INTERFACE](#) [String](#) [toString](#) ([IsNaN](#)< double long > in)
- [DOCTEST_INTERFACE](#) const [ContextOptions](#) * [getContextOptions](#) ()
- [DOCTEST_INTERFACE](#) const char * [assertString](#) ([assertType::Enum](#) at)
- [DOCTEST_INTERFACE](#) const char * [failureString](#) ([assertType::Enum](#) at)
- [DOCTEST_DEFINE_DECORATOR](#) (test_suite, const char *, "")
- [DOCTEST_DEFINE_DECORATOR](#) (description, const char *, "")
- [DOCTEST_DEFINE_DECORATOR](#) (skip, bool, true)
- [DOCTEST_DEFINE_DECORATOR](#) (no_breaks, bool, true)
- [DOCTEST_DEFINE_DECORATOR](#) (no_output, bool, true)
- [DOCTEST_DEFINE_DECORATOR](#) (timeout, double, 0)
- [DOCTEST_DEFINE_DECORATOR](#) (may_fail, bool, true)
- [DOCTEST_DEFINE_DECORATOR](#) (should_fail, bool, true)
- [DOCTEST_DEFINE_DECORATOR](#) (expected_failures, int, 0)
- [template](#)<typename T>
int [registerExceptionTranslator](#) ([String](#)(*translateFunction)(T)) noexcept
- [DOCTEST_INTERFACE](#) const char * [skipPathFromFilename](#) (const char *file)
- [template](#)<typename Reporter>
int [registerReporter](#) (const char *name, int priority, bool isReporter) noexcept

Variables

- template struct [DOCTEST_INTERFACE_DECL](#) [IsNaN](#)< float >
- template struct [DOCTEST_INTERFACE_DECL](#) [IsNaN](#)< double >
- template struct [DOCTEST_INTERFACE_DECL](#) [IsNaN](#)< long double >
- struct [DOCTEST_INTERFACE](#) [TestCaseData](#)
- [DOCTEST_INTERFACE](#) bool [is_running_in_test](#)

7.1.1 Typedef Documentation

7.1.1.1 [size_t](#)

```
typedef decltype(sizeof(void *)) std::size\_t
```

7.1.2 Function Documentation

7.1.2.1 [assertString\(\)](#)

```
DOCTEST\_INTERFACE const char * doctest::assertString (
    assertType::Enum at)
```

7.1.2.2 [DOCTEST_DEFINE_DECORATOR\(\)](#) [1/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    description ,
    const char * ,
    "" )
```

7.1.2.3 [DOCTEST_DEFINE_DECORATOR\(\)](#) [2/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    expected_failures ,
    int ,
    0 )
```

7.1.2.4 [DOCTEST_DEFINE_DECORATOR\(\)](#) [3/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    may_fail ,
    bool ,
    true )
```

7.1.2.5 [DOCTEST_DEFINE_DECORATOR\(\)](#) [4/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    no_breaks ,
    bool ,
    true )
```

7.1.2.6 DOCTEST_DEFINE_DECORATOR() [5/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    no_output ,
    bool ,
    true )
```

7.1.2.7 DOCTEST_DEFINE_DECORATOR() [6/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    should_fail ,
    bool ,
    true )
```

7.1.2.8 DOCTEST_DEFINE_DECORATOR() [7/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    skip ,
    bool ,
    true )
```

7.1.2.9 DOCTEST_DEFINE_DECORATOR() [8/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    test_suite ,
    const char * ,
    "" )
```

7.1.2.10 DOCTEST_DEFINE_DECORATOR() [9/9]

```
doctest::DOCTEST_DEFINE_DECORATOR (
    timeout ,
    double ,
    0 )
```

7.1.2.11 failureString()

```
DOCTEST_INTERFACE const char * doctest::failureString (
    assertType::Enum at)
```

7.1.2.12 getContextOptions()

```
DOCTEST_INTERFACE const ContextOptions * doctest::getContextOptions ()
```

7.1.2.13 operator"!=([1/3]

```
DOCTEST_INTERFACE bool doctest::operator!= (
    const Contains & lhs,
    const String & rhs)
```

7.1.2.14 operator"!=([2/3]

```
DOCTEST_INTERFACE bool doctest::operator!= (
    const String & lhs,
    const Contains & rhs)
```

7.1.2.15 operator"!=([3/3]

```
DOCTEST_INTERFACE bool doctest::operator!= (
    const String & lhs,
    const String & rhs)
```

7.1.2.16 operator+()

```
DOCTEST_INTERFACE String doctest::operator+ (
    const String & lhs,
    const String & rhs)
```

7.1.2.17 operator<()

```
DOCTEST_INTERFACE bool doctest::operator< (
    const String & lhs,
    const String & rhs)
```

7.1.2.18 operator<=()

```
DOCTEST_INTERFACE bool doctest::operator<= (
    const String & lhs,
    const String & rhs)
```

7.1.2.19 operator==([1/3]

```
DOCTEST_INTERFACE bool doctest::operator==(
    const Contains & lhs,
    const String & rhs)
```

7.1.2.20 operator==([2/3]

```
DOCTEST_INTERFACE bool doctest::operator==(
    const String & lhs,
    const Contains & rhs)
```

7.1.2.21 operator==() [3/3]

```
DOCTEST_INTERFACE bool doctest::operator== (
    const String & lhs,
    const String & rhs)
```

7.1.2.22 operator>()

```
DOCTEST_INTERFACE bool doctest::operator> (
    const String & lhs,
    const String & rhs)
```

7.1.2.23 operator>=()

```
DOCTEST_INTERFACE bool doctest::operator>= (
    const String & lhs,
    const String & rhs)
```

7.1.2.24 registerExceptionTranslator()

```
template<typename T>
int doctest::registerExceptionTranslator (
    String(* translateFunction ) (T)) [noexcept]
```

7.1.2.25 registerReporter()

```
template<typename Reporter>
int doctest::registerReporter (
    const char * name,
    int priority,
    bool isReporter) [noexcept]
```

7.1.2.26 skipPathFromFilename()

```
DOCTEST_INTERFACE const char * doctest::skipPathFromFilename (
    const char * file)
```

7.1.2.27 toString() [1/25]

```
template<typename T>
String doctest::toString ( )
```

7.1.2.28 toString() [2/25]

```
DOCTEST_INTERFACE String doctest::toString (
    bool in)
```

7.1.2.29 toString() [3/25]

```
DOCTEST_INTERFACE String doctest::toString (
    char in)
```

7.1.2.30 toString() [4/25]

```
DOCTEST_INTERFACE String doctest::toString (
    char signed in)
```

7.1.2.31 toString() [5/25]

```
DOCTEST_INTERFACE String doctest::toString (
    char unsigned in)
```

7.1.2.32 toString() [6/25]

```
DOCTEST_INTERFACE String doctest::toString (
    const Approx & in)
```

7.1.2.33 toString() [7/25]

```
DOCTEST_INTERFACE String doctest::toString (
    const Contains & in)
```

7.1.2.34 toString() [8/25]

```
template<typename T, typename detail::types::enable_if<!detail::should_stringify_as_underlying_type<
T >::value, bool >::type = true>
String doctest::toString (
    const DOCTEST_REF_WRAP(T) value)
```

7.1.2.35 toString() [9/25]

```
DOCTEST_INTERFACE String doctest::toString (
    const String & in)
```

7.1.2.36 toString() [10/25]

```
DOCTEST_INTERFACE String doctest::toString (
    double in)
```

7.1.2.37 toString() [11/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    double long in)
```

7.1.2.38 toString() [12/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    float in)
```

7.1.2.39 toString() [13/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    IsNaN< double > in)
```

7.1.2.40 toString() [14/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    IsNaN< double long > in)
```

7.1.2.41 toString() [15/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    IsNaN< float > in)
```

7.1.2.42 toString() [16/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    long in)
```

7.1.2.43 toString() [17/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    long long in)
```

7.1.2.44 toString() [18/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    long long unsigned in)
```

7.1.2.45 toString() [19/25]

```
DOCTEST_INTERFACE String doctest::toString (  
    long unsigned in)
```


7.1.2.46 toString() [20/25]

```
DOCTEST_INTERFACE String doctest::toString (
    short in)
```

7.1.2.47 toString() [21/25]

```
DOCTEST_INTERFACE String doctest::toString (
    short unsigned in)
```

7.1.2.48 toString() [22/25]

```
DOCTEST_INTERFACE String doctest::toString (
    signed in)
```

7.1.2.49 toString() [23/25]

```
DOCTEST_INTERFACE String doctest::toString (
    std::nullptr_t )
```

7.1.2.50 toString() [24/25]

```
String && doctest::toString (
    String && in) [inline]
```

7.1.2.51 toString() [25/25]

```
DOCTEST_INTERFACE String doctest::toString (
    unsigned in)
```

7.1.3 Variable Documentation**7.1.3.1 is_running_in_test**

```
DOCTEST_INTERFACE bool doctest::is_running_in_test [extern]
```

7.1.3.2 IsNaN< double >

```
template struct DOCTEST_INTERFACE_DECL doctest::IsNaN< double > [extern]
```

7.1.3.3 IsNaN< float >

```
template struct DOCTEST_INTERFACE_DECL doctest::IsNaN< float > [extern]
```

7.1.3.4 IsNaN< long double >

```
template struct DOCTEST_INTERFACE_DECL doctest::IsNaN< long double > [extern]
```

7.1.3.5 TestCaseData

```
struct DOCTEST_INTERFACE doctest::TestCaseData
```

7.2 doctest::assertType Namespace Reference

Enumerations

- enum Enum {

is_warn = 1, is_check = 2 * is_warn, is_require = 2 * is_check, is_normal = 2 * is_require,

is_throws = 2 * is_normal, is_throws_as = 2 * is_throws, is_throws_with = 2 * is_throws_as, is_nothrow =

2 * is_throws_with,

is_false = 2 * is_nothrow, is_unary = 2 * is_false, is_eq = 2 * is_unary, is_ne = 2 * is_eq,

is_lt = 2 * is_ne, is_gt = 2 * is_lt, is_ge = 2 * is_gt, is_le = 2 * is_ge,

DT_WARN = is_normal | is_warn, DT_CHECK = is_normal | is_check, DT_REQUIRE = is_normal | is_↵

require, DT_WARN_FALSE = is_normal | is_false | is_warn,

DT_CHECK_FALSE = is_normal | is_false | is_check, DT_REQUIRE_FALSE = is_normal | is_false | is_↵

require, DT_WARN_THROWS = is_throws | is_warn, DT_CHECK_THROWS = is_throws | is_check,

DT_REQUIRE_THROWS = is_throws | is_require, DT_WARN_THROWS_AS = is_throws_as | is_warn

, DT_CHECK_THROWS_AS = is_throws_as | is_check, DT_REQUIRE_THROWS_AS = is_throws_as |

is_require,

DT_WARN_THROWS_WITH = is_throws_with | is_warn, DT_CHECK_THROWS_WITH = is_throws_with |

is_check, DT_REQUIRE_THROWS_WITH = is_throws_with | is_require, DT_WARN_THROWS_WITH_AS

= is_throws_with | is_throws_as | is_warn,

DT_CHECK_THROWS_WITH_AS = is_throws_with | is_throws_as | is_check, DT_REQUIRE_THROWS_WITH_AS

= is_throws_with | is_throws_as | is_require, DT_WARN_NOTHROW = is_nothrow | is_warn,

DT_CHECK_NOTHROW = is_nothrow | is_check,

DT_REQUIRE_NOTHROW = is_nothrow | is_require, DT_WARN_EQ = is_normal | is_eq | is_warn,

DT_CHECK_EQ = is_normal | is_eq | is_check, DT_REQUIRE_EQ = is_normal | is_eq | is_require,

DT_WARN_NE = is_normal | is_ne | is_warn, DT_CHECK_NE = is_normal | is_ne | is_check,

DT_REQUIRE_NE = is_normal | is_ne | is_require, DT_WARN_GT = is_normal | is_gt | is_warn,

DT_CHECK_GT = is_normal | is_gt | is_check, DT_REQUIRE_GT = is_normal | is_gt | is_require,

DT_WARN_LT = is_normal | is_lt | is_warn, DT_CHECK_LT = is_normal | is_lt | is_check,

DT_REQUIRE_LT = is_normal | is_lt | is_require, DT_WARN_GE = is_normal | is_ge | is_warn,

DT_CHECK_GE = is_normal | is_ge | is_check, DT_REQUIRE_GE = is_normal | is_ge | is_require

,

DT_WARN_LE = is_normal | is_le | is_warn, DT_CHECK_LE = is_normal | is_le | is_check,

DT_REQUIRE_LE = is_normal | is_le | is_require, DT_WARN_UNARY = is_normal | is_unary | is_↵

warn,

DT_CHECK_UNARY = is_normal | is_unary | is_check, DT_REQUIRE_UNARY = is_normal | is_↵

_unary | is_require, DT_WARN_UNARY_FALSE = is_normal | is_false | is_unary | is_warn,

DT_CHECK_UNARY_FALSE = is_normal | is_false | is_unary | is_check,

DT_REQUIRE_UNARY_FALSE = is_normal | is_false | is_unary | is_require }

7.2.1 Enumeration Type Documentation

7.2.1.1 Enum

enum `doctest::assertType::Enum`

Enumerator

is_warn	
is_check	
is_require	
is_normal	
is_throws	
is_throws_as	
is_throws_with	
is_nothrow	
is_false	
is_unary	
is_eq	
is_ne	
is_lt	
is_gt	
is_ge	
is_le	
DT_WARN	
DT_CHECK	
DT_REQUIRE	
DT_WARN_FALSE	
DT_CHECK_FALSE	
DT_REQUIRE_FALSE	
DT_WARN_THROWS	
DT_CHECK_THROWS	
DT_REQUIRE_THROWS	
DT_WARN_THROWS_AS	
DT_CHECK_THROWS_AS	
DT_REQUIRE_THROWS_AS	
DT_WARN_THROWS_WITH	
DT_CHECK_THROWS_WITH	
DT_REQUIRE_THROWS_WITH	
DT_WARN_THROWS_WITH_AS	
DT_CHECK_THROWS_WITH_AS	
DT_REQUIRE_THROWS_WITH_AS	

DT_WARN_NOTHROW	
DT_CHECK_NOTHROW	
DT_REQUIRE_NOTHROW	
DT_WARN_EQ	
DT_CHECK_EQ	
DT_REQUIRE_EQ	
DT_WARN_NE	
DT_CHECK_NE	
DT_REQUIRE_NE	
DT_WARN_GT	
DT_CHECK_GT	
DT_REQUIRE_GT	
DT_WARN_LT	
DT_CHECK_LT	
DT_REQUIRE_LT	
DT_WARN_GE	
DT_CHECK_GE	
DT_REQUIRE_GE	
DT_WARN_LE	
DT_CHECK_LE	
DT_REQUIRE_LE	
DT_WARN_UNARY	
DT_CHECK_UNARY	
DT_REQUIRE_UNARY	
DT_WARN_UNARY_FALSE	
DT_CHECK_UNARY_FALSE	
DT_REQUIRE_UNARY_FALSE	

7.3 doctest::Color Namespace Reference

Enumerations

- enum [Enum](#) {
[None](#) = 0 , [White](#) , [Red](#) , [Green](#) ,
[Blue](#) , [Cyan](#) , [Yellow](#) , [Grey](#) ,
[Bright](#) = 0x10 , [BrightRed](#) = Bright | Red , [BrightGreen](#) = Bright | Green , [LightGrey](#) = Bright | Grey ,
[BrightWhite](#) = Bright | White }

Functions

- [DOCTEST_INTERFACE](#) [std::ostream](#) & [operator<<](#) ([std::ostream](#) &s, [Color::Enum](#) code)

7.3.1 Enumeration Type Documentation

7.3.1.1 Enum

```
enum doctest::Color::Enum
```

Enumerator

None	
White	
Red	
Green	
Blue	
Cyan	
Yellow	
Grey	
Bright	
BrightRed	
BrightGreen	
LightGrey	
BrightWhite	

7.3.2 Function Documentation

7.3.2.1 operator<<()

```
DOCTEST_INTERFACE std::ostream & doctest::Color::operator<< (
    std::ostream & s,
    Color::Enum code)
```

7.4 doctest::detail Namespace Reference

Namespaces

- namespace [types](#)
- namespace [binaryAssertComparison](#)

Classes

- struct [deferred_false](#)
- struct [is_std_string](#)
- struct [is_std_string< T, typename types::enable_if< types::is_same< decltype\(declval< const T >\(\).c_str\(\)\), const char * >::value &&types::is_same< decltype\(declval< const T >\(\).size\(\)\), size_t >::value >::type >](#)
- struct [has_insertion_operator](#)

- struct [has_insertion_operator](#)< T, decltype(operator<<(declval< std::ostream & >(), declval< const T & >()), void())>
- struct [should_stringify_as_underlying_type](#)
- struct [is_pair](#)
- struct [is_pair](#)< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)> >
- struct [is_container](#)
- struct [is_container](#)< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())> >
- struct [StringMakerBase](#)
- struct [filldata](#)
- struct [StringMakerBase](#)< true >
- struct [filldata](#)< T[N]>
- struct [filldata](#)< const char[N]>
- struct [filldata](#)< const void * >
- struct [filldata](#)< const volatile void * >
- struct [filldata](#)< T * >
- struct [filldata](#)< T, typename detail::types::enable_if<!detail::has_insertion_operator< T >::value &&detail::is_pair< T >::value >::type >
- struct [filldata](#)< T, typename detail::types::enable_if<!detail::has_insertion_operator< T >::value &&detail::is_container< T >::value >::type >
- struct [RelationalComparator](#)
- struct [Result](#)
- struct [ResultBuilder](#)
- struct [Expression_lhs](#)
- struct [ExpressionDecomposer](#)
- struct [Subcase](#)
- struct [TestSuite](#)
- struct [TestCase](#)
- struct [IExceptionTranslator](#)
- class [ExceptionTranslator](#)
- struct [ContextScopeBase](#)
- class [ContextScope](#)
- struct [MessageBuilder](#)
- struct [TestFailureException](#)

Typedefs

- using [funcType](#) = void (*)()
- using [assert_handler](#) = void (*)(const [AssertData](#) &)
- using [reporterCreatorFunc](#) = [IReporter](#) (*)(const [ContextOptions](#) &)

Functions

- [DOCTEST_INTERFACE](#) bool [isDebuggerActive](#) ()
- template<typename T>
T && [declval](#) ()
- template<class T>
[DOCTEST_CONSTEXPR_FUNC](#) T && [forward](#) (typename [types::remove_reference](#)< T >::type &t)
[DOCTEST_NOEXCEPT](#)
- template<class T>
[DOCTEST_CONSTEXPR_FUNC](#) T && [forward](#) (typename [types::remove_reference](#)< T >::type &&t)
[DOCTEST_NOEXCEPT](#)

- [DOCTEST_INTERFACE](#) [std::ostream](#) * [tlssPush](#) ()
- [DOCTEST_INTERFACE](#) [String](#) [tlssPop](#) ()
- [template](#)<typename T>
void [filloss](#) ([std::ostream](#) *stream, const T &in)
- [template](#)<typename T, [size_t](#) N>
void [filloss](#) ([std::ostream](#) *stream, const T(&in)[N])
- [template](#)<typename T>
[String](#) [toStream](#) (const T &in)
- [template](#)<typename L, typename R>
[String](#) [stringifyBinaryExpr](#) (const [DOCTEST_REF_WRAP](#)(L) lhs, const char *op, const [DOCTEST_REF_WRAP](#)(R) rhs)
- [DOCTEST_INTERFACE](#) int [setTestSuite](#) (const [TestSuite](#) &ts) noexcept
- [DOCTEST_INTERFACE](#) int [regTest](#) (const [TestCase](#) &tc) noexcept
- [DOCTEST_INTERFACE](#) void [registerExceptionTranslatorImpl](#) (const [IExceptionTranslator](#) *et) noexcept
- [template](#)<typename L>
[ContextScope](#)< L > [MakeContextScope](#) (const L &lambda)
- [DOCTEST_INTERFACE](#) bool [checkIfShouldThrow](#) ([assertType::Enum](#) at)
- [DOCTEST_INTERFACE](#) void [throwException](#) ()
- [DOCTEST_INTERFACE](#) void [failed_out_of_a_testing_context](#) (const [AssertData](#) &ad)
- [DOCTEST_INTERFACE](#) bool [decomp_assert](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const [Result](#) &result)
- [template](#)<int comparison, typename L, typename R>
[DOCTEST_NOINLINE](#) bool [binary_assert](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const [DOCTEST_REF_WRAP](#)(L) lhs, const [DOCTEST_REF_WRAP](#)(R) rhs)
- [template](#)<typename L>
[DOCTEST_NOINLINE](#) bool [unary_assert](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const [DOCTEST_REF_WRAP](#)(L) val)
- [DOCTEST_INTERFACE](#) void [registerReporterImpl](#) (const char *name, int prio, [reporterCreatorFunc](#) c, bool isReporter) noexcept
- [template](#)<typename Reporter>
[IReporter](#) * [reporterCreator](#) (const [ContextOptions](#) &o)
- [DOCTEST_INTERFACE](#) [size_t](#) [acquireGeneratorDecisionIndex](#) ([size_t](#) count)
- [template](#)<typename T, typename... Rest>
T [acquireGeneratorValue](#) (T first, Rest... rest)
- [template](#)<typename T>
int [instantiationHelper](#) (const T &) noexcept

Variables

- struct [DOCTEST_INTERFACE](#) [TestCase](#)

7.4.1 Typedef Documentation

7.4.1.1 assert_handler

```
using doctest::detail::assert\_handler = void (*) (const AssertData &)
```

7.4.1.2 funcType

```
using doctest::detail::funcType = void (*) ()
```

7.4.1.3 reporterCreatorFunc

```
using doctest::detail::reporterCreatorFunc = IReporter (*)(const ContextOptions &)
```

7.4.2 Function Documentation

7.4.2.1 acquireGeneratorDecisionIndex()

```
DOCTEST_INTERFACE size_t doctest::detail::acquireGeneratorDecisionIndex (
    size_t count)
```

7.4.2.2 acquireGeneratorValue()

```
template<typename T, typename... Rest>
T doctest::detail::acquireGeneratorValue (
    T first,
    Rest... rest)
```

7.4.2.3 binary_assert()

```
template<int comparison, typename L, typename R>
DOCTEST_NOINLINE bool doctest::detail::binary_assert (
    assertType::Enum at,
    const char * file,
    int line,
    const char * expr,
    const DOCTEST_REF_WRAP(L) lhs,
    const DOCTEST_REF_WRAP(R) rhs)
```

7.4.2.4 checkIfShouldThrow()

```
DOCTEST_INTERFACE bool doctest::detail::checkIfShouldThrow (
    assertType::Enum at)
```

7.4.2.5 declval()

```
template<typename T>
T && doctest::detail::declval ()
```

7.4.2.6 decomp_assert()

```
DOCTEST_INTERFACE bool doctest::detail::decomp_assert (
    assertType::Enum at,
    const char * file,
    int line,
    const char * expr,
    const Result & result)
```


7.4.2.7 failed_out_of_a_testing_context()

```
DOCTEST_INTERFACE void doctest::detail::failed_out_of_a_testing_context (
    const AssertData & ad)
```

7.4.2.8 filloss() [1/2]

```
template<typename T>
void doctest::detail::filloss (
    std::ostream * stream,
    const T & in)
```

7.4.2.9 filloss() [2/2]

```
template<typename T, size_t N>
void doctest::detail::filloss (
    std::ostream * stream,
    const T(&) in[N])
```

7.4.2.10 forward() [1/2]

```
template<class T>
DOCTEST_CONSTEXPR_FUNC T && doctest::detail::forward (
    typename types::remove_reference< T >::type && t)
```

7.4.2.11 forward() [2/2]

```
template<class T>
DOCTEST_CONSTEXPR_FUNC T && doctest::detail::forward (
    typename types::remove_reference< T >::type & t)
```

7.4.2.12 instantiationHelper()

```
template<typename T>
int doctest::detail::instantiationHelper (
    const T & ) [noexcept]
```

7.4.2.13 isDebuggerActive()

```
DOCTEST_INTERFACE bool doctest::detail::isDebuggerActive ()
```

7.4.2.14 MakeContextScope()

```
template<typename L>
ContextScope< L > doctest::detail::MakeContextScope (
    const L & lambda)
```

7.4.2.15 registerExceptionTranslatorImpl()

```
DOCTEST_INTERFACE void doctest::detail::registerExceptionTranslatorImpl (
    const IExceptionTranslator * et) [noexcept]
```

7.4.2.16 registerReporterImpl()

```
DOCTEST_INTERFACE void doctest::detail::registerReporterImpl (
    const char * name,
    int prio,
    reporterCreatorFunc c,
    bool isReporter) [noexcept]
```

7.4.2.17 regTest()

```
DOCTEST_INTERFACE int doctest::detail::regTest (
    const TestCase & tc) [noexcept]
```

7.4.2.18 reporterCreator()

```
template<typename Reporter>
IReporter * doctest::detail::reporterCreator (
    const ContextOptions & o)
```

7.4.2.19 setTestSuite()

```
DOCTEST_INTERFACE int doctest::detail::setTestSuite (
    const TestSuite & ts) [noexcept]
```

7.4.2.20 stringifyBinaryExpr()

```
template<typename L, typename R>
String doctest::detail::stringifyBinaryExpr (
    const DOCTEST_REF_WRAP(L) lhs,
    const char * op,
    const DOCTEST_REF_WRAP(R) rhs)
```

7.4.2.21 throwException()

```
DOCTEST_INTERFACE void doctest::detail::throwException ()
```

7.4.2.22 tlssPop()

```
DOCTEST_INTERFACE String doctest::detail::tlssPop ()
```

7.4.2.23 tlssPush()

```
DOCTEST_INTERFACE std::ostream * doctest::detail::tlssPush ()
```

7.4.2.24 toStream()

```
template<typename T>
String doctest::detail::toStream (
    const T & in)
```

7.4.2.25 unary_assert()

```
template<typename L>
DOCTEST_NOINLINE bool doctest::detail::unary_assert (
    assertType::Enum at,
    const char * file,
    int line,
    const char * expr,
    const DOCTEST_REF_WRAP(L) val)
```

7.4.3 Variable Documentation

7.4.3.1 TestCase

```
struct DOCTEST_INTERFACE doctest::detail::TestCase
```

7.5 doctest::detail::binaryAssertComparison Namespace Reference

Enumerations

- enum Enum {
eq = 0, ne, gt, lt,
ge, le }

7.5.1 Enumeration Type Documentation

7.5.1.1 Enum

```
enum doctest::detail::binaryAssertComparison::Enum
```

Enumerator

eq	
----	--

ne	
gt	
lt	
ge	
le	

7.6 doctest::detail::types Namespace Reference

Classes

- struct [enable_if](#)
- struct [enable_if< true, T >](#)
- struct [true_type](#)
- struct [false_type](#)
- struct [remove_reference](#)
- struct [remove_reference< T & >](#)
- struct [remove_reference< T && >](#)
- struct [is_same](#)
- struct [is_same< T, T >](#)
- struct [is_rvalue_reference](#)
- struct [is_rvalue_reference< T && >](#)
- struct [remove_const](#)
- struct [remove_const< const T >](#)
- struct [is_enum](#)
- struct [underlying_type](#)
- struct [is_pointer](#)
- struct [is_pointer< T * >](#)
- struct [is_array](#)
- struct [is_array< T\[SIZE\]>](#)

Typedefs

- [template<typename... Unused>](#)
using [void_t](#) = void

7.6.1 Typedef Documentation

7.6.1.1 void_t

```
template<typename... Unused>
using doctest::detail::types::void\_t = void
```

7.7 doctest::TestCaseFailureReason Namespace Reference

Enumerations

- enum [Enum](#) {
[None](#) = 0 , [AssertFailure](#) = 1 , [Exception](#) = 2 , [Crash](#) = 4 ,
[TooManyFailedAsserts](#) = 8 , [Timeout](#) = 16 , [ShouldHaveFailedButDidnt](#) = 32 , [ShouldHaveFailedAndDid](#) = 64
 ,
[DidntFailExactlyNumTimes](#) = 128 , [FailedExactlyNumTimes](#) = 256 , [CouldHaveFailedAndDid](#) = 512 }

7.7.1 Enumeration Type Documentation

7.7.1.1 Enum

```
enum doctest::TestCaseFailureReason::Enum
```

Enumerator

None	
AssertFailure	
Exception	
Crash	
TooManyFailedAsserts	
Timeout	
ShouldHaveFailedButDidnt	
ShouldHaveFailedAndDid	
DidntFailExactlyNumTimes	
FailedExactlyNumTimes	
CouldHaveFailedAndDid	

7.8 doctest_detail_test_suite_ns Namespace Reference

Functions

- [DOCTEST_INTERFACE doctest::detail::TestSuite & getCurrentTestSuite \(\)](#) noexcept

7.8.1 Function Documentation

7.8.1.1 getCurrentTestSuite()

```
DOCTEST_INTERFACE doctest::detail::TestSuite & doctest_detail_test_suite_ns::getCurrentTestSuite() [noexcept]
```

7.9 std Namespace Reference

Classes

- struct [char_traits](#)
- class [basic_ostream](#)
- class [basic_istream](#)
- class [tuple](#)

Typedefs

- typedef decltype(nullptr) [nullptr_t](#)
- typedef decltype(sizeof(void*)) [size_t](#)
- typedef [basic_ostream](#)< char, [char_traits](#)< char > > [ostream](#)
- typedef [basic_istream](#)< char, [char_traits](#)< char > > [istream](#)

Functions

- template<class traits>
[basic_ostream](#)< char, traits > & [operator<<](#) ([basic_ostream](#)< char, traits > &, const char *)

7.9.1 Typedef Documentation

7.9.1.1 istream

```
typedef basic\_istream<char, char\_traits<char> > std::istream
```

7.9.1.2 nullptr_t

```
typedef decltype(nullptr) std::nullptr\_t
```

7.9.1.3 ostream

```
typedef basic\_ostream<char, char\_traits<char> > std::ostream
```

7.9.1.4 size_t

```
typedef decltype(sizeof(void*)) std::size\_t
```

7.9.2 Function Documentation

7.9.2.1 operator<<()

```
template<class traits>
basic\_ostream< char, traits > & std::operator<< (
    basic\_ostream< char, traits > & ,
    const char * )
```

Chapter 8

Class Documentation

8.1 doctest::Approx Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- [Approx](#) (double value)
- [Approx operator\(\)](#) (double value) const
- [Approx & epsilon](#) (double newEpsilon)
- [Approx & scale](#) (double newScale)

Public Attributes

- double [m_epsilon](#)
- double [m_scale](#)
- double [m_value](#)

Friends

- [DOCTEST_INTERFACE](#) friend bool [operator==](#) (double lhs, const [Approx](#) &rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator==](#) (const [Approx](#) &lhs, double rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator!=](#) (double lhs, const [Approx](#) &rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator!=](#) (const [Approx](#) &lhs, double rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator<=](#) (double lhs, const [Approx](#) &rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator<=](#) (const [Approx](#) &lhs, double rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator>=](#) (double lhs, const [Approx](#) &rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator>=](#) (const [Approx](#) &lhs, double rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator<](#) (double lhs, const [Approx](#) &rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator<](#) (const [Approx](#) &lhs, double rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator>](#) (double lhs, const [Approx](#) &rhs)
- [DOCTEST_INTERFACE](#) friend bool [operator>](#) (const [Approx](#) &lhs, double rhs)

8.1.1 Constructor & Destructor Documentation

8.1.1.1 Approx()

```
doctest::Approx::Approx (
    double value)
```

8.1.2 Member Function Documentation

8.1.2.1 epsilon()

```
Approx & doctest::Approx::epsilon (
    double newEpsilon)
```

8.1.2.2 operator>()

```
Approx doctest::Approx::operator() (
    double value) const
```

8.1.2.3 scale()

```
Approx & doctest::Approx::scale (
    double newScale)
```

8.1.3 Friends And Related Symbol Documentation

8.1.3.1 operator"!=" [1/2]

```
DOCTEST_INTERFACE friend bool operator!= (
    const Approx & lhs,
    double rhs) [friend]
```

8.1.3.2 operator"!=" [2/2]

```
DOCTEST_INTERFACE friend bool operator!= (
    double lhs,
    const Approx & rhs) [friend]
```

8.1.3.3 operator< [1/2]

```
DOCTEST_INTERFACE friend bool operator< (
    const Approx & lhs,
    double rhs) [friend]
```


8.1.3.4 operator< [2/2]

```
DOCTEST_INTERFACE friend bool operator< (  
    double lhs,  
    const Approx & rhs) [friend]
```

8.1.3.5 operator<= [1/2]

```
DOCTEST_INTERFACE friend bool operator<= (  
    const Approx & lhs,  
    double rhs) [friend]
```

8.1.3.6 operator<= [2/2]

```
DOCTEST_INTERFACE friend bool operator<= (  
    double lhs,  
    const Approx & rhs) [friend]
```

8.1.3.7 operator== [1/2]

```
DOCTEST_INTERFACE friend bool operator== (  
    const Approx & lhs,  
    double rhs) [friend]
```

8.1.3.8 operator== [2/2]

```
DOCTEST_INTERFACE friend bool operator== (  
    double lhs,  
    const Approx & rhs) [friend]
```

8.1.3.9 operator> [1/2]

```
DOCTEST_INTERFACE friend bool operator> (  
    const Approx & lhs,  
    double rhs) [friend]
```

8.1.3.10 operator> [2/2]

```
DOCTEST_INTERFACE friend bool operator> (  
    double lhs,  
    const Approx & rhs) [friend]
```

8.1.3.11 operator>= [1/2]

```
DOCTEST_INTERFACE friend bool operator>= (  
    const Approx & lhs,  
    double rhs) [friend]
```

8.1.3.12 operator>= [2/2]

```
DOCTEST_INTERFACE friend bool operator>= (
    double lhs,
    const Approx & rhs) [friend]
```

8.1.4 Member Data Documentation

8.1.4.1 m_epsilon

```
double doctest::Approx::m_epsilon
```

8.1.4.2 m_scale

```
double doctest::Approx::m_scale
```

8.1.4.3 m_value

```
double doctest::Approx::m_value
```

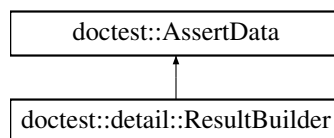
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.2 doctest::AssertData Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::AssertData:



Classes

- class [StringContains](#)

Public Member Functions

- [AssertData](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const char *exception_type, const [StringContains](#) &exception_string)

Public Attributes

- const [TestCaseData](#) * [m_test_case](#)
- [assertType::Enum](#) [m_at](#)
- const char * [m_file](#)
- int [m_line](#)
- const char * [m_expr](#)
- bool [m_failed](#)
- bool [m_threw](#)
- [String](#) [m_exception](#)
- [String](#) [m_decomp](#)
- bool [m_threw_as](#)
- const char * [m_exception_type](#)
- class [DOCTEST_INTERFACE](#) [doctest::AssertData::StringContains](#) [m_exception_string](#)

8.2.1 Constructor & Destructor Documentation

8.2.1.1 AssertData()

```
doctest::AssertData::AssertData (
    assertType::Enum at,
    const char * file,
    int line,
    const char * expr,
    const char * exception_type,
    const StringContains & exception_string)
```

8.2.2 Member Data Documentation

8.2.2.1 m_at

```
assertType::Enum doctest::AssertData::m_at
```

8.2.2.2 m_decomp

```
String doctest::AssertData::m_decomp
```

8.2.2.3 m_exception

```
String doctest::AssertData::m_exception
```

8.2.2.4 m_exception_string

```
class DOCTEST\_INTERFACE doctest::AssertData::StringContains doctest::AssertData::m_exception←  
_string
```

8.2.2.5 m_exception_type

```
const char* doctest::AssertData::m_exception_type
```

8.2.2.6 m_expr

```
const char* doctest::AssertData::m_expr
```

8.2.2.7 m_failed

```
bool doctest::AssertData::m_failed
```

8.2.2.8 m_file

```
const char* doctest::AssertData::m_file
```

8.2.2.9 m_line

```
int doctest::AssertData::m_line
```

8.2.2.10 m_test_case

```
const TestCaseData* doctest::AssertData::m_test_case
```

8.2.2.11 m_threw

```
bool doctest::AssertData::m_threw
```

8.2.2.12 m_threw_as

```
bool doctest::AssertData::m_threw_as
```

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

8.3 std::basic_istream< charT, traits > Class Template Reference

The documentation for this class was generated from the following file:

- [tests/doctest.h](#)

8.4 std::basic_ostream< charT, traits > Class Template Reference

The documentation for this class was generated from the following file:

- tests/[doctest.h](#)

8.5 std::char_traits< charT > Struct Template Reference

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.6 doctest::Contains Class Reference

```
#include <doctest.h>
```

Public Member Functions

- [Contains](#) (const [String](#) &string)
- bool [checkWith](#) (const [String](#) &other) const

Public Attributes

- [String](#) string

8.6.1 Constructor & Destructor Documentation

8.6.1.1 Contains()

```
doctest::Contains::Contains (  
    const String & string) [explicit]
```

8.6.2 Member Function Documentation

8.6.2.1 checkWith()

```
bool doctest::Contains::checkWith (  
    const String & other) const
```

8.6.3 Member Data Documentation

8.6.3.1 string

`String doctest::Contains::string`

The documentation for this class was generated from the following file:

- tests/[doctest.h](#)

8.7 doctest::Context Class Reference

```
#include <doctest.h>
```

Public Member Functions

- [Context](#) (int argc=0, const char *const *argv=nullptr)
- [Context](#) (const Context &)=delete
- [Context](#) (Context &&)=delete
- [Context](#) & operator= (const [Context](#) &)=delete
- [Context](#) & operator= ([Context](#) &&)=delete
- [~Context](#) ()
- void [applyCommandLine](#) (int argc, const char *const *argv)
- void [addFilter](#) (const char *filter, const char *value)
- void [clearFilters](#) ()
- void [setOption](#) (const char *option, bool value)
- void [setOption](#) (const char *option, int value)
- void [setOption](#) (const char *option, const char *value)
- bool [shouldExit](#) ()
- void [setAsDefaultForAssertsOutOfTestCases](#) ()
- void [setAssertHandler](#) (detail::assert_handler ah)
- void [setCout](#) (std::ostream *out)
- int [run](#) ()

8.7.1 Constructor & Destructor Documentation

8.7.1.1 Context() [1/3]

```
doctest::Context::Context (
    int argc = 0,
    const char *const * argv = nullptr) [explicit]
```

8.7.1.2 Context() [2/3]

```
doctest::Context::Context (
    const Context & ) [delete]
```

8.7.1.3 Context() [3/3]

```
doctest::Context::Context (
    Context && ) [delete]
```

8.7.1.4 ~Context()

```
doctest::Context::~~Context ()
```

8.7.2 Member Function Documentation

8.7.2.1 addFilter()

```
void doctest::Context::addFilter (
    const char * filter,
    const char * value)
```

8.7.2.2 applyCommandLine()

```
void doctest::Context::applyCommandLine (
    int argc,
    const char *const * argv)
```

8.7.2.3 clearFilters()

```
void doctest::Context::clearFilters ()
```

8.7.2.4 operator=() [1/2]

```
Context & doctest::Context::operator= (
    const Context & ) [delete]
```

8.7.2.5 operator=() [2/2]

```
Context & doctest::Context::operator= (
    Context && ) [delete]
```

8.7.2.6 run()

```
int doctest::Context::run ()
```

8.7.2.7 setAsDefaultForAssertsOutOfTestCases()

```
void doctest::Context::setAsDefaultForAssertsOutOfTestCases ()
```

8.7.2.8 setAssertHandler()

```
void doctest::Context::setAssertHandler (
    detail::assert_handler ah)
```

8.7.2.9 setCout()

```
void doctest::Context::setCout (
    std::ostream * out)
```

8.7.2.10 setOption() [1/3]

```
void doctest::Context::setOption (
    const char * option,
    bool value)
```

8.7.2.11 setOption() [2/3]

```
void doctest::Context::setOption (
    const char * option,
    const char * value)
```

8.7.2.12 setOption() [3/3]

```
void doctest::Context::setOption (
    const char * option,
    int value)
```

8.7.2.13 shouldExit()

```
bool doctest::Context::shouldExit ()
```

The documentation for this class was generated from the following file:

- tests/[doctest.h](#)

8.8 doctest::ContextOptions Struct Reference

```
#include <doctest.h>
```


Public Attributes

- `std::ostream * cout = nullptr`
- `String binary_name`
- `const detail::TestCase * currentTest = nullptr`
- `String out`
- `String order_by`
- `unsigned rand_seed`
- `unsigned first`
- `unsigned last`
- `int abort_after`
- `int subcase_filter_levels`
- `bool success`
- `bool case_sensitive`
- `bool exit`
- `bool duration`
- `bool minimal`
- `bool quiet`
- `bool no_throw`
- `bool no_exitcode`
- `bool no_run`
- `bool no_intro`
- `bool no_version`
- `bool no_colors`
- `bool force_colors`
- `bool no_breaks`
- `bool no_skip`
- `bool gnu_file_line`
- `bool no_path_in_filenames`
- `String strip_file_prefixes`
- `bool no_line_numbers`
- `bool no_debug_output`
- `bool no_skipped_summary`
- `bool no_time_in_output`
- `bool help`
- `bool version`
- `bool count`
- `bool list_test_cases`
- `bool list_test_suites`
- `bool list_reporters`

8.8.1 Member Data Documentation

8.8.1.1 abort_after

```
int doctest::ContextOptions::abort_after
```

8.8.1.2 binary_name

```
String doctest::ContextOptions::binary_name
```

8.8.1.3 case_sensitive

```
bool doctest::ContextOptions::case_sensitive
```

8.8.1.4 count

```
bool doctest::ContextOptions::count
```

8.8.1.5 cout

```
std::ostream* doctest::ContextOptions::cout = nullptr
```

8.8.1.6 currentTest

```
const detail::TestCase* doctest::ContextOptions::currentTest = nullptr
```

8.8.1.7 duration

```
bool doctest::ContextOptions::duration
```

8.8.1.8 exit

```
bool doctest::ContextOptions::exit
```

8.8.1.9 first

```
unsigned doctest::ContextOptions::first
```

8.8.1.10 force_colors

```
bool doctest::ContextOptions::force_colors
```

8.8.1.11 gnu_file_line

```
bool doctest::ContextOptions::gnu_file_line
```

8.8.1.12 help

```
bool doctest::ContextOptions::help
```

8.8.1.13 last

unsigned doctest::ContextOptions::last

8.8.1.14 list_reporters

bool doctest::ContextOptions::list_reporters

8.8.1.15 list_test_cases

bool doctest::ContextOptions::list_test_cases

8.8.1.16 list_test_suites

bool doctest::ContextOptions::list_test_suites

8.8.1.17 minimal

bool doctest::ContextOptions::minimal

8.8.1.18 no_breaks

bool doctest::ContextOptions::no_breaks

8.8.1.19 no_colors

bool doctest::ContextOptions::no_colors

8.8.1.20 no_debug_output

bool doctest::ContextOptions::no_debug_output

8.8.1.21 no_exitcode

bool doctest::ContextOptions::no_exitcode

8.8.1.22 no_intro

bool doctest::ContextOptions::no_intro

8.8.1.23 no_line_numbers

```
bool doctest::ContextOptions::no_line_numbers
```

8.8.1.24 no_path_in_filenames

```
bool doctest::ContextOptions::no_path_in_filenames
```

8.8.1.25 no_run

```
bool doctest::ContextOptions::no_run
```

8.8.1.26 no_skip

```
bool doctest::ContextOptions::no_skip
```

8.8.1.27 no_skipped_summary

```
bool doctest::ContextOptions::no_skipped_summary
```

8.8.1.28 no_throw

```
bool doctest::ContextOptions::no_throw
```

8.8.1.29 no_time_in_output

```
bool doctest::ContextOptions::no_time_in_output
```

8.8.1.30 no_version

```
bool doctest::ContextOptions::no_version
```

8.8.1.31 order_by

```
String doctest::ContextOptions::order_by
```

8.8.1.32 out

```
String doctest::ContextOptions::out
```

8.8.1.33 quiet

```
bool doctest::ContextOptions::quiet
```

8.8.1.34 rand_seed

```
unsigned doctest::ContextOptions::rand_seed
```

8.8.1.35 strip_file_prefixes

```
String doctest::ContextOptions::strip_file_prefixes
```

8.8.1.36 subcase_filter_levels

```
int doctest::ContextOptions::subcase_filter_levels
```

8.8.1.37 success

```
bool doctest::ContextOptions::success
```

8.8.1.38 version

```
bool doctest::ContextOptions::version
```

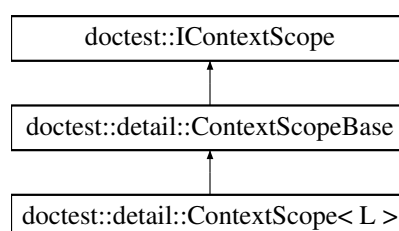
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.9 doctest::detail::ContextScope< L > Class Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::ContextScope< L >:



Public Member Functions

- [ContextScope](#) (const L &lambda)
- [ContextScope](#) (L &&lambda)
- [ContextScope](#) (const ContextScope &)=delete
- [ContextScope](#) (ContextScope &&) noexcept=default
- [ContextScope](#) & [operator=](#) (const [ContextScope](#) &)=delete
- [ContextScope](#) & [operator=](#) ([ContextScope](#) &&)=delete
- void [stringify](#) ([std::ostream](#) *s) const override
- [~ContextScope](#) () override

Public Member Functions inherited from [doctest::detail::ContextScopeBase](#)

- [ContextScopeBase](#) (const ContextScopeBase &)=delete
- [ContextScopeBase](#) & [operator=](#) (const [ContextScopeBase](#) &)=delete
- [ContextScopeBase](#) & [operator=](#) ([ContextScopeBase](#) &&)=delete
- [~ContextScopeBase](#) () override=default

Additional Inherited Members

Protected Member Functions inherited from [doctest::detail::ContextScopeBase](#)

- [ContextScopeBase](#) ()
- [ContextScopeBase](#) (ContextScopeBase &&other) noexcept
- void [destroy](#) ()

Protected Attributes inherited from [doctest::detail::ContextScopeBase](#)

- bool [need_to_destroy](#) {true}

8.9.1 Constructor & Destructor Documentation

8.9.1.1 ContextScope() [1/4]

```
template<typename L>
doctest::detail::ContextScope< L >::ContextScope (
    const L & lambda) [inline], [explicit]
```

8.9.1.2 ContextScope() [2/4]

```
template<typename L>
doctest::detail::ContextScope< L >::ContextScope (
    L && lambda) [inline], [explicit]
```

8.9.1.3 ContextScope() [3/4]

```
template<typename L>
doctest::detail::ContextScope< L >::ContextScope (
    const ContextScope< L > & ) [delete]
```

8.9.1.4 ContextScope() [4/4]

```
template<typename L>
doctest::detail::ContextScope< L >::ContextScope (
    ContextScope< L > && ) [default], [noexcept]
```

8.9.1.5 ~ContextScope()

```
template<typename L>
doctest::detail::ContextScope< L >::~~ContextScope () [inline], [override]
```

8.9.2 Member Function Documentation

8.9.2.1 operator=() [1/2]

```
template<typename L>
ContextScope & doctest::detail::ContextScope< L >::operator= (
    const ContextScope< L > & ) [delete]
```

8.9.2.2 operator=() [2/2]

```
template<typename L>
ContextScope & doctest::detail::ContextScope< L >::operator= (
    ContextScope< L > && ) [delete]
```

8.9.2.3 stringify()

```
template<typename L>
void doctest::detail::ContextScope< L >::stringify (
    std::ostream * s) const [inline], [override], [virtual]
```

Implements [doctest::!ContextScope](#).

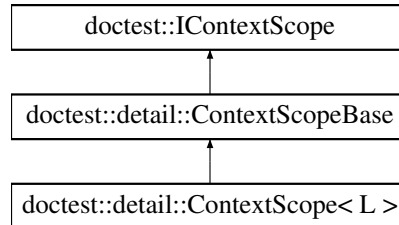
The documentation for this class was generated from the following file:

- [tests/doctest.h](#)

8.10 doctest::detail::ContextScopeBase Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::ContextScopeBase:



Public Member Functions

- [ContextScopeBase](#) (const ContextScopeBase &)=delete
- [ContextScopeBase](#) & [operator=](#) (const [ContextScopeBase](#) &)=delete
- [ContextScopeBase](#) & [operator=](#) ([ContextScopeBase](#) &&)=delete
- [~ContextScopeBase](#) () override=default

Public Member Functions inherited from [doctest::IContextScope](#)

- virtual void [stringify](#) (std::ostream *) const =0

Protected Member Functions

- [ContextScopeBase](#) ()
- [ContextScopeBase](#) (ContextScopeBase &&other) noexcept
- void [destroy](#) ()

Protected Attributes

- bool [need_to_destroy](#) {true}

8.10.1 Constructor & Destructor Documentation

8.10.1.1 ContextScopeBase() [1/3]

```
doctest::detail::ContextScopeBase::ContextScopeBase (
    const ContextScopeBase & ) [delete]
```

8.10.1.2 ~ContextScopeBase()

```
doctest::detail::ContextScopeBase::~~ContextScopeBase () [override], [default]
```


8.10.1.3 ContextScopeBase() [2/3]

```
doctest::detail::ContextScopeBase::ContextScopeBase () [protected]
```

8.10.1.4 ContextScopeBase() [3/3]

```
doctest::detail::ContextScopeBase::ContextScopeBase (
    ContextScopeBase && other) [protected], [noexcept]
```

8.10.2 Member Function Documentation

8.10.2.1 destroy()

```
void doctest::detail::ContextScopeBase::destroy () [protected]
```

8.10.2.2 operator=() [1/2]

```
ContextScopeBase & doctest::detail::ContextScopeBase::operator= (
    const ContextScopeBase & ) [delete]
```

8.10.2.3 operator=() [2/2]

```
ContextScopeBase & doctest::detail::ContextScopeBase::operator= (
    ContextScopeBase && ) [delete]
```

8.10.3 Member Data Documentation

8.10.3.1 need_to_destroy

```
bool doctest::detail::ContextScopeBase::need_to_destroy {true} [protected]
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.11 doctest::CurrentTestCaseStats Struct Reference

```
#include <doctest.h>
```

Public Attributes

- int [numAssertsCurrentTest](#)
- int [numAssertsFailedCurrentTest](#)
- double [seconds](#)
- int [failure_flags](#)
- bool [testCaseSuccess](#)

8.11.1 Member Data Documentation

8.11.1.1 failure_flags

```
int doctest::CurrentTestCaseStats::failure_flags
```

8.11.1.2 numAssertsCurrentTest

```
int doctest::CurrentTestCaseStats::numAssertsCurrentTest
```

8.11.1.3 numAssertsFailedCurrentTest

```
int doctest::CurrentTestCaseStats::numAssertsFailedCurrentTest
```

8.11.1.4 seconds

```
double doctest::CurrentTestCaseStats::seconds
```

8.11.1.5 testCaseSuccess

```
bool doctest::CurrentTestCaseStats::testCaseSuccess
```

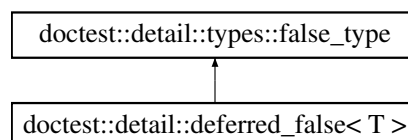
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

8.12 doctest::detail::deferred_false< T > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::deferred_false< T >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.13 doctest::detail::types::enable_if< COND, T > Struct Template Reference

```
#include <doctest.h>
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.14 doctest::detail::types::enable_if< true, T > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = T

8.14.1 Member Typedef Documentation

8.14.1.1 type

```
template<typename T>  
using doctest::detail::types::enable\_if< true, T >::type = T
```

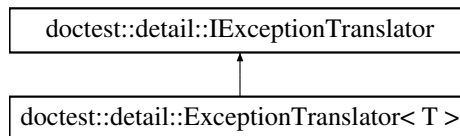
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.15 doctest::detail::ExceptionTranslator< T > Class Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::ExceptionTranslator< T >:



Public Member Functions

- [ExceptionTranslator](#) ([String](#)(*translateFunction)(T)) noexcept
- bool [translate](#) ([String](#) &res) const override

8.15.1 Constructor & Destructor Documentation

8.15.1.1 ExceptionTranslator()

```
template<typename T>
doctest::detail::ExceptionTranslator< T >::ExceptionTranslator (
    String(* translateFunction ) (T)) [inline], [explicit], [noexcept]
```

8.15.2 Member Function Documentation

8.15.2.1 translate()

```
template<typename T>
bool doctest::detail::ExceptionTranslator< T >::translate (
    String & res) const [inline], [override], [virtual]
```

Implements [doctest::detail::IExceptionTranslator](#).

The documentation for this class was generated from the following file:

- tests/[doctest.h](#)

8.16 doctest::detail::Expression_lhs< L > Struct Template Reference

```
#include <doctest.h>
```

Public Member Functions

- [Expression_lhs](#) (L &&in, [assertType::Enum](#) at)
- [DOCTEST_NOINLINE](#) [operator Result](#) ()
- [operator L](#) () const

Public Attributes

- [L lhs](#)
- [assertType::Enum m_at](#)

8.16.1 Constructor & Destructor Documentation

8.16.1.1 Expression_lhs()

```
template<typename L>
doctest::detail::Expression_lhs< L >::Expression_lhs (
    L && in,
    assertType::Enum at) [inline], [explicit]
```

8.16.2 Member Function Documentation

8.16.2.1 operator L()

```
template<typename L>
doctest::detail::Expression_lhs< L >::operator L () const [inline]
```

8.16.2.2 operator Result()

```
template<typename L>
DOCTEST_NOINLINE doctest::detail::Expression_lhs< L >::operator Result () [inline]
```

8.16.3 Member Data Documentation

8.16.3.1 lhs

```
template<typename L>
L doctest::detail::Expression_lhs< L >::lhs
```

8.16.3.2 m_at

```
template<typename L>
assertType::Enum doctest::detail::Expression_lhs< L >::m_at
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.17 doctest::detail::ExpressionDecomposer Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- [ExpressionDecomposer](#) ([assertType::Enum](#) at)
- [template<typename L>](#)
[Expression_lhs](#)< const L && > [operator<<](#) (const L &&operand)
- [template<typename L, typename](#) [types::enable_if<!doctest::detail::types::is_rvalue_reference< L >::value, void >::type](#) * = nullptr>
[Expression_lhs](#)< const L & > [operator<<](#) (const L &operand)

Public Attributes

- [assertType::Enum](#) m_at

8.17.1 Constructor & Destructor Documentation

8.17.1.1 ExpressionDecomposer()

```
doctest::detail::ExpressionDecomposer::ExpressionDecomposer (
    assertType::Enum at)
```

8.17.2 Member Function Documentation

8.17.2.1 operator<<() [1/2]

```
template<typename L>
Expression\_lhs< const L && > doctest::detail::ExpressionDecomposer::operator<< (
    const L && operand) [inline]
```

8.17.2.2 operator<<() [2/2]

```
template<typename L, typename types::enable\_if<!doctest::detail::types::is\_rvalue\_reference<
L >::value, void >::type * = nullptr>
Expression\_lhs< const L & > doctest::detail::ExpressionDecomposer::operator<< (
    const L & operand) [inline]
```

8.17.3 Member Data Documentation

8.17.3.1 m_at

```
assertType::Enum doctest::detail::ExpressionDecomposer::m_at
```

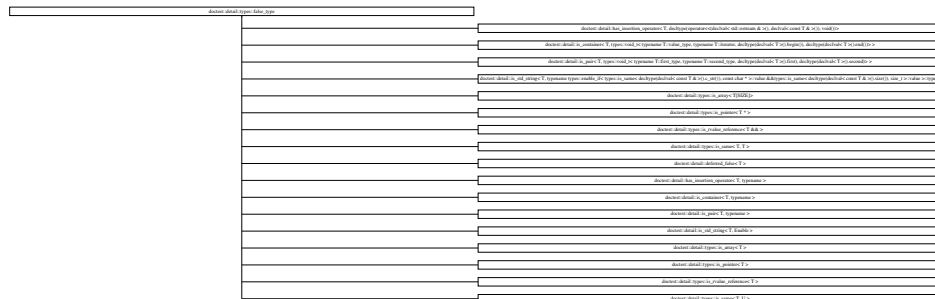
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.18 doctest::detail::types::false_type Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::false_type:



Static Public Attributes

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

8.18.1 Member Data Documentation

8.18.1.1 value

[DOCTEST_CONSTEXPR](#) bool doctest::detail::types::false_type::value = false [static]

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.19 doctest::detail::filldata< T, Enable > Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static void [fill](#) ([std::ostream](#) *stream, const T &in)

8.19.1 Member Function Documentation

8.19.1.1 fill()

```

template<typename T, typename Enable>
void doctest::detail::filldata< T, Enable >::fill (
    std::ostream * stream,
    const T & in) [inline], [static]

```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.20 doctest::detail::filldata< const char[N]> Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static void [fill](#) ([std::ostream](#) *stream, const char(&in)[N])

8.20.1 Member Function Documentation

8.20.1.1 fill()

```
template<size_t N>
void doctest::detail::filldata< const char[N]>::fill (
    std::ostream * stream,
    const char(& in[N]) [inline], [static]
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.21 doctest::detail::filldata< const void * > Struct Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static [DOCTEST_INTERFACE](#) void [fill](#) ([std::ostream](#) *stream, const void *in)

8.21.1 Member Function Documentation

8.21.1.1 fill()

```
DOCTEST_INTERFACE void doctest::detail::filldata< const void * >::fill (
    std::ostream * stream,
    const void * in) [static]
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.22 doctest::detail::filldata< const volatile void * > Struct Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static [DOCTEST_INTERFACE](#) void [fill](#) ([std::ostream](#) *stream, const volatile void *in)

8.22.1 Member Function Documentation

8.22.1.1 fill()

```
DOCTEST\_INTERFACE void doctest::detail::filldata< const volatile void * >::fill (  
    std::ostream * stream,  
    const volatile void * in) [static]
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.23 doctest::detail::filldata< T * > Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static void [fill](#) ([std::ostream](#) *stream, const T *in)

8.23.1 Member Function Documentation

8.23.1.1 fill()

```
template<typename T>  
void doctest::detail::filldata< T * >::fill (  
    std::ostream * stream,  
    const T * in) [inline], [static]
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.24 doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value &&detail::is_container< T >::value >::type > Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static void [fill](#) ([std::ostream](#) *stream, const [DOCTEST_REF_WRAP](#)(T) in)

8.24.1 Member Function Documentation

8.24.1.1 fill()

```
template<typename T>
void doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator<
T >::value &&detail::is_container< T >::value >::type >::fill (
    std::ostream * stream,
    const DOCTEST_REF_WRAP(T) in) [inline], [static]
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.25 doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value &&detail::is_pair< T >::value >::type > Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static void [fill](#) ([std::ostream](#) *stream, const T &in)

8.25.1 Member Function Documentation

8.25.1.1 fill()

```
template<typename T>
void doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator<
T >::value &&detail::is_pair< T >::value >::type >::fill (
    std::ostream * stream,
    const T & in) [inline], [static]
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.26 doctest::detail::filldata< T[N]> Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- static void [fill](#) ([std::ostream](#) *stream, const T(&in)[N])

8.26.1 Member Function Documentation

8.26.1.1 fill()

```
template<typename T, size\_t N>
void doctest::detail::filldata< T[N]>::fill (
    std::ostream * stream,
    const T(&) in[N]) [inline], [static]
```

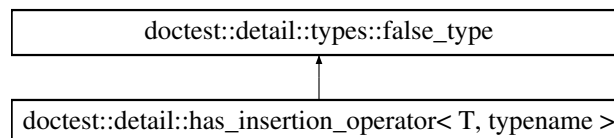
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.27 doctest::detail::has_insertion_operator< T, typename > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::has_insertion_operator< T, typename >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

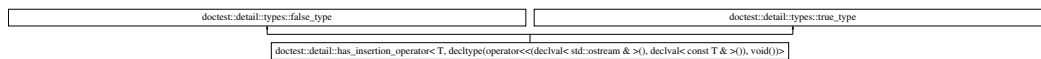
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.28 doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >()), declval< const T & >()), void())> Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >()), declval< const T & >()), void())>:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = true

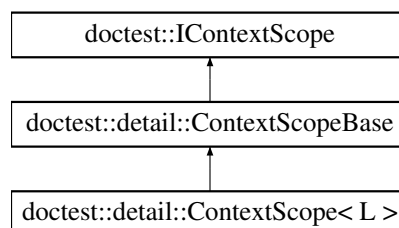
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.29 doctest::IContextScope Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::IContextScope:



Public Member Functions

- virtual void [stringify](#) ([std::ostream](#) *) const =0

8.29.1 Member Function Documentation

8.29.1.1 stringify()

```
virtual void doctest::IContextScope::stringify (
    std::ostream * ) const [pure virtual]
```

Implemented in [doctest::detail::ContextScope< L >](#).

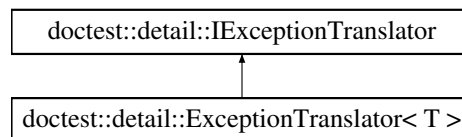
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.30 doctest::detail::IExceptionTranslator Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::IExceptionTranslator:



Public Member Functions

- virtual bool [translate](#) ([String](#) &) const =0

8.30.1 Member Function Documentation

8.30.1.1 translate()

```
virtual bool doctest::detail::IExceptionTranslator::translate (
    String & ) const [pure virtual]
```

Implemented in [doctest::detail::ExceptionTranslator< T >](#).

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.31 doctest::IReporter Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- virtual void [report_query](#) (const [QueryData](#) &)=0
- virtual void [test_run_start](#) ()=0
- virtual void [test_run_end](#) (const [TestRunStats](#) &)=0
- virtual void [test_case_start](#) (const [TestCaseData](#) &)=0
- virtual void [test_case_reenter](#) (const [TestCaseData](#) &)=0
- virtual void [test_case_end](#) (const [CurrentTestCaseStats](#) &)=0
- virtual void [test_case_exception](#) (const [TestCaseException](#) &)=0
- virtual void [subcase_start](#) (const [SubcaseSignature](#) &)=0
- virtual void [subcase_end](#) ()=0
- virtual void [log_assert](#) (const [AssertData](#) &)=0
- virtual void [log_message](#) (const [MessageData](#) &)=0
- virtual void [test_case_skipped](#) (const [TestCaseData](#) &)=0

Static Public Member Functions

- static int [get_num_active_contexts](#) ()
- static const [IContextScope](#) *const * [get_active_contexts](#) ()
- static int [get_num_stringified_contexts](#) ()
- static const [String](#) * [get_stringified_contexts](#) ()

8.31.1 Member Function Documentation

8.31.1.1 [get_active_contexts\(\)](#)

```
const IContextScope *const * doctest::IReporter::get_active_contexts () [static]
```

8.31.1.2 [get_num_active_contexts\(\)](#)

```
int doctest::IReporter::get_num_active_contexts () [static]
```

8.31.1.3 [get_num_stringified_contexts\(\)](#)

```
int doctest::IReporter::get_num_stringified_contexts () [static]
```

8.31.1.4 [get_stringified_contexts\(\)](#)

```
const String * doctest::IReporter::get_stringified_contexts () [static]
```

8.31.1.5 [log_assert\(\)](#)

```
virtual void doctest::IReporter::log_assert (
    const AssertData & ) [pure virtual]
```

8.31.1.6 log_message()

```
virtual void doctest::IReporter::log_message (
    const MessageData & ) [pure virtual]
```

8.31.1.7 report_query()

```
virtual void doctest::IReporter::report_query (
    const QueryData & ) [pure virtual]
```

8.31.1.8 subcase_end()

```
virtual void doctest::IReporter::subcase_end () [pure virtual]
```

8.31.1.9 subcase_start()

```
virtual void doctest::IReporter::subcase_start (
    const SubcaseSignature & ) [pure virtual]
```

8.31.1.10 test_case_end()

```
virtual void doctest::IReporter::test_case_end (
    const CurrentTestCaseStats & ) [pure virtual]
```

8.31.1.11 test_case_exception()

```
virtual void doctest::IReporter::test_case_exception (
    const TestCaseException & ) [pure virtual]
```

8.31.1.12 test_case_reenter()

```
virtual void doctest::IReporter::test_case_reenter (
    const TestCaseData & ) [pure virtual]
```

8.31.1.13 test_case_skipped()

```
virtual void doctest::IReporter::test_case_skipped (
    const TestCaseData & ) [pure virtual]
```

8.31.1.14 test_case_start()

```
virtual void doctest::IReporter::test_case_start (
    const TestCaseData & ) [pure virtual]
```

8.31.1.15 test_run_end()

```
virtual void doctest::IReporter::test_run_end (
    const TestRunStats & ) [pure virtual]
```

8.31.1.16 test_run_start()

```
virtual void doctest::IReporter::test_run_start () [pure virtual]
```

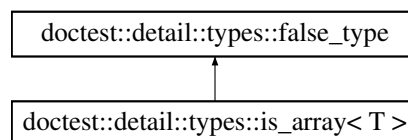
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.32 doctest::detail::types::is_array< T > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::is_array< T >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

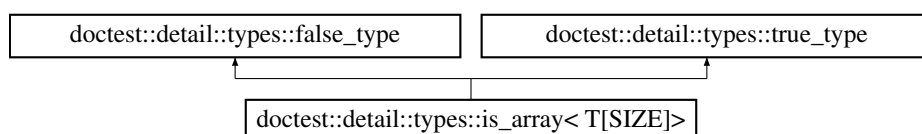
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.33 doctest::detail::types::is_array< T[SIZE]> Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::is_array< T[SIZE]>:



Additional Inherited Members**Static Public Attributes inherited from [doctest::detail::types::false_type](#)**

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = true

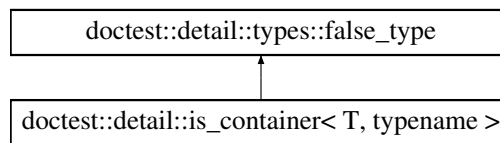
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.34 doctest::detail::is_container< T, typename > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::is_container< T, typename >:

**Additional Inherited Members****Static Public Attributes inherited from [doctest::detail::types::false_type](#)**

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.35 doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())> > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())> >:



Additional Inherited Members

Static Public Attributes inherited from `doctest::detail::types::false_type`

- static `DOCTEST_CONSTEXPR` `bool value` = false

Static Public Attributes inherited from `doctest::detail::types::true_type`

- static `DOCTEST_CONSTEXPR` `bool value` = true

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.36 `doctest::detail::types::is_enum< T >` Struct Template Reference

```
#include <doctest.h>
```

Static Public Attributes

- static `DOCTEST_CONSTEXPR` `bool value` = `__is_enum(T)`

8.36.1 Member Data Documentation

8.36.1.1 `value`

```
template<typename T>
DOCTEST_CONSTEXPR bool doctest::detail::types::is_enum< T >::value = __is_enum(T) [static]
```

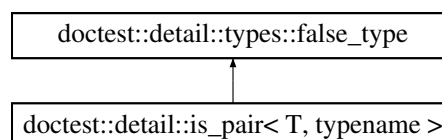
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.37 `doctest::detail::is_pair< T, typename >` Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for `doctest::detail::is_pair< T, typename >`:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

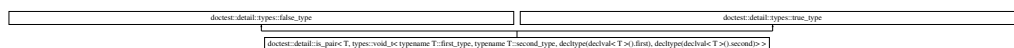
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.38 doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)> > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)> >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = true

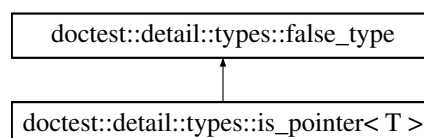
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.39 doctest::detail::types::is_pointer< T > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::is_pointer< T >:



Additional Inherited Members

Static Public Attributes inherited from `doctest::detail::types::false_type`

- static `DOCTEST_CONSTEXPR` bool `value` = false

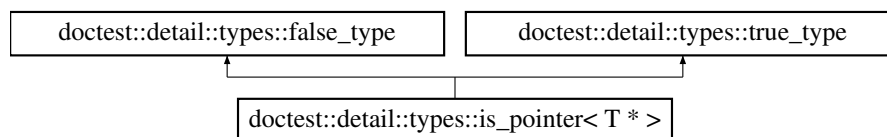
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.40 `doctest::detail::types::is_pointer< T * >` Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for `doctest::detail::types::is_pointer< T * >`:



Additional Inherited Members

Static Public Attributes inherited from `doctest::detail::types::false_type`

- static `DOCTEST_CONSTEXPR` bool `value` = false

Static Public Attributes inherited from `doctest::detail::types::true_type`

- static `DOCTEST_CONSTEXPR` bool `value` = true

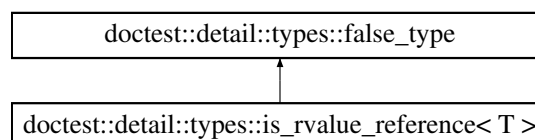
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.41 `doctest::detail::types::is_rvalue_reference< T >` Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for `doctest::detail::types::is_rvalue_reference< T >`:



Additional Inherited Members**Static Public Attributes inherited from [doctest::detail::types::false_type](#)**

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

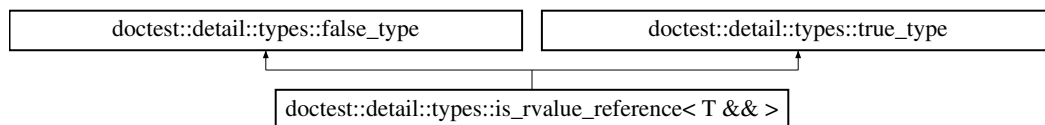
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.42 doctest::detail::types::is_rvalue_reference< T && > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::is_rvalue_reference< T && >:

**Additional Inherited Members****Static Public Attributes inherited from [doctest::detail::types::false_type](#)**

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = true

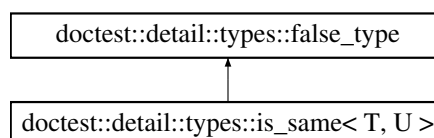
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.43 doctest::detail::types::is_same< T, U > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::is_same< T, U >:



Additional Inherited Members

Static Public Attributes inherited from `doctest::detail::types::false_type`

- static `DOCTEST_CONSTEXPR` bool `value` = false

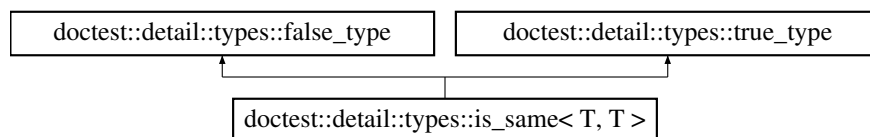
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.44 `doctest::detail::types::is_same< T, T >` Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for `doctest::detail::types::is_same< T, T >`:



Additional Inherited Members

Static Public Attributes inherited from `doctest::detail::types::false_type`

- static `DOCTEST_CONSTEXPR` bool `value` = false

Static Public Attributes inherited from `doctest::detail::types::true_type`

- static `DOCTEST_CONSTEXPR` bool `value` = true

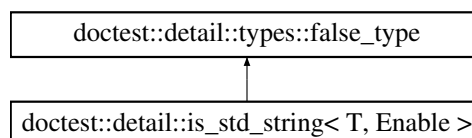
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.45 `doctest::detail::is_std_string< T, Enable >` Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for `doctest::detail::is_std_string< T, Enable >`:



Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.46 doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype(declval< const T & >().c_str()), const char * >::value &&types::is_same< decltype(declval< const T & >().size()), size_t >::value >::type > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype(declval< const T & >().c_str()), const char * >::value &&types::is_same< decltype(declval< const T & >().size()), size_t >::value >::type >:



Additional Inherited Members

Static Public Attributes inherited from [doctest::detail::types::false_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = false

Static Public Attributes inherited from [doctest::detail::types::true_type](#)

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = true

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.47 doctest::IsNaN< F > Struct Template Reference

```
#include <doctest.h>
```

Public Member Functions

- [IsNaN](#) (F f, bool flip=false)
- [IsNaN](#)< F > [operator!](#) () const
- [operator bool](#) () const

Public Attributes

- F [value](#)
- bool [flipped](#)

8.47.1 Constructor & Destructor Documentation

8.47.1.1 [IsNaN\(\)](#)

```
template<typename F>
doctest::IsNaN< F >::IsNaN (
    F f,
    bool flip = false) [inline]
```

8.47.2 Member Function Documentation

8.47.2.1 [operator bool\(\)](#)

```
template<typename F>
doctest::IsNaN< F >::operator bool () const
```

8.47.2.2 [operator"!"\(\)](#)

```
template<typename F>
IsNaN< F > doctest::IsNaN< F >::operator! () const [inline]
```

8.47.3 Member Data Documentation

8.47.3.1 [flipped](#)

```
template<typename F>
bool doctest::IsNaN< F >::flipped
```

8.47.3.2 [value](#)

```
template<typename F>
F doctest::IsNaN< F >::value
```

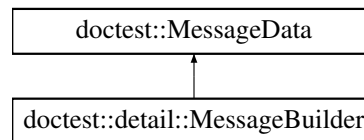
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.48 doctest::detail::MessageBuilder Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::MessageBuilder:



Public Member Functions

- [MessageBuilder](#) (const char *file, int line, [assertType::Enum](#) severity)
- [MessageBuilder](#) (const MessageBuilder &)=delete
- [MessageBuilder](#) (MessageBuilder &&)=delete
- [MessageBuilder](#) & [operator=](#) (const [MessageBuilder](#) &)=delete
- [MessageBuilder](#) & [operator=](#) ([MessageBuilder](#) &&)=delete
- [~MessageBuilder](#) () noexcept(false)
- template<typename T>
[MessageBuilder](#) & [operator,](#) (const T &in)
- template<typename T>
[DOctest_Msvc_Suppress_Warning_Pop](#) [MessageBuilder](#) & [operator<<](#) (const T &in)
- template<typename T>
[MessageBuilder](#) & [operator*](#) (const T &in)
- bool [log](#) ()
- void [react](#) ()

Public Attributes

- [std::ostream](#) * [m_stream](#)
- bool [logged](#) = false

Public Attributes inherited from [doctest::MessageData](#)

- [String](#) [m_string](#)
- const char * [m_file](#)
- int [m_line](#)
- [assertType::Enum](#) [m_severity](#)

8.48.1 Constructor & Destructor Documentation

8.48.1.1 MessageBuilder() [1/3]

```
doctest::detail::MessageBuilder::MessageBuilder (
    const char * file,
    int line,
    assertType::Enum severity)
```

8.48.1.2 MessageBuilder() [2/3]

```
doctest::detail::MessageBuilder::MessageBuilder (
    const MessageBuilder & ) [delete]
```

8.48.1.3 MessageBuilder() [3/3]

```
doctest::detail::MessageBuilder::MessageBuilder (
    MessageBuilder && ) [delete]
```

8.48.1.4 ~MessageBuilder()

```
doctest::detail::MessageBuilder::~~MessageBuilder ()
```

8.48.2 Member Function Documentation**8.48.2.1 log()**

```
bool doctest::detail::MessageBuilder::log ()
```

8.48.2.2 operator*()

```
template<typename T>
MessageBuilder & doctest::detail::MessageBuilder::operator* (
    const T & in) [inline]
```

8.48.2.3 operator,()

```
template<typename T>
MessageBuilder & doctest::detail::MessageBuilder::operator, (
    const T & in) [inline]
```

8.48.2.4 operator<<()

```
template<typename T>
DOCTEST_MSVC_SUPPRESS_WARNING_POP MessageBuilder & doctest::detail::MessageBuilder::operator<<
(
    const T & in) [inline]
```

8.48.2.5 operator=() [1/2]

```
MessageBuilder & doctest::detail::MessageBuilder::operator= (
    const MessageBuilder & ) [delete]
```

8.48.2.6 operator=() [2/2]

```
MessageBuilder & doctest::detail::MessageBuilder::operator= (
    MessageBuilder && ) [delete]
```

8.48.2.7 react()

```
void doctest::detail::MessageBuilder::react ()
```

8.48.3 Member Data Documentation

8.48.3.1 logged

```
bool doctest::detail::MessageBuilder::logged = false
```

8.48.3.2 m_stream

```
std::ostream* doctest::detail::MessageBuilder::m_stream
```

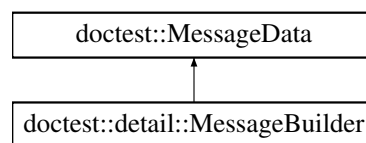
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.49 doctest::MessageData Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::MessageData:



Public Attributes

- [String m_string](#)
- [const char * m_file](#)
- [int m_line](#)
- [assertType::Enum m_severity](#)

8.49.1 Member Data Documentation

8.49.1.1 m_file

```
const char* doctest::MessageData::m_file
```

8.49.1.2 m_line

```
int doctest::MessageData::m_line
```

8.49.1.3 m_severity

```
assertType::Enum doctest::MessageData::m_severity
```

8.49.1.4 m_string

```
String doctest::MessageData::m_string
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.50 doctest::QueryData Struct Reference

```
#include <doctest.h>
```

Public Attributes

- const [TestRunStats](#) * [run_stats](#) = nullptr
- const [TestCaseData](#) ** [data](#) = nullptr
- unsigned [num_data](#) = 0

8.50.1 Member Data Documentation

8.50.1.1 data

```
const TestCaseData** doctest::QueryData::data = nullptr
```

8.50.1.2 num_data

```
unsigned doctest::QueryData::num_data = 0
```

8.50.1.3 run_stats

```
const TestRunStats* doctest::QueryData::run_stats = nullptr
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.51 doctest::detail::RelationalComparator< int, L, R > Struct Template Reference

```
#include <doctest.h>
```

Public Member Functions

- bool [operator\(\)](#) (const DOCTEST_REF_WRAP(L), const DOCTEST_REF_WRAP(R)) const

8.51.1 Member Function Documentation

8.51.1.1 operator()()

```
template<int, class L, class R>
bool doctest::detail::RelationalComparator< int, L, R >::operator() (
    const DOCTEST_REF_WRAP(L),
    const DOCTEST_REF_WRAP(R)) const [inline]
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.52 doctest::detail::types::remove_const< T > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = T

8.52.1 Member Typedef Documentation

8.52.1.1 type

```
template<typename T>
using doctest::detail::types::remove_const< T >::type = T
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.53 doctest::detail::types::remove_const< const T > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = T

8.53.1 Member Typedef Documentation

8.53.1.1 type

```
template<typename T>
using doctest::detail::types::remove_const< const T >::type = T
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.54 doctest::detail::types::remove_reference< T > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = T

8.54.1 Member Typedef Documentation

8.54.1.1 type

```
template<typename T>
using doctest::detail::types::remove_reference< T >::type = T
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.55 doctest::detail::types::remove_reference< T & > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = T

8.55.1 Member Typedef Documentation

8.55.1.1 type

```
template<typename T>
using doctest::detail::types::remove_reference< T & >::type = T
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.56 doctest::detail::types::remove_reference< T && > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = T

8.56.1 Member Typedef Documentation

8.56.1.1 type

```
template<typename T>
using doctest::detail::types::remove_reference< T && >::type = T
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.57 doctest::detail::Result Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- [Result](#) ()=default
- [Result](#) (bool passed, const [String](#) &decomposition=[String](#)())

Public Attributes

- bool [m_passed](#)
- [String](#) [m_decomp](#)

8.57.1 Constructor & Destructor Documentation

8.57.1.1 Result() [1/2]

```
doctest::detail::Result::Result () [default]
```

8.57.1.2 Result() [2/2]

```
doctest::detail::Result::Result (
    bool passed,
    const String & decomposition = String())
```

8.57.2 Member Data Documentation

8.57.2.1 m_decomp

```
String doctest::detail::Result::m_decomp
```


8.57.2.2 m_passed

```
bool doctest::detail::Result::m_passed
```

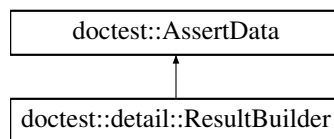
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.58 doctest::detail::ResultBuilder Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::ResultBuilder:



Public Member Functions

- [ResultBuilder](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const char *exception_type="", const [String](#) &exception_string="")
- [ResultBuilder](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const char *exception_type, const [Contains](#) &exception_string)
- void [setResult](#) (const [Result](#) &res)
- template<int comparison, typename L, typename R>
[DOCTEST_NOINLINE](#) bool [binary_assert](#) (const [DOCTEST_REF_WRAP](#)(L) lhs, const [DOCTEST_REF_WRAP](#)(R) rhs)
- template<typename L>
[DOCTEST_NOINLINE](#) bool [unary_assert](#) (const [DOCTEST_REF_WRAP](#)(L) val)
- void [translateException](#) ()
- bool [log](#) ()
- void [react](#) () const

Public Member Functions inherited from [doctest::AssertData](#)

- [AssertData](#) ([assertType::Enum](#) at, const char *file, int line, const char *expr, const char *exception_type, const [StringContains](#) &exception_string)

Additional Inherited Members

Public Attributes inherited from [doctest::AssertData](#)

- const [TestCaseData](#) * [m_test_case](#)
- [assertType::Enum](#) [m_at](#)
- const char * [m_file](#)
- int [m_line](#)
- const char * [m_expr](#)
- bool [m_failed](#)
- bool [m_threw](#)
- [String](#) [m_exception](#)
- [String](#) [m_decomp](#)
- bool [m_threw_as](#)
- const char * [m_exception_type](#)
- class [DOCTEST_INTERFACE](#) [doctest::AssertData::StringContains](#) [m_exception_string](#)

8.58.1 Constructor & Destructor Documentation

8.58.1.1 ResultBuilder() [1/2]

```
doctest::detail::ResultBuilder::ResultBuilder (
    assertType::Enum at,
    const char * file,
    int line,
    const char * expr,
    const char * exception_type = "",
    const String & exception_string = "")
```

8.58.1.2 ResultBuilder() [2/2]

```
doctest::detail::ResultBuilder::ResultBuilder (
    assertType::Enum at,
    const char * file,
    int line,
    const char * expr,
    const char * exception_type,
    const Contains & exception_string)
```

8.58.2 Member Function Documentation

8.58.2.1 binary_assert()

```
template<int comparison, typename L, typename R>
DOCTEST_NOINLINE bool doctest::detail::ResultBuilder::binary_assert (
    const DOCTEST_REF_WRAP(L) lhs,
    const DOCTEST_REF_WRAP(R) rhs) [inline]
```

8.58.2.2 log()

```
bool doctest::detail::ResultBuilder::log ()
```

8.58.2.3 react()

```
void doctest::detail::ResultBuilder::react () const
```

8.58.2.4 setResult()

```
void doctest::detail::ResultBuilder::setResult (
    const Result & res)
```

8.58.2.5 translateException()

```
void doctest::detail::ResultBuilder::translateException ()
```

8.58.2.6 unary_assert()

```
template<typename L>
DOCTEST_NOINLINE bool doctest::detail::ResultBuilder::unary_assert (
    const DOCTEST_REF_WRAP(L) val) [inline]
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.59 doctest::detail::should_stringify_as_underlying_type< T > Struct Template Reference

```
#include <doctest.h>
```

Static Public Attributes

- static [DOCTEST_CONSTEXPR](#) bool [value](#)

8.59.1 Member Data Documentation

8.59.1.1 value

```
template<typename T>
DOCTEST_CONSTEXPR bool doctest::detail::should_stringify_as_underlying_type< T >::value [static]
```

Initial value:

```

=
detail::types::is_enum<T>::value && !doctest::detail::has_insertion_operator<T>::value
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.60 doctest::String Class Reference

```
#include <doctest.h>
```

Public Types

- using [size_type](#) = [DOCTEST_CONFIG_STRING_SIZE_TYPE](#)

Public Member Functions

- [String](#) () noexcept
- [~String](#) ()
- [String](#) (const char *in)
- [String](#) (const char *in, [size_type](#) in_size)
- [String](#) (std::istream &in, [size_type](#) in_size)
- template<typename T, typename [detail::types::enable_if](#)< [detail::is_std_string](#)< T >::value, bool >::type = true>
[String](#) (const T &in)
- [String](#) (const [String](#) &other)
- [String](#) & [operator=](#) (const [String](#) &other)
- [String](#) & [operator+=](#) (const [String](#) &other)
- [String](#) ([String](#) &&other) noexcept
- [String](#) & [operator=](#) ([String](#) &&other) noexcept
- char [operator\[\]](#) ([size_type](#) i) const
- char & [operator\[\]](#) ([size_type](#) i)
- const char * [c_str](#) () const
- char * [c_str](#) ()
- [size_type](#) [size](#) () const
- [size_type](#) [capacity](#) () const
- [String](#) [substr](#) ([size_type](#) pos, [size_type](#) cnt=[npos](#)) &&
- [String](#) [substr](#) ([size_type](#) pos, [size_type](#) cnt=[npos](#)) const &
- [size_type](#) [find](#) (char ch, [size_type](#) pos=0) const
- [size_type](#) [rfind](#) (char ch, [size_type](#) pos=[npos](#)) const
- int [compare](#) (const char *other, bool no_case=false) const
- int [compare](#) (const [String](#) &other, bool no_case=false) const

Static Public Attributes

- static [DOCTEST_CONSTEXPR](#) [size_type](#) [npos](#) = static_cast<[size_type](#)>(-1)

Friends

- [DOCTEST_INTERFACE](#) std::ostream & [operator<<](#) (std::ostream &s, const [String](#) &in)

8.60.1 Member Typedef Documentation

8.60.1.1 [size_type](#)

```
using doctest::String::size\_type = DOCTEST\_CONFIG\_STRING\_SIZE\_TYPE
```

8.60.2 Constructor & Destructor Documentation

8.60.2.1 [String\(\)](#) [1/7]

```
doctest::String::String () [noexcept]
```

8.60.2.2 ~String()

```
doctest::String::~~String ()
```

8.60.2.3 String() [2/7]

```
doctest::String::String (  
    const char * in)
```

8.60.2.4 String() [3/7]

```
doctest::String::String (  
    const char * in,  
    size_type in_size)
```

8.60.2.5 String() [4/7]

```
doctest::String::String (  
    std::istream & in,  
    size_type in_size)
```

8.60.2.6 String() [5/7]

```
template<typename T, typename detail::types::enable_if< detail::is_std_string< T >::value,  
bool >::type = true>  
doctest::String::String (  
    const T & in) [inline]
```

8.60.2.7 String() [6/7]

```
doctest::String::String (  
    const String & other)
```

8.60.2.8 String() [7/7]

```
doctest::String::String (  
    String && other) [noexcept]
```

8.60.3 Member Function Documentation

8.60.3.1 c_str() [1/2]

```
char * doctest::String::c_str () [inline]
```

8.60.3.2 c_str() [2/2]

```
const char * doctest::String::c_str () const [inline]
```

8.60.3.3 capacity()

```
size_type doctest::String::capacity () const
```

8.60.3.4 compare() [1/2]

```
int doctest::String::compare (  
    const char * other,  
    bool no_case = false) const
```

8.60.3.5 compare() [2/2]

```
int doctest::String::compare (  
    const String & other,  
    bool no_case = false) const
```

8.60.3.6 find()

```
size_type doctest::String::find (  
    char ch,  
    size_type pos = 0) const
```

8.60.3.7 operator+=()

```
String & doctest::String::operator+= (  
    const String & other)
```

8.60.3.8 operator=() [1/2]

```
String & doctest::String::operator= (  
    const String & other)
```

8.60.3.9 operator=() [2/2]

```
String & doctest::String::operator= (  
    String && other) [noexcept]
```

8.60.3.10 operator[]() [1/2]

```
char & doctest::String::operator[] (  
    size_type i)
```

8.60.3.11 operator[]() [2/2]

```
char doctest::String::operator[] (
    size_type i) const
```

8.60.3.12 rfind()

```
size_type doctest::String::rfind (
    char ch,
    size_type pos = npos) const
```

8.60.3.13 size()

```
size_type doctest::String::size () const
```

8.60.3.14 substr() [1/2]

```
String doctest::String::substr (
    size_type pos,
    size_type cnt = npos) &&
```

8.60.3.15 substr() [2/2]

```
String doctest::String::substr (
    size_type pos,
    size_type cnt = npos) const &
```

8.60.4 Friends And Related Symbol Documentation

8.60.4.1 operator<<

```
DOCTEST_INTERFACE std::ostream & operator<< (
    std::ostream & s,
    const String & in) [friend]
```

8.60.5 Member Data Documentation

8.60.5.1 buf

```
char doctest::String::buf[len]
```

8.60.5.2 data

```
view doctest::String::data
```

8.60.5.3 npos

```
DOCTEST_CONSTEXPR size_type doctest::String::npos = static_cast<size_type>(-1) [static]
```

The documentation for this class was generated from the following file:

- tests/[doctest.h](#)

8.61 doctest::AssertData::StringContains Class Reference

```
#include <doctest.h>
```

Public Member Functions

- [StringContains](#) (const [String](#) &str)
- [StringContains](#) ([Contains](#) cntn)
- bool [check](#) (const [String](#) &str)
- [operator const String & \(\)](#) const
- const char * [c_str](#) () const

8.61.1 Constructor & Destructor Documentation

8.61.1.1 StringContains() [1/2]

```
doctest::AssertData::StringContains::StringContains (
    const String & str) [inline]
```

8.61.1.2 StringContains() [2/2]

```
doctest::AssertData::StringContains::StringContains (
    Contains cntn) [inline]
```

8.61.2 Member Function Documentation

8.61.2.1 c_str()

```
const char * doctest::AssertData::StringContains::c_str () const [inline]
```

8.61.2.2 check()

```
bool doctest::AssertData::StringContains::check (
    const String & str) [inline]
```


8.61.2.3 operator const String &()

```
doctest::AssertData::StringContains::operator const String & () const [inline]
```

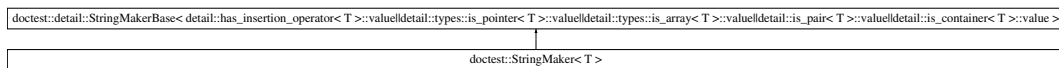
The documentation for this class was generated from the following file:

- tests/[doctest.h](#)

8.62 doctest::StringMaker< T > Struct Template Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::StringMaker< T >:



Additional Inherited Members

Static Public Member Functions inherited from [doctest::detail::StringMakerBase< detail::has_insertion_operator< T >::value||detail::types::is_pointer< T >::value||detail::types::is_array< T >::value||detail::is_pair< T >::value||detail::is_container< T >::value >](#)

- static [String convert](#) (const DOCTEST_REF_WRAP(T))

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.63 doctest::detail::StringMakerBase< C > Struct Template Reference

```
#include <doctest.h>
```

Static Public Member Functions

- [template<typename T>](#)
static [String convert](#) (const DOCTEST_REF_WRAP(T))

8.63.1 Member Function Documentation

8.63.1.1 convert()

```
template<bool C>
template<typename T>
String doctest::detail::StringMakerBase< C >::convert (
    const DOCTEST_REF_WRAP(T) [inline], [static]
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.64 doctest::detail::StringMakerBase< true > Struct Reference

```
#include <doctest.h>
```

Static Public Member Functions

- template<typename T>
static [String convert](#) (const [DOCTEST_REF_WRAP](#)(T) in)

8.64.1 Member Function Documentation

8.64.1.1 convert()

```
template<typename T>
String doctest::detail::StringMakerBase< true >::convert (
    const DOCTEST_REF_WRAP(T) in) [inline], [static]
```

The documentation for this struct was generated from the following file:

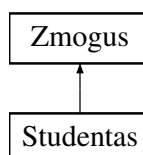
- tests/[doctest.h](#)

8.65 Studentas Class Reference

Klase, skirta saugoti studento duomenis, paveldi [Zmogus](#) klase.

```
#include <student.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- [Studentas](#) ()
Konstruktorius.
- [Studentas](#) ([std::istream](#) &is)
Konstruktorius, kuris nuskaito studento duomenis is ivesties srauto.
- [~Studentas](#) ()=default
Destruktorius.
- [Studentas](#) (const [Studentas](#) &other)
Kopijavimo konstruktorius.
- [Studentas](#) & [operator=](#) (const [Studentas](#) &other)
Kopijavimo priskyrimo operatorius.
- [Studentas](#) ([Studentas](#) &&other) noexcept
Perkelimo konstruktorius.
- [Studentas](#) & [operator=](#) ([Studentas](#) &&other) noexcept
Perkelimo priskyrimo operatorius.
- const vector< int > [paz](#) () const
grazina Studento Pazymius
- int [egz](#) () const
grazina Studento egzamino pazymi
- double [vid](#) () const
grazina Studento pazymiu vidurki
- double [med](#) () const
grazina Studento pazymiu mediana
- [std::istream](#) & [ReadStudent](#) ([std::istream](#) &)
- void [SetEgz](#) (int egz)
Nustato studento egzamino pazymi.
- void [SetVid](#) (double vid)
Nustato galutini bala pagal vidurki.
- void [SetMed](#) (double med)
Nustato galutini bala pagal mediana.
- void [AddPaz](#) (int paz)
Prideda viena namu darbu pazymi.
- void [ClearPaz](#) ()
Isvalo visus namu darbu pazymius.
- void [MedIrVidSkaciavimas](#) (int sum)
Apskaiciuoja galutinius balus pagal vidurki ir mediana.
- void [spausdinti](#) () const override
Isveda studento duomenis.

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()
Konstruktorius.
- [Zmogus](#) (const string &vardas, const string &pavarde)
Konstruktorius su vardu ir pavarde.
- virtual [~Zmogus](#) ()=default
Destruktorius.
- const string [vardas](#) () const
grazina Zmogaus Vardas

- const string [pavarde](#) () const
grazina Zmogaus Pavarde
- void [SetVardas](#) (const string &vardas)
Iraso zmogaus varda.
- void [SetPavarde](#) (const string &pavarde)
Iraso zmogaus pavarde.

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- string [_vardas](#)
- string [_pavarde](#)

8.65.1 Detailed Description

Klase, skirta saugoti studento duomenis, paveldi [Zmogus](#) klase.

[Studentas](#) klase saugo studento pazymius, egzamina, vidurki ir mediana Ji paveldi varda ir pavarde is zmogus klases

8.65.2 Constructor & Destructor Documentation

8.65.2.1 Studentas() [1/4]

```
Studentas::Studentas () [inline]
```

Konstruktorius.

Sukuria tuscia studento objekta. Nustato pazymius 0

8.65.2.2 Studentas() [2/4]

```
Studentas::Studentas (
    std::istream & is)
```

Konstruktorius, kuris nuskaito studento duomenis is ivesties srauto.

Parameters

<i>is</i>	Ivesties srautas, is kurio bus skaitomi studento duomenys
-----------	---

8.65.2.3 ~Studentas()

```
Studentas::~~Studentas () [default]
```

Destruktorius.

8.65.2.4 Studentas() [3/4]

```
Studentas::Studentas (
    const Studentas & other)
```

Kopijavimo konstruktorius.

Parameters

<i>other</i>	<code>Studentas</code> objektas, is kurio kopijuojami duomenys.
--------------	---

8.65.2.5 Studentas() [4/4]

```
Studentas::Studentas (
    Studentas && other) [noexcept]
```

Perkelimo konstruktorius.

Parameters

<i>other</i>	<code>Studentas</code> objektas, is kurio perkeliama duomenys.
--------------	--

8.65.3 Member Function Documentation

8.65.3.1 AddPaz()

```
void Studentas::AddPaz (
    int paz) [inline]
```

Prideda viena namu darbu pazymi.

Parameters

<i>paz</i>	Namu darbu pazymys.
------------	---------------------

8.65.3.2 ClearPaz()

```
void Studentas::ClearPaz () [inline]
```

Isvalo visus namu darbu pazymius.

8.65.3.3 egz()

```
int Studentas::egz () const [inline]
```

grazina Studento egzamino pazymi

Returns

Studento egzamino pazymis

8.65.3.4 med()

```
double Studentas::med () const [inline]
```

grazina Studento pazymiu mediana

Returns

Studento pazymiu mediana

8.65.3.5 MedIrVidSkaciavimas()

```
void Studentas::MedIrVidSkaciavimas (  
    int sum)
```

Apskaiciuoja galutinius balus pagal vidurki ir mediana.

Parameters

<i>sum</i>	Namu darbu pazymiu suma.
------------	--------------------------

8.65.3.6 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other)
```

Kopijavimo priskyrimo operatorius.

Parameters

<i>other</i>	Studentas objektas, is kurio kopijuojami duomenys.
--------------	--

Returns

Nuoroda i si objekta.

8.65.3.7 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [noexcept]
```

Perkelimo priskyrimo operatorius.

Parameters

<i>other</i>	Studentas objektas, is kurio perkeliama duomenys.
--------------	---

Returns

Nuoroda i si objekta.

8.65.3.8 paz()

```
const vector< int > Studentas::paz () const [inline]
```

grazina Studento Pazymius

Returns

Studento pazymius vector<int>

8.65.3.9 ReadStudent()

```
std::istream & Studentas::ReadStudent (
    std::istream & is)
```

8.65.3.10 SetEgz()

```
void Studentas::SetEgz (
    int egz) [inline]
```

Nustato studento egzamino pazymi.

Parameters

<i>egz</i>	Egzamino pazymys.
------------	-------------------

8.65.3.11 SetMed()

```
void Studentas::SetMed (
    double med) [inline]
```

Nustato galutini bala pagal mediana.

Parameters

<i>med</i>	Galutinis balas pagal mediana.
------------	--------------------------------

8.65.3.12 SetVid()

```
void Studentas::SetVid (
    double vid) [inline]
```

Nustato galutini bala pagal vidurki.

Parameters

<i>vid</i>	Galutinis balas pagal vidurki.
------------	--------------------------------

8.65.3.13 spausdinti()

```
void Studentas::spausdinti () const [override], [virtual]
```

Isveda studento duomenis.

Implements [Zmogus](#).

8.65.3.14 vid()

```
double Studentas::vid () const [inline]
```

grazina Studento pazymiu vidurki

Returns

Studento pazymiu vidurkis

The documentation for this class was generated from the following files:

- include/[student.h](#)
- src/[student.cpp](#)

8.66 doctest::detail::Subcase Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- [Subcase](#) (const [String](#) &name, const char *file, int line)
- [Subcase](#) (const Subcase &)=delete
- [Subcase](#) (Subcase &&)=delete
- [Subcase](#) & [operator=](#) (const [Subcase](#) &)=delete
- [Subcase](#) & [operator=](#) ([Subcase](#) &&)=delete
- [~Subcase](#) ()
- [operator bool](#) () const

Public Attributes

- [SubcaseSignature m_signature](#)
- bool [m_entered](#) = false

8.66.1 Constructor & Destructor Documentation

8.66.1.1 Subcase() [1/3]

```
doctest::detail::Subcase::Subcase (  
    const String & name,  
    const char * file,  
    int line)
```

8.66.1.2 Subcase() [2/3]

```
doctest::detail::Subcase::Subcase (  
    const Subcase & ) [delete]
```

8.66.1.3 Subcase() [3/3]

```
doctest::detail::Subcase::Subcase (  
    Subcase && ) [delete]
```

8.66.1.4 ~Subcase()

```
doctest::detail::Subcase::~~Subcase ()
```

8.66.2 Member Function Documentation

8.66.2.1 operator bool()

```
doctest::detail::Subcase::operator bool () const
```

8.66.2.2 operator=() [1/2]

```
Subcase & doctest::detail::Subcase::operator= (  
    const Subcase & ) [delete]
```

8.66.2.3 operator=() [2/2]

```
Subcase & doctest::detail::Subcase::operator= (  
    Subcase && ) [delete]
```

8.66.3 Member Data Documentation

8.66.3.1 m_entered

```
bool doctest::detail::Subcase::m_entered = false
```

8.66.3.2 m_signature

```
SubcaseSignature doctest::detail::Subcase::m_signature
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.67 doctest::SubcaseSignature Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- bool [operator==](#) (const [SubcaseSignature](#) &other) const
- bool [operator<](#) (const [SubcaseSignature](#) &other) const

Public Attributes

- [String](#) [m_name](#)
- const char * [m_file](#)
- int [m_line](#)

8.67.1 Member Function Documentation

8.67.1.1 operator<()

```
bool doctest::SubcaseSignature::operator< (  
    const SubcaseSignature & other) const
```

8.67.1.2 operator==()

```
bool doctest::SubcaseSignature::operator== (  
    const SubcaseSignature & other) const
```

8.67.2 Member Data Documentation

8.67.2.1 m_file

```
const char* doctest::SubcaseSignature::m_file
```

8.67.2.2 m_line

```
int doctest::SubcaseSignature::m_line
```

8.67.2.3 m_name

```
String doctest::SubcaseSignature::m_name
```

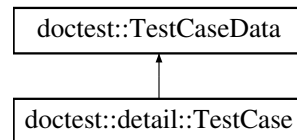
The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

8.68 doctest::detail::TestCase Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::TestCase:



Public Member Functions

- [TestCase](#) ([funcType](#) test, const char *file, unsigned line, const [TestSuite](#) &test_suite, const [String](#) &type=[String](#)(), int template_id=-1) noexcept
- [TestCase](#) (const [TestCase](#) &other) noexcept
- [TestCase](#) ([TestCase](#) &&)=delete
- [TestCase](#) & [operator=](#) (const [TestCase](#) &other) noexcept
- [DOctest_MSVC_SUPPRESS_WARNING_POP](#) [TestCase](#) & [operator=](#) ([TestCase](#) &&)=delete
- [TestCase](#) & [operator*](#) (const char *in) noexcept
- template<typename T>
 [TestCase](#) & [operator*](#) (const T &in) noexcept
- bool [operator<](#) (const [TestCase](#) &other) const noexcept
- [~TestCase](#) ()=default

Public Attributes

- [funcType m_test](#)
- [String m_type](#)
- [int m_template_id](#)
- [String m_full_name](#)

Public Attributes inherited from [doctest::TestCaseData](#)

- [String m_file](#)
- [unsigned m_line](#)
- [const char * m_name](#)
- [const char * m_test_suite](#)
- [const char * m_description](#)
- [bool m_skip](#)
- [bool m_no_breaks](#)
- [bool m_no_output](#)
- [bool m_may_fail](#)
- [bool m_should_fail](#)
- [int m_expected_failures](#)
- [double m_timeout](#)

8.68.1 Constructor & Destructor Documentation**8.68.1.1 [TestCase\(\)](#) [1/3]**

```
doctest::detail::TestCase::TestCase (
    funcType test,
    const char * file,
    unsigned line,
    const TestSuite & test_suite,
    const String & type = String(),
    int template_id = -1) [noexcept]
```

8.68.1.2 [TestCase\(\)](#) [2/3]

```
doctest::detail::TestCase::TestCase (
    const TestCase & other) [noexcept]
```

8.68.1.3 [TestCase\(\)](#) [3/3]

```
doctest::detail::TestCase::TestCase (
    TestCase && ) [delete]
```

8.68.1.4 [~TestCase\(\)](#)

```
doctest::detail::TestCase::~~TestCase () [default]
```

8.68.2 Member Function Documentation

8.68.2.1 operator*() [1/2]

```
TestCase & doctest::detail::TestCase::operator* (
    const char * in) [noexcept]
```

8.68.2.2 operator*() [2/2]

```
template<typename T>
TestCase & doctest::detail::TestCase::operator* (
    const T & in) [inline], [noexcept]
```

8.68.2.3 operator<()

```
bool doctest::detail::TestCase::operator< (
    const TestCase & other) const [noexcept]
```

8.68.2.4 operator=() [1/2]

```
TestCase & doctest::detail::TestCase::operator= (
    const TestCase & other) [noexcept]
```

8.68.2.5 operator=() [2/2]

```
DOCTEST_MSVC_SUPPRESS_WARNING_POP TestCase & doctest::detail::TestCase::operator= (
    TestCase && ) [delete]
```

8.68.3 Member Data Documentation

8.68.3.1 m_full_name

```
String doctest::detail::TestCase::m_full_name
```

8.68.3.2 m_template_id

```
int doctest::detail::TestCase::m_template_id
```

8.68.3.3 m_test

```
funcType doctest::detail::TestCase::m_test
```

8.68.3.4 m_type

```
String doctest::detail::TestCase::m_type
```

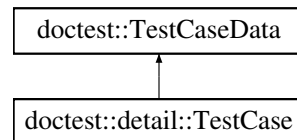
The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.69 doctest::TestCaseData Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::TestCaseData:



Public Attributes

- [String m_file](#)
- unsigned [m_line](#)
- const char * [m_name](#)
- const char * [m_test_suite](#)
- const char * [m_description](#)
- bool [m_skip](#)
- bool [m_no_breaks](#)
- bool [m_no_output](#)
- bool [m_may_fail](#)
- bool [m_should_fail](#)
- int [m_expected_failures](#)
- double [m_timeout](#)

8.69.1 Member Data Documentation

8.69.1.1 m_description

```
const char* doctest::TestCaseData::m_description
```

8.69.1.2 m_expected_failures

```
int doctest::TestCaseData::m_expected_failures
```

8.69.1.3 m_file

`String doctest::TestCaseData::m_file`

8.69.1.4 m_line

`unsigned doctest::TestCaseData::m_line`

8.69.1.5 m_may_fail

`bool doctest::TestCaseData::m_may_fail`

8.69.1.6 m_name

`const char* doctest::TestCaseData::m_name`

8.69.1.7 m_no_breaks

`bool doctest::TestCaseData::m_no_breaks`

8.69.1.8 m_no_output

`bool doctest::TestCaseData::m_no_output`

8.69.1.9 m_should_fail

`bool doctest::TestCaseData::m_should_fail`

8.69.1.10 m_skip

`bool doctest::TestCaseData::m_skip`

8.69.1.11 m_test_suite

`const char* doctest::TestCaseData::m_test_suite`

8.69.1.12 m_timeout

`double doctest::TestCaseData::m_timeout`

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

8.70 doctest::TestCaseException Struct Reference

```
#include <doctest.h>
```

Public Attributes

- [String error_string](#)
- [bool is_crash](#)

8.70.1 Member Data Documentation

8.70.1.1 error_string

```
String doctest::TestCaseException::error_string
```

8.70.1.2 is_crash

```
bool doctest::TestCaseException::is_crash
```

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

8.71 doctest::detail::TestFailureException Struct Reference

```
#include <doctest.h>
```

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

8.72 doctest::TestRunStats Struct Reference

```
#include <doctest.h>
```

Public Attributes

- [unsigned numTestCases](#)
- [unsigned numTestCasesPassingFilters](#)
- [unsigned numTestSuitesPassingFilters](#)
- [unsigned numTestCasesFailed](#)
- [int numAsserts](#)
- [int numAssertsFailed](#)

8.72.1 Member Data Documentation

8.72.1.1 numAsserts

```
int doctest::TestRunStats::numAsserts
```

8.72.1.2 numAssertsFailed

```
int doctest::TestRunStats::numAssertsFailed
```

8.72.1.3 numTestCases

```
unsigned doctest::TestRunStats::numTestCases
```

8.72.1.4 numTestCasesFailed

```
unsigned doctest::TestRunStats::numTestCasesFailed
```

8.72.1.5 numTestCasesPassingFilters

```
unsigned doctest::TestRunStats::numTestCasesPassingFilters
```

8.72.1.6 numTestSuitesPassingFilters

```
unsigned doctest::TestRunStats::numTestSuitesPassingFilters
```

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

8.73 doctest::detail::TestSuite Struct Reference

```
#include <doctest.h>
```

Public Member Functions

- [TestSuite](#) & [operator*](#) (const char *in) noexcept
- [template<typename T>](#)
[TestSuite](#) & [operator*](#) (const T &in) noexcept

Public Attributes

- `const char * m_test_suite` = nullptr
- `const char * m_description` = nullptr
- `bool m_skip` = false
- `bool m_no_breaks` = false
- `bool m_no_output` = false
- `bool m_may_fail` = false
- `bool m_should_fail` = false
- `int m_expected_failures` = 0
- `double m_timeout` = 0

8.73.1 Member Function Documentation

8.73.1.1 `operator*()` [1/2]

```
TestSuite & doctest::detail::TestSuite::operator* (  
    const char * in) [noexcept]
```

8.73.1.2 `operator*()` [2/2]

```
template<typename T>  
TestSuite & doctest::detail::TestSuite::operator* (  
    const T & in) [inline], [noexcept]
```

8.73.2 Member Data Documentation

8.73.2.1 `m_description`

```
const char* doctest::detail::TestSuite::m_description = nullptr
```

8.73.2.2 `m_expected_failures`

```
int doctest::detail::TestSuite::m_expected_failures = 0
```

8.73.2.3 `m_may_fail`

```
bool doctest::detail::TestSuite::m_may_fail = false
```

8.73.2.4 `m_no_breaks`

```
bool doctest::detail::TestSuite::m_no_breaks = false
```

8.73.2.5 m_no_output

```
bool doctest::detail::TestSuite::m_no_output = false
```

8.73.2.6 m_should_fail

```
bool doctest::detail::TestSuite::m_should_fail = false
```

8.73.2.7 m_skip

```
bool doctest::detail::TestSuite::m_skip = false
```

8.73.2.8 m_test_suite

```
const char* doctest::detail::TestSuite::m_test_suite = nullptr
```

8.73.2.9 m_timeout

```
double doctest::detail::TestSuite::m_timeout = 0
```

The documentation for this struct was generated from the following file:

- [tests/doctest.h](#)

8.74 Timer Class Reference

Klase, skirta programos vykdymo laikui matuoti.

```
#include <timer.h>
```

Public Member Functions

- [Timer](#) ()
Sukria laikmatį ir pradeda laiko matavimą.
- void [reset](#) ()
Atnaujina laikmatį.
- double [elapsed](#) () const
Grazina kiek laiko praėjo.

8.74.1 Detailed Description

Klase, skirta programos vykdymo laikui matuoti.

8.74.2 Constructor & Destructor Documentation

8.74.2.1 Timer()

```
Timer::Timer () [inline]
```

Sukria laikmati ir pradeda laiko matavima.

8.74.3 Member Function Documentation

8.74.3.1 elapsed()

```
double Timer::elapsed () const [inline]
```

Grazina kiek laiko praejo.

Returns

Praeitas laikas

8.74.3.2 reset()

```
void Timer::reset () [inline]
```

Atnaujina laikmati.

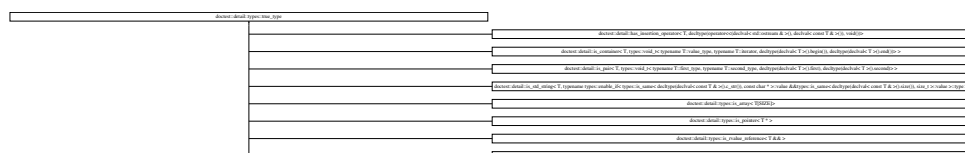
The documentation for this class was generated from the following file:

- [include/timer.h](#)

8.75 doctest::detail::types::true_type Struct Reference

```
#include <doctest.h>
```

Inheritance diagram for doctest::detail::types::true_type:



Static Public Attributes

- static [DOCTEST_CONSTEXPR](#) bool [value](#) = true

8.75.1 Member Data Documentation

8.75.1.1 value

```
DOCTEST_CONSTEXPR bool doctest::detail::types::true_type::value = true [static]
```

The documentation for this struct was generated from the following file:

- tests/[doctest.h](#)

8.76 std::tuple< Types > Class Template Reference

The documentation for this class was generated from the following file:

- tests/[doctest.h](#)

8.77 doctest::detail::types::underlying_type< T > Struct Template Reference

```
#include <doctest.h>
```

Public Types

- using [type](#) = __underlying_type(T)

8.77.1 Member Typedef Documentation

8.77.1.1 type

```
template<typename T>
using doctest::detail::types::underlying_type< T >::type = __underlying_type(T)
```

The documentation for this struct was generated from the following file:

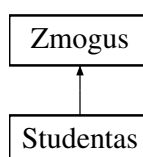
- tests/[doctest.h](#)

8.78 Zmogus Class Reference

Abstrakti bazine klase, sauganti bendrus zmogaus duomenis.

```
#include <zmogus.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) ()
Konstruktorius.
- [Zmogus](#) (const string &vardas, const string &pavarde)
Konstruktorius su vardu ir pavarde.
- virtual [~Zmogus](#) ()=default
Destruktorius.
- virtual void [spausdinti](#) () const =0
- const string [vardas](#) () const
grazina Zmogaus Vardas
- const string [pavarde](#) () const
grazina Zmogaus Pavarde
- void [SetVardas](#) (const string &vardas)
Iraso zmogaus varda.
- void [SetPavarde](#) (const string &pavarde)
Iraso zmogaus pavarde.

Protected Attributes

- string [_vardas](#)
- string [_pavarde](#)

8.78.1 Detailed Description

Abstrakti bazine klase, sauganti bendrus zmogaus duomenis.

Sia klase paveldi [Studentas](#) klase Is jos negalima kurti objektu tiesiogiai [Zmogus](#) klase yra abstrakti, nes turi metoda [spausdinti\(\)](#), kuri [Studentas](#) klase turi perrasyti.

8.78.2 Constructor & Destructor Documentation

8.78.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [inline]
```

Konstruktorius.

8.78.2.2 Zmogus() [2/2]

```
Zmogus::Zmogus (
    const string & vardas,
    const string & pavarde) [inline]
```

Konstruktorius su vardu ir pavarde.

Parameters

<i>vardas</i>	Zmogus vardas
<i>pavarde</i>	Zmogus pavarde

8.78.2.3 ~Zmogus()

```
virtual Zmogus::~~Zmogus () [virtual], [default]
```

Destruktorius.

8.78.3 Member Function Documentation

8.78.3.1 pavarde()

```
const string Zmogus::pavarde () const [inline]
```

grazina Zmogaus Pavarde

Returns

Zmogaus pavarde

8.78.3.2 SetPavarde()

```
void Zmogus::SetPavarde (
    const string & pavarde) [inline]
```

Iraso zmogaus pavarde.

Parameters

<i>pavarde</i>	naujas Pavarde
----------------	----------------

8.78.3.3 SetVardas()

```
void Zmogus::SetVardas (
    const string & vardas) [inline]
```

Iraso zmogaus vardą.

Parameters

<i>vardas</i>	naujas Vardas
---------------	---------------

8.78.3.4 spausdinti()

```
virtual void Zmogus::spausdinti () const [pure virtual]
```

Implemented in [Studentas](#).

8.78.3.5 vardas()

```
const string Zmogus::vardas () const [inline]
```

grazina Zmogaus Vardas

Returns

Zmogaus vardas

8.78.4 Member Data Documentation

8.78.4.1 _pavarde

```
string Zmogus::_pavarde [protected]
```

Pavarde

8.78.4.2 _vardas

```
string Zmogus::_vardas [protected]
```

Vardas

The documentation for this class was generated from the following file:

- [include/zmogus.h](#)

Chapter 9

File Documentation

9.1 include/student.h File Reference

[Studentas](#) klases aprasymas.

```
#include <string>
#include <vector>
#include <iostream>
#include "zmogus.h"
```

Classes

- class [Studentas](#)

Klase, skirta saugoti studento duomenis, paveldi [Zmogus](#) klase.

Functions

- [std::istream & operator>>](#) ([std::istream](#) &is, [Studentas](#) &s)
Ivesties operatorius studento duomenims nuskaityti.
- [std::ostream & operator<<](#) ([std::ostream](#) &os, const [Studentas](#) &s)
Išvesties operatorius studento duomenims išvesti.

9.1.1 Detailed Description

[Studentas](#) klases aprasymas.

9.1.2 Function Documentation

9.1.2.1 operator<<()

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s)
```

Išvesties operatorius studento duomenims išvesti.

Parameters

<i>os</i>	Išvesties srautas.
<i>s</i>	Studentas objektas.

Returns

Išvesties srautas.

9.1.2.2 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s)
```

Išvesties operatorius studento duomenims nuskaityti.

Parameters

<i>is</i>	Išvesties srautas.
<i>s</i>	Studentas objektas.

Returns

Išvesties srautas.

9.2 student.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STUDENT_H
00002 #define STUDENT_H
00003
00004 #include <string>
00005 #include <vector>
00006 #include <iostream>
00007 #include "zmogus.h"
00008
00009 using std::string;
00010 using std::vector;
00011
00016
00024
00025 class Studentas : public Zmogus
```

```

00026 {
00027 private:
00028     vector<int> _paz;
00029     int _egz;
00030     double _vid;
00031     double _med;
00032
00033 public:
00040     Studentas() : Zmogus(), _paz(), _egz(0), _vid(0.0), _med(0.0) {}
00046     Studentas(std::istream &is);
00047
00051     ~Studentas() = default; // destruktorius
00056     Studentas(const Studentas &other); // copy konstruktorius - sukuria nauja objekta
00062     Studentas &operator=(const Studentas &other); // copy priskyrimo konstruktorius - kopijuojame
    egzistuojancia objekta
00067     Studentas(Studentas &&other) noexcept; // noexcept - neturetu mesti isimciu
00073     Studentas &operator=(Studentas &&other) noexcept; // noexcept - neturetu mesti isimciu
00074
00079     inline const vector<int> paz() const { return _paz; }
00084     inline int egz() const { return _egz; }
00089     inline double vid() const { return _vid; }
00094     inline double med() const { return _med; }
00095
00096     std::istream &ReadStudent(std::istream &);
00097
00102     void SetEgz(int egz) { _egz = egz; }
00107     void SetVid(double vid) { _vid = vid; }
00112     void SetMed(double med) { _med = med; }
00117     void AddPaz(int paz) { _paz.push_back(paz); }
00121     void ClearPaz() { _paz.clear(); }
00122
00127     void MedIrVidSkaciavimas(int sum);
00128
00132     void spausdinti() const override;
00133 };
00134
00141 std::istream& operator>(std::istream& is, Studentas& s);
00142
00149 std::ostream& operator<(std::ostream& os, const Studentas& s);
00150
00151 #endif

```

9.3 include/student_io_deque.h File Reference

Funkcijos darbu su `Studentas` objektais naudojant deque konteineri.

```

#include "student.h"
#include <string>
#include <deque>

```

Functions

- void `inputas` (deque< `Studentas` > &grupe)
Leidžia vartotojui iversti arba nuskaityti studento duomenis.
- void `outputas` (deque< `Studentas` > &grupe)
Leidžia vartotojui pasirinkti studentu isvedimo buda.
- void `fileRead` (deque< `Studentas` > &grupe, string file_name)
Nuskaito studentu duomenis is failo.
- void `fileTest` (deque< `Studentas` > &grupe, string file_name, int &testKiekis)
Testuoja studentu nuskaityma is failo kelis kartus.
- void `sortByUser` (deque< `Studentas` > &grupe, int temp)
Grazina vartotojo pasirinkta rikiavimo kriteriju.
- void `duomenulrasymasFaile` (const deque< `Studentas` > &grupe, int temp, const string &fileName)
Iraso studentu duomenis i faila.
- void `duomenulrasymasKonsole` (deque< `Studentas` > &grupe, int temp)

- Isveda studentu duomenis i konsolę.*
- bool [containsDigit](#) (const string &str)
Patikrina, ar eilutę sudaryta tik iš skaitmenų.
- void [GenerateStudentsFile](#) (int n)
Sugeneruoja studentų duomenų failą.
- void [SplitStudentsStrategy1](#) (const deque< [Studentas](#) > &grupe, deque< [Studentas](#) > &failed, deque< [Studentas](#) > &passed)
Padalina studentus į dvi grupes: neįslaukusius ir įslaukusius.
- void [SplitStudentsStrategy2](#) (deque< [Studentas](#) > &grupe, deque< [Studentas](#) > &failed)
Padalina studentus į dvi grupes, paliekant įslaukusius pradinėje grupėje.
- void [SplitStudentsStrategy3](#) (deque< [Studentas](#) > &grupe, deque< [Studentas](#) > &failed)
Padalina studentus į dvi grupes naudojant stable_partition.
- int [getSortChoice](#) (int temp)
Rusiuoja pagal vartotojo pasirinktą variantą.
- void [benchmarkProcessingFile](#) (int n, int testKiekis, int strategy)
Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greičio testą.
- void [RuleOfFiveTestas](#) ()
Atlieka Rule of Five demonstracinį testą [Studentas](#) klasei.

9.3.1 Detailed Description

Funkcijos darbui su [Studentas](#) objektais naudojant deque konteinerį.

9.3.2 Function Documentation

9.3.2.1 benchmarkProcessingFile()

```
void benchmarkProcessingFile (
    int n,
    int testKiekis,
    int strategy)
```

Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greičio testą.

Parameters

<i>n</i>	Studentų skaičius.
<i>testKiekis</i>	Kiek kartu kartojamas testas.
<i>strategy</i>	Studentų skirstymo strategijos numeris.

9.3.2.2 containsDigit()

```
bool containsDigit (
    const string & str)
```

Patikrina, ar eilute sudaryta tik is skaitmenu.

Parameters

<i>str</i>	Tikrinama eilute.
------------	-------------------

Returns

true, jei eilute sudaryta tik is skaitmenu, false kitu atveju.

9.3.2.3 duomenulrasymasFaile()

```
void duomenulrasymasFaile (
    const deque< Studentas > & grupe,
    int temp,
    const string & fileName)
```

Iraso studentu duomenis i faila.

Parameters

<i>grupe</i>	Studentu deque konteineris.
<i>temp</i>	Isvedimo formato pasirinkimas.
<i>fileName</i>	Failo pavadinimas.

9.3.2.4 duomenulrasymasKonsole()

```
void duomenulrasymasKonsole (
    deque< Studentas > & grupe,
    int temp)
```

Isveda studentu duomenis i konsole.

Parameters

<i>grupe</i>	Studentu deque konteineris.
<i>temp</i>	Isvedimo formato pasirinkimas.

9.3.2.5 fileRead()

```
void fileRead (
    deque< Studentas > & grupe,
    string file_name)
```

Nuskaito studentu duomenis is failo.

Parameters

<i>grupe</i>	Studentu deque konteineris, i kuri irasomi nuskaityti studentai.
<i>file_name</i>	Failo pavadinimas.

9.3.2.6 fileTest()

```
void fileTest (
    deque< Studentas > & grupe,
    string file_name,
    int & testKiekis)
```

Testuoja studentu nuskaityma is failo kelis kartus.

Parameters

<i>grupe</i>	Studentu deque konteineris.
<i>file_name</i>	Failo pavadinimas.
<i>testKiekis</i>	Testu skaicius.

9.3.2.7 GenerateStudentsFile()

```
void GenerateStudentsFile (
    int n)
```

Sugeneruoja studentu duomenu faila.

Parameters

<i>n</i>	Studentu skaicius faile.
----------	--------------------------

9.3.2.8 getSortChoice()

```
int getSortChoice (
    int temp)
```

Rusiuoja pagal vartotojo pasirinkta varianta.

Parameters

<i>temp</i>	Rusiavimo formato pasirinkimas.
-------------	---------------------------------

9.3.2.9 inputas()

```
void inputas (  
    deque< Studentas > & grupe)
```

Leidžia vartotojui iversti arba nuskaityti studento duomenis.

Parameters

<i>grupe</i>	Studentu deque konteineris
--------------	----------------------------

9.3.2.10 outputas()

```
void outputas (  
    deque< Studentas > & grupe)
```

Leidžia vartotojui pasirinkti studentu isvedimo buda.

Parameters

<i>grupe</i>	Studentu deque konteineris
--------------	----------------------------

9.3.2.11 RuleOfFiveTestas()

```
void RuleOfFiveTestas ()
```

Atlieka Rule of Five demonstracini testa [Studentas](#) klasei.

9.3.2.12 sortByUser()

```
void sortByUser (  
    deque< Studentas > & grupe,  
    int temp)
```

Grazina vartotojo pasirinkta rikiavimo kriteriju.

Parameters

<i>temp</i>	Pasirinktas isvedimo formatas.
-------------	--------------------------------

Returns

Rikiavimo pasirinkimo numeris.

9.3.2.13 SplitStudentsStrategy1()

```
void SplitStudentsStrategy1 (  
    const deque< Studentas > & grupe,  
    deque< Studentas > & failed,  
    deque< Studentas > & passed)
```

Padalina studentus i dvi grupes: neislaikiusius ir islaikiusius.

Parameters

<i>grupe</i>	Pradine studentu grupe.
<i>failed</i>	Neislaikiusiu studentu grupe.
<i>passed</i>	Islaikiusiu studentu grupe.

9.3.2.14 SplitStudentsStrategy2()

```
void SplitStudentsStrategy2 (  
    deque< Studentas > & grupe,  
    deque< Studentas > & failed)
```

Padalina studentus i dvi grupes, paliekant islaikiusius pradineje grupeje.

Parameters

<i>grupe</i>	Pradine studentu grupe, po funkcijos joje lieka islaike studentai.
<i>failed</i>	Neislaikiusiu studentu grupe.

9.3.2.15 SplitStudentsStrategy3()

```
void SplitStudentsStrategy3 (  
    deque< Studentas > & grupe,  
    deque< Studentas > & failed)
```

Padalina studentus i dvi grupes naudojant `stable_partition`.

Parameters

<i>grupe</i>	Pradine studentu grupe, po funkcijos joje lieka islaike studentai.
<i>failed</i>	Neislaikiusiu studentu grupe.

9.4 student_io_deque.h

[Go to the documentation of this file.](#)

```

00001 #ifndef STUDENT_IO_H_DEQUE
00002 #define STUDENT_IO_H_DEQUE
00003
00004 #include "student.h"
00005 #include <string>
00006 #include <deque>
00007
00008 using std::deque;
00009 using std::string;
00010
00015
00020 void inputas(deque<Studentas> &grupe);
00021
00026 void outputas(deque<Studentas>& grupe);
00027
00033 void fileRead(deque<Studentas> &grupe, string file_name);
00040 void fileTest(deque<Studentas> &grupe, string file_name, int &testKiekis);
00046 void sortByUser(deque<Studentas> &grupe, int temp);
00053 void duomenulrasyamasFaile(const deque<Studentas>& grupe, int temp, const string& fileName);
00059 void duomenulrasyamasKonsole(deque<Studentas> &grupe, int temp);
00060
00066 bool containsDigit(const string &str);
00071 void GenerateStudentsFile(int n);
00078 void SplitStudentsStrategy1(const deque<Studentas> &grupe, deque<Studentas> &failed, deque<Studentas>
    &passed);
00084 void SplitStudentsStrategy2(deque<Studentas> &grupe, deque<Studentas> &failed);
00090 void SplitStudentsStrategy3(deque<Studentas> &grupe, deque<Studentas> &failed);
00091
00092
00097 int getSortChoice(int temp);
00098
00105 void benchmarkProcessingFile(int n, int testKiekis, int strategy);
00106
00110 void RuleOfFiveTestas();
00111
00112 #endif

```

9.5 include/student_io_list.h File Reference

```

#include "student.h"
#include <string>
#include <list>

```

Functions

- void `inputas` (list< `Studentas` > &grupe)
- void `outputas` (list< `Studentas` > &grupe)
- void `fileRead` (list< `Studentas` > &grupe, string file_name)
- void `fileTest` (list< `Studentas` > &grupe, string file_name, int &testKiekis)
- void `sortByUser` (list< `Studentas` > &grupe, int temp)
- void `duomenulrasyamasFaile` (list< `Studentas` > &grupe, int temp, string fileName)
- void `duomenulrasyamasKonsole` (list< `Studentas` > &grupe, int temp)
- bool `containsDigit` (const string &str)
Patikrina, ar eilute sudaryta tik is skaitmenu.
- void `GenerateStudentsFile` (int n)
Sugeneruoja studentu duomeni faila.
- void `SplitStudentsStrategy1` (const list< `Studentas` > &grupe, list< `Studentas` > &failed, list< `Studentas` > &passed)
- void `SplitStudentsStrategy2` (list< `Studentas` > &grupe, list< `Studentas` > &failed)
- void `SplitStudentsStrategy3` (list< `Studentas` > &grupe, list< `Studentas` > &failed)

- int `getSortChoice` (int temp)
Rusiuoja pagal vartotojo pasirinkta varianta.
- void `benchmarkProcessingFile` (int n, int testKiekis, int strategy)
Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

9.5.1 Function Documentation

9.5.1.1 `benchmarkProcessingFile()`

```
void benchmarkProcessingFile (
    int n,
    int testKiekis,
    int strategy)
```

Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

Parameters

<i>n</i>	Studentu skaicius.
<i>testKiekis</i>	Kiek kartu kartojamas testas.
<i>strategy</i>	Studentu skirstymo strategijos numeris.

9.5.1.2 `containsDigit()`

```
bool containsDigit (
    const string & str)
```

Patikrina, ar eilute sudaryta tik is skaitmenu.

Parameters

<i>str</i>	Tikrinama eilute.
------------	-------------------

Returns

true, jei eilute sudaryta tik is skaitmenu, false kitu atveju.

9.5.1.3 `duomenulrasymasFaile()`

```
void duomenulrasymasFaile (
    list< Studentas > & grupe,
    int temp,
    string fileName)
```

9.5.1.4 duomenulrasymasKonsole()

```
void duomenulrasymasKonsole (
    list< Studentas > & grupe,
    int temp)
```

9.5.1.5 fileRead()

```
void fileRead (
    list< Studentas > & grupe,
    string file_name)
```

9.5.1.6 fileTest()

```
void fileTest (
    list< Studentas > & grupe,
    string file_name,
    int & testKiekis)
```

9.5.1.7 GenerateStudentsFile()

```
void GenerateStudentsFile (
    int n)
```

Sugeneruoja studentu duomenu faila.

Parameters

<i>n</i>	Studentu skaicius faile.
----------	--------------------------

9.5.1.8 getSortChoice()

```
int getSortChoice (
    int temp)
```

Rusiuoja pagal vartotojo pasirinkta varianta.

Parameters

<i>temp</i>	Rusiavimo formato pasirinkimas.
-------------	---------------------------------

9.5.1.9 inputas()

```
void inputas (
    list< Studentas > & grupe)
```

9.5.1.10 outputas()

```
void outputas (
    list< Studentas > & grupe)
```

9.5.1.11 sortByUser()

```
void sortByUser (
    list< Studentas > & grupe,
    int temp)
```

9.5.1.12 SplitStudentsStrategy1()

```
void SplitStudentsStrategy1 (
    const list< Studentas > & grupe,
    list< Studentas > & failed,
    list< Studentas > & passed)
```

9.5.1.13 SplitStudentsStrategy2()

```
void SplitStudentsStrategy2 (
    list< Studentas > & grupe,
    list< Studentas > & failed)
```

9.5.1.14 SplitStudentsStrategy3()

```
void SplitStudentsStrategy3 (
    list< Studentas > & grupe,
    list< Studentas > & failed)
```

9.6 student_io_list.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STUDENT_IO_H_LIST
00002 #define STUDENT_IO_H_LIST
00003
00004 #include "student.h"
00005 #include <string>
00006 #include <list>
00007
00008 using std::string;
00009 using std::list;
00010
00011 void inputas(list<Studentas> &grupe);
00012 void outputas(list<Studentas> &grupe);
00013
00014 void fileRead(list<Studentas> &grupe, string file_name);
00015 void fileTest(list<Studentas> &grupe, string file_name, int &testKiekis);
00016
00017 void sortByUser(list<Studentas> &grupe, int temp);
00018
00019 void duomenuIrasymasFaile(list<Studentas> &grupe, int temp, string fileName);
00020 void duomenuIrasymasKonsole(list<Studentas> &grupe, int temp);
00021
00022 bool containsDigit(const string &str);
00023 void GenerateStudentsFile(int n);
00024 void SplitStudentsStrategy1(const list<Studentas> &grupe, list<Studentas> &failed, list<Studentas>
    &passed);
00025 void SplitStudentsStrategy2(list<Studentas> &grupe, list<Studentas> &failed);
00026 void SplitStudentsStrategy3(list<Studentas> &grupe, list<Studentas> &failed);
00027
00028 int getSortChoice(int temp);
00029
00030 void benchmarkProcessingFile(int n, int testKiekis, int strategy);
00031
00032 #endif
```

9.7 include/student_io_vector.h File Reference

```
#include "student.h"
#include <string>
#include <vector>
```

Functions

- void [inputas](#) (vector< [Studentas](#) > &grupe)
- void [outputas](#) (vector< [Studentas](#) > &grupe)
- void [fileRead](#) (vector< [Studentas](#) > &grupe, string file_name)
- void [fileTest](#) (vector< [Studentas](#) > &grupe, string file_name, int &testKiekis)
- void [sortByUser](#) (vector< [Studentas](#) > &grupe, int temp)
- void [duomenulrasymasFaile](#) (vector< [Studentas](#) > &grupe, int temp, string fileName)
- void [duomenulrasymasKonsole](#) (vector< [Studentas](#) > &grupe, int temp)
- bool [containsDigit](#) (const string &str)
Patikrina, ar eilute sudaryta tik is skaitmenu.
- void [GenerateStudentsFile](#) (int n)
Sugeneruoja studentu duomeniu faila.
- void [SplitStudentsStrategy1](#) (const vector< [Studentas](#) > &grupe, vector< [Studentas](#) > &failed, vector< [Studentas](#) > &passed)
- void [SplitStudentsStrategy2](#) (vector< [Studentas](#) > &grupe, vector< [Studentas](#) > &failed)
- void [SplitStudentsStrategy3](#) (vector< [Studentas](#) > &grupe, vector< [Studentas](#) > &failed)
- int [getSortChoice](#) (int temp)
Rusiuoja pagal vartotojo pasirinkta varianta.
- void [benchmarkProcessingFile](#) (int n, int testKiekis, int strategy)
Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

9.7.1 Function Documentation

9.7.1.1 benchmarkProcessingFile()

```
void benchmarkProcessingFile (
    int n,
    int testKiekis,
    int strategy)
```

Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

Parameters

<i>n</i>	Studentu skaicius.
<i>testKiekis</i>	Kiek kartu kartojamas testas.
<i>strategy</i>	Studentu skirstymo strategijos numeris.

9.7.1.2 containsDigit()

```
bool containsDigit (
    const string & str)
```

Patikrina, ar eilute sudaryta tik is skaitmenu.

Parameters

<i>str</i>	Tikrinama eilute.
------------	-------------------

Returns

true, jei eilute sudaryta tik is skaitmenu, false kitu atveju.

9.7.1.3 duomenulrasymasFaile()

```
void duomenulrasymasFaile (
    vector< Studentas > & grupe,
    int temp,
    string fileName)
```

9.7.1.4 duomenulrasymasKonsole()

```
void duomenulrasymasKonsole (
    vector< Studentas > & grupe,
    int temp)
```

9.7.1.5 fileRead()

```
void fileRead (
    vector< Studentas > & grupe,
    string file_name)
```

9.7.1.6 fileTest()

```
void fileTest (
    vector< Studentas > & grupe,
    string file_name,
    int & testKiekis)
```

9.7.1.7 GenerateStudentsFile()

```
void GenerateStudentsFile (  
    int n)
```

Sugeneruoja studentu duomenų failą.

Parameters

<i>n</i>	Studentų skaičius failė.
----------	--------------------------

9.7.1.8 getSortChoice()

```
int getSortChoice (  
    int temp)
```

Rusiuoja pagal vartotojo pasirinktą variantą.

Parameters

<i>temp</i>	Rusavimo formato pasirinkimas.
-------------	--------------------------------

9.7.1.9 inputas()

```
void inputas (  
    vector< Studentas > & grupe)
```

9.7.1.10 outputas()

```
void outputas (  
    vector< Studentas > & grupe)
```

9.7.1.11 sortByUser()

```
void sortByUser (  
    vector< Studentas > & grupe,  
    int temp)
```

9.7.1.12 SplitStudentsStrategy1()

```
void SplitStudentsStrategy1 (  
    const vector< Studentas > & grupe,  
    vector< Studentas > & failed,  
    vector< Studentas > & passed)
```

9.7.1.13 SplitStudentsStrategy2()

```
void SplitStudentsStrategy2 (
    vector< Studentas > & grupe,
    vector< Studentas > & failed)
```

9.7.1.14 SplitStudentsStrategy3()

```
void SplitStudentsStrategy3 (
    vector< Studentas > & grupe,
    vector< Studentas > & failed)
```

9.8 student_io_vector.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STUDENT_IO_H_VECTOR
00002 #define STUDENT_IO_H_VECTOR
00003
00004 #include "student.h"
00005 #include <string>
00006 #include <vector>
00007
00008 using std::vector;
00009 using std::string;
00010
00011
00012 void inputas(vector<Studentas> &grupe);
00013 void outputas(vector<Studentas> &grupe);
00014
00015 void fileRead(vector<Studentas> &grupe, string file_name);
00016 void fileTest(vector<Studentas> &grupe, string file_name, int &testKiekis);
00017
00018 void sortByUser(vector<Studentas> &grupe, int temp);
00019
00020 void duomenuIrasymasFaile(vector<Studentas> &grupe, int temp, string fileName);
00021 void duomenuIrasymasKonsole(vector<Studentas> &grupe, int temp);
00022
00023 bool containsDigit(const string &str);
00024 void GenerateStudentsFile(int n);
00025
00026 void SplitStudentsStrategy1(const vector<Studentas> &grupe, vector<Studentas> &failed,
    vector<Studentas> &passed);
00027 void SplitStudentsStrategy2(vector<Studentas> &grupe, vector<Studentas> &failed);
00028 void SplitStudentsStrategy3(vector<Studentas> &grupe, vector<Studentas> &failed);
00029
00030 int getSortChoice(int temp);
00031
00032 void benchmarkProcessingFile(int n, int testKiekis, int strategy);
00033
00034 #endif
```

9.9 include/timer.h File Reference

Laiko matavimo klases aprasymas.

```
#include <chrono>
```

Classes

- class [Timer](#)

Klase, skirta programos vykdymo laikui matuoti.

9.9.1 Detailed Description

Laiko matavimo klases aprasymas.

9.10 timer.h

[Go to the documentation of this file.](#)

```
00001 #ifndef TIMER_H
00002 #define TIMER_H
00003
00004 #include <chrono>
00005
00010
00015
00016 class Timer
00017 {
00018     // usage of using
00019     using hrClock = std::chrono::high_resolution_clock;
00021     using durationDouble = std::chrono::duration<double>;
00022
00023 private:
00024     std::chrono::time_point<hrClock> start;
00025
00026 public:
00030     Timer() : start{hrClock::now()} {}
00034     void reset()
00035     {
00036         start = hrClock::now();
00037     }
00042     double elapsed() const
00043     {
00044         return durationDouble(hrClock::now() - start).count();
00045     }
00046 };
00047
00048 #endif
```

9.11 include/utils.h File Reference

Pagalbiniu funkciju ir duomenu aprasymas.

```
#include <string>
#include <vector>
```

Functions

- void [intInput](#) (int &temp)
Nuskaityto sveikaji skaiciu ir patikrina ivenessi.
- void [randomVardasPavarde](#) (string &vardas, string &pavarde)
Sugeneruoja atsitiktini varda ir pavarde.

Variables

- const vector< string > [vardai](#)
Vardu sarasas atsitiktiniam generavimui.
- const vector< string > [vyr_pavardes](#)
Vyrisku pavardziu sarasas atsitiktiniam generavimui.
- const vector< string > [mot_pavardes](#)
Moterisku pavardziu sarasas atsitiktiniam generavimui.

9.11.1 Detailed Description

Pagalbiniu funkciju ir duomenu aprasymas.

9.11.2 Function Documentation

9.11.2.1 intInput()

```
void intInput (  
    int & temp)
```

Nuskaito sveikaji skaiciu ir patikrina ivedi.

Parameters

<i>temp</i>	Kintamasis, i kuri irasomas nuskaitytas skaicius.
-------------	---

9.11.2.2 randomVardasPavarde()

```
void randomVardasPavarde (  
    string & vardas,  
    string & pavarde)
```

Sugeneruoja atsitiktini varda ir pavarde.

Parameters

<i>vardas</i>	Sugeneruotas vardas.
<i>pavarde</i>	Sugeneruota pavarde.

9.11.3 Variable Documentation

9.11.3.1 mot_pavardes

```
const vector<string> mot_pavardes [extern]
```

Moterisku pavardziu sarasas atsitiktiniam generavimui.

9.11.3.2 vardai

```
const vector<string> vardai [extern]
```

Vardu sarasas atsitiktiniam generavimui.

9.11.3.3 vyr_pavardes

```
const vector<string> vyr_pavardes [extern]
```

Vyriskų pavardžių sąrašas atsitiktiniam generavimui.

9.12 utils.h

[Go to the documentation of this file.](#)

```
00001 #ifndef UTILS_H
00002 #define UTILS_H
00003
00004 #include <string>
00005 #include <vector>
00006
00007 using std::vector;
00008 using std::string;
00009
00014
00018 extern const vector<string> vardai;
00019
00023 extern const vector<string> vyr_pavardes;
00024
00028 extern const vector<string> mot_pavardes;
00029
00034 void intInput(int& temp);
00035
00041 void randomVardasPavarde(string& vardas, string& pavarde);
00042
00043 #endif
```

9.13 include/zmogus.h File Reference

[Zmogus](#) abstrakcijos bazinės klasės aprašymas.

```
#include <string>
```

Classes

- class [Zmogus](#)
Abstrakti bazinė klasė, sauganti bendrus žmogaus duomenis.

9.13.1 Detailed Description

[Zmogus](#) abstrakcijos bazinės klasės aprašymas.

9.14 zmogus.h

[Go to the documentation of this file.](#)

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005
00006 using std::string;
00007
00012
00022
00023 class Zmogus
00024 {
00025 protected:
00026     string _vardas;
00027     string _pavarde;
00028
00029 public:
00033     Zmogus() : _vardas(""), _pavarde("") {}
00039     Zmogus(const string &vardas, const string &pavarde)
00040         : _vardas(vardas), _pavarde(pavarde) {}
00044     virtual ~Zmogus() = default;
00045
00046     virtual void spausdinti() const = 0;
00051     inline const string vardas() const { return _vardas; }
00056     inline const string pavarde() const { return _pavarde; }
00061     void SetVardas(const string &vardas) { _vardas = vardas; }
00066     void SetPavarde(const string &pavarde) { _pavarde = pavarde; }
00067 };
00068
00069 #endif

```

9.15 src/io_deque.cpp File Reference

```

#include "student_io_deque.h"
#include "utils.h"
#include "timer.h"
#include <iostream>
#include <iomanip>
#include <algorithm>
#include <limits>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <ctime>
#include <stdlib.h>
#include <utility>
#include <filesystem>

```

Functions

- void **outputas** (deque< **Studentas** > &grupe)
Leidžia vartotojui pasirinkti studentu isvedimo buda.
- void **inputas** (deque< **Studentas** > &grupe)
Leidžia vartotojui iversti arba nuskaityti studento duomenis.
- void **fileTest** (deque< **Studentas** > &grupe, string file_name, int &testKiekis)
Testuoja studentu nuskaityma is failo kelis kartus.
- void **fileRead** (deque< **Studentas** > &grupe, string file_name)
Nuskaity studentu duomenis is failo.
- void **sortByUser** (deque< **Studentas** > &grupe, int temp)

- Grazina vartotojo pasirinkta rikiavimo kriteriju.*
- int [getSortChoice](#) (int temp)
 - Rusiuoja pagal vartotojo pasirinkta varianta.*
- void [duomenulrasymasFaile](#) (const deque< [Studentas](#) > &grupe, int temp, const string &fileName)
 - Iraso studentu duomenis i faila.*
- void [duomenulrasymasKonsole](#) (deque< [Studentas](#) > &grupe, int temp)
 - Isveda studentu duomenis i konsole.*
- bool [containsDigit](#) (const string &str)
 - Patikrina, ar eilute sudaryta tik is skaitmenu.*
- void [GenerateStudentsFile](#) (int n)
 - Sugeneruoja studentu duomeniu faila.*
- void [SplitStudentsStrategy1](#) (const deque< [Studentas](#) > &grupe, deque< [Studentas](#) > &failed, deque< [Studentas](#) > &passed)
 - Padalina studentus i dvi grupes: neislaikiusius ir islaikiusius.*
- void [SplitStudentsStrategy2](#) (deque< [Studentas](#) > &grupe, deque< [Studentas](#) > &failed)
 - Padalina studentus i dvi grupes, paliekant islaikiusius pradineje grupeje.*
- void [SplitStudentsStrategy3](#) (deque< [Studentas](#) > &grupe, deque< [Studentas](#) > &failed)
 - Padalina studentus i dvi grupes naudojant stable_partition.*
- void [benchmarkProcessingFile](#) (int n, int testKiekis, int strategy)
 - Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.*
- void [RuleOfFiveTestas](#) ()
 - Atlieka Rule of Five demonstracini testa [Studentas](#) klasei.*

9.15.1 Function Documentation

9.15.1.1 benchmarkProcessingFile()

```
void benchmarkProcessingFile (
    int n,
    int testKiekis,
    int strategy)
```

Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

Parameters

<i>n</i>	Studentu skaicius.
<i>testKiekis</i>	Kiek kartu kartojamas testas.
<i>strategy</i>	Studentu skirstymo strategijos numeris.

9.15.1.2 containsDigit()

```
bool containsDigit (
    const string & str)
```

Patikrina, ar eilute sudaryta tik is skaitmenu.

Parameters

<i>str</i>	Tikrinama eilute.
------------	-------------------

Returns

true, jei eilute sudaryta tik is skaitmenu, false kitu atveju.

9.15.1.3 duomenulrasymasFaile()

```
void duomenulrasymasFaile (
    const deque< Studentas > & grupe,
    int temp,
    const string & fileName)
```

Iraso studentu duomenis i faila.

Parameters

<i>grupe</i>	Studentu deque konteineris.
<i>temp</i>	Isvedimo formato pasirinkimas.
<i>fileName</i>	Failo pavadinimas.

9.15.1.4 duomenulrasymasKonsole()

```
void duomenulrasymasKonsole (
    deque< Studentas > & grupe,
    int temp)
```

Isveda studentu duomenis i konsole.

Parameters

<i>grupe</i>	Studentu deque konteineris.
<i>temp</i>	Isvedimo formato pasirinkimas.

9.15.1.5 fileRead()

```
void fileRead (
    deque< Studentas > & grupe,
    string file_name)
```

Nuskaito studentu duomenis is failo.

Parameters

<i>grupe</i>	Studentu deque konteineris, i kuri irasomi nuskaityti studentai.
<i>file_name</i>	Failo pavadinimas.

9.15.1.6 fileTest()

```
void fileTest (
    deque< Studentas > & grupe,
    string file_name,
    int & testKiekis)
```

Testuoja studentu nuskaityma is failo kelis kartus.

Parameters

<i>grupe</i>	Studentu deque konteineris.
<i>file_name</i>	Failo pavadinimas.
<i>testKiekis</i>	Testu skaicius.

9.15.1.7 GenerateStudentsFile()

```
void GenerateStudentsFile (
    int n)
```

Sugeneruoja studentu duomenu faila.

Parameters

<i>n</i>	Studentu skaicius faile.
----------	--------------------------

9.15.1.8 getSortChoice()

```
int getSortChoice (
    int temp)
```

Rusiuoja pagal vartotojo pasirinkta varianta.

Parameters

<i>temp</i>	Rusiavimo formato pasirinkimas.
-------------	---------------------------------

9.15.1.9 inputas()

```
void inputas (  
    deque< Studentas > & grupe)
```

Leidžia vartotojui iversti arba nuskaityti studento duomenis.

Parameters

<i>grupe</i>	Studentu deque konteineris
--------------	----------------------------

9.15.1.10 outputas()

```
void outputas (  
    deque< Studentas > & grupe)
```

Leidžia vartotojui pasirinkti studentu isvedimo buda.

Parameters

<i>grupe</i>	Studentu deque konteineris
--------------	----------------------------

9.15.1.11 RuleOfFiveTestas()

```
void RuleOfFiveTestas ()
```

Atlieka Rule of Five demonstracini testa [Studentas](#) klasei.

9.15.1.12 sortByUser()

```
void sortByUser (  
    deque< Studentas > & grupe,  
    int temp)
```

Grazina vartotojo pasirinkta rikiavimo kriteriju.

Parameters

<i>temp</i>	Pasirinktas isvedimo formatas.
-------------	--------------------------------

Returns

Rikiavimo pasirinkimo numeris.

9.15.1.13 SplitStudentsStrategy1()

```
void SplitStudentsStrategy1 (
    const deque< Studentas > & grupe,
    deque< Studentas > & failed,
    deque< Studentas > & passed)
```

Padalina studentus i dvi grupes: neislaikiusius ir islaikiusius.

Parameters

<i>grupe</i>	Pradine studentu grupe.
<i>failed</i>	Neislaikiusiu studentu grupe.
<i>passed</i>	Islaikiusiu studentu grupe.

9.15.1.14 SplitStudentsStrategy2()

```
void SplitStudentsStrategy2 (
    deque< Studentas > & grupe,
    deque< Studentas > & failed)
```

Padalina studentus i dvi grupes, paliekant islaikiusius pradineje grupeje.

Parameters

<i>grupe</i>	Pradine studentu grupe, po funkcijos joje lieka islaike studentai.
<i>failed</i>	Neislaikiusiu studentu grupe.

9.15.1.15 SplitStudentsStrategy3()

```
void SplitStudentsStrategy3 (
    deque< Studentas > & grupe,
    deque< Studentas > & failed)
```

Padalina studentus i dvi grupes naudojant `stable_partition`.

Parameters

<i>grupe</i>	Pradine studentu grupe, po funkcijos joje lieka islaike studentai.
<i>failed</i>	Neislaikiusiu studentu grupe.

9.16 src/io_list.cpp File Reference

```
#include "student_io_list.h"
#include "utils.h"
#include "timer.h"
#include <iostream>
#include <iomanip>
#include <algorithm>
#include <limits>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <ctime>
#include <stdlib.h>
#include <filesystem>
```

Functions

- void [outputas](#) (list< [Studentas](#) > &grupe)
- void [inputas](#) (list< [Studentas](#) > &grupe)
- void [fileTest](#) (list< [Studentas](#) > &grupe, string file_name, int &testKiekis)
- void [fileRead](#) (list< [Studentas](#) > &grupe, string file_name)
- void [sortByUser](#) (list< [Studentas](#) > &grupe, int temp)
- int [getSortChoice](#) (int temp)
Rusiuoja pagal vartotojo pasirinkta varianta.
- void [duomenulrasymasFaile](#) (list< [Studentas](#) > &grupe, int temp, string fileName)
- void [duomenulrasymasKonsole](#) (list< [Studentas](#) > &grupe, int temp)
- bool [containsDigit](#) (const string &str)
Patikrina, ar eilute sudaryta tik is skaitmenu.
- void [GenerateStudentsFile](#) (int n)
Sugeneruoja studentu duomeni faila.
- void [SplitStudentsStrategy1](#) (const list< [Studentas](#) > &grupe, list< [Studentas](#) > &failed, list< [Studentas](#) > &passed)
- void [SplitStudentsStrategy2](#) (list< [Studentas](#) > &grupe, list< [Studentas](#) > &failed)
- void [SplitStudentsStrategy3](#) (list< [Studentas](#) > &grupe, list< [Studentas](#) > &failed)
- void [benchmarkProcessingFile](#) (int n, int testKiekis, int strategy)
Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

9.16.1 Function Documentation

9.16.1.1 benchmarkProcessingFile()

```
void benchmarkProcessingFile (
    int n,
    int testKiekis,
    int strategy)
```

Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

Parameters

<i>n</i>	Studentu skaicius.
<i>testKiekis</i>	Kiek kartu kartojamas testas.
<i>strategy</i>	Studentu skirstymo strategijos numeris.

9.16.1.2 containsDigit()

```
bool containsDigit (  
    const string & str)
```

Patikrina, ar eilute sudaryta tik is skaitmenu.

Parameters

<i>str</i>	Tikrinama eilute.
------------	-------------------

Returns

true, jei eilute sudaryta tik is skaitmenu, false kitu atveju.

9.16.1.3 duomenulrasymasFaile()

```
void duomenulrasymasFaile (  
    list< Studentas > & grupe,  
    int temp,  
    string fileName)
```

9.16.1.4 duomenulrasymasKonsole()

```
void duomenulrasymasKonsole (  
    list< Studentas > & grupe,  
    int temp)
```

9.16.1.5 fileRead()

```
void fileRead (  
    list< Studentas > & grupe,  
    string file_name)
```

9.16.1.6 fileTest()

```
void fileTest (  
    list< Studentas > & grupe,  
    string file_name,  
    int & testKiekis)
```

9.16.1.7 GenerateStudentsFile()

```
void GenerateStudentsFile (  
    int n)
```

Sugeneruoja studentu duomenų failą.

Parameters

<i>n</i>	Studentų skaičius failė.
----------	--------------------------

9.16.1.8 getSortChoice()

```
int getSortChoice (  
    int temp)
```

Rusiuoja pagal vartotojo pasirinktą variantą.

Parameters

<i>temp</i>	Rusavimo formato pasirinkimas.
-------------	--------------------------------

9.16.1.9 inputas()

```
void inputas (  
    list< Studentas > & grupe)
```

9.16.1.10 outputas()

```
void outputas (  
    list< Studentas > & grupe)
```

9.16.1.11 sortByUser()

```
void sortByUser (  
    list< Studentas > & grupe,  
    int temp)
```

9.16.1.12 SplitStudentsStrategy1()

```
void SplitStudentsStrategy1 (  
    const list< Studentas > & grupe,  
    list< Studentas > & failed,  
    list< Studentas > & passed)
```

9.16.1.13 SplitStudentsStrategy2()

```
void SplitStudentsStrategy2 (
    list< Studentas > & grupe,
    list< Studentas > & failed)
```

9.16.1.14 SplitStudentsStrategy3()

```
void SplitStudentsStrategy3 (
    list< Studentas > & grupe,
    list< Studentas > & failed)
```

9.17 src/io_vector.cpp File Reference

```
#include "student_io_vector.h"
#include "utils.h"
#include "timer.h"
#include <iostream>
#include <iomanip>
#include <algorithm>
#include <limits>
#include <fstream>
#include <sstream>
#include <stdexcept>
#include <ctime>
#include <stdlib.h>
#include <filesystem>
```

Functions

- void **outputas** (vector< **Studentas** > &grupe)
- void **inputas** (vector< **Studentas** > &grupe)
- void **fileTest** (vector< **Studentas** > &grupe, string file_name, int &testKiekis)
- void **fileRead** (vector< **Studentas** > &grupe, string file_name)
- void **sortByUser** (vector< **Studentas** > &grupe, int temp)
- int **getSortChoice** (int temp)
Rusiuoja pagal vartotojo pasirinkta varianta.
- void **duomenulrasymasFaile** (vector< **Studentas** > &grupe, int temp, string fileName)
- void **duomenulrasymasKonsole** (vector< **Studentas** > &grupe, int temp)
- bool **containsDigit** (const string &str)
Patikrina, ar eilute sudaryta tik is skaitmenu.
- void **GenerateStudentsFile** (int n)
Sugeneruoja studentu duomenu faila.
- void **SplitStudentsStrategy1** (const vector< **Studentas** > &grupe, vector< **Studentas** > &failed, vector< **Studentas** > &passed)
- void **SplitStudentsStrategy2** (vector< **Studentas** > &grupe, vector< **Studentas** > &failed)
- void **SplitStudentsStrategy3** (vector< **Studentas** > &grupe, vector< **Studentas** > &failed)
- void **benchmarkProcessingFile** (int n, int testKiekis, int strategy)
Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

9.17.1 Function Documentation

9.17.1.1 benchmarkProcessingFile()

```
void benchmarkProcessingFile (  
    int n,  
    int testKiekis,  
    int strategy)
```

Atlieka failo generavimo, nuskaitymo, skirstymo ir rasymo greicio testa.

Parameters

<i>n</i>	Studentu skaicius.
<i>testKiekis</i>	Kiek kartu kartojamas testas.
<i>strategy</i>	Studentu skirstymo strategijos numeris.

9.17.1.2 containsDigit()

```
bool containsDigit (  
    const string & str)
```

Patikrina, ar eilute sudaryta tik is skaitmenu.

Parameters

<i>str</i>	Tikrinama eilute.
------------	-------------------

Returns

true, jei eilute sudaryta tik is skaitmenu, false kitu atveju.

9.17.1.3 duomenulrasymasFaile()

```
void duomenulrasymasFaile (  
    vector< Studentas > & grupe,  
    int temp,  
    string fileName)
```

9.17.1.4 duomenulrasymasKonsole()

```
void duomenulrasymasKonsole (  
    vector< Studentas > & grupe,  
    int temp)
```

9.17.1.5 fileRead()

```
void fileRead (
    vector< Studentas > & grupe,
    string file_name)
```

9.17.1.6 fileTest()

```
void fileTest (
    vector< Studentas > & grupe,
    string file_name,
    int & testKiekis)
```

9.17.1.7 GenerateStudentsFile()

```
void GenerateStudentsFile (
    int n)
```

Sugeneruoja studentu duomenu faila.

Parameters

<i>n</i>	Studentu skaicius faile.
----------	--------------------------

9.17.1.8 getSortChoice()

```
int getSortChoice (
    int temp)
```

Rusiuoja pagal vartotojo pasirinkta varianta.

Parameters

<i>temp</i>	Rusiavimo formato pasirinkimas.
-------------	---------------------------------

9.17.1.9 inputas()

```
void inputas (
    vector< Studentas > & grupe)
```

9.17.1.10 outputas()

```
void outputas (
    vector< Studentas > & grupe)
```

9.17.1.11 sortByUser()

```
void sortByUser (
    vector< Studentas > & grupe,
    int temp)
```

9.17.1.12 SplitStudentsStrategy1()

```
void SplitStudentsStrategy1 (
    const vector< Studentas > & grupe,
    vector< Studentas > & failed,
    vector< Studentas > & passed)
```

9.17.1.13 SplitStudentsStrategy2()

```
void SplitStudentsStrategy2 (
    vector< Studentas > & grupe,
    vector< Studentas > & failed)
```

9.17.1.14 SplitStudentsStrategy3()

```
void SplitStudentsStrategy3 (
    vector< Studentas > & grupe,
    vector< Studentas > & failed)
```

9.18 src/main_deque.cpp File Reference

```
#include "student_io_deque.h"
#include <deque>
```

Functions

- int [main](#) ()

9.18.1 Function Documentation

9.18.1.1 main()

```
int main ()
```


9.19 src/main_list.cpp File Reference

```
#include "student_io_list.h"  
#include <vector>
```

Functions

- int [main](#) ()

9.19.1 Function Documentation

9.19.1.1 main()

```
int main ()
```

9.20 src/main_vector.cpp File Reference

```
#include "student_io_vector.h"  
#include <vector>
```

Functions

- int [main](#) ()

9.20.1 Function Documentation

9.20.1.1 main()

```
int main ()
```

9.21 src/student.cpp File Reference

```
#include "student.h"  
#include <algorithm>  
#include <iostream>  
#include <utility>
```

Functions

- `std::istream & operator>> (std::istream &is, Studentas &s)`
Ivesties operatorius studento duomenims nuskaityti.
- `std::ostream & operator<< (std::ostream &os, const Studentas &s)`
Ivesties operatorius studento duomenims isvesti.

9.21.1 Function Documentation

9.21.1.1 operator<<()

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Studentas & s)
```

Ivesties operatorius studento duomenims isvesti.

Parameters

<code>os</code>	Ivesties srautas.
<code>s</code>	<code>Studentas</code> objektas.

Returns

Ivesties srautas.

9.21.1.2 operator>>()

```
std::istream & operator>> (  
    std::istream & is,  
    Studentas & s)
```

Ivesties operatorius studento duomenims nuskaityti.

Parameters

<code>is</code>	Ivesties srautas.
<code>s</code>	<code>Studentas</code> objektas.

Returns

Ivesties srautas.

9.22 src/utils.cpp File Reference

```
#include "utils.h"
#include <iostream>
#include <limits>
#include <cstdlib>
```

Functions

- void [intInput](#) (int &temp)
Nuskaityto sveikąjį skaičių ir patikrina investiciją.
- void [randomVardasPavarde](#) (string &vardas, string &pavarde)
Sugeneruoja atsitiktinį vardą ir pavardę.

Variables

- const vector< string > [vardai](#)
Vardų sąrašas atsitiktiniam generavimui.
- const vector< string > [vyr_pavardes](#) = {"Kazlauskas", "Jankauskas", "Petrauskas", "Stankevicius", "Zukauskas", "Butkus", "Pocius", "Urbonas", "Mockus", "Savickas"}
Vyrų pavardžių sąrašas atsitiktiniam generavimui.
- const vector< string > [mot_pavardes](#) = {"Kazlauskiene", "Jankauskiene", "Petrauskiene", "Stankeviciene", "Zukauskiene", "Butkiene", "Pociene", "Urboniene", "Mockiene", "Savickiene"}
Motėrių pavardžių sąrašas atsitiktiniam generavimui.

9.22.1 Function Documentation

9.22.1.1 intInput()

```
void intInput (  
    int & temp)
```

Nuskaityto sveikąjį skaičių ir patikrina investiciją.

Parameters

<i>temp</i>	Kintamasis, į kurį įrašomas nuskaitytas skaičius.
-------------	---

9.22.1.2 randomVardasPavarde()

```
void randomVardasPavarde (
    string & vardas,
    string & pavarde)
```

Sugeneruoja atsitiktini vardą ir pavardę.

Parameters

<i>vardas</i>	Sugeneruotas vardas.
<i>pavarde</i>	Sugeneruota pavardė.

9.22.2 Variable Documentation

9.22.2.1 mot_pavardes

```
const vector<string> mot_pavardes = {"Kazlauskienė", "Jankauskienė", "Petrauskienė", "Stankevičienė",
    "Zukauskienė", "Butkienė", "Pocienė", "Urbonienė", "Mockienė", "Savickienė"}
```

Moteriskų pavardžių sąrašas atsitiktiniam generavimui.

9.22.2.2 vardai

```
const vector<string> vardai
```

Initial value:

```
= {"Jonas", "Mantas", "Tomas", "Lukas", "Karolis", "Darius", "Paulius", "Mindaugas", "Justas", "Rokas",
    "Agne", "Ieva", "Egle", "Gabija", "Monika", "Karolina", "Viktorija", "Emilija", "Justina",
    "Greta"}
```

Vardų sąrašas atsitiktiniam generavimui.

9.22.2.3 vyr_pavardes

```
const vector<string> vyr_pavardes = {"Kazlauskas", "Jankauskas", "Petrauskas", "Stankevičius",
    "Zukauskas", "Butkus", "Pocius", "Urbonas", "Mockus", "Savickas"}
```

Vyriskų pavardžių sąrašas atsitiktiniam generavimui.

9.23 tests/doctest.h File Reference

```
#include <signal.h>
```

Classes

- struct `doctest::detail::types::enable_if< COND, T >`
- struct `doctest::detail::types::enable_if< true, T >`
- struct `doctest::detail::types::true_type`
- struct `doctest::detail::types::false_type`
- struct `doctest::detail::types::remove_reference< T >`
- struct `doctest::detail::types::remove_reference< T & >`
- struct `doctest::detail::types::remove_reference< T && >`
- struct `doctest::detail::types::is_same< T, U >`
- struct `doctest::detail::types::is_same< T, T >`
- struct `doctest::detail::types::is_rvalue_reference< T >`
- struct `doctest::detail::types::is_rvalue_reference< T && >`
- struct `doctest::detail::types::remove_const< T >`
- struct `doctest::detail::types::remove_const< const T >`
- struct `doctest::detail::types::is_enum< T >`
- struct `doctest::detail::types::underlying_type< T >`
- struct `doctest::detail::types::is_pointer< T >`
- struct `doctest::detail::types::is_pointer< T * >`
- struct `doctest::detail::types::is_array< T >`
- struct `doctest::detail::types::is_array< T[SIZE]>`
- struct `doctest::detail::deferred_false< T >`
- struct `doctest::detail::is_std_string< T, Enable >`
- struct `doctest::detail::is_std_string< T, typename types::enable_if< types::is_same< decltype(declval< const T & >().c_str()), const char * >::value &&types::is_same< decltype(declval< const T & >().size()), size_t >::value >::type >`
- class `doctest::String`
- struct `doctest::detail::has_insertion_operator< T, typename >`
- struct `doctest::detail::has_insertion_operator< T, decltype(operator<<(declval< std::ostream & >()), declval< const T & >()), void())>`
- struct `doctest::detail::should_stringify_as_underlying_type< T >`
- struct `doctest::detail::is_pair< T, typename >`
- struct `doctest::detail::is_pair< T, types::void_t< typename T::first_type, typename T::second_type, decltype(declval< T >().first), decltype(declval< T >().second)> >`
- struct `doctest::detail::is_container< T, typename >`
- struct `doctest::detail::is_container< T, types::void_t< typename T::value_type, typename T::iterator, decltype(declval< T >().begin()), decltype(declval< T >().end())> >`
- struct `doctest::detail::StringMakerBase< C >`
- struct `doctest::detail::StringMakerBase< true >`
- struct `doctest::StringMaker< T >`
- struct `doctest::detail::filldata< T, Enable >`
- struct `doctest::detail::filldata< T[N]>`
- struct `doctest::detail::filldata< const char[N]>`
- struct `doctest::detail::filldata< const void * >`
- struct `doctest::detail::filldata< const volatile void * >`
- struct `doctest::detail::filldata< T * >`
- struct `doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value &&detail::is_pair< T >::value >::type >`
- struct `doctest::detail::filldata< T, typename detail::types::enable_if< !detail::has_insertion_operator< T >::value &&detail::is_container< T >::value >::type >`
- class `doctest::Contains`
- struct `doctest::Approx`
- struct `doctest::IsNaN< F >`
- struct `doctest::ContextOptions`
- struct `doctest::AssertData`

- class `doctest::AssertData::StringContains`
- struct `doctest::detail::RelationalComparator< int, L, R >`
- struct `doctest::detail::Result`
- struct `doctest::detail::ResultBuilder`
- struct `doctest::detail::Expression_lhs< L >`
- struct `doctest::detail::ExpressionDecomposer`
- struct `doctest::SubcaseSignature`
- struct `doctest::detail::Subcase`
- struct `doctest::detail::TestSuite`
- struct `doctest::TestCaseData`
- struct `doctest::detail::TestCase`
- struct `doctest::detail::IExceptionTranslator`
- class `doctest::detail::ExceptionTranslator< T >`
- struct `doctest::IContextScope`
- struct `doctest::detail::ContextScopeBase`
- class `doctest::detail::ContextScope< L >`
- struct `doctest::MessageData`
- struct `doctest::detail::MessageBuilder`
- struct `doctest::detail::TestFailureException`
- class `doctest::Context`
- struct `doctest::CurrentTestCaseStats`
- struct `doctest::TestCaseException`
- struct `doctest::TestRunStats`
- struct `doctest::QueryData`
- struct `doctest::IReporter`

Namespaces

- namespace `doctest`
- namespace `doctest::detail`
- namespace `std`
- namespace `doctest::detail::types`
- namespace `doctest::assertType`
- namespace `doctest::detail::binaryAssertComparison`
- namespace `doctest::Color`
- namespace `doctest_detail_test_suite_ns`
- namespace `doctest::TestCaseFailureReason`

Macros

- `#define DOCTEST_PARTS_PUBLIC_VERSION`
- `#define DOCTEST_VERSION_MAJOR 2`
- `#define DOCTEST_VERSION_MINOR 5`
- `#define DOCTEST_VERSION_PATCH 0`
- `#define DOCTEST_TOSTR_IMPL(x)`
- `#define DOCTEST_TOSTR(x)`
- `#define DOCTEST_VERSION_STR`
- `#define DOCTEST_VERSION (DOCTEST_VERSION_MAJOR * 10000 + DOCTEST_VERSION_MINOR * 100 + DOCTEST_VERSION_PATCH)`
- `#define DOCTEST_PARTS_PUBLIC_COMPILER`
- `#define DOCTEST_CPLUSPLUS __cplusplus`
- `#define DOCTEST_COMPILER(MAJOR, MINOR, PATCH)`
- `#define DOCTEST_MSVC 0`

- #define [DOCTEST_CLANG](#) 0
- #define [DOCTEST_GCC](#) 0
- #define [DOCTEST_ICC](#) 0
- #define [DOCTEST_PARTS_PUBLIC_WARNINGS](#)
- #define [DOCTEST_CLANG_SUPPRESS_WARNING_PUSH](#)
- #define [DOCTEST_CLANG_SUPPRESS_WARNING\(w\)](#)
- #define [DOCTEST_CLANG_SUPPRESS_WARNING_POP](#)
- #define [DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH\(w\)](#)
- #define [DOCTEST_GCC_SUPPRESS_WARNING_PUSH](#)
- #define [DOCTEST_GCC_SUPPRESS_WARNING\(w\)](#)
- #define [DOCTEST_GCC_SUPPRESS_WARNING_POP](#)
- #define [DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH\(w\)](#)
- #define [DOCTEST_MSVC_SUPPRESS_WARNING_PUSH](#)
- #define [DOCTEST_MSVC_SUPPRESS_WARNING\(w\)](#)
- #define [DOCTEST_MSVC_SUPPRESS_WARNING_POP](#)
- #define [DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH\(w\)](#)
- #define [DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH](#)
- #define [DOCTEST_SUPPRESS_COMMON_WARNINGS_POP](#)
- #define [DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH](#)
- #define [DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP](#) [DOCTEST_SUPPRESS_COMMON_WARNINGS_POP](#)
- #define [DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH](#)
- #define [DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP](#) [DOCTEST_SUPPRESS_COMMON_WARNINGS_POP](#)
- #define [DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN](#)
- #define [DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END](#) [DOCTEST_MSVC_SUPPRESS_](#)
- #define [DOCTEST_PARTS_PUBLIC_CONFIG](#)
- #define [DOCTEST_PARTS_PUBLIC_PLATFORM](#)
- #define [DOCTEST_PLATFORM_LINUX](#)
- #define [DOCTEST_CONFIG_POSIX_SIGNALS](#)
- #define [DOCTEST_CONFIG_NO_EXCEPTIONS](#)
- #define [DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS](#)
- #define [DOCTEST_SYMBOL_EXPORT](#) __attribute__((visibility("default")))
- #define [DOCTEST_SYMBOL_IMPORT](#)
- #define [DOCTEST_INTERFACE](#)
- #define [DOCTEST_INTERFACE_DECL](#) [DOCTEST_INTERFACE](#)
- #define [DOCTEST_INTERFACE_DEF](#)
- #define [DOCTEST_EMPTY](#)
- #define [DOCTEST_NOINLINE](#) __attribute__((noinline))
- #define [DOCTEST_UNUSED](#) __attribute__((unused))
- #define [DOCTEST_ALIGNMENT\(x\)](#)
- #define [DOCTEST_INLINE_NOINLINE](#) inline [DOCTEST_NOINLINE](#)
- #define [DOCTEST_NORETURN](#) [[noreturn]]
- #define [DOCTEST_NOEXCEPT](#) noexcept
- #define [DOCTEST_CONSTEXPR](#) constexpr
- #define [DOCTEST_CONSTEXPR_FUNC](#) constexpr
- #define [DOCTEST_NO_SANITIZE_INTEGER](#)
- #define [DOCTEST_HAS_BUILTIN\(x\)](#)
- #define [DOCTEST_PARTS_PUBLIC_UTILITY](#)
- #define [DOCTEST_DECLARE_INTERFACE](#)(name)
- #define [DOCTEST_DEFINE_INTERFACE](#)(name)
- #define [DOCTEST_COUNTER](#) __LINE__
- #define [DOCTEST_CAT_IMPL](#)(s1, s2)
- #define [DOCTEST_CAT](#)(s1, s2)
- #define [DOCTEST_ANONYMOUS\(x\)](#)
- #define [DOCTEST_REF_WRAP\(x\)](#)
- #define [DOCTEST_GLOBAL_NO_WARNINGS](#)(var, ...)

- `#define DOCTEST_PARTS_PUBLIC_DEBUGGER`
- `#define DOCTEST_BREAK_INTO_DEBUGGER()`
- `#define DOCTEST_PARTS_PUBLIC_STD_FWD`
- `#define DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS`
- `#define DOCTEST_PARTS_PUBLIC_STD_UTILITY`
- `#define DOCTEST_PARTS_PUBLIC_STRING`
- `#define DOCTEST_CONFIG_STRING_SIZE_TYPE unsigned`
- `#define DOCTEST_STRINGIFY(...)`
- `#define DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS`
- `#define DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX`
- `#define DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN`
- `#define DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS`
- `#define DOCTEST_PARTS_PUBLIC_ASSERT_TYPE`
- `#define DOCTEST_PARTS_PUBLIC_ASSERT_DATA`
- `#define DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR`
- `#define DOCTEST_COMPARISON_RETURN_TYPE bool`
- `#define DOCTEST_RELATIONAL_OP(name, op)`
- `#define DOCTEST_CMP_EQ(l, r)`
- `#define DOCTEST_CMP_NE(l, r)`
- `#define DOCTEST_CMP_GT(l, r)`
- `#define DOCTEST_CMP_LT(l, r)`
- `#define DOCTEST_CMP_GE(l, r)`
- `#define DOCTEST_CMP_LE(l, r)`
- `#define DOCTEST_BINARY_RELATIONAL_OP(n, op)`
- `#define DOCTEST_PARTS_PUBLIC_ASSERT_RESULT`
- `#define DOCTEST_FORBIT_EXPRESSION(rt, op)`
- `#define DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION`
- `#define SFINAE_OP(ret, op)`
- `#define DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(op, op_str, op_macro)`
- `#define DOCTEST_PARTS_PUBLIC_COLOR`
- `#define DOCTEST_PARTS_PUBLIC_SUBCASE`
- `#define DOCTEST_PARTS_PUBLIC_TEST_SUITE`
- `#define DOCTEST_PARTS_PUBLIC_TEST_CASE`
- `#define DOCTEST_PARTS_PUBLIC_DECORATORS`
- `#define DOCTEST_DEFINE_DECORATOR(name, type, def)`
- `#define DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR`
- `#define DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE`
- `#define DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE`
- `#define DOCTEST_PARTS_PUBLIC_PATH`
- `#define DOCTEST_PARTS_PUBLIC_EXCEPTIONS`
- `#define DOCTEST_PARTS_PUBLIC_CONTEXT`
- `#define DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER`
- `#define DOCTEST_ASSERT_OUT_OF_TESTS(decomp)`
- `#define DOCTEST_ASSERT_IN_TESTS(decomp)`
- `#define DOCTEST_PARTS_PUBLIC_REPORTER`
- `#define DOCTEST_PARTS_PUBLIC_GENERATOR`
- `#define DOCTEST_PARTS_PUBLIC_MACROS`
- `#define DOCTEST_FUNC_EMPTY (void)0`
- `#define DOCTEST_FUNC_SCOPE_BEGIN do /* NOLINT(cppcoreguidelines-avoid-do-while)*/`
- `#define DOCTEST_FUNC_SCOPE_END while (false)`
- `#define DOCTEST_FUNC_SCOPE_RET(v)`
- `#define DOCTEST_ASSERT_LOG_REACT_RETURN(b)`
- `#define DOCTEST_WRAP_IN_TRY(x)`
- `#define DOCTEST_CAST_TO_VOID(...)`
- `#define DOCTEST_REGISTER_FUNCTION(global_prefix, f, decorators)`

- #define [DOCTEST_IMPLEMENT_FIXTURE](#)(der, base, func, decorators)
- #define [DOCTEST_CREATE_AND_REGISTER_FUNCTION](#)(f, decorators)
- #define [DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS](#)(f, proxy, decorators)
- #define [DOCTEST_TEST_CASE](#)(decorators)
- #define [DOCTEST_TEST_CASE_CLASS](#)(...)
- #define [DOCTEST_TEST_CASE_FIXTURE](#)(c, decorators)
- #define [DOCTEST_TYPE_TO_STRING_AS](#)(str, ...)
- #define [DOCTEST_TYPE_TO_STRING](#)(...)
- #define [DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL](#)(dec, T, iter, func)
- #define [DOCTEST_TEST_CASE_TEMPLATE_DEFINE](#)(dec, T, id)
- #define [DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL](#)(id, anon, ...)
- #define [DOCTEST_TEST_CASE_TEMPLATE_INVOKE](#)(id, ...)
- #define [DOCTEST_TEST_CASE_TEMPLATE_APPLY](#)(id, ...)
- #define [DOCTEST_TEST_CASE_TEMPLATE_IMPL](#)(dec, T, anon, ...)
- #define [DOCTEST_TEST_CASE_TEMPLATE](#)(dec, T, ...)
- #define [DOCTEST_SUBCASE](#)(name)
- #define [DOCTEST_GENERATE](#)(...)
- #define [DOCTEST_TEST_SUITE_IMPL](#)(decorators, ns_name)
- #define [DOCTEST_TEST_SUITE](#)(decorators)
- #define [DOCTEST_TEST_SUITE_BEGIN](#)(decorators)
- #define [DOCTEST_TEST_SUITE_END](#)
- #define [DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL](#)(translatorName, signature)
- #define [DOCTEST_REGISTER_EXCEPTION_TRANSLATOR](#)(signature)
- #define [DOCTEST_REGISTER_REPORTER](#)(name, priority, reporter)
- #define [DOCTEST_REGISTER_LISTENER](#)(name, priority, reporter)
- #define [DOCTEST_INFO](#)(...)
- #define [DOCTEST_INFO_IMPL](#)(mb_name, s_name, ...)
- #define [DOCTEST_CAPTURE](#)(x)
- #define [DOCTEST_ADD_AT_IMPL](#)(type, file, line, mb, ...)
- #define [DOCTEST_ADD_MESSAGE_AT](#)(file, line, ...)
- #define [DOCTEST_ADD_FAIL_CHECK_AT](#)(file, line, ...)
- #define [DOCTEST_ADD_FAIL_AT](#)(file, line, ...)
- #define [DOCTEST_MESSAGE](#)(...)
- #define [DOCTEST_FAIL_CHECK](#)(...)
- #define [DOCTEST_FAIL](#)(...)
- #define [DOCTEST_TO_LVALUE](#)(...)
- #define [DOCTEST_ASSERT_IMPLEMENT_2](#)(assert_type, ...)
- #define [DOCTEST_ASSERT_IMPLEMENT_1](#)(assert_type, ...)
- #define [DOCTEST_BINARY_ASSERT](#)(assert_type, comp, ...)
- #define [DOCTEST_UNARY_ASSERT](#)(assert_type, ...)
- #define [DOCTEST_WARN](#)(...)
- #define [DOCTEST_CHECK](#)(...)
- #define [DOCTEST_REQUIRE](#)(...)
- #define [DOCTEST_WARN_FALSE](#)(...)
- #define [DOCTEST_CHECK_FALSE](#)(...)
- #define [DOCTEST_REQUIRE_FALSE](#)(...)
- #define [DOCTEST_WARN_MESSAGE](#)(cond, ...)
- #define [DOCTEST_CHECK_MESSAGE](#)(cond, ...)
- #define [DOCTEST_REQUIRE_MESSAGE](#)(cond, ...)
- #define [DOCTEST_WARN_FALSE_MESSAGE](#)(cond, ...)
- #define [DOCTEST_CHECK_FALSE_MESSAGE](#)(cond, ...)
- #define [DOCTEST_REQUIRE_FALSE_MESSAGE](#)(cond, ...)
- #define [DOCTEST_WARN_EQ](#)(...)
- #define [DOCTEST_CHECK_EQ](#)(...)
- #define [DOCTEST_REQUIRE_EQ](#)(...)

- #define DOCTEST_WARN_NE(...)
- #define DOCTEST_CHECK_NE(...)
- #define DOCTEST_REQUIRE_NE(...)
- #define DOCTEST_WARN_GT(...)
- #define DOCTEST_CHECK_GT(...)
- #define DOCTEST_REQUIRE_GT(...)
- #define DOCTEST_WARN_LT(...)
- #define DOCTEST_CHECK_LT(...)
- #define DOCTEST_REQUIRE_LT(...)
- #define DOCTEST_WARN_GE(...)
- #define DOCTEST_CHECK_GE(...)
- #define DOCTEST_REQUIRE_GE(...)
- #define DOCTEST_WARN_LE(...)
- #define DOCTEST_CHECK_LE(...)
- #define DOCTEST_REQUIRE_LE(...)
- #define DOCTEST_WARN_UNARY(...)
- #define DOCTEST_CHECK_UNARY(...)
- #define DOCTEST_REQUIRE_UNARY(...)
- #define DOCTEST_WARN_UNARY_FALSE(...)
- #define DOCTEST_CHECK_UNARY_FALSE(...)
- #define DOCTEST_REQUIRE_UNARY_FALSE(...)
- #define DOCTEST_EXCEPTION_EMPTY_FUNC
- #define DOCTEST_REQUIRE DOCTEST_EXCEPTION_EMPTY_FUNC
- #define DOCTEST_REQUIRE_FALSE DOCTEST_EXCEPTION_EMPTY_FUNC
- #define DOCTEST_REQUIRE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC
- #define DOCTEST_REQUIRE_FALSE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC
- #define DOCTEST_REQUIRE_EQ DOCTEST_EXCEPTION_EMPTY_FUNC
- #define DOCTEST_REQUIRE_NE DOCTEST_EXCEPTION_EMPTY_FUNC
- #define DOCTEST_REQUIRE_GT DOCTEST_EXCEPTION_EMPTY_FUNC
- #define DOCTEST_REQUIRE_LT DOCTEST_EXCEPTION_EMPTY_FUNC
- #define DOCTEST_REQUIRE_GE DOCTEST_EXCEPTION_EMPTY_FUNC
- #define DOCTEST_REQUIRE_LE DOCTEST_EXCEPTION_EMPTY_FUNC
- #define DOCTEST_REQUIRE_UNARY DOCTEST_EXCEPTION_EMPTY_FUNC
- #define DOCTEST_REQUIRE_UNARY_FALSE DOCTEST_EXCEPTION_EMPTY_FUNC
- #define DOCTEST_WARN_THROWS(...)
- #define DOCTEST_CHECK_THROWS(...)
- #define DOCTEST_REQUIRE_THROWS(...)
- #define DOCTEST_WARN_THROWS_AS(expr, ...)
- #define DOCTEST_CHECK_THROWS_AS(expr, ...)
- #define DOCTEST_REQUIRE_THROWS_AS(expr, ...)
- #define DOCTEST_WARN_THROWS_WITH(expr, ...)
- #define DOCTEST_CHECK_THROWS_WITH(expr, ...)
- #define DOCTEST_REQUIRE_THROWS_WITH(expr, ...)
- #define DOCTEST_WARN_THROWS_WITH_AS(expr, with, ...)
- #define DOCTEST_CHECK_THROWS_WITH_AS(expr, with, ...)
- #define DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, ...)
- #define DOCTEST_WARN_NOTHROW(...)
- #define DOCTEST_CHECK_NOTHROW(...)
- #define DOCTEST_REQUIRE_NOTHROW(...)
- #define DOCTEST_WARN_THROWS_MESSAGE(expr, ...)
- #define DOCTEST_CHECK_THROWS_MESSAGE(expr, ...)
- #define DOCTEST_REQUIRE_THROWS_MESSAGE(expr, ...)
- #define DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, ...)
- #define DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, ...)
- #define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...)

- `#define DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with, ...)`
- `#define DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, ...)`
- `#define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...)`
- `#define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...)`
- `#define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...)`
- `#define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...)`
- `#define DOCTEST_WARN_NOTHROW_MESSAGE(expr, ...)`
- `#define DOCTEST_CHECK_NOTHROW_MESSAGE(expr, ...)`
- `#define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...)`
- `#define DOCTEST_FAST_WARN_EQ DOCTEST_WARN_EQ`
- `#define DOCTEST_FAST_CHECK_EQ DOCTEST_CHECK_EQ`
- `#define DOCTEST_FAST_REQUIRE_EQ DOCTEST_REQUIRE_EQ`
- `#define DOCTEST_FAST_WARN_NE DOCTEST_WARN_NE`
- `#define DOCTEST_FAST_CHECK_NE DOCTEST_CHECK_NE`
- `#define DOCTEST_FAST_REQUIRE_NE DOCTEST_REQUIRE_NE`
- `#define DOCTEST_FAST_WARN_GT DOCTEST_WARN_GT`
- `#define DOCTEST_FAST_CHECK_GT DOCTEST_CHECK_GT`
- `#define DOCTEST_FAST_REQUIRE_GT DOCTEST_REQUIRE_GT`
- `#define DOCTEST_FAST_WARN_LT DOCTEST_WARN_LT`
- `#define DOCTEST_FAST_CHECK_LT DOCTEST_CHECK_LT`
- `#define DOCTEST_FAST_REQUIRE_LT DOCTEST_REQUIRE_LT`
- `#define DOCTEST_FAST_WARN_GE DOCTEST_WARN_GE`
- `#define DOCTEST_FAST_CHECK_GE DOCTEST_CHECK_GE`
- `#define DOCTEST_FAST_REQUIRE_GE DOCTEST_REQUIRE_GE`
- `#define DOCTEST_FAST_WARN_LE DOCTEST_WARN_LE`
- `#define DOCTEST_FAST_CHECK_LE DOCTEST_CHECK_LE`
- `#define DOCTEST_FAST_REQUIRE_LE DOCTEST_REQUIRE_LE`
- `#define DOCTEST_FAST_WARN_UNARY DOCTEST_WARN_UNARY`
- `#define DOCTEST_FAST_CHECK_UNARY DOCTEST_CHECK_UNARY`
- `#define DOCTEST_FAST_REQUIRE_UNARY DOCTEST_REQUIRE_UNARY`
- `#define DOCTEST_FAST_WARN_UNARY_FALSE DOCTEST_WARN_UNARY_FALSE`
- `#define DOCTEST_FAST_CHECK_UNARY_FALSE DOCTEST_CHECK_UNARY_FALSE`
- `#define DOCTEST_FAST_REQUIRE_UNARY_FALSE DOCTEST_REQUIRE_UNARY_FALSE`
- `#define DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE(id, ...)`
- `#define DOCTEST_SCENARIO(name)`
- `#define DOCTEST_SCENARIO_CLASS(name)`
- `#define DOCTEST_SCENARIO_METHOD(x, name)`
- `#define DOCTEST_SCENARIO_TEMPLATE(name, T, ...)`
- `#define DOCTEST_SCENARIO_TEMPLATE_DEFINE(name, T, id)`
- `#define DOCTEST_GIVEN(name)`
- `#define DOCTEST_AND_GIVEN(name)`
- `#define DOCTEST_WHEN(name)`
- `#define DOCTEST_AND_WHEN(name)`
- `#define DOCTEST_THEN(name)`
- `#define DOCTEST_AND_THEN(name)`
- `#define TEST_CASE(name)`
- `#define TEST_CASE_CLASS(name)`
- `#define TEST_CASE_FIXTURE(x, name)`
- `#define TYPE_TO_STRING_AS(str, ...)`
- `#define TYPE_TO_STRING(...)`
- `#define TEST_CASE_TEMPLATE(name, T, ...)`
- `#define TEST_CASE_TEMPLATE_DEFINE(name, T, id)`
- `#define TEST_CASE_TEMPLATE_INVOKE(id, ...)`
- `#define TEST_CASE_TEMPLATE_APPLY(id, ...)`
- `#define SUBCASE(name)`

- #define [GENERATE\(...\)](#)
- #define [TEST_SUITE\(decorators\)](#)
- #define [TEST_SUITE_BEGIN\(name\)](#)
- #define [TEST_SUITE_END DOCTEST_TEST_SUITE_END](#)
- #define [REGISTER_EXCEPTION_TRANSLATOR\(signature\)](#)
- #define [REGISTER_REPORTER\(name, priority, reporter\)](#)
- #define [REGISTER_LISTENER\(name, priority, reporter\)](#)
- #define [INFO\(...\)](#)
- #define [CAPTURE\(x\)](#)
- #define [ADD_MESSAGE_AT\(file, line, ...\)](#)
- #define [ADD_FAIL_CHECK_AT\(file, line, ...\)](#)
- #define [ADD_FAIL_AT\(file, line, ...\)](#)
- #define [MESSAGE\(...\)](#)
- #define [FAIL_CHECK\(...\)](#)
- #define [FAIL\(...\)](#)
- #define [TO_LVALUE\(...\)](#)
- #define [WARN\(...\)](#)
- #define [WARN_FALSE\(...\)](#)
- #define [WARN_THROWS\(...\)](#)
- #define [WARN_THROWS_AS\(expr, ...\)](#)
- #define [WARN_THROWS_WITH\(expr, ...\)](#)
- #define [WARN_THROWS_WITH_AS\(expr, with, ...\)](#)
- #define [WARN_NOTHROW\(...\)](#)
- #define [CHECK\(...\)](#)
- #define [CHECK_FALSE\(...\)](#)
- #define [CHECK_THROWS\(...\)](#)
- #define [CHECK_THROWS_AS\(expr, ...\)](#)
- #define [CHECK_THROWS_WITH\(expr, ...\)](#)
- #define [CHECK_THROWS_WITH_AS\(expr, with, ...\)](#)
- #define [CHECK_NOTHROW\(...\)](#)
- #define [REQUIRE\(...\)](#)
- #define [REQUIRE_FALSE\(...\)](#)
- #define [REQUIRE_THROWS\(...\)](#)
- #define [REQUIRE_THROWS_AS\(expr, ...\)](#)
- #define [REQUIRE_THROWS_WITH\(expr, ...\)](#)
- #define [REQUIRE_THROWS_WITH_AS\(expr, with, ...\)](#)
- #define [REQUIRE_NOTHROW\(...\)](#)
- #define [WARN_MESSAGE\(cond, ...\)](#)
- #define [WARN_FALSE_MESSAGE\(cond, ...\)](#)
- #define [WARN_THROWS_MESSAGE\(expr, ...\)](#)
- #define [WARN_THROWS_AS_MESSAGE\(expr, ex, ...\)](#)
- #define [WARN_THROWS_WITH_MESSAGE\(expr, with, ...\)](#)
- #define [WARN_THROWS_WITH_AS_MESSAGE\(expr, with, ex, ...\)](#)
- #define [WARN_NOTHROW_MESSAGE\(expr, ...\)](#)
- #define [CHECK_MESSAGE\(cond, ...\)](#)
- #define [CHECK_FALSE_MESSAGE\(cond, ...\)](#)
- #define [CHECK_THROWS_MESSAGE\(expr, ...\)](#)
- #define [CHECK_THROWS_AS_MESSAGE\(expr, ex, ...\)](#)
- #define [CHECK_THROWS_WITH_MESSAGE\(expr, with, ...\)](#)
- #define [CHECK_THROWS_WITH_AS_MESSAGE\(expr, with, ex, ...\)](#)
- #define [CHECK_NOTHROW_MESSAGE\(expr, ...\)](#)
- #define [REQUIRE_MESSAGE\(cond, ...\)](#)
- #define [REQUIRE_FALSE_MESSAGE\(cond, ...\)](#)
- #define [REQUIRE_THROWS_MESSAGE\(expr, ...\)](#)
- #define [REQUIRE_THROWS_AS_MESSAGE\(expr, ex, ...\)](#)

- #define [REQUIRE_THROWS_WITH_MESSAGE](#)(expr, with, ...)
- #define [REQUIRE_THROWS_WITH_AS_MESSAGE](#)(expr, with, ex, ...)
- #define [REQUIRE_NOTHROW_MESSAGE](#)(expr, ...)
- #define [SCENARIO](#)(name)
- #define [SCENARIO_METHOD](#)(x, name)
- #define [SCENARIO_CLASS](#)(name)
- #define [SCENARIO_TEMPLATE](#)(name, T, ...)
- #define [SCENARIO_TEMPLATE_DEFINE](#)(name, T, id)
- #define [GIVEN](#)(name)
- #define [AND_GIVEN](#)(name)
- #define [WHEN](#)(name)
- #define [AND_WHEN](#)(name)
- #define [THEN](#)(name)
- #define [AND_THEN](#)(name)
- #define [WARN_EQ](#)(...)
- #define [CHECK_EQ](#)(...)
- #define [REQUIRE_EQ](#)(...)
- #define [WARN_NE](#)(...)
- #define [CHECK_NE](#)(...)
- #define [REQUIRE_NE](#)(...)
- #define [WARN_GT](#)(...)
- #define [CHECK_GT](#)(...)
- #define [REQUIRE_GT](#)(...)
- #define [WARN_LT](#)(...)
- #define [CHECK_LT](#)(...)
- #define [REQUIRE_LT](#)(...)
- #define [WARN_GE](#)(...)
- #define [CHECK_GE](#)(...)
- #define [REQUIRE_GE](#)(...)
- #define [WARN_LE](#)(...)
- #define [CHECK_LE](#)(...)
- #define [REQUIRE_LE](#)(...)
- #define [WARN_UNARY](#)(...)
- #define [CHECK_UNARY](#)(...)
- #define [REQUIRE_UNARY](#)(...)
- #define [WARN_UNARY_FALSE](#)(...)
- #define [CHECK_UNARY_FALSE](#)(...)
- #define [REQUIRE_UNARY_FALSE](#)(...)
- #define [FAST_WARN_EQ](#)(...)
- #define [FAST_CHECK_EQ](#)(...)
- #define [FAST_REQUIRE_EQ](#)(...)
- #define [FAST_WARN_NE](#)(...)
- #define [FAST_CHECK_NE](#)(...)
- #define [FAST_REQUIRE_NE](#)(...)
- #define [FAST_WARN_GT](#)(...)
- #define [FAST_CHECK_GT](#)(...)
- #define [FAST_REQUIRE_GT](#)(...)
- #define [FAST_WARN_LT](#)(...)
- #define [FAST_CHECK_LT](#)(...)
- #define [FAST_REQUIRE_LT](#)(...)
- #define [FAST_WARN_GE](#)(...)
- #define [FAST_CHECK_GE](#)(...)
- #define [FAST_REQUIRE_GE](#)(...)
- #define [FAST_WARN_LE](#)(...)
- #define [FAST_CHECK_LE](#)(...)

- `#define FAST_REQUIRE_LE(...)`
- `#define FAST_WARN_UNARY(...)`
- `#define FAST_CHECK_UNARY(...)`
- `#define FAST_REQUIRE_UNARY(...)`
- `#define FAST_WARN_UNARY_FALSE(...)`
- `#define FAST_CHECK_UNARY_FALSE(...)`
- `#define FAST_REQUIRE_UNARY_FALSE(...)`
- `#define TEST_CASE_TEMPLATE_INSTANTIATE(id, ...)`

Typedefs

- `typedef decltype(nullptr) std::nullptr_t`
- `typedef decltype(sizeof(void*)) std::size_t`
- `typedef basic_ostream< char, char_traits< char > > std::ostream`
- `typedef basic_istream< char, char_traits< char > > std::istream`
- `template<typename... Unused>`
`using doctest::detail::types::void_t = void`
- `using doctest::detail::funcType = void (*)()`
- `using doctest::detail::assert_handler = void (*)(const AssertData &)`
- `using doctest::detail::reporterCreatorFunc = IReporter (*)(const ContextOptions &)`
- `typedef decltype(sizeof(void*)) doctest::size_t`

Enumerations

- `enum doctest::assertType::Enum {`
`doctest::assertType::is_warn = 1, doctest::assertType::is_check = 2 * is_warn, doctest::assertType::is_require`
`= 2 * is_check, doctest::assertType::is_normal = 2 * is_require,`
`doctest::assertType::is_throws = 2 * is_normal, doctest::assertType::is_throws_as = 2 * is_throws,`
`doctest::assertType::is_throws_with = 2 * is_throws_as, doctest::assertType::is_nothrow = 2 * is_throws_↵`
`with,`
`doctest::assertType::is_false = 2 * is_nothrow, doctest::assertType::is_unary = 2 * is_false, doctest::assertType::is_eq`
`= 2 * is_unary, doctest::assertType::is_ne = 2 * is_eq,`
`doctest::assertType::is_lt = 2 * is_ne, doctest::assertType::is_gt = 2 * is_lt, doctest::assertType::is_ge = 2 *`
`is_gt, doctest::assertType::is_le = 2 * is_ge,`
`doctest::assertType::DT_WARN = is_normal | is_warn, doctest::assertType::DT_CHECK = is_normal | is_↵`
`check, doctest::assertType::DT_REQUIRE = is_normal | is_require, doctest::assertType::DT_WARN_FALSE`
`= is_normal | is_false | is_warn,`
`doctest::assertType::DT_CHECK_FALSE = is_normal | is_false | is_check, doctest::assertType::DT_REQUIRE_FALSE`
`= is_normal | is_false | is_require, doctest::assertType::DT_WARN_THROWS = is_throws | is_warn,`
`doctest::assertType::DT_CHECK_THROWS = is_throws | is_check,`
`doctest::assertType::DT_REQUIRE_THROWS = is_throws | is_require, doctest::assertType::DT_WARN_THROWS_AS`
`= is_throws_as | is_warn, doctest::assertType::DT_CHECK_THROWS_AS = is_throws_as | is_check,`
`doctest::assertType::DT_REQUIRE_THROWS_AS = is_throws_as | is_require,`
`doctest::assertType::DT_WARN_THROWS_WITH = is_throws_with | is_warn, doctest::assertType::DT_CHECK_THROWS_W`
`= is_throws_with | is_check, doctest::assertType::DT_REQUIRE_THROWS_WITH = is_throws_with | is_↵`
`require, doctest::assertType::DT_WARN_THROWS_WITH_AS = is_throws_with | is_throws_as | is_warn,`
`doctest::assertType::DT_CHECK_THROWS_WITH_AS = is_throws_with | is_throws_as | is_check,`
`doctest::assertType::DT_REQUIRE_THROWS_WITH_AS = is_throws_with | is_throws_as | is_require,`
`doctest::assertType::DT_WARN_NOTHROW = is_nothrow | is_warn, doctest::assertType::DT_CHECK_NOTHROW`
`= is_nothrow | is_check,`
`doctest::assertType::DT_REQUIRE_NOTHROW = is_nothrow | is_require, doctest::assertType::DT_WARN_EQ`
`= is_normal | is_eq | is_warn, doctest::assertType::DT_CHECK_EQ = is_normal | is_eq | is_check,`
`doctest::assertType::DT_REQUIRE_EQ = is_normal | is_eq | is_require,`
`doctest::assertType::DT_WARN_NE = is_normal | is_ne | is_warn, doctest::assertType::DT_CHECK_NE`
`= is_normal | is_ne | is_check, doctest::assertType::DT_REQUIRE_NE = is_normal | is_ne | is_require,`

```

doctest::assertType::DT_WARN_GT = is_normal | is_gt | is_warn ,
doctest::assertType::DT_CHECK_GT = is_normal | is_gt | is_check , doctest::assertType::DT_REQUIRE_GT
= is_normal | is_gt | is_require , doctest::assertType::DT_WARN_LT = is_normal | is_lt | is_warn ,
doctest::assertType::DT_CHECK_LT = is_normal | is_lt | is_check ,
doctest::assertType::DT_REQUIRE_LT = is_normal | is_lt | is_require , doctest::assertType::DT_WARN_GE
= is_normal | is_ge | is_warn , doctest::assertType::DT_CHECK_GE = is_normal | is_ge | is_check ,
doctest::assertType::DT_REQUIRE_GE = is_normal | is_ge | is_require ,
doctest::assertType::DT_WARN_LE = is_normal | is_le | is_warn , doctest::assertType::DT_CHECK_LE
= is_normal | is_le | is_check , doctest::assertType::DT_REQUIRE_LE = is_normal | is_le | is_require ,
doctest::assertType::DT_WARN_UNARY = is_normal | is_unary | is_warn ,
doctest::assertType::DT_CHECK_UNARY = is_normal | is_unary | is_check , doctest::assertType::DT_REQUIRE_UNARY
= is_normal | is_unary | is_require , doctest::assertType::DT_WARN_UNARY_FALSE = is_normal | is_false
| is_unary | is_warn , doctest::assertType::DT_CHECK_UNARY_FALSE = is_normal | is_false | is_unary |
is_check ,
doctest::assertType::DT_REQUIRE_UNARY_FALSE = is_normal | is_false | is_unary | is_require }
• enum doctest::detail::binaryAssertComparison::Enum {
doctest::detail::binaryAssertComparison::eq = 0 , doctest::detail::binaryAssertComparison::ne , doctest::detail::binaryAssertComparison::lt ,
doctest::detail::binaryAssertComparison::le }
• enum doctest::Color::Enum {
doctest::Color::None = 0 , doctest::Color::White , doctest::Color::Red , doctest::Color::Green ,
doctest::Color::Blue , doctest::Color::Cyan , doctest::Color::Yellow , doctest::Color::Grey ,
doctest::Color::Bright = 0x10 , doctest::Color::BrightRed = Bright | Red , doctest::Color::BrightGreen = Bright
| Green , doctest::Color::LightGrey = Bright | Grey ,
doctest::Color::BrightWhite = Bright | White }
• enum doctest::TestCaseFailureReason::Enum {
doctest::TestCaseFailureReason::None = 0 , doctest::TestCaseFailureReason::AssertFailure = 1 ,
doctest::TestCaseFailureReason::Exception = 2 , doctest::TestCaseFailureReason::Crash = 4 ,
doctest::TestCaseFailureReason::TooManyFailedAsserts = 8 , doctest::TestCaseFailureReason::Timeout =
16 , doctest::TestCaseFailureReason::ShouldHaveFailedButDidnt = 32 , doctest::TestCaseFailureReason::ShouldHaveFailedAndDidnt = 64 ,
doctest::TestCaseFailureReason::DidntFailExactlyNumTimes = 128 , doctest::TestCaseFailureReason::FailedExactlyNumTimes = 256 , doctest::TestCaseFailureReason::CouldHaveFailedAndDid = 512 }

```

Functions

- DOCTEST_INTERFACE bool doctest::detail::isDebuggerActive ()
- template<class traits>
basic_ostream< char, traits > & std::operator<< (basic_ostream< char, traits > &, const char *)
- template<typename T>
T && doctest::detail::declval ()
- template<class T>
DOCTEST_CONSTEXPR_FUNC T && doctest::detail::forward (typename types::remove_reference< T >::type &t) DOCTEST_NOEXCEPT
- template<class T>
DOCTEST_CONSTEXPR_FUNC T && doctest::detail::forward (typename types::remove_reference< T >::type &&t) DOCTEST_NOEXCEPT
- DOCTEST_INTERFACE String doctest::operator+ (const String &lhs, const String &rhs)
- DOCTEST_INTERFACE bool doctest::operator== (const String &lhs, const String &rhs)
- DOCTEST_INTERFACE bool doctest::operator!= (const String &lhs, const String &rhs)
- DOCTEST_INTERFACE bool doctest::operator< (const String &lhs, const String &rhs)
- DOCTEST_INTERFACE bool doctest::operator> (const String &lhs, const String &rhs)
- DOCTEST_INTERFACE bool doctest::operator<= (const String &lhs, const String &rhs)
- DOCTEST_INTERFACE bool doctest::operator>= (const String &lhs, const String &rhs)
- DOCTEST_INTERFACE std::ostream * doctest::detail::tlssPush ()
- DOCTEST_INTERFACE String doctest::detail::tlssPop ()

- `template<typename T>`
`void doctest::detail::filloss (std::ostream *stream, const T &in)`
- `template<typename T, size_t N>`
`void doctest::detail::filloss (std::ostream *stream, const T(&in)[N])`
- `template<typename T>`
`String doctest::detail::toStream (const T &in)`
- `template<typename T>`
`String doctest::toString ()`
- `template<typename T, typename detail::types::enable_if<!detail::should_stringify_as_underlying_type< T >::value, bool >::type = true>`
`String doctest::toString (const DOCTEST_REF_WRAP(T) value)`
- `String && doctest::toString (String &&in)`
- `DOCTEST_INTERFACE String doctest::toString (const String &in)`
- `DOCTEST_INTERFACE String doctest::toString (std::nullptr_t)`
- `DOCTEST_INTERFACE String doctest::toString (bool in)`
- `DOCTEST_INTERFACE String doctest::toString (float in)`
- `DOCTEST_INTERFACE String doctest::toString (double in)`
- `DOCTEST_INTERFACE String doctest::toString (double long in)`
- `DOCTEST_INTERFACE String doctest::toString (char in)`
- `DOCTEST_INTERFACE String doctest::toString (char signed in)`
- `DOCTEST_INTERFACE String doctest::toString (char unsigned in)`
- `DOCTEST_INTERFACE String doctest::toString (short in)`
- `DOCTEST_INTERFACE String doctest::toString (short unsigned in)`
- `DOCTEST_INTERFACE String doctest::toString (signed in)`
- `DOCTEST_INTERFACE String doctest::toString (unsigned in)`
- `DOCTEST_INTERFACE String doctest::toString (long in)`
- `DOCTEST_INTERFACE String doctest::toString (long unsigned in)`
- `DOCTEST_INTERFACE String doctest::toString (long long in)`
- `DOCTEST_INTERFACE String doctest::toString (long long unsigned in)`
- `template<typename L, typename R>`
`String doctest::detail::stringifyBinaryExpr (const DOCTEST_REF_WRAP(L) lhs, const char *op, const DOCTEST_REF_WRAP(R) rhs)`
- `DOCTEST_INTERFACE String doctest::toString (const Contains &in)`
- `DOCTEST_INTERFACE bool doctest::operator== (const String &lhs, const Contains &rhs)`
- `DOCTEST_INTERFACE bool doctest::operator== (const Contains &lhs, const String &rhs)`
- `DOCTEST_INTERFACE bool doctest::operator!= (const String &lhs, const Contains &rhs)`
- `DOCTEST_INTERFACE bool doctest::operator!= (const Contains &lhs, const String &rhs)`
- `DOCTEST_INTERFACE String doctest::toString (const Approx &in)`
- `DOCTEST_INTERFACE String doctest::toString (IsNaN< float > in)`
- `DOCTEST_INTERFACE String doctest::toString (IsNaN< double > in)`
- `DOCTEST_INTERFACE String doctest::toString (IsNaN< double long > in)`
- `DOCTEST_INTERFACE const ContextOptions * doctest::getContextOptions ()`
- `DOCTEST_INTERFACE const char * doctest::assertString (assertType::Enum at)`
- `DOCTEST_INTERFACE const char * doctest::failureString (assertType::Enum at)`
- `DOCTEST_INTERFACE std::ostream & doctest::Color::operator<< (std::ostream &s, Color::Enum code)`
- `DOCTEST_INTERFACE int doctest::detail::setTestSuite (const TestSuite &ts) noexcept`
- `DOCTEST_INTERFACE doctest::detail::TestSuite & doctest_detail_test_suite_ns::getCurrentTestSuite ()`
`noexcept`
- `DOCTEST_INTERFACE int doctest::detail::regTest (const TestCase &tc) noexcept`
- `doctest::DOCTEST_DEFINE_DECORATOR (test_suite, const char *, "")`
- `doctest::DOCTEST_DEFINE_DECORATOR (description, const char *, "")`
- `doctest::DOCTEST_DEFINE_DECORATOR (skip, bool, true)`
- `doctest::DOCTEST_DEFINE_DECORATOR (no_breaks, bool, true)`
- `doctest::DOCTEST_DEFINE_DECORATOR (no_output, bool, true)`
- `doctest::DOCTEST_DEFINE_DECORATOR (timeout, double, 0)`

- `doctest::DOCTEST_DEFINE_DECORATOR` (may_fail, bool, true)
- `doctest::DOCTEST_DEFINE_DECORATOR` (should_fail, bool, true)
- `doctest::DOCTEST_DEFINE_DECORATOR` (expected_failures, int, 0)
- `DOCTEST_INTERFACE` void `doctest::detail::registerExceptionTranslatorImpl` (const `IExceptionTranslator` *et) noexcept
- template<typename T>
int `doctest::registerExceptionTranslator` (String(*translateFunction)(T)) noexcept
- template<typename L>
`ContextScope`< L > `doctest::detail::MakeContextScope` (const L &lambda)
- `DOCTEST_INTERFACE` const char * `doctest::skipPathFromFilename` (const char *file)
- `DOCTEST_INTERFACE` bool `doctest::detail::checkIfShouldThrow` (`assertType::Enum` at)
- `DOCTEST_INTERFACE` void `doctest::detail::throwException` ()
- `DOCTEST_INTERFACE` void `doctest::detail::failed_out_of_a_testing_context` (const `AssertData` &ad)
- `DOCTEST_INTERFACE` bool `doctest::detail::decomp_assert` (`assertType::Enum` at, const char *file, int line, const char *expr, const `Result` &result)
- template<int comparison, typename L, typename R>
`DOCTEST_NOINLINE` bool `doctest::detail::binary_assert` (`assertType::Enum` at, const char *file, int line, const char *expr, const `DOCTEST_REF_WRAP`(L) lhs, const `DOCTEST_REF_WRAP`(R) rhs)
- template<typename L>
`DOCTEST_NOINLINE` bool `doctest::detail::unary_assert` (`assertType::Enum` at, const char *file, int line, const char *expr, const `DOCTEST_REF_WRAP`(L) val)
- `DOCTEST_INTERFACE` void `doctest::detail::registerReporterImpl` (const char *name, int prio, `reporterCreatorFunc` c, bool isReporter) noexcept
- template<typename Reporter>
`IReporter` * `doctest::detail::reporterCreator` (const `ContextOptions` &o)
- template<typename Reporter>
int `doctest::registerReporter` (const char *name, int priority, bool isReporter) noexcept
- `DOCTEST_INTERFACE` size_t `doctest::detail::acquireGeneratorDecisionIndex` (size_t count)
- template<typename T, typename... Rest>
T `doctest::detail::acquireGeneratorValue` (T first, Rest... rest)
- template<typename T>
int `doctest::detail::instantiationHelper` (const T &) noexcept

Variables

- template struct `DOCTEST_INTERFACE_DECL` `doctest::IsNaN`< float >
- template struct `DOCTEST_INTERFACE_DECL` `doctest::IsNaN`< double >
- template struct `DOCTEST_INTERFACE_DECL` `doctest::IsNaN`< long double >
- struct `DOCTEST_INTERFACE` `doctest::detail::TestCase`
- struct `DOCTEST_INTERFACE` `doctest::TestCaseData`
- `DOCTEST_INTERFACE` bool `doctest::is_running_in_test`

9.23.1 Macro Definition Documentation

9.23.1.1 ADD_FAIL_AT

```
#define ADD_FAIL_AT(
    file,
    line,
    ...)
```

Value:

```
DOCTEST_ADD_FAIL_AT(file, line, __VA_ARGS__)
```

9.23.1.2 ADD_FAIL_CHECK_AT

```
#define ADD_FAIL_CHECK_AT(  
    file,  
    line,  
    ...)
```

Value:

[DOCTEST_ADD_FAIL_CHECK_AT](#)(*file*, *line*, __VA_ARGS__)

9.23.1.3 ADD_MESSAGE_AT

```
#define ADD_MESSAGE_AT(  
    file,  
    line,  
    ...)
```

Value:

[DOCTEST_ADD_MESSAGE_AT](#)(*file*, *line*, __VA_ARGS__)

9.23.1.4 AND_GIVEN

```
#define AND_GIVEN(  
    name)
```

Value:

[DOCTEST_AND_GIVEN](#)(*name*)

9.23.1.5 AND_THEN

```
#define AND_THEN(  
    name)
```

Value:

[DOCTEST_AND_THEN](#)(*name*)

9.23.1.6 AND_WHEN

```
#define AND_WHEN(  
    name)
```

Value:

[DOCTEST_AND_WHEN](#)(*name*)

9.23.1.7 CAPTURE

```
#define CAPTURE(  
    x)
```

Value:

[DOCTEST_CAPTURE](#)(*x*)

9.23.1.8 CHECK

```
#define CHECK(  
    ...)
```

Value:

[DOCTEST_CHECK](#)(__VA_ARGS__)

9.23.1.9 CHECK_EQ

```
#define CHECK_EQ(  
    ...)
```

Value:

[DOCTEST_CHECK_EQ](#)(__VA_ARGS__)

9.23.1.10 CHECK_FALSE

```
#define CHECK_FALSE(  
    ...)
```

Value:

[DOCTEST_CHECK_FALSE](#)(__VA_ARGS__)

9.23.1.11 CHECK_FALSE_MESSAGE

```
#define CHECK_FALSE_MESSAGE(  
    cond,  
    ...)
```

Value:

[DOCTEST_CHECK_FALSE_MESSAGE](#)(cond, __VA_ARGS__)

9.23.1.12 CHECK_GE

```
#define CHECK_GE(  
    ...)
```

Value:

[DOCTEST_CHECK_GE](#)(__VA_ARGS__)

9.23.1.13 CHECK_GT

```
#define CHECK_GT(  
    ...)
```

Value:

[DOCTEST_CHECK_GT](#)(__VA_ARGS__)

9.23.1.14 CHECK_LE

```
#define CHECK_LE(  
    ...)
```

Value:

[DOCTEST_CHECK_LE](#)(__VA_ARGS__)

9.23.1.15 CHECK_LT

```
#define CHECK_LT(  
    ...)
```

Value:

[DOCTEST_CHECK_LT](#)(__VA_ARGS__)

9.23.1.16 CHECK_MESSAGE

```
#define CHECK_MESSAGE(  
    cond,  
    ...)
```

Value:

[DOCTEST_CHECK_MESSAGE](#)(cond, __VA_ARGS__)

9.23.1.17 CHECK_NE

```
#define CHECK_NE(  
    ...)
```

Value:

[DOCTEST_CHECK_NE](#)(__VA_ARGS__)

9.23.1.18 CHECK_NOTHROW

```
#define CHECK_NOTHROW(  
    ...)
```

Value:

[DOCTEST_CHECK_NOTHROW](#)(__VA_ARGS__)

9.23.1.19 CHECK_NOTHROW_MESSAGE

```
#define CHECK_NOTHROW_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_CHECK_NOTHROW_MESSAGE](#)(expr, __VA_ARGS__)

9.23.1.20 CHECK_THROWS

```
#define CHECK_THROWS(  
    ...)
```

Value:

`DOCTEST_CHECK_THROWS(__VA_ARGS__)`

9.23.1.21 CHECK_THROWS_AS

```
#define CHECK_THROWS_AS(  
    expr,  
    ...)
```

Value:

`DOCTEST_CHECK_THROWS_AS(expr, __VA_ARGS__)`

9.23.1.22 CHECK_THROWS_AS_MESSAGE

```
#define CHECK_THROWS_AS_MESSAGE(  
    expr,  
    ex,  
    ...)
```

Value:

`DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, __VA_ARGS__)`

9.23.1.23 CHECK_THROWS_MESSAGE

```
#define CHECK_THROWS_MESSAGE(  
    expr,  
    ...)
```

Value:

`DOCTEST_CHECK_THROWS_MESSAGE(expr, __VA_ARGS__)`

9.23.1.24 CHECK_THROWS_WITH

```
#define CHECK_THROWS_WITH(  
    expr,  
    ...)
```

Value:

`DOCTEST_CHECK_THROWS_WITH(expr, __VA_ARGS__)`

9.23.1.25 CHECK_THROWS_WITH_AS

```
#define CHECK_THROWS_WITH_AS(  
    expr,  
    with,  
    ...)
```

Value:

`DOCTEST_CHECK_THROWS_WITH_AS(expr, with, __VA_ARGS__)`

9.23.1.26 CHECK_THROWS_WITH_AS_MESSAGE

```
#define CHECK_THROWS_WITH_AS_MESSAGE(  
    expr,  
    with,  
    ex,  
    ...)
```

Value:

`DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, __VA_ARGS__)`

9.23.1.27 CHECK_THROWS_WITH_MESSAGE

```
#define CHECK_THROWS_WITH_MESSAGE(  
    expr,  
    with,  
    ...)
```

Value:

`DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, __VA_ARGS__)`

9.23.1.28 CHECK_UNARY

```
#define CHECK_UNARY(  
    ...)
```

Value:

`DOCTEST_CHECK_UNARY(__VA_ARGS__)`

9.23.1.29 CHECK_UNARY_FALSE

```
#define CHECK_UNARY_FALSE(  
    ...)
```

Value:

`DOCTEST_CHECK_UNARY_FALSE(__VA_ARGS__)`

9.23.1.30 DOCTEST_ADD_AT_IMPL

```
#define DOCTEST_ADD_AT_IMPL(
    type,
    file,
    line,
    mb,
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN {
    doctest::detail::MessageBuilder mb(file, line, doctest::assertType::type);
    mb * __VA_ARGS__;
    if (mb.log())
        DOCTEST_BREAK_INTO_DEBUGGER();
    mb.react();
}
DOCTEST_FUNC_SCOPE_END
```

9.23.1.31 DOCTEST_ADD_FAIL_AT

```
#define DOCTEST_ADD_FAIL_AT(
    file,
    line,
    ...)
```

Value:

```
DOCTEST_ADD_AT_IMPL(is_require, file, line, DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)
```

9.23.1.32 DOCTEST_ADD_FAIL_CHECK_AT

```
#define DOCTEST_ADD_FAIL_CHECK_AT(
    file,
    line,
    ...)
```

Value:

```
DOCTEST_ADD_AT_IMPL(is_check, file, line, DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)
```

9.23.1.33 DOCTEST_ADD_MESSAGE_AT

```
#define DOCTEST_ADD_MESSAGE_AT(
    file,
    line,
    ...)
```

Value:

```
DOCTEST_ADD_AT_IMPL(is_warn, file, line, DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)
```

9.23.1.34 DOCTEST_ALIGNMENT

```
#define DOCTEST_ALIGNMENT(  
    x)
```

Value:

```
__attribute__((aligned(x)))
```

9.23.1.35 DOCTEST_AND_GIVEN

```
#define DOCTEST_AND_GIVEN(  
    name)
```

Value:

```
DOCTEST_SUBCASE("    And: " name)
```

9.23.1.36 DOCTEST_AND_THEN

```
#define DOCTEST_AND_THEN(  
    name)
```

Value:

```
DOCTEST_SUBCASE("    And: " name)
```

9.23.1.37 DOCTEST_AND_WHEN

```
#define DOCTEST_AND_WHEN(  
    name)
```

Value:

```
DOCTEST_SUBCASE("    And: " name)
```

9.23.1.38 DOCTEST_ANONYMOUS

```
#define DOCTEST_ANONYMOUS(  
    x)
```

Value:

```
DOCTEST_CAT(x, DOCTEST_COUNTER)
```

9.23.1.39 DOCTEST_ASSERT_IMPLEMENT_1

```
#define DOCTEST_ASSERT_IMPLEMENT_1(  
    assert_type,  
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN {  
    \ DOCTEST_ASSERT_IMPLEMENT_2(assert_type, __VA_ARGS__);  
    \  
}  
    \  
DOCTEST_FUNC_SCOPE_END
```


9.23.1.40 DOCTEST_ASSERT_IMPLEMENT_2

```
#define DOCTEST_ASSERT_IMPLEMENT_2(
    assert_type,
    ...)
```

Value:

```
DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Woverloaded-shift-op-parentheses")
\
/* NOLINTNEXTLINE(clang-analyzer-cplusplus.NewDeleteLeaks) */
\
doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__, __LINE__,
    #__VA_ARGS__);
DOCTEST_WRAP_IN_TRY(
    DOCTEST_RB.setResult(doctest::detail::ExpressionDecomposer(doctest::assertType::assert_type) «
    __VA_ARGS__)
) /* NOLINTNEXTLINE(clang-analyzer-cplusplus.NewDeleteLeaks) */
\
DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB)
\
DOCTEST_CLANG_SUPPRESS_WARNING_POP
```

9.23.1.41 DOCTEST_ASSERT_IN_TESTS

```
#define DOCTEST_ASSERT_IN_TESTS(
    decomp)
```

Value:

```
ResultBuilder rb(at, file, line, expr);
\
rb.m_failed = failed;
\
if (rb.m_failed || getContextOptions()->success)
\
    rb.m_decomp = decomp;
\
if (rb.log())
\
    DOCTEST_BREAK_INTO_DEBUGGER();
\
if (rb.m_failed && checkIfShouldThrow(at))
\
    throwException()
```

9.23.1.42 DOCTEST_ASSERT_LOG_REACT_RETURN

```
#define DOCTEST_ASSERT_LOG_REACT_RETURN(
    b)
```

Value:

```
if (b.log())
\
    DOCTEST_BREAK_INTO_DEBUGGER();
\
b.react();
\
DOCTEST_FUNC_SCOPE_RET(!b.m_failed)
```

9.23.1.43 DOCTEST_ASSERT_OUT_OF_TESTS

```
#define DOCTEST_ASSERT_OUT_OF_TESTS(  
    decomp)
```

Value:

```
do { /* NOLINT(cppcoreguidelines-avoid-do-while) */  
    \  
    if (!is_running_in_test) {  
        \  
        if (failed) {  
            \  
            ResultBuilder rb(at, file, line, expr);  
            \  
            rb.m_failed = failed;  
            \  
            rb.m_decomp = decomp;  
            \  
            failed_out_of_a_testing_context(rb);  
            \  
            if (isDebuggerActive() && !getContextOptions()->no_breaks)  
                \  
                DOCTEST_BREAK_INTO_DEBUGGER();  
            \  
            if (checkIfShouldThrow(at))  
                \  
                throwException();  
            \  
        }  
        \  
        return !failed;  
    }  
    \  
} while (false)
```

9.23.1.44 DOCTEST_BINARY_ASSERT

```
#define DOCTEST_BINARY_ASSERT(  
    assert_type,  
    comp,  
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN {  
    \  
    doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__, __LINE__,  
        #__VA_ARGS__); \  
    DOCTEST_WRAP_IN_TRY(DOCTEST_RB.binary_assert<doctest::detail::binaryAssertComparison::comp>(__VA_ARGS__))  
    \  
    DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);  
    \  
}  
DOCTEST_FUNC_SCOPE_END
```

9.23.1.45 DOCTEST_BINARY_RELATIONAL_OP

```
#define DOCTEST_BINARY_RELATIONAL_OP(  
    n,  
    op)
```

Value:

```
template <class L, class R> struct RelationalComparator<n, L, R> { bool operator()(const DOCTEST_REF_WRAP(L)  
    lhs, const DOCTEST_REF_WRAP(R) rhs) const { return op(lhs, rhs); } };
```

9.23.1.46 DOCTEST_BREAK_INTO_DEBUGGER

```
#define DOCTEST_BREAK_INTO_DEBUGGER()
```

Value:

```
raise(SIGTRAP)
```

9.23.1.47 DOCTEST_CAPTURE

```
#define DOCTEST_CAPTURE(  
    x)
```

Value:

```
DOCTEST_INFO(#x " := ", x)
```

9.23.1.48 DOCTEST_CAST_TO_VOID

```
#define DOCTEST_CAST_TO_VOID(  
    ...)
```

Value:

```
__VA_ARGS__;
```

9.23.1.49 DOCTEST_CAT

```
#define DOCTEST_CAT(  
    s1,  
    s2)
```

Value:

```
DOCTEST_CAT_IMPL(s1, s2)
```

9.23.1.50 DOCTEST_CAT_IMPL

```
#define DOCTEST_CAT_IMPL(  
    s1,  
    s2)
```

Value:

```
s1##s2
```

9.23.1.51 DOCTEST_CHECK

```
#define DOCTEST_CHECK(  
    ...)
```

Value:

```
DOCTEST_ASSERT_IMPLEMENT_1(DT_CHECK, __VA_ARGS__)
```

9.23.1.52 DOCTEST_CHECK_EQ

```
#define DOCTEST_CHECK_EQ(  
    ...)
```

Value:

`DOCTEST_BINARY_ASSERT(DT_CHECK_EQ, eq, __VA_ARGS__)`

9.23.1.53 DOCTEST_CHECK_FALSE

```
#define DOCTEST_CHECK_FALSE(  
    ...)
```

Value:

`DOCTEST_ASSERT_IMPLEMENT_1(DT_CHECK_FALSE, __VA_ARGS__)`

9.23.1.54 DOCTEST_CHECK_FALSE_MESSAGE

```
#define DOCTEST_CHECK_FALSE_MESSAGE(  
    cond,  
    ...)
```

Value:

`DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); DOCTEST_ASSERT_IMPLEMENT_2(DT_CHECK_FALSE, cond); }
DOCTEST_FUNC_SCOPE_END`

9.23.1.55 DOCTEST_CHECK_GE

```
#define DOCTEST_CHECK_GE(  
    ...)
```

Value:

`DOCTEST_BINARY_ASSERT(DT_CHECK_GE, ge, __VA_ARGS__)`

9.23.1.56 DOCTEST_CHECK_GT

```
#define DOCTEST_CHECK_GT(  
    ...)
```

Value:

`DOCTEST_BINARY_ASSERT(DT_CHECK_GT, gt, __VA_ARGS__)`

9.23.1.57 DOCTEST_CHECK_LE

```
#define DOCTEST_CHECK_LE(  
    ...)
```

Value:

`DOCTEST_BINARY_ASSERT(DT_CHECK_LE, le, __VA_ARGS__)`

9.23.1.58 DOCTEST_CHECK_LT

```
#define DOCTEST_CHECK_LT(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_CHECK_LT, lt, __VA_ARGS__)
```

9.23.1.59 DOCTEST_CHECK_MESSAGE

```
#define DOCTEST_CHECK_MESSAGE(  
    cond,  
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); DOCTEST_ASSERT_IMPLEMENT_2(DT_CHECK, cond); }  
DOCTEST_FUNC_SCOPE_END
```

9.23.1.60 DOCTEST_CHECK_NE

```
#define DOCTEST_CHECK_NE(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_CHECK_NE, ne, __VA_ARGS__)
```

9.23.1.61 DOCTEST_CHECK_NOTHROW

```
#define DOCTEST_CHECK_NOTHROW(  
    ...)
```

Value:

```
DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.62 DOCTEST_CHECK_NOTHROW_MESSAGE

```
#define DOCTEST_CHECK_NOTHROW_MESSAGE(  
    expr,  
    ...)
```

Value:

```
DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.63 DOCTEST_CHECK_THROWS

```
#define DOCTEST_CHECK_THROWS(  
    ...)
```

Value:

```
DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.64 DOCTEST_CHECK_THROWS_AS

```
#define DOCTEST_CHECK_THROWS_AS(  
    expr,  
    ...)
```

Value:

`DOCTEST_EXCEPTION_EMPTY_FUNC`

9.23.1.65 DOCTEST_CHECK_THROWS_AS_MESSAGE

```
#define DOCTEST_CHECK_THROWS_AS_MESSAGE(  
    expr,  
    ex,  
    ...)
```

Value:

`DOCTEST_EXCEPTION_EMPTY_FUNC`

9.23.1.66 DOCTEST_CHECK_THROWS_MESSAGE

```
#define DOCTEST_CHECK_THROWS_MESSAGE(  
    expr,  
    ...)
```

Value:

`DOCTEST_EXCEPTION_EMPTY_FUNC`

9.23.1.67 DOCTEST_CHECK_THROWS_WITH

```
#define DOCTEST_CHECK_THROWS_WITH(  
    expr,  
    ...)
```

Value:

`DOCTEST_EXCEPTION_EMPTY_FUNC`

9.23.1.68 DOCTEST_CHECK_THROWS_WITH_AS

```
#define DOCTEST_CHECK_THROWS_WITH_AS(  
    expr,  
    with,  
    ...)
```

Value:

`DOCTEST_EXCEPTION_EMPTY_FUNC`

9.23.1.69 DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE

```
#define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(  
    expr,  
    with,  
    ex,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

9.23.1.70 DOCTEST_CHECK_THROWS_WITH_MESSAGE

```
#define DOCTEST_CHECK_THROWS_WITH_MESSAGE(  
    expr,  
    with,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

9.23.1.71 DOCTEST_CHECK_UNARY

```
#define DOCTEST_CHECK_UNARY(  
    ...)
```

Value:

[DOCTEST_UNARY_ASSERT](#)(DT_CHECK_UNARY, __VA_ARGS__)

9.23.1.72 DOCTEST_CHECK_UNARY_FALSE

```
#define DOCTEST_CHECK_UNARY_FALSE(  
    ...)
```

Value:

[DOCTEST_UNARY_ASSERT](#)(DT_CHECK_UNARY_FALSE, __VA_ARGS__)

9.23.1.73 DOCTEST_CLANG

```
#define DOCTEST_CLANG 0
```

9.23.1.74 DOCTEST_CLANG_SUPPRESS_WARNING

```
#define DOCTEST_CLANG_SUPPRESS_WARNING(  
    w)
```

9.23.1.75 DOCTEST_CLANG_SUPPRESS_WARNING_POP

```
#define DOCTEST_CLANG_SUPPRESS_WARNING_POP
```

9.23.1.76 DOCTEST_CLANG_SUPPRESS_WARNING_PUSH

```
#define DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
```

9.23.1.77 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH

```
#define DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH(  
    w)
```

9.23.1.78 DOCTEST_CMP_EQ

```
#define DOCTEST_CMP_EQ(  
    l,  
    r)
```

Value:

```
l == r
```

9.23.1.79 DOCTEST_CMP_GE

```
#define DOCTEST_CMP_GE(  
    l,  
    r)
```

Value:

```
l >= r
```

9.23.1.80 DOCTEST_CMP_GT

```
#define DOCTEST_CMP_GT(  
    l,  
    r)
```

Value:

```
l > r
```

9.23.1.81 DOCTEST_CMP_LE

```
#define DOCTEST_CMP_LE(  
    l,  
    r)
```

Value:

```
l <= r
```


9.23.1.82 DOCTEST_CMP_LT

```
#define DOCTEST_CMP_LT(  
    l,  
    r)
```

Value:

```
l < r
```

9.23.1.83 DOCTEST_CMP_NE

```
#define DOCTEST_CMP_NE(  
    l,  
    r)
```

Value:

```
l != r
```

9.23.1.84 DOCTEST_COMPARISON_RETURN_TYPE

```
#define DOCTEST_COMPARISON_RETURN_TYPE bool
```

9.23.1.85 DOCTEST_COMPILER

```
#define DOCTEST_COMPILER(  
    MAJOR,  
    MINOR,  
    PATCH)
```

Value:

```
((MAJOR) * 10000000 + (MINOR) * 100000 + (PATCH))
```

9.23.1.86 DOCTEST_CONFIG_NO_EXCEPTIONS

```
#define DOCTEST_CONFIG_NO_EXCEPTIONS
```

9.23.1.87 DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS

```
#define DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
```

9.23.1.88 DOCTEST_CONFIG_POSIX_SIGNALS

```
#define DOCTEST_CONFIG_POSIX_SIGNALS
```

9.23.1.89 DOCTEST_CONFIG_STRING_SIZE_TYPE

```
#define DOCTEST_CONFIG_STRING_SIZE_TYPE unsigned
```

9.23.1.90 DOCTEST_CONSTEXPR

```
#define DOCTEST_CONSTEXPR constexpr
```

9.23.1.91 DOCTEST_CONSTEXPR_FUNC

```
#define DOCTEST_CONSTEXPR_FUNC constexpr
```

9.23.1.92 DOCTEST_COUNTER

```
#define DOCTEST_COUNTER __LINE__
```

9.23.1.93 DOCTEST_CPLUSPLUS

```
#define DOCTEST_CPLUSPLUS __cplusplus
```

9.23.1.94 DOCTEST_CREATE_AND_REGISTER_FUNCTION

```
#define DOCTEST_CREATE_AND_REGISTER_FUNCTION(  
    f,  
    decorators)
```

Value:

```
static void f();  
    \  
DOCTEST_REGISTER_FUNCTION(DOCTEST_EMPTY, f, decorators)  
    \  
static void f()
```

9.23.1.95 DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS

```
#define DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS(  
    f,  
    proxy,  
    decorators)
```

Value:

```
static doctest::detail::funcType proxy() {  
    \  
    return f;  
    \  
}  
    \  
DOCTEST_REGISTER_FUNCTION(inline, proxy(), decorators)  
    \  
static void f()
```

9.23.1.96 DOCTEST_DECLARE_INTERFACE

```
#define DOCTEST_DECLARE_INTERFACE(  
    name)
```

Value:

```
virtual ~name();  
name() = default;  
name(const name &) = delete;  
name(name &&) = delete;  
name &operator=(const name &) = delete;  
name &operator=(name &&) = delete;
```

9.23.1.97 DOCTEST_DEFINE_DECORATOR

```
#define DOCTEST_DEFINE_DECORATOR(  
    name,  
    type,  
    def)
```

Value:

```
struct name {  
    type data;  
    name(type in = def) noexcept  
        : data(in) {}  
    void fill(detail::TestCase &state) const noexcept {  
        state.DOCTEST_CAT(m_, name) = data;  
    }  
    void fill(detail::TestSuite &state) const noexcept {  
        state.DOCTEST_CAT(m_, name) = data;  
    }  
}
```

9.23.1.98 DOCTEST_DEFINE_INTERFACE

```
#define DOCTEST_DEFINE_INTERFACE(  
    name)
```

Value:

```
name::~~name() = default;
```

9.23.1.99 DOCTEST_DO_BINARY_EXPRESSION_COMPARISON

```
#define DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(
    op,
    op_str,
    op_macro)
```

Value:

```
/* NOLINTBEGIN(cppcoreguidelines-missing-std-forward) */
template <typename R>
DOCTEST_NOINLINE SFINAE_OP(Result, op) operator op(R &&rhs) {
    bool res = op_macro(doctest::detail::forward<const L>(lhs), doctest::detail::forward<R>(rhs));
    if (m_at & assertType::is_false)
        res = !res;
    if (!res || doctest::getContextOptions()->success)
        return Result(res, stringifyBinaryExpr(lhs, op_str, rhs));
    return Result(res);
}
/* NOLINTEND(cppcoreguidelines-missing-std-forward) */
```

9.23.1.100 DOCTEST_EMPTY

```
#define DOCTEST_EMPTY
```

9.23.1.101 DOCTEST_EXCEPTION_EMPTY_FUNC

```
#define DOCTEST_EXCEPTION_EMPTY_FUNC
```

Value:

```
[] {
    static_assert(
        false,
        "Exceptions are disabled! "
        "Use DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS if you want "
        "to compile with exceptions disabled."
    );
    return false;
}()
```

9.23.1.102 DOCTEST_FAIL

```
#define DOCTEST_FAIL(
    ...)
```

Value:

```
DOCTEST_ADD_FAIL_AT(__FILE__, __LINE__, __VA_ARGS__)
```

9.23.1.103 DOCTEST_FAIL_CHECK

```
#define DOCTEST_FAIL_CHECK(  
    ...)
```

Value:

```
DOCTEST_ADD_FAIL_CHECK_AT(__FILE__, __LINE__, __VA_ARGS__)
```

9.23.1.104 DOCTEST_FAST_CHECK_EQ

```
#define DOCTEST_FAST_CHECK_EQ DOCTEST_CHECK_EQ
```

9.23.1.105 DOCTEST_FAST_CHECK_GE

```
#define DOCTEST_FAST_CHECK_GE DOCTEST_CHECK_GE
```

9.23.1.106 DOCTEST_FAST_CHECK_GT

```
#define DOCTEST_FAST_CHECK_GT DOCTEST_CHECK_GT
```

9.23.1.107 DOCTEST_FAST_CHECK_LE

```
#define DOCTEST_FAST_CHECK_LE DOCTEST_CHECK_LE
```

9.23.1.108 DOCTEST_FAST_CHECK_LT

```
#define DOCTEST_FAST_CHECK_LT DOCTEST_CHECK_LT
```

9.23.1.109 DOCTEST_FAST_CHECK_NE

```
#define DOCTEST_FAST_CHECK_NE DOCTEST_CHECK_NE
```

9.23.1.110 DOCTEST_FAST_CHECK_UNARY

```
#define DOCTEST_FAST_CHECK_UNARY DOCTEST_CHECK_UNARY
```

9.23.1.111 DOCTEST_FAST_CHECK_UNARY_FALSE

```
#define DOCTEST_FAST_CHECK_UNARY_FALSE DOCTEST_CHECK_UNARY_FALSE
```

9.23.1.112 DOCTEST_FAST_REQUIRE_EQ

```
#define DOCTEST_FAST_REQUIRE_EQ DOCTEST_REQUIRE_EQ
```

9.23.1.113 DOCTEST_FAST_REQUIRE_GE

```
#define DOCTEST_FAST_REQUIRE_GE DOCTEST_REQUIRE_GE
```

9.23.1.114 DOCTEST_FAST_REQUIRE_GT

```
#define DOCTEST_FAST_REQUIRE_GT DOCTEST_REQUIRE_GT
```

9.23.1.115 DOCTEST_FAST_REQUIRE_LE

```
#define DOCTEST_FAST_REQUIRE_LE DOCTEST_REQUIRE_LE
```

9.23.1.116 DOCTEST_FAST_REQUIRE_LT

```
#define DOCTEST_FAST_REQUIRE_LT DOCTEST_REQUIRE_LT
```

9.23.1.117 DOCTEST_FAST_REQUIRE_NE

```
#define DOCTEST_FAST_REQUIRE_NE DOCTEST_REQUIRE_NE
```

9.23.1.118 DOCTEST_FAST_REQUIRE_UNARY

```
#define DOCTEST_FAST_REQUIRE_UNARY DOCTEST_REQUIRE_UNARY
```

9.23.1.119 DOCTEST_FAST_REQUIRE_UNARY_FALSE

```
#define DOCTEST_FAST_REQUIRE_UNARY_FALSE DOCTEST_REQUIRE_UNARY_FALSE
```

9.23.1.120 DOCTEST_FAST_WARN_EQ

```
#define DOCTEST_FAST_WARN_EQ DOCTEST_WARN_EQ
```

9.23.1.121 DOCTEST_FAST_WARN_GE

```
#define DOCTEST_FAST_WARN_GE DOCTEST_WARN_GE
```

9.23.1.122 DOCTEST_FAST_WARN_GT

```
#define DOCTEST_FAST_WARN_GT DOCTEST_WARN_GT
```

9.23.1.123 DOCTEST_FAST_WARN_LE

```
#define DOCTEST_FAST_WARN_LE DOCTEST_WARN_LE
```

9.23.1.124 DOCTEST_FAST_WARN_LT

```
#define DOCTEST_FAST_WARN_LT DOCTEST_WARN_LT
```

9.23.1.125 DOCTEST_FAST_WARN_NE

```
#define DOCTEST_FAST_WARN_NE DOCTEST_WARN_NE
```

9.23.1.126 DOCTEST_FAST_WARN_UNARY

```
#define DOCTEST_FAST_WARN_UNARY DOCTEST_WARN_UNARY
```

9.23.1.127 DOCTEST_FAST_WARN_UNARY_FALSE

```
#define DOCTEST_FAST_WARN_UNARY_FALSE DOCTEST_WARN_UNARY_FALSE
```

9.23.1.128 DOCTEST_FORBIT_EXPRESSION

```
#define DOCTEST_FORBIT_EXPRESSION(  
    rt,  
    op)
```

Value:

```
template <typename R>  
    \  
    rt &operator op(const R &) {  
        \  
        static_assert(deferred_false<R>::value, "Expression Too Complex Please Rewrite As Binary Comparison!");  
        \  
        return *this;  
    }  
}
```

9.23.1.129 DOCTEST_FUNC_EMPTY

```
#define DOCTEST_FUNC_EMPTY (void)0
```

9.23.1.130 DOCTEST_FUNC_SCOPE_BEGIN

```
#define DOCTEST_FUNC_SCOPE_BEGIN do /* NOLINT(cppcoreguidelines-avoid-do-while)*/
```

9.23.1.131 DOCTEST_FUNC_SCOPE_END

```
#define DOCTEST_FUNC_SCOPE_END while (false)
```

9.23.1.132 DOCTEST_FUNC_SCOPE_RET

```
#define DOCTEST_FUNC_SCOPE_RET(  
    v)
```

Value:

```
(void)0
```

9.23.1.133 DOCTEST_GCC

```
#define DOCTEST_GCC 0
```

9.23.1.134 DOCTEST_GCC_SUPPRESS_WARNING

```
#define DOCTEST_GCC_SUPPRESS_WARNING(  
    w)
```

9.23.1.135 DOCTEST_GCC_SUPPRESS_WARNING_POP

```
#define DOCTEST_GCC_SUPPRESS_WARNING_POP
```

9.23.1.136 DOCTEST_GCC_SUPPRESS_WARNING_PUSH

```
#define DOCTEST_GCC_SUPPRESS_WARNING_PUSH
```

9.23.1.137 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH

```
#define DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH(  
    w)
```

9.23.1.138 DOCTEST_GENERATE

```
#define DOCTEST_GENERATE(  
    ...)
```

Value:

```
doctest::detail::acquireGeneratorValue(__VA_ARGS__)
```


9.23.1.139 DOCTEST_GIVEN

```
#define DOCTEST_GIVEN(  
    name)
```

Value:

```
DOCTEST_SUBCASE("    Given: " name)
```

9.23.1.140 DOCTEST_GLOBAL_NO_WARNINGS

```
#define DOCTEST_GLOBAL_NO_WARNINGS(  
    var,  
    ...)
```

Value:

```
DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wglobal-constructors")  
    \  
static const int var = doctest::detail::consume(&var, __VA_ARGS__);  
    \  
DOCTEST_CLANG_SUPPRESS_WARNING_POP
```

9.23.1.141 DOCTEST_HAS_BUILTIN

```
#define DOCTEST_HAS_BUILTIN(  
    x)
```

Value:

```
0
```

9.23.1.142 DOCTEST_ICC

```
#define DOCTEST_ICC 0
```

9.23.1.143 DOCTEST_IMPLEMENT_FIXTURE

```
#define DOCTEST_IMPLEMENT_FIXTURE(  
    der,  
    base,  
    func,  
    decorators)
```

Value:

```
namespace { /* NOLINT */  
    \  
struct der : public base {  
    \  
    void f();  
    \  
};  
    \  
static DOCTEST_INLINE_NOINLINE void func() {  
    \  
    der v;  
    \  
    v.f();  
    \  
}  
    \  
DOCTEST_REGISTER_FUNCTION(DOCTEST_EMPTY, func, decorators)  
    \  
}  
    \  
DOCTEST_INLINE_NOINLINE void der::f()
```

9.23.1.144 DOCTEST_INFO

```
#define DOCTEST_INFO(
    ...)
```

Value:

```
DOCTEST_INFO_IMPL(DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_MB_), DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_OTHER_),
    __VA_ARGS__)
```

9.23.1.145 DOCTEST_INFO_IMPL

```
#define DOCTEST_INFO_IMPL(
    mb_name,
    s_name,
    ...)
```

Value:

```
auto DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_) = doctest::detail::MakeContextScope([&](std::ostream *s_name) {
    \
    doctest::detail::MessageBuilder mb_name(__FILE__, __LINE__, doctest::assertType::is_warn);
    \
    mb_name.m_stream = s_name;
    \
    mb_name *__VA_ARGS__;
    \
})
```

9.23.1.146 DOCTEST_INLINE_NOINLINE

```
#define DOCTEST_INLINE_NOINLINE inline DOCTEST_NOINLINE
```

9.23.1.147 DOCTEST_INTERFACE

```
#define DOCTEST_INTERFACE
```

9.23.1.148 DOCTEST_INTERFACE_DECL

```
#define DOCTEST_INTERFACE_DECL DOCTEST_INTERFACE
```

9.23.1.149 DOCTEST_INTERFACE_DEF

```
#define DOCTEST_INTERFACE_DEF
```

9.23.1.150 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN

```
#define DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
```

Value:

```
DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
DOCTEST_MSVC_SUPPRESS_WARNING(4548) /* before comma no effect; expected side - effect */
DOCTEST_MSVC_SUPPRESS_WARNING(4265) /* virtual functions, but destructor is not virtual */
DOCTEST_MSVC_SUPPRESS_WARNING(4986) /* exception specification does not match previous */
DOCTEST_MSVC_SUPPRESS_WARNING(4350) /* 'member1' called instead of 'member2' */
DOCTEST_MSVC_SUPPRESS_WARNING(4668) /* not defined as a preprocessor macro */
DOCTEST_MSVC_SUPPRESS_WARNING(4365) /* signed/unsigned mismatch */
DOCTEST_MSVC_SUPPRESS_WARNING(4774) /* format string not a string literal */
DOCTEST_MSVC_SUPPRESS_WARNING(4820) /* padding */
DOCTEST_MSVC_SUPPRESS_WARNING(4625) /* copy constructor was implicitly deleted */
DOCTEST_MSVC_SUPPRESS_WARNING(4626) /* assignment operator was implicitly deleted */
DOCTEST_MSVC_SUPPRESS_WARNING(5027) /* move assignment operator implicitly deleted */
DOCTEST_MSVC_SUPPRESS_WARNING(5026) /* move constructor was implicitly deleted */
DOCTEST_MSVC_SUPPRESS_WARNING(4623) /* default constructor was implicitly deleted */
DOCTEST_MSVC_SUPPRESS_WARNING(5039) /* pointer to pot. throwing function passed to extern C */
DOCTEST_MSVC_SUPPRESS_WARNING(5045) /* Spectre mitigation for memory load */
DOCTEST_MSVC_SUPPRESS_WARNING(5105) /* macro producing 'defined' has undefined behavior */
DOCTEST_MSVC_SUPPRESS_WARNING(4738) /* storing float result in memory, loss of performance */
DOCTEST_MSVC_SUPPRESS_WARNING(5262) /* implicit fall-through */
```

9.23.1.151 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END

```
#define DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END DOCTEST_MSVC_SUPPRESS_WARNING_POP
```

9.23.1.152 DOCTEST_MESSAGE

```
#define DOCTEST_MESSAGE(
    ...)
```

Value:

```
DOCTEST_ADD_MESSAGE_AT(__FILE__, __LINE__, __VA_ARGS__)
```

9.23.1.153 DOCTEST_MSVC

```
#define DOCTEST_MSVC 0
```

9.23.1.154 DOCTEST_MSVC_SUPPRESS_WARNING

```
#define DOCTEST_MSVC_SUPPRESS_WARNING(
    w)
```

9.23.1.155 DOCTEST_MSVC_SUPPRESS_WARNING_POP

```
#define DOCTEST_MSVC_SUPPRESS_WARNING_POP
```

9.23.1.156 DOCTEST_MSVC_SUPPRESS_WARNING_PUSH

```
#define DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
```

9.23.1.157 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH

```
#define DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(  
    w)
```

9.23.1.158 DOCTEST_NO_SANITIZE_INTEGER

```
#define DOCTEST_NO_SANITIZE_INTEGER
```

9.23.1.159 DOCTEST_NOEXCEPT

```
#define DOCTEST_NOEXCEPT noexcept
```

9.23.1.160 DOCTEST_NOINLINE

```
#define DOCTEST_NOINLINE __attribute__((noinline))
```

9.23.1.161 DOCTEST_NORETURN

```
#define DOCTEST_NORETURN [[noreturn]]
```

9.23.1.162 DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR
```

9.23.1.163 DOCTEST_PARTS_PUBLIC_ASSERT_DATA

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_DATA
```

9.23.1.164 DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION
```

9.23.1.165 DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER
```

9.23.1.166 DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE
```

9.23.1.167 DOCTEST_PARTS_PUBLIC_ASSERT_RESULT

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_RESULT
```

9.23.1.168 DOCTEST_PARTS_PUBLIC_ASSERT_TYPE

```
#define DOCTEST_PARTS_PUBLIC_ASSERT_TYPE
```

9.23.1.169 DOCTEST_PARTS_PUBLIC_COLOR

```
#define DOCTEST_PARTS_PUBLIC_COLOR
```

9.23.1.170 DOCTEST_PARTS_PUBLIC_COMPILER

```
#define DOCTEST_PARTS_PUBLIC_COMPILER
```

9.23.1.171 DOCTEST_PARTS_PUBLIC_CONFIG

```
#define DOCTEST_PARTS_PUBLIC_CONFIG
```

9.23.1.172 DOCTEST_PARTS_PUBLIC_CONTEXT

```
#define DOCTEST_PARTS_PUBLIC_CONTEXT
```

9.23.1.173 DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS

```
#define DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS
```

9.23.1.174 DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE

```
#define DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE
```

9.23.1.175 DOCTEST_PARTS_PUBLIC_DEBUGGER

```
#define DOCTEST_PARTS_PUBLIC_DEBUGGER
```

9.23.1.176 DOCTEST_PARTS_PUBLIC_DECORATORS

```
#define DOCTEST_PARTS_PUBLIC_DECORATORS
```

9.23.1.177 DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR

```
#define DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR
```

9.23.1.178 DOCTEST_PARTS_PUBLIC_EXCEPTIONS

```
#define DOCTEST_PARTS_PUBLIC_EXCEPTIONS
```

9.23.1.179 DOCTEST_PARTS_PUBLIC_GENERATOR

```
#define DOCTEST_PARTS_PUBLIC_GENERATOR
```

9.23.1.180 DOCTEST_PARTS_PUBLIC_MACROS

```
#define DOCTEST_PARTS_PUBLIC_MACROS
```

9.23.1.181 DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX

```
#define DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX
```

9.23.1.182 DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS

```
#define DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS
```

9.23.1.183 DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN

```
#define DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN
```

9.23.1.184 DOCTEST_PARTS_PUBLIC_PATH

```
#define DOCTEST_PARTS_PUBLIC_PATH
```

9.23.1.185 DOCTEST_PARTS_PUBLIC_PLATFORM

```
#define DOCTEST_PARTS_PUBLIC_PLATFORM
```

9.23.1.186 DOCTEST_PARTS_PUBLIC_REPORTER

```
#define DOCTEST_PARTS_PUBLIC_REPORTER
```

9.23.1.187 DOCTEST_PARTS_PUBLIC_STD_FWD

```
#define DOCTEST_PARTS_PUBLIC_STD_FWD
```

9.23.1.188 DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS

```
#define DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS
```

9.23.1.189 DOCTEST_PARTS_PUBLIC_STD_UTILITY

```
#define DOCTEST_PARTS_PUBLIC_STD_UTILITY
```

9.23.1.190 DOCTEST_PARTS_PUBLIC_STRING

```
#define DOCTEST_PARTS_PUBLIC_STRING
```

9.23.1.191 DOCTEST_PARTS_PUBLIC_SUBCASE

```
#define DOCTEST_PARTS_PUBLIC_SUBCASE
```

9.23.1.192 DOCTEST_PARTS_PUBLIC_TEST_CASE

```
#define DOCTEST_PARTS_PUBLIC_TEST_CASE
```

9.23.1.193 DOCTEST_PARTS_PUBLIC_TEST_SUITE

```
#define DOCTEST_PARTS_PUBLIC_TEST_SUITE
```

9.23.1.194 DOCTEST_PARTS_PUBLIC_UTILITY

```
#define DOCTEST_PARTS_PUBLIC_UTILITY
```

9.23.1.195 DOCTEST_PARTS_PUBLIC_VERSION

```
#define DOCTEST_PARTS_PUBLIC_VERSION
```

9.23.1.196 DOCTEST_PARTS_PUBLIC_WARNINGS

```
#define DOCTEST_PARTS_PUBLIC_WARNINGS
```

9.23.1.197 DOCTEST_PLATFORM_LINUX

```
#define DOCTEST_PLATFORM_LINUX
```

9.23.1.198 DOCTEST_REF_WRAP

```
#define DOCTEST_REF_WRAP(  
    x)
```

Value:

```
x &
```

9.23.1.199 DOCTEST_REGISTER_EXCEPTION_TRANSLATOR

```
#define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR(  
    signature)
```

Value:

```
DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL(DOCTEST_ANONYMOUS(DOCTEST_ANON_TRANSLATOR_), signature)
```

9.23.1.200 DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL

```
#define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL(  
    translatorName,  
    signature)
```

Value:

```
inline doctest::String translatorName(signature);  
DOCTEST_GLOBAL_NO_WARNINGS(  
    DOCTEST_ANONYMOUS(DOCTEST_ANON_TRANSLATOR_VAR_), /* NOLINT(cert-err58-cpp) */  
    doctest::registerExceptionTranslator(translatorName)  
)  
doctest::String translatorName(signature)
```


9.23.1.201 DOCTEST_REGISTER_FUNCTION

```
#define DOCTEST_REGISTER_FUNCTION(
    global_prefix,
    f,
    decorators)
```

Value:

```
global_prefix DOCTEST_GLOBAL_NO_WARNINGS(
    \
    DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT */
    \
    doctest::detail::regTest(
        \
        doctest::detail::TestCase(f, __FILE__, __LINE__,
        doctest_detail_test_suite_ns::getCurrentTestSuite()) * \
        decorators
    \
    )
    \
)
```

9.23.1.202 DOCTEST_REGISTER_LISTENER

```
#define DOCTEST_REGISTER_LISTENER(
    name,
    priority,
    reporter)
```

Value:

```
DOCTEST_GLOBAL_NO_WARNINGS(
    \
    DOCTEST_ANONYMOUS(DOCTEST_ANON_REPORTER_VAR_), /* NOLINT(cert-err58-cpp) */
    \
    doctest::registerReporter<reporter>(name, priority, false)
    \
)
    \
static_assert(true, "")
```

9.23.1.203 DOCTEST_REGISTER_REPORTER

```
#define DOCTEST_REGISTER_REPORTER(
    name,
    priority,
    reporter)
```

Value:

```
DOCTEST_GLOBAL_NO_WARNINGS(
    \
    DOCTEST_ANONYMOUS(DOCTEST_ANON_REPORTER_VAR_), /* NOLINT(cert-err58-cpp) */
    \
    doctest::registerReporter<reporter>(name, priority, true)
    \
)
    \
static_assert(true, "")
```

9.23.1.204 DOCTEST_RELATIONAL_OP

```
#define DOCTEST_RELATIONAL_OP(
    name,
    op)
```

Value:

```
template <typename L, typename R>
DOCTEST_COMPARISON_RETURN_TYPE name(const DOCTEST_REF_WRAP(L) lhs, const DOCTEST_REF_WRAP(R) rhs) {
    return lhs op rhs;
}
```

9.23.1.205 DOCTEST_REQUIRE [1/2]

```
#define DOCTEST_REQUIRE DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.206 DOCTEST_REQUIRE [2/2]

```
#define DOCTEST_REQUIRE(
    ...)
```

Value:

```
DOCTEST_ASSERT_IMPLEMENT_1(DT_REQUIRE, __VA_ARGS__)
```

9.23.1.207 DOCTEST_REQUIRE_EQ [1/2]

```
#define DOCTEST_REQUIRE_EQ DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.208 DOCTEST_REQUIRE_EQ [2/2]

```
#define DOCTEST_REQUIRE_EQ(
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_REQUIRE_EQ, eq, __VA_ARGS__)
```

9.23.1.209 DOCTEST_REQUIRE_FALSE [1/2]

```
#define DOCTEST_REQUIRE_FALSE DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.210 DOCTEST_REQUIRE_FALSE [2/2]

```
#define DOCTEST_REQUIRE_FALSE(
    ...)
```

Value:

```
DOCTEST_ASSERT_IMPLEMENT_1(DT_REQUIRE_FALSE, __VA_ARGS__)
```

9.23.1.211 DOCTEST_REQUIRE_FALSE_MESSAGE [1/2]

```
#define DOCTEST_REQUIRE_FALSE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.212 DOCTEST_REQUIRE_FALSE_MESSAGE [2/2]

```
#define DOCTEST_REQUIRE_FALSE_MESSAGE(  
    cond,  
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); DOCTEST_ASSERT_IMPLEMENT_2(DT_REQUIRE_FALSE, cond); }  
DOCTEST_FUNC_SCOPE_END
```

9.23.1.213 DOCTEST_REQUIRE_GE [1/2]

```
#define DOCTEST_REQUIRE_GE DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.214 DOCTEST_REQUIRE_GE [2/2]

```
#define DOCTEST_REQUIRE_GE(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_REQUIRE_GE, ge, __VA_ARGS__)
```

9.23.1.215 DOCTEST_REQUIRE_GT [1/2]

```
#define DOCTEST_REQUIRE_GT DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.216 DOCTEST_REQUIRE_GT [2/2]

```
#define DOCTEST_REQUIRE_GT(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_REQUIRE_GT, gt, __VA_ARGS__)
```

9.23.1.217 DOCTEST_REQUIRE_LE [1/2]

```
#define DOCTEST_REQUIRE_LE DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.218 DOCTEST_REQUIRE_LE [2/2]

```
#define DOCTEST_REQUIRE_LE(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_REQUIRE_LE, le, __VA_ARGS__)
```

9.23.1.219 DOCTEST_REQUIRE_LT [1/2]

```
#define DOCTEST_REQUIRE_LT DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.220 DOCTEST_REQUIRE_LT [2/2]

```
#define DOCTEST_REQUIRE_LT(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_REQUIRE_LT, lt, __VA_ARGS__)
```

9.23.1.221 DOCTEST_REQUIRE_MESSAGE [1/2]

```
#define DOCTEST_REQUIRE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.222 DOCTEST_REQUIRE_MESSAGE [2/2]

```
#define DOCTEST_REQUIRE_MESSAGE(  
    cond,  
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); DOCTEST_ASSERT_IMPLEMENT_2(DT_REQUIRE, cond); }  
DOCTEST_FUNC_SCOPE_END
```

9.23.1.223 DOCTEST_REQUIRE_NE [1/2]

```
#define DOCTEST_REQUIRE_NE DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.224 DOCTEST_REQUIRE_NE [2/2]

```
#define DOCTEST_REQUIRE_NE(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_REQUIRE_NE, ne, __VA_ARGS__)
```

9.23.1.225 DOCTEST_REQUIRE_NOTHROW

```
#define DOCTEST_REQUIRE_NOTHROW(  
    ...)
```

Value:

```
DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.226 DOCTEST_REQUIRE_NOTHROW_MESSAGE

```
#define DOCTEST_REQUIRE_NOTHROW_MESSAGE(  
    expr,  
    ...)
```

Value:

`DOCTEST_EXCEPTION_EMPTY_FUNC`

9.23.1.227 DOCTEST_REQUIRE_THROWS

```
#define DOCTEST_REQUIRE_THROWS(  
    ...)
```

Value:

`DOCTEST_EXCEPTION_EMPTY_FUNC`

9.23.1.228 DOCTEST_REQUIRE_THROWS_AS

```
#define DOCTEST_REQUIRE_THROWS_AS(  
    expr,  
    ...)
```

Value:

`DOCTEST_EXCEPTION_EMPTY_FUNC`

9.23.1.229 DOCTEST_REQUIRE_THROWS_AS_MESSAGE

```
#define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(  
    expr,  
    ex,  
    ...)
```

Value:

`DOCTEST_EXCEPTION_EMPTY_FUNC`

9.23.1.230 DOCTEST_REQUIRE_THROWS_MESSAGE

```
#define DOCTEST_REQUIRE_THROWS_MESSAGE(  
    expr,  
    ...)
```

Value:

`DOCTEST_EXCEPTION_EMPTY_FUNC`

9.23.1.231 DOCTEST_REQUIRE_THROWS_WITH

```
#define DOCTEST_REQUIRE_THROWS_WITH(  
    expr,  
    ...)
```

Value:

`DOCTEST_EXCEPTION_EMPTY_FUNC`

9.23.1.232 DOCTEST_REQUIRE_THROWS_WITH_AS

```
#define DOCTEST_REQUIRE_THROWS_WITH_AS(  
    expr,  
    with,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

9.23.1.233 DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE

```
#define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(  
    expr,  
    with,  
    ex,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

9.23.1.234 DOCTEST_REQUIRE_THROWS_WITH_MESSAGE

```
#define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(  
    expr,  
    with,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

9.23.1.235 DOCTEST_REQUIRE_UNARY [1/2]

```
#define DOCTEST_REQUIRE_UNARY DOCTEST\_EXCEPTION\_EMPTY\_FUNC
```

9.23.1.236 DOCTEST_REQUIRE_UNARY [2/2]

```
#define DOCTEST_REQUIRE_UNARY(  
    ...)
```

Value:

[DOCTEST_UNARY_ASSERT](#)(DT_REQUIRE_UNARY, __VA_ARGS__)

9.23.1.237 DOCTEST_REQUIRE_UNARY_FALSE [1/2]

```
#define DOCTEST_REQUIRE_UNARY_FALSE DOCTEST\_EXCEPTION\_EMPTY\_FUNC
```

9.23.1.238 DOCTEST_REQUIRE_UNARY_FALSE [2/2]

```
#define DOCTEST_REQUIRE_UNARY_FALSE(  
    ...)
```

Value:

```
DOCTEST_UNARY_ASSERT(DT_REQUIRE_UNARY_FALSE, __VA_ARGS__)
```

9.23.1.239 DOCTEST_SCENARIO

```
#define DOCTEST_SCENARIO(  
    name)
```

Value:

```
DOCTEST_TEST_CASE(" Scenario: " name)
```

9.23.1.240 DOCTEST_SCENARIO_CLASS

```
#define DOCTEST_SCENARIO_CLASS(  
    name)
```

Value:

```
DOCTEST_TEST_CASE_CLASS(" Scenario: " name)
```

9.23.1.241 DOCTEST_SCENARIO_METHOD

```
#define DOCTEST_SCENARIO_METHOD(  
    x,  
    name)
```

Value:

```
DOCTEST_TEST_CASE_FIXTURE(x, " Scenario: " name)
```

9.23.1.242 DOCTEST_SCENARIO_TEMPLATE

```
#define DOCTEST_SCENARIO_TEMPLATE(  
    name,  
    T,  
    ...)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE(" Scenario: " name, T, __VA_ARGS__)
```

9.23.1.243 DOCTEST_SCENARIO_TEMPLATE_DEFINE

```
#define DOCTEST_SCENARIO_TEMPLATE_DEFINE(  
    name,  
    T,  
    id)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE_DEFINE(" Scenario: " name, T, id)
```

9.23.1.244 DOCTEST_STRINGIFY

```
#define DOCTEST_STRINGIFY(  
    ...)
```

Value:

```
toString(__VA_ARGS__)
```

9.23.1.245 DOCTEST_SUBCASE

```
#define DOCTEST_SUBCASE(  
    name)
```

Value:

```
if (const doctest::detail::Subcase &DOCTEST_ANONYMOUS(DOCTEST_ANON_SUBCASE_) DOCTEST_UNUSED =  
    \  
    doctest::detail::Subcase(name, __FILE__, __LINE__))
```

9.23.1.246 DOCTEST_SUPPRESS_COMMON_WARNINGS_POP

```
#define DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
```

Value:

```
DOCTEST_CLANG_SUPPRESS_WARNING_POP  
    \  
DOCTEST_GCC_SUPPRESS_WARNING_POP  
    \  
DOCTEST_MSVC_SUPPRESS_WARNING_POP
```

9.23.1.247 DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH

```
#define DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH
```

9.23.1.248 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP

```
#define DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
```

9.23.1.249 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH

```
#define DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
```

9.23.1.250 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP

```
#define DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
```


9.23.1.251 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH

```
#define DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
```

Value:

```
DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH
\
DOCTEST_CLANG_SUPPRESS_WARNING("-Wnon-virtual-dtor")
\
DOCTEST_CLANG_SUPPRESS_WARNING("-Wdeprecated")
\
DOCTEST_GCC_SUPPRESS_WARNING("-Wctor-dtor-privacy")
\
DOCTEST_GCC_SUPPRESS_WARNING("-Wnon-virtual-dtor")
\
DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-promo")
\
DOCTEST_MSVC_SUPPRESS_WARNING(4623) /* default constructor was implicitly deleted */
```

9.23.1.252 DOCTEST_SYMBOL_EXPORT

```
#define DOCTEST_SYMBOL_EXPORT __attribute__((visibility("default")))
```

9.23.1.253 DOCTEST_SYMBOL_IMPORT

```
#define DOCTEST_SYMBOL_IMPORT
```

9.23.1.254 DOCTEST_TEST_CASE

```
#define DOCTEST_TEST_CASE(
    decorators)
```

Value:

```
DOCTEST_CREATE_AND_REGISTER_FUNCTION(DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), decorators)
```

9.23.1.255 DOCTEST_TEST_CASE_CLASS

```
#define DOCTEST_TEST_CASE_CLASS(
    ...)
```

Value:

```
TEST_CASES_CAN_BE_REGISTERED_IN_CLASSES_ONLY_IN_CPPL7_MODE_OR_WITH_VS_2017_OR_NEWER
```

9.23.1.256 DOCTEST_TEST_CASE_FIXTURE

```
#define DOCTEST_TEST_CASE_FIXTURE(
    c,
    decorators)
```

Value:

```
DOCTEST_IMPLEMENT_FIXTURE(
    \
    DOCTEST_ANONYMOUS(DOCTEST_ANON_CLASS_), c, DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), decorators
    \
)
```

9.23.1.257 DOCTEST_TEST_CASE_TEMPLATE

```
#define DOCTEST_TEST_CASE_TEMPLATE(
    dec,
    T,
    ...)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE_IMPL(dec, T, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_), __VA_ARGS__)
```

9.23.1.258 DOCTEST_TEST_CASE_TEMPLATE_APPLY

```
#define DOCTEST_TEST_CASE_TEMPLATE_APPLY(
    id,
    ...)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_), __VA_ARGS__)
\
static_assert(true, "")
```

9.23.1.259 DOCTEST_TEST_CASE_TEMPLATE_DEFINE

```
#define DOCTEST_TEST_CASE_TEMPLATE_DEFINE(
    dec,
    T,
    id)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, DOCTEST_CAT(id, ITERATOR),
    DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_))
```

9.23.1.260 DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL

```
#define DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(
    dec,
    T,
    iter,
    func)
```

Value:

```
template <typename T>
\
static void func();
\
namespace { /* NOLINT */
\
template <typename Tuple>
\
struct iter;
\
template <typename Type, typename... Rest>
\
struct iter<std::tuple<Type, Rest...> {
\
    iter(const char *file, unsigned line, int index) noexcept {
\
        doctest::detail::regTest(
\
```

```

        doctest::detail::TestCase(
            \
            func<Type>,
            \
            file,
            \
            line,
            \
            doctest_detail_test_suite_ns::getCurrentTestSuite(),
            \
            doctest::toString<Type>(),
            \
            int(line) * 1000 + index
        \
        ) *
        \
        dec
    \
    );
    \
    iter<std::tuple<Rest...>>(file, line, index + 1);
}
\
};
template <>
struct iter<std::tuple<>> {
    \
    iter(const char *, unsigned, int) {}
};
\
}
template <typename T>
\
static void func()

```

9.23.1.261 DOCTEST_TEST_CASE_TEMPLATE_IMPL

```

#define DOCTEST_TEST_CASE_TEMPLATE_IMPL(
    \
    dec,
    \
    T,
    \
    anon,
    \
    ...)

```

Value:

```

DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, DOCTEST_CAT(anon, ITERATOR), anon);
\
DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(anon, anon, std::tuple<__VA_ARGS__>)
\
template <typename T>
\
static void anon()

```

9.23.1.262 DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE

```

#define DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE(
    \
    id,
    \
    ...)

```

Value:

```

DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, __VA_ARGS__)

```

9.23.1.263 DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL

```
#define DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(
    id,
    anon,
    ...)
```

Value:

```
DOCTEST_GLOBAL_NO_WARNINGS(
    \
    DOCTEST_CAT(anon, DUMMY), /* NOLINT(cert-err58-cpp, fuchsia-statically-constructed-objects) */
    \
    doctest::detail::instantiationHelper(DOCTEST_CAT(id, ITERATOR) < __VA_ARGS__ > (__FILE__, __LINE__, 0))
    \
)
```

9.23.1.264 DOCTEST_TEST_CASE_TEMPLATE_INVOKE

```
#define DOCTEST_TEST_CASE_TEMPLATE_INVOKE(
    id,
    ...)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_),
    \
    std::tuple<__VA_ARGS__>) \
static_assert(true, "")
```

9.23.1.265 DOCTEST_TEST_SUITE

```
#define DOCTEST_TEST_SUITE(
    decorators)
```

Value:

```
DOCTEST_TEST_SUITE_IMPL(decorators, DOCTEST_ANONYMOUS(DOCTEST_ANON_SUITE_))
```

9.23.1.266 DOCTEST_TEST_SUITE_BEGIN

```
#define DOCTEST_TEST_SUITE_BEGIN(
    decorators)
```

Value:

```
DOCTEST_GLOBAL_NO_WARNINGS(
    \
    DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT(cert-err58-cpp) */
    \
    doctest::detail::setTestSuite(doctest::detail::TestSuite() * decorators)
    \
)
    \
static_assert(true, "")
```

9.23.1.267 DOCTEST_TEST_SUITE_END

```
#define DOCTEST_TEST_SUITE_END
```

Value:

```
DOCTEST_GLOBAL_NO_WARNINGS(
    \
    DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT(cert-err58-cpp) */
    \
    doctest::detail::setTestSuite(doctest::detail::TestSuite() * "")
    \
)
    \
using DOCTEST_ANONYMOUS(DOCTEST_ANON_FOR_SEMICOLON_) = int
```

9.23.1.268 DOCTEST_TEST_SUITE_IMPL

```
#define DOCTEST_TEST_SUITE_IMPL(
    decorators,
    ns_name)
```

Value:

```
namespace ns_name {
    namespace doctest_detail_test_suite_ns {
        static DOCTEST_NOINLINE doctest::detail::TestSuite &getCurrentTestSuite() noexcept {
            DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4640)
            DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wexit-time-destructors")
            DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wmissing-field-initializers")
            static doctest::detail::TestSuite data{};
            static bool initied = false;
            DOCTEST_MSVC_SUPPRESS_WARNING_POP
            DOCTEST_CLANG_SUPPRESS_WARNING_POP
            DOCTEST_GCC_SUPPRESS_WARNING_POP
            if (!initied) {
                data *decorators;
                initied = true;
            }
            return data;
        }
    }
}
namespace ns_name
```

9.23.1.269 DOCTEST_THEN

```
#define DOCTEST_THEN(
    name)
```

Value:

```
DOCTEST_SUBCASE("    Then: " name)
```

9.23.1.270 DOCTEST_TO_LVALUE

```
#define DOCTEST_TO_LVALUE(
    ...)
```

Value:

```
__VA_ARGS__
```

9.23.1.271 DOCTEST_TOSTR

```
#define DOCTEST_TOSTR(  
    x)
```

Value:

```
DOCTEST_TOSTR_IMPL(x)
```

9.23.1.272 DOCTEST_TOSTR_IMPL

```
#define DOCTEST_TOSTR_IMPL(  
    x)
```

Value:

```
#x
```

9.23.1.273 DOCTEST_TYPE_TO_STRING

```
#define DOCTEST_TYPE_TO_STRING(  
    ...)
```

Value:

```
DOCTEST_TYPE_TO_STRING_AS(__VA_ARGS__, __VA_ARGS__)
```

9.23.1.274 DOCTEST_TYPE_TO_STRING_AS

```
#define DOCTEST_TYPE_TO_STRING_AS(  
    str,  
    ...)
```

Value:

```
namespace doctest {  
    \  
    template <>  
    \  
    inline String toString<__VA_ARGS__>() {  
        \  
        return str;  
        \  
    }  
    \  
}  
    \  
static_assert(true, "")
```

9.23.1.275 DOCTEST_UNARY_ASSERT

```
#define DOCTEST_UNARY_ASSERT(  
    assert_type,  
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN {  
    \  
    doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__, __LINE__,  
        #__VA_ARGS__); \  
    DOCTEST_WRAP_IN_TRY(DOCTEST_RB.unary_assert(__VA_ARGS__))  
    \  
    DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);  
    \  
}  
    \  
DOCTEST_FUNC_SCOPE_END
```

9.23.1.276 DOCTEST_UNUSED

```
#define DOCTEST_UNUSED __attribute__((unused))
```

9.23.1.277 DOCTEST_VERSION

```
#define DOCTEST_VERSION (DOCTEST_VERSION_MAJOR * 10000 + DOCTEST_VERSION_MINOR * 100 + DOCTEST_VERSION_PATCH)
```

9.23.1.278 DOCTEST_VERSION_MAJOR

```
#define DOCTEST_VERSION_MAJOR 2
```

9.23.1.279 DOCTEST_VERSION_MINOR

```
#define DOCTEST_VERSION_MINOR 5
```

9.23.1.280 DOCTEST_VERSION_PATCH

```
#define DOCTEST_VERSION_PATCH 0
```

9.23.1.281 DOCTEST_VERSION_STR

```
#define DOCTEST_VERSION_STR
```

Value:

```
DOCTEST_TOSTR(DOCTEST_VERSION_MAJOR) "."  
DOCTEST_TOSTR(DOCTEST_VERSION_MINOR) "."  
DOCTEST_TOSTR(DOCTEST_VERSION_PATCH)
```

9.23.1.282 DOCTEST_WARN

```
#define DOCTEST_WARN(  
    ...)
```

Value:

```
DOCTEST_ASSERT_IMPLEMENT_1(DT_WARN, __VA_ARGS__)
```

9.23.1.283 DOCTEST_WARN_EQ

```
#define DOCTEST_WARN_EQ(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_WARN_EQ, eq, __VA_ARGS__)
```

9.23.1.284 DOCTEST_WARN_FALSE

```
#define DOCTEST_WARN_FALSE(  
    ...)
```

Value:

```
DOCTEST_ASSERT_IMPLEMENT_1(DT_WARN_FALSE, __VA_ARGS__)
```

9.23.1.285 DOCTEST_WARN_FALSE_MESSAGE

```
#define DOCTEST_WARN_FALSE_MESSAGE(  
    cond,  
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); DOCTEST_ASSERT_IMPLEMENT_2(DT_WARN_FALSE, cond); }  
DOCTEST_FUNC_SCOPE_END
```

9.23.1.286 DOCTEST_WARN_GE

```
#define DOCTEST_WARN_GE(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_WARN_GE, ge, __VA_ARGS__)
```

9.23.1.287 DOCTEST_WARN_GT

```
#define DOCTEST_WARN_GT(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_WARN_GT, gt, __VA_ARGS__)
```

9.23.1.288 DOCTEST_WARN_LE

```
#define DOCTEST_WARN_LE(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_WARN_LE, le, __VA_ARGS__)
```

9.23.1.289 DOCTEST_WARN_LT

```
#define DOCTEST_WARN_LT(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_WARN_LT, lt, __VA_ARGS__)
```


9.23.1.290 DOCTEST_WARN_MESSAGE

```
#define DOCTEST_WARN_MESSAGE(  
    cond,  
    ...)
```

Value:

```
DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__); DOCTEST_ASSERT_IMPLEMENT_2(DT_WARN, cond); }  
DOCTEST_FUNC_SCOPE_END
```

9.23.1.291 DOCTEST_WARN_NE

```
#define DOCTEST_WARN_NE(  
    ...)
```

Value:

```
DOCTEST_BINARY_ASSERT(DT_WARN_NE, ne, __VA_ARGS__)
```

9.23.1.292 DOCTEST_WARN_NOTHROW

```
#define DOCTEST_WARN_NOTHROW(  
    ...)
```

Value:

```
DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.293 DOCTEST_WARN_NOTHROW_MESSAGE

```
#define DOCTEST_WARN_NOTHROW_MESSAGE(  
    expr,  
    ...)
```

Value:

```
DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.294 DOCTEST_WARN_THROWS

```
#define DOCTEST_WARN_THROWS(  
    ...)
```

Value:

```
DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.295 DOCTEST_WARN_THROWS_AS

```
#define DOCTEST_WARN_THROWS_AS(  
    expr,  
    ...)
```

Value:

```
DOCTEST_EXCEPTION_EMPTY_FUNC
```

9.23.1.296 DOCTEST_WARN_THROWS_AS_MESSAGE

```
#define DOCTEST_WARN_THROWS_AS_MESSAGE(  
    expr,  
    ex,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

9.23.1.297 DOCTEST_WARN_THROWS_MESSAGE

```
#define DOCTEST_WARN_THROWS_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

9.23.1.298 DOCTEST_WARN_THROWS_WITH

```
#define DOCTEST_WARN_THROWS_WITH(  
    expr,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

9.23.1.299 DOCTEST_WARN_THROWS_WITH_AS

```
#define DOCTEST_WARN_THROWS_WITH_AS(  
    expr,  
    with,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

9.23.1.300 DOCTEST_WARN_THROWS_WITH_AS_MESSAGE

```
#define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(  
    expr,  
    with,  
    ex,  
    ...)
```

Value:

[DOCTEST_EXCEPTION_EMPTY_FUNC](#)

9.23.1.301 DOCTEST_WARN_THROWS_WITH_MESSAGE

```
#define DOCTEST_WARN_THROWS_WITH_MESSAGE(  
    expr,  
    with,  
    ...)
```

Value:

`DOCTEST_EXCEPTION_EMPTY_FUNC`

9.23.1.302 DOCTEST_WARN_UNARY

```
#define DOCTEST_WARN_UNARY(  
    ...)
```

Value:

`DOCTEST_UNARY_ASSERT`(`DT_WARN_UNARY`, `__VA_ARGS__`)

9.23.1.303 DOCTEST_WARN_UNARY_FALSE

```
#define DOCTEST_WARN_UNARY_FALSE(  
    ...)
```

Value:

`DOCTEST_UNARY_ASSERT`(`DT_WARN_UNARY_FALSE`, `__VA_ARGS__`)

9.23.1.304 DOCTEST_WHEN

```
#define DOCTEST_WHEN(  
    name)
```

Value:

`DOCTEST_SUBCASE`(" When: " `name`)

9.23.1.305 DOCTEST_WRAP_IN_TRY

```
#define DOCTEST_WRAP_IN_TRY(  
    x)
```

Value:

`x;`

9.23.1.306 FAIL

```
#define FAIL(  
    ...)
```

Value:

`DOCTEST_FAIL`(`__VA_ARGS__`)

9.23.1.307 FAIL_CHECK

```
#define FAIL_CHECK(  
    ...)
```

Value:

`DOCTEST_FAIL_CHECK(__VA_ARGS__)`

9.23.1.308 FAST_CHECK_EQ

```
#define FAST_CHECK_EQ(  
    ...)
```

Value:

`DOCTEST_FAST_CHECK_EQ(__VA_ARGS__)`

9.23.1.309 FAST_CHECK_GE

```
#define FAST_CHECK_GE(  
    ...)
```

Value:

`DOCTEST_FAST_CHECK_GE(__VA_ARGS__)`

9.23.1.310 FAST_CHECK_GT

```
#define FAST_CHECK_GT(  
    ...)
```

Value:

`DOCTEST_FAST_CHECK_GT(__VA_ARGS__)`

9.23.1.311 FAST_CHECK_LE

```
#define FAST_CHECK_LE(  
    ...)
```

Value:

`DOCTEST_FAST_CHECK_LE(__VA_ARGS__)`

9.23.1.312 FAST_CHECK_LT

```
#define FAST_CHECK_LT(  
    ...)
```

Value:

`DOCTEST_FAST_CHECK_LT(__VA_ARGS__)`

9.23.1.313 FAST_CHECK_NE

```
#define FAST_CHECK_NE(  
    ...)
```

Value:

`DOCTEST_FAST_CHECK_NE(__VA_ARGS__)`

9.23.1.314 FAST_CHECK_UNARY

```
#define FAST_CHECK_UNARY(  
    ...)
```

Value:

`DOCTEST_FAST_CHECK_UNARY(__VA_ARGS__)`

9.23.1.315 FAST_CHECK_UNARY_FALSE

```
#define FAST_CHECK_UNARY_FALSE(  
    ...)
```

Value:

`DOCTEST_FAST_CHECK_UNARY_FALSE(__VA_ARGS__)`

9.23.1.316 FAST_REQUIRE_EQ

```
#define FAST_REQUIRE_EQ(  
    ...)
```

Value:

`DOCTEST_FAST_REQUIRE_EQ(__VA_ARGS__)`

9.23.1.317 FAST_REQUIRE_GE

```
#define FAST_REQUIRE_GE(  
    ...)
```

Value:

`DOCTEST_FAST_REQUIRE_GE(__VA_ARGS__)`

9.23.1.318 FAST_REQUIRE_GT

```
#define FAST_REQUIRE_GT(  
    ...)
```

Value:

`DOCTEST_FAST_REQUIRE_GT(__VA_ARGS__)`

9.23.1.319 FAST_REQUIRE_LE

```
#define FAST_REQUIRE_LE(  
    ...)
```

Value:

`DOCTEST_FAST_REQUIRE_LE(__VA_ARGS__)`

9.23.1.320 FAST_REQUIRE_LT

```
#define FAST_REQUIRE_LT(  
    ...)
```

Value:

`DOCTEST_FAST_REQUIRE_LT(__VA_ARGS__)`

9.23.1.321 FAST_REQUIRE_NE

```
#define FAST_REQUIRE_NE(  
    ...)
```

Value:

`DOCTEST_FAST_REQUIRE_NE(__VA_ARGS__)`

9.23.1.322 FAST_REQUIRE_UNARY

```
#define FAST_REQUIRE_UNARY(  
    ...)
```

Value:

`DOCTEST_FAST_REQUIRE_UNARY(__VA_ARGS__)`

9.23.1.323 FAST_REQUIRE_UNARY_FALSE

```
#define FAST_REQUIRE_UNARY_FALSE(  
    ...)
```

Value:

`DOCTEST_FAST_REQUIRE_UNARY_FALSE(__VA_ARGS__)`

9.23.1.324 FAST_WARN_EQ

```
#define FAST_WARN_EQ(  
    ...)
```

Value:

`DOCTEST_FAST_WARN_EQ(__VA_ARGS__)`

9.23.1.325 FAST_WARN_GE

```
#define FAST_WARN_GE(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_GE](#)(__VA_ARGS__)

9.23.1.326 FAST_WARN_GT

```
#define FAST_WARN_GT(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_GT](#)(__VA_ARGS__)

9.23.1.327 FAST_WARN_LE

```
#define FAST_WARN_LE(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_LE](#)(__VA_ARGS__)

9.23.1.328 FAST_WARN_LT

```
#define FAST_WARN_LT(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_LT](#)(__VA_ARGS__)

9.23.1.329 FAST_WARN_NE

```
#define FAST_WARN_NE(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_NE](#)(__VA_ARGS__)

9.23.1.330 FAST_WARN_UNARY

```
#define FAST_WARN_UNARY(  
    ...)
```

Value:

[DOCTEST_FAST_WARN_UNARY](#)(__VA_ARGS__)

9.23.1.331 FAST_WARN_UNARY_FALSE

```
#define FAST_WARN_UNARY_FALSE(  
    ...)
```

Value:

`DOCTEST_FAST_WARN_UNARY_FALSE(__VA_ARGS__)`

9.23.1.332 GENERATE

```
#define GENERATE(  
    ...)
```

Value:

`DOCTEST_GENERATE(__VA_ARGS__)`

9.23.1.333 GIVEN

```
#define GIVEN(  
    name)
```

Value:

`DOCTEST_GIVEN(name)`

9.23.1.334 INFO

```
#define INFO(  
    ...)
```

Value:

`DOCTEST_INFO(__VA_ARGS__)`

9.23.1.335 MESSAGE

```
#define MESSAGE(  
    ...)
```

Value:

`DOCTEST_MESSAGE(__VA_ARGS__)`

9.23.1.336 REGISTER_EXCEPTION_TRANSLATOR

```
#define REGISTER_EXCEPTION_TRANSLATOR(  
    signature)
```

Value:

`DOCTEST_REGISTER_EXCEPTION_TRANSLATOR(signature)`

9.23.1.337 REGISTER_LISTENER

```
#define REGISTER_LISTENER(  
    name,  
    priority,  
    reporter)
```

Value:

`DOCTEST_REGISTER_LISTENER`(name, priority, reporter)

9.23.1.338 REGISTER_REPORTER

```
#define REGISTER_REPORTER(  
    name,  
    priority,  
    reporter)
```

Value:

`DOCTEST_REGISTER_REPORTER`(name, priority, reporter)

9.23.1.339 REQUIRE

```
#define REQUIRE(  
    ...)
```

Value:

`DOCTEST_REQUIRE`(__VA_ARGS__)

9.23.1.340 REQUIRE_EQ

```
#define REQUIRE_EQ(  
    ...)
```

Value:

`DOCTEST_REQUIRE_EQ`(__VA_ARGS__)

9.23.1.341 REQUIRE_FALSE

```
#define REQUIRE_FALSE(  
    ...)
```

Value:

`DOCTEST_REQUIRE_FALSE`(__VA_ARGS__)

9.23.1.342 REQUIRE_FALSE_MESSAGE

```
#define REQUIRE_FALSE_MESSAGE(  
    cond,  
    ...)
```

Value:

`DOCTEST_REQUIRE_FALSE_MESSAGE`(cond, __VA_ARGS__)

9.23.1.343 REQUIRE_GE

```
#define REQUIRE_GE(  
    ...)
```

Value:

`DOCTEST_REQUIRE_GE(__VA_ARGS__)`

9.23.1.344 REQUIRE_GT

```
#define REQUIRE_GT(  
    ...)
```

Value:

`DOCTEST_REQUIRE_GT(__VA_ARGS__)`

9.23.1.345 REQUIRE_LE

```
#define REQUIRE_LE(  
    ...)
```

Value:

`DOCTEST_REQUIRE_LE(__VA_ARGS__)`

9.23.1.346 REQUIRE_LT

```
#define REQUIRE_LT(  
    ...)
```

Value:

`DOCTEST_REQUIRE_LT(__VA_ARGS__)`

9.23.1.347 REQUIRE_MESSAGE

```
#define REQUIRE_MESSAGE(  
    cond,  
    ...)
```

Value:

`DOCTEST_REQUIRE_MESSAGE(cond, __VA_ARGS__)`

9.23.1.348 REQUIRE_NE

```
#define REQUIRE_NE(  
    ...)
```

Value:

`DOCTEST_REQUIRE_NE(__VA_ARGS__)`

9.23.1.349 REQUIRE_NOTHROW

```
#define REQUIRE_NOTHROW(  
    ...)
```

Value:

[DOCTEST_REQUIRE_NOTHROW](#)(__VA_ARGS__)

9.23.1.350 REQUIRE_NOTHROW_MESSAGE

```
#define REQUIRE_NOTHROW_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_REQUIRE_NOTHROW_MESSAGE](#)(*expr*, __VA_ARGS__)

9.23.1.351 REQUIRE_THROWS

```
#define REQUIRE_THROWS(  
    ...)
```

Value:

[DOCTEST_REQUIRE_THROWS](#)(__VA_ARGS__)

9.23.1.352 REQUIRE_THROWS_AS

```
#define REQUIRE_THROWS_AS(  
    expr,  
    ...)
```

Value:

[DOCTEST_REQUIRE_THROWS_AS](#)(*expr*, __VA_ARGS__)

9.23.1.353 REQUIRE_THROWS_AS_MESSAGE

```
#define REQUIRE_THROWS_AS_MESSAGE(  
    expr,  
    ex,  
    ...)
```

Value:

[DOCTEST_REQUIRE_THROWS_AS_MESSAGE](#)(*expr*, *ex*, __VA_ARGS__)

9.23.1.354 REQUIRE_THROWS_MESSAGE

```
#define REQUIRE_THROWS_MESSAGE(  
    expr,  
    ...)
```

Value:

[DOCTEST_REQUIRE_THROWS_MESSAGE](#)(*expr*, __VA_ARGS__)

9.23.1.355 REQUIRE_THROWS_WITH

```
#define REQUIRE_THROWS_WITH(  
    expr,  
    ...)
```

Value:

`DOCTEST_REQUIRE_THROWS_WITH(expr, __VA_ARGS__)`

9.23.1.356 REQUIRE_THROWS_WITH_AS

```
#define REQUIRE_THROWS_WITH_AS(  
    expr,  
    with,  
    ...)
```

Value:

`DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, __VA_ARGS__)`

9.23.1.357 REQUIRE_THROWS_WITH_AS_MESSAGE

```
#define REQUIRE_THROWS_WITH_AS_MESSAGE(  
    expr,  
    with,  
    ex,  
    ...)
```

Value:

`DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, __VA_ARGS__)`

9.23.1.358 REQUIRE_THROWS_WITH_MESSAGE

```
#define REQUIRE_THROWS_WITH_MESSAGE(  
    expr,  
    with,  
    ...)
```

Value:

`DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, __VA_ARGS__)`

9.23.1.359 REQUIRE_UNARY

```
#define REQUIRE_UNARY(  
    ...)
```

Value:

`DOCTEST_REQUIRE_UNARY(__VA_ARGS__)`

9.23.1.360 REQUIRE_UNARY_FALSE

```
#define REQUIRE_UNARY_FALSE(  
    ...)
```

Value:

`DOCTEST_REQUIRE_UNARY_FALSE(__VA_ARGS__)`

9.23.1.361 SCENARIO

```
#define SCENARIO(  
    name)
```

Value:

`DOCTEST_SCENARIO(name)`

9.23.1.362 SCENARIO_CLASS

```
#define SCENARIO_CLASS(  
    name)
```

Value:

`DOCTEST_SCENARIO_CLASS(name)`

9.23.1.363 SCENARIO_METHOD

```
#define SCENARIO_METHOD(  
    x,  
    name)
```

Value:

`DOCTEST_SCENARIO_METHOD(x, name)`

9.23.1.364 SCENARIO_TEMPLATE

```
#define SCENARIO_TEMPLATE(  
    name,  
    T,  
    ...)
```

Value:

`DOCTEST_SCENARIO_TEMPLATE(name, T, __VA_ARGS__)`

9.23.1.365 SCENARIO_TEMPLATE_DEFINE

```
#define SCENARIO_TEMPLATE_DEFINE(  
    name,  
    T,  
    id)
```

Value:

`DOCTEST_SCENARIO_TEMPLATE_DEFINE(name, T, id)`

9.23.1.366 SFINAE_OP

```
#define SFINAE_OP(  
    ret,  
    op)
```

Value:

```
decltype((void) (doctest::detail::declval<L>() op doctest::detail::declval<R>()), ret{})
```

9.23.1.367 SUBCASE

```
#define SUBCASE(  
    name)
```

Value:

```
DOCTEST_SUBCASE(name)
```

9.23.1.368 TEST_CASE

```
#define TEST_CASE(  
    name)
```

Value:

```
DOCTEST_TEST_CASE(name)
```

9.23.1.369 TEST_CASE_CLASS

```
#define TEST_CASE_CLASS(  
    name)
```

Value:

```
DOCTEST_TEST_CASE_CLASS(name)
```

9.23.1.370 TEST_CASE_FIXTURE

```
#define TEST_CASE_FIXTURE(  
    x,  
    name)
```

Value:

```
DOCTEST_TEST_CASE_FIXTURE(x, name)
```

9.23.1.371 TEST_CASE_TEMPLATE

```
#define TEST_CASE_TEMPLATE(  
    name,  
    T,  
    ...)
```

Value:

```
DOCTEST_TEST_CASE_TEMPLATE(name, T, __VA_ARGS__)
```

9.23.1.372 TEST_CASE_TEMPLATE_APPLY

```
#define TEST_CASE_TEMPLATE_APPLY(  
    id,  
    ...)
```

Value:

`DOCTEST_TEST_CASE_TEMPLATE_APPLY(id, __VA_ARGS__)`

9.23.1.373 TEST_CASE_TEMPLATE_DEFINE

```
#define TEST_CASE_TEMPLATE_DEFINE(  
    name,  
    T,  
    id)
```

Value:

`DOCTEST_TEST_CASE_TEMPLATE_DEFINE(name, T, id)`

9.23.1.374 TEST_CASE_TEMPLATE_INSTANTIATE

```
#define TEST_CASE_TEMPLATE_INSTANTIATE(  
    id,  
    ...)
```

Value:

`DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE(id, __VA_ARGS__)`

9.23.1.375 TEST_CASE_TEMPLATE_INVOKE

```
#define TEST_CASE_TEMPLATE_INVOKE(  
    id,  
    ...)
```

Value:

`DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, __VA_ARGS__)`

9.23.1.376 TEST_SUITE

```
#define TEST_SUITE(  
    decorators)
```

Value:

`DOCTEST_TEST_SUITE(decorators)`

9.23.1.377 TEST_SUITE_BEGIN

```
#define TEST_SUITE_BEGIN(  
    name)
```

Value:

`DOCTEST_TEST_SUITE_BEGIN(name)`

9.23.1.378 TEST_SUITE_END

```
#define TEST_SUITE_END DOCTEST_TEST_SUITE_END
```

9.23.1.379 THEN

```
#define THEN(  
    name)
```

Value:

`DOCTEST_THEN`(*name*)

9.23.1.380 TO_LVALUE

```
#define TO_LVALUE(  
    ...)
```

Value:

`DOCTEST_TO_LVALUE`(__VA_ARGS__)

9.23.1.381 TYPE_TO_STRING

```
#define TYPE_TO_STRING(  
    ...)
```

Value:

`DOCTEST_TYPE_TO_STRING`(__VA_ARGS__)

9.23.1.382 TYPE_TO_STRING_AS

```
#define TYPE_TO_STRING_AS(  
    str,  
    ...)
```

Value:

`DOCTEST_TYPE_TO_STRING_AS`(*str*, __VA_ARGS__)

9.23.1.383 WARN

```
#define WARN(  
    ...)
```

Value:

`DOCTEST_WARN`(__VA_ARGS__)

9.23.1.384 WARN_EQ

```
#define WARN_EQ(  
    ...)
```

Value:

[DOCTEST_WARN_EQ](#)(__VA_ARGS__)

9.23.1.385 WARN_FALSE

```
#define WARN_FALSE(  
    ...)
```

Value:

[DOCTEST_WARN_FALSE](#)(__VA_ARGS__)

9.23.1.386 WARN_FALSE_MESSAGE

```
#define WARN_FALSE_MESSAGE(  
    cond,  
    ...)
```

Value:

[DOCTEST_WARN_FALSE_MESSAGE](#)(cond, __VA_ARGS__)

9.23.1.387 WARN_GE

```
#define WARN_GE(  
    ...)
```

Value:

[DOCTEST_WARN_GE](#)(__VA_ARGS__)

9.23.1.388 WARN_GT

```
#define WARN_GT(  
    ...)
```

Value:

[DOCTEST_WARN_GT](#)(__VA_ARGS__)

9.23.1.389 WARN_LE

```
#define WARN_LE(  
    ...)
```

Value:

[DOCTEST_WARN_LE](#)(__VA_ARGS__)

9.23.1.390 WARN_LT

```
#define WARN_LT(  
    ...)
```

Value:

`DOCTEST_WARN_LT` (`__VA_ARGS__`)

9.23.1.391 WARN_MESSAGE

```
#define WARN_MESSAGE(  
    cond,  
    ...)
```

Value:

`DOCTEST_WARN_MESSAGE` (`cond`, `__VA_ARGS__`)

9.23.1.392 WARN_NE

```
#define WARN_NE(  
    ...)
```

Value:

`DOCTEST_WARN_NE` (`__VA_ARGS__`)

9.23.1.393 WARN_NOTHROW

```
#define WARN_NOTHROW(  
    ...)
```

Value:

`DOCTEST_WARN_NOTHROW` (`__VA_ARGS__`)

9.23.1.394 WARN_NOTHROW_MESSAGE

```
#define WARN_NOTHROW_MESSAGE(  
    expr,  
    ...)
```

Value:

`DOCTEST_WARN_NOTHROW_MESSAGE` (`expr`, `__VA_ARGS__`)

9.23.1.395 WARN_THROWS

```
#define WARN_THROWS(  
    ...)
```

Value:

`DOCTEST_WARN_THROWS` (`__VA_ARGS__`)

9.23.1.396 WARN_THROWS_AS

```
#define WARN_THROWS_AS(  
    expr,  
    ...)
```

Value:

`DOCTEST_WARN_THROWS_AS(expr, __VA_ARGS__)`

9.23.1.397 WARN_THROWS_AS_MESSAGE

```
#define WARN_THROWS_AS_MESSAGE(  
    expr,  
    ex,  
    ...)
```

Value:

`DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, __VA_ARGS__)`

9.23.1.398 WARN_THROWS_MESSAGE

```
#define WARN_THROWS_MESSAGE(  
    expr,  
    ...)
```

Value:

`DOCTEST_WARN_THROWS_MESSAGE(expr, __VA_ARGS__)`

9.23.1.399 WARN_THROWS_WITH

```
#define WARN_THROWS_WITH(  
    expr,  
    ...)
```

Value:

`DOCTEST_WARN_THROWS_WITH(expr, __VA_ARGS__)`

9.23.1.400 WARN_THROWS_WITH_AS

```
#define WARN_THROWS_WITH_AS(  
    expr,  
    with,  
    ...)
```

Value:

`DOCTEST_WARN_THROWS_WITH_AS(expr, with, __VA_ARGS__)`

9.23.1.401 WARN_THROWS_WITH_AS_MESSAGE

```
#define WARN_THROWS_WITH_AS_MESSAGE(  
    expr,  
    with,  
    ex,  
    ...)
```

Value:

`DOCTEST_WARN_THROWS_WITH_AS_MESSAGE`(*expr*, *with*, *ex*, __VA_ARGS__)

9.23.1.402 WARN_THROWS_WITH_MESSAGE

```
#define WARN_THROWS_WITH_MESSAGE(  
    expr,  
    with,  
    ...)
```

Value:

`DOCTEST_WARN_THROWS_WITH_MESSAGE`(*expr*, *with*, __VA_ARGS__)

9.23.1.403 WARN_UNARY

```
#define WARN_UNARY(  
    ...)
```

Value:

`DOCTEST_WARN_UNARY`(__VA_ARGS__)

9.23.1.404 WARN_UNARY_FALSE

```
#define WARN_UNARY_FALSE(  
    ...)
```

Value:

`DOCTEST_WARN_UNARY_FALSE`(__VA_ARGS__)

9.23.1.405 WHEN

```
#define WHEN(  
    name)
```

Value:

`DOCTEST_WHEN`(*name*)

9.24 doctest.h

[Go to the documentation of this file.](#)

```

00001 // =====
00002 // == DO NOT MODIFY THIS FILE BY HAND - IT IS AUTO GENERATED! ==
00003 // =====
00004 //
00005 // doctest.h - the lightest feature-rich C++ single-header testing framework for unit tests and TDD
00006 //
00007 // Copyright (c) 2016-2023 Viktor Kirilov
00008 //
00009 // Distributed under the MIT Software License
00010 // See accompanying file LICENSE.txt or copy at
00011 // https://opensource.org/licenses/MIT
00012 //
00013 // The documentation can be found at the library's page:
00014 // https://github.com/doctest/doctest/blob/master/doc/markdown/readme.md
00015 //
00016 // =====
00017 // =====
00018 // =====
00019 //
00020 // The library is heavily influenced by Catch - https://github.com/catchorg/Catch2
00021 // which uses the Boost Software License - Version 1.0
00022 // see here - https://github.com/catchorg/Catch2/blob/master/LICENSE.txt
00023 //
00024 // The concept of subcases (sections in Catch) and expression decomposition are from there.
00025 // Some parts of the code are taken directly:
00026 // - stringification - the detection of "ostream& operator<<(ostream&, const T&)" and StringMaker<>
00027 // - the Approx() helper class for floating point comparison
00028 // - colors in the console
00029 // - breaking into a debugger
00030 // - signal / SEH handling
00031 // - timer
00032 // - XmlWriter class - thanks to Phil Nash for allowing the direct reuse (AKA copy/paste)
00033 //
00034 // The expression decomposing templates are taken from lest - https://github.com/martinmoene/lest
00035 // which uses the Boost Software License - Version 1.0
00036 // see here - https://github.com/martinmoene/lest/blob/master/LICENSE.txt
00037 //
00038 // =====
00039 // =====
00040 // =====
00041 //
00042 #ifndef DOCTEST_LIBRARY_INCLUDED
00043 #define DOCTEST_LIBRARY_INCLUDED
00044 //
00045 // =====
00046 // == VERSION =====
00047 // =====
00048 //
00049 #ifndef DOCTEST_PARTS_PUBLIC_VERSION
00050 #define DOCTEST_PARTS_PUBLIC_VERSION
00051 //
00052 // NOLINTBEGIN(cppcoreguidelines-macro-to-enum, modernize-macro-to-enum)
00053 #define DOCTEST_VERSION_MAJOR 2
00054 #define DOCTEST_VERSION_MINOR 5
00055 #define DOCTEST_VERSION_PATCH 0
00056 // NOLINTEND(cppcoreguidelines-macro-to-enum, modernize-macro-to-enum)
00057 //
00058 // util we need here
00059 #define DOCTEST_TOSTR_IMPL(x) #x
00060 #define DOCTEST_TOSTR(x) DOCTEST_TOSTR_IMPL(x)
00061 //
00062 // clang-format off
00063 #define DOCTEST_VERSION_STR
00064 \
00065 \
00066 \
00067 // clang-format on
00068 //
00069 #define DOCTEST_VERSION (DOCTEST_VERSION_MAJOR * 10000 + DOCTEST_VERSION_MINOR * 100 +
00070 \
00071 \
00072 \
00073 // == COMPILER VERSION =====
00074 // =====
00075 //
00076 #ifndef DOCTEST_PARTS_PUBLIC_COMPILER
00077 #define DOCTEST_PARTS_PUBLIC_COMPILER
00078 //

```

```

00079 // ideas for the version stuff are taken from here: https://github.com/cxxstuff/cxx_detect
00080
00081 #ifdef _MSC_VER
00082 #define DOCTEST_CPLUSPLUS _MSVC_LANG
00083 #else
00084 #define DOCTEST_CPLUSPLUS __cplusplus
00085 #endif
00086
00087 #define DOCTEST_COMPILER(MAJOR, MINOR, PATCH) ((MAJOR) * 10000000 + (MINOR) * 100000 + (PATCH))
00088
00089 // GCC/Clang and GCC/MSVC are mutually exclusive, but Clang/MSVC are not because of clang-cl...
00090 #if defined(_MSC_VER) && defined(_MSC_FULL_VER)
00091 #if _MSC_VER == _MSC_FULL_VER / 10000
00092 #define DOCTEST_MSVC DOCTEST_COMPILER(_MSC_VER / 100, _MSC_VER % 100, _MSC_FULL_VER % 10000)
00093 #else // MSVC
00094 #define DOCTEST_MSVC DOCTEST_COMPILER(_MSC_VER / 100, (_MSC_FULL_VER / 100000) % 100, _MSC_FULL_VER % 100000)
00095 #endif // MSVC
00096 #endif // MSVC
00097 #if defined(__clang__) && defined(__clang_minor__) && defined(__clang_patchlevel__)
00098 #define DOCTEST_CLANG DOCTEST_COMPILER(__clang_major__, __clang_minor__, __clang_patchlevel__)
00099 #elif defined(__GNUC__) && defined(__GNUC_MINOR__) && defined(__GNUC_PATCHLEVEL__) &&
!defined(__INTEL_COMPILER)
00100 #define DOCTEST_GCC DOCTEST_COMPILER(__GNUC__, __GNUC_MINOR__, __GNUC_PATCHLEVEL__)
00101 #endif // GCC
00102 #if defined(__INTEL_COMPILER)
00103 #define DOCTEST_ICC DOCTEST_COMPILER(__INTEL_COMPILER / 100, __INTEL_COMPILER % 100, 0)
00104 #endif // ICC
00105
00106 #ifndef DOCTEST_MSVC
00107 #define DOCTEST_MSVC 0
00108 #endif // DOCTEST_MSVC
00109 #ifndef DOCTEST_CLANG
00110 #define DOCTEST_CLANG 0
00111 #endif // DOCTEST_CLANG
00112 #ifndef DOCTEST_GCC
00113 #define DOCTEST_GCC 0
00114 #endif // DOCTEST_GCC
00115 #ifndef DOCTEST_ICC
00116 #define DOCTEST_ICC 0
00117 #endif // DOCTEST_ICC
00118
00119 #endif // DOCTEST_PARTS_PUBLIC_COMPILER
00120 // =====
00121 // == COMPILER WARNINGS HELPERS ==
00122 // =====
00123
00124 #ifndef DOCTEST_PARTS_PUBLIC_WARNINGS
00125 #define DOCTEST_PARTS_PUBLIC_WARNINGS
00126
00127
00128 #if DOCTEST_CLANG && !DOCTEST_ICC
00129 #define DOCTEST_PRAGMA_TO_STR(x) _Pragma(#x)
00130 #define DOCTEST_CLANG_SUPPRESS_WARNING_PUSH _Pragma("clang diagnostic push")
00131 #define DOCTEST_CLANG_SUPPRESS_WARNING(w) DOCTEST_PRAGMA_TO_STR(clang diagnostic ignored w)
00132 #define DOCTEST_CLANG_SUPPRESS_WARNING_POP _Pragma("clang diagnostic pop")
00133 #define DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH(w)
\
DOCTEST_CLANG_SUPPRESS_WARNING_PUSH DOCTEST_CLANG_SUPPRESS_WARNING(w)
00135 #else // DOCTEST_CLANG
00136 #define DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
00137 #define DOCTEST_CLANG_SUPPRESS_WARNING(w)
00138 #define DOCTEST_CLANG_SUPPRESS_WARNING_POP
00139 #define DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH(w)
00140 #endif // DOCTEST_CLANG
00141
00142 #if DOCTEST_GCC
00143 #define DOCTEST_PRAGMA_TO_STR(x) _Pragma(#x)
00144 #define DOCTEST_GCC_SUPPRESS_WARNING_PUSH _Pragma("GCC diagnostic push")
00145 #define DOCTEST_GCC_SUPPRESS_WARNING(w) DOCTEST_PRAGMA_TO_STR(GCC diagnostic ignored w)
00146 #define DOCTEST_GCC_SUPPRESS_WARNING_POP _Pragma("GCC diagnostic pop")
00147 #define DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH(w) DOCTEST_GCC_SUPPRESS_WARNING_PUSH
DOCTEST_GCC_SUPPRESS_WARNING(w)
00148 #else // DOCTEST_GCC
00149 #define DOCTEST_GCC_SUPPRESS_WARNING_PUSH
00150 #define DOCTEST_GCC_SUPPRESS_WARNING(w)
00151 #define DOCTEST_GCC_SUPPRESS_WARNING_POP
00152 #define DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH(w)
00153 #endif // DOCTEST_GCC
00154
00155 #if DOCTEST_MSVC
00156 #define DOCTEST_MSVC_SUPPRESS_WARNING_PUSH __pragma(warning(push))
00157 #define DOCTEST_MSVC_SUPPRESS_WARNING(w) __pragma(warning(disable : w))
00158 #define DOCTEST_MSVC_SUPPRESS_WARNING_POP __pragma(warning(pop))
00159 #define DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(w) DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
DOCTEST_MSVC_SUPPRESS_WARNING(w)
00160 #else // DOCTEST_MSVC

```

```

00161 #define DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
00162 #define DOCTEST_MSVC_SUPPRESS_WARNING(w)
00163 #define DOCTEST_MSVC_SUPPRESS_WARNING_POP
00164 #define DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(w)
00165 #endif // DOCTEST_MSVC
00166
00167 // =====
00168 // == COMPILER WARNINGS ==
00169 // =====
00170
00171 // both the header and the implementation suppress all of these,
00172 // so it only makes sense to aggregate them like so
00173 #define DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH
00174 \
00175 \
00176 \
00177 \
00178 \
00179 \
00180 \
00181 \
00182 \
00183 \
00184 \
00185 \
00186 \
00187 \
00188 \
00189 \
00190 \
00191 \
00192 \
00193 \
00194 \
00195 \
00196 \
00197 \
00198 \
00199 \
00200 \
00201 \
00202 \
00203 \
00204 \
00205 \
00206 \
00207 \
00208 \
00209 \
00210 \
DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
DOCTEST_CLANG_SUPPRESS_WARNING("-Wunknown-pragmas")
DOCTEST_CLANG_SUPPRESS_WARNING("-Wunknown-warning-option")
DOCTEST_CLANG_SUPPRESS_WARNING("-Wweak-vtables")
DOCTEST_CLANG_SUPPRESS_WARNING("-Wpadded")
DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-prototypes")
DOCTEST_CLANG_SUPPRESS_WARNING("-Wc++98-compat")
DOCTEST_CLANG_SUPPRESS_WARNING("-Wc++98-compat-pedantic")
DOCTEST_CLANG_SUPPRESS_WARNING("-Wunsafe-buffer-usage")
DOCTEST_CLANG_SUPPRESS_WARNING("-Wunused-macros")
DOCTEST_GCC_SUPPRESS_WARNING_PUSH
DOCTEST_GCC_SUPPRESS_WARNING("-Wunknown-pragmas")
DOCTEST_GCC_SUPPRESS_WARNING("-Wpragmas")
DOCTEST_GCC_SUPPRESS_WARNING("-Weffc++")
DOCTEST_GCC_SUPPRESS_WARNING("-Wstrict-overflow")
DOCTEST_GCC_SUPPRESS_WARNING("-Wstrict-aliasing")
DOCTEST_GCC_SUPPRESS_WARNING("-Wmissing-declarations")
DOCTEST_GCC_SUPPRESS_WARNING("-Wuseless-cast")
DOCTEST_GCC_SUPPRESS_WARNING("-Wnoexcept")
DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
/* these 4 also disabled globally via cmake: */
DOCTEST_MSVC_SUPPRESS_WARNING(4514) /* unreferenced inline function has been removed */
DOCTEST_MSVC_SUPPRESS_WARNING(4571) /* SEH related */
DOCTEST_MSVC_SUPPRESS_WARNING(4710) /* function not inlined */
DOCTEST_MSVC_SUPPRESS_WARNING(4711) /* function selected for inline expansion*/
/* common ones */
DOCTEST_MSVC_SUPPRESS_WARNING(4616) /* invalid compiler warning */
DOCTEST_MSVC_SUPPRESS_WARNING(4619) /* invalid compiler warning */
DOCTEST_MSVC_SUPPRESS_WARNING(4996) /* The compiler encountered a deprecated declaration */
DOCTEST_MSVC_SUPPRESS_WARNING(4706) /* assignment within conditional expression */
DOCTEST_MSVC_SUPPRESS_WARNING(4512) /* 'class' : assignment operator could not be generated */
DOCTEST_MSVC_SUPPRESS_WARNING(4127) /* conditional expression is constant */
DOCTEST_MSVC_SUPPRESS_WARNING(4820) /* padding */
DOCTEST_MSVC_SUPPRESS_WARNING(4625) /* copy constructor was implicitly deleted */
DOCTEST_MSVC_SUPPRESS_WARNING(4626) /* assignment operator was implicitly deleted */

```

```

00211 \ DOCTEST_MSVC_SUPPRESS_WARNING(5027) /* move assignment operator implicitly deleted */
00212 \ DOCTEST_MSVC_SUPPRESS_WARNING(5026) /* move constructor was implicitly deleted */
00213 \ DOCTEST_MSVC_SUPPRESS_WARNING(4640) /* construction of local static object not thread-safe */
00214 \ DOCTEST_MSVC_SUPPRESS_WARNING(5045) /* Spectre mitigation for memory load */
00215 \ DOCTEST_MSVC_SUPPRESS_WARNING(5264) /* 'variable-name': 'const' variable is not used */
00216 \ /* static analysis */
00217 \ DOCTEST_MSVC_SUPPRESS_WARNING(26439) /* Function may not throw. Declare it 'noexcept' */
00218 \ DOCTEST_MSVC_SUPPRESS_WARNING(26495) /* Always initialize a member variable */
00219 \ DOCTEST_MSVC_SUPPRESS_WARNING(26451) /* Arithmetic overflow ... */
00220 \ DOCTEST_MSVC_SUPPRESS_WARNING(26444) /* Avoid unnamed objects with custom ctor and dtor... */
00221 \ DOCTEST_MSVC_SUPPRESS_WARNING(26812) /* Prefer 'enum class' over 'enum' */
00222
00223 #define DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
00224 \ DOCTEST_CLANG_SUPPRESS_WARNING_POP
00225 \ DOCTEST_GCC_SUPPRESS_WARNING_POP
00226 \ DOCTEST_MSVC_SUPPRESS_WARNING_POP
00227
00228 #define DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
00229 \ DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH
00230
00231 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wnon-virtual-dtor")
00232 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wdeprecated")
00233
00234 \ DOCTEST_GCC_SUPPRESS_WARNING("-Wctor-dtor-privacy")
00235 \ DOCTEST_GCC_SUPPRESS_WARNING("-Wnon-virtual-dtor")
00236 \ DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-promo")
00237
00238 \ DOCTEST_MSVC_SUPPRESS_WARNING(4623) /* default constructor was implicitly deleted */
00239
00240 #define DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
00241
00242 #define DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
00243 \ DOCTEST_SUPPRESS_COMMON_WARNINGS_PUSH
00244
00245 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wglobal-constructors")
00246 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wexit-time-destructors")
00247 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wsign-conversion")
00248 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wshorten-64-to-32")
00249 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-variable-declarations")
00250 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wswitch")
00251 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wswitch-enum")
00252 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wcovered-switch-default")
00253 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-noreturn")
00254 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wdisabled-macro-expansion")
00255 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-braces")
00256 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wmissing-field-initializers")
00257 \ DOCTEST_CLANG_SUPPRESS_WARNING("-Wunused-member-function")

```



```

00258     DOCTEST_CLANG_SUPPRESS_WARNING("-Wunused-function")
00259 \
00260     DOCTEST_CLANG_SUPPRESS_WARNING("-Wnonportable-system-include-path")
00261 \
00262     DOCTEST_GCC_SUPPRESS_WARNING("-Wconversion")
00263 \
00264     DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-conversion")
00265 \
00266     DOCTEST_GCC_SUPPRESS_WARNING("-Wmissing-field-initializers")
00267 \
00268     DOCTEST_GCC_SUPPRESS_WARNING("-Wmissing-braces")
00269 \
00270     DOCTEST_GCC_SUPPRESS_WARNING("-Wswitch")
00271 \
00272     DOCTEST_GCC_SUPPRESS_WARNING("-Wswitch-enum")
00273 \
00274     DOCTEST_GCC_SUPPRESS_WARNING("-Wswitch-default")
00275 \
00276     DOCTEST_GCC_SUPPRESS_WARNING("-Wunsafe-loop-optimizations")
00277 \
00278     DOCTEST_GCC_SUPPRESS_WARNING("-Wold-style-cast")
00279 \
00280     DOCTEST_GCC_SUPPRESS_WARNING("-Wunused-function")
00281 \
00282     DOCTEST_GCC_SUPPRESS_WARNING("-Wmultiple-inheritance")
00283 \
00284     DOCTEST_GCC_SUPPRESS_WARNING("-Wsuggest-attribute")
00285 \
00286     DOCTEST_GCC_SUPPRESS_WARNING("-Wnrvo")
00287 \
00288     DOCTEST_MSVC_SUPPRESS_WARNING(4267) /* conversion from 'x' to 'y', possible loss of data */
00289 \
00290     DOCTEST_MSVC_SUPPRESS_WARNING(4530) /* exception handler, but unwind semantics not enabled */
00291 \
00292     DOCTEST_MSVC_SUPPRESS_WARNING(4577) /* 'noexcept' with no exception handling mode specified */
00293 \
00294     DOCTEST_MSVC_SUPPRESS_WARNING(4774) /* format string in argument is not a string literal */
00295 \
00296     DOCTEST_MSVC_SUPPRESS_WARNING(4365) /* signed/unsigned mismatch */
00297 \
00298     DOCTEST_MSVC_SUPPRESS_WARNING(5039) /* pointer to pot. throwing function passed to extern C */
00299 \
00300     DOCTEST_MSVC_SUPPRESS_WARNING(4800) /* forcing value to bool (performance warning) */
00301 \
00302     DOCTEST_MSVC_SUPPRESS_WARNING(5245) /* unreferenced function with internal linkage removed */
00303 \
00304     #define DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP DOCTEST_SUPPRESS_COMMON_WARNINGS_POP
00305 \
00306     #define DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
00307 \
00308     DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
00309 \
00310     DOCTEST_MSVC_SUPPRESS_WARNING(4548) /* before comma no effect; expected side - effect */
00311 \
00312     DOCTEST_MSVC_SUPPRESS_WARNING(4265) /* virtual functions, but destructor is not virtual */
00313 \
00314     DOCTEST_MSVC_SUPPRESS_WARNING(4986) /* exception specification does not match previous */
00315 \
00316     DOCTEST_MSVC_SUPPRESS_WARNING(4350) /* 'member1' called instead of 'member2' */
00317 \
00318     DOCTEST_MSVC_SUPPRESS_WARNING(4668) /* not defined as a preprocessor macro */
00319 \
00320     DOCTEST_MSVC_SUPPRESS_WARNING(4365) /* signed/unsigned mismatch */
00321 \
00322     DOCTEST_MSVC_SUPPRESS_WARNING(4774) /* format string not a string literal */
00323 \
00324     DOCTEST_MSVC_SUPPRESS_WARNING(4820) /* padding */
00325 \
00326     DOCTEST_MSVC_SUPPRESS_WARNING(4625) /* copy constructor was implicitly deleted */
00327 \
00328     DOCTEST_MSVC_SUPPRESS_WARNING(4626) /* assignment operator was implicitly deleted */
00329 \
00330     DOCTEST_MSVC_SUPPRESS_WARNING(5027) /* move assignment operator implicitly deleted */
00331 \
00332     DOCTEST_MSVC_SUPPRESS_WARNING(5026) /* move constructor was implicitly deleted */
00333 \
00334     DOCTEST_MSVC_SUPPRESS_WARNING(4623) /* default constructor was implicitly deleted */
00335 \
00336     DOCTEST_MSVC_SUPPRESS_WARNING(5039) /* pointer to pot. throwing function passed to extern C */
00337 \
00338     DOCTEST_MSVC_SUPPRESS_WARNING(5045) /* Spectre mitigation for memory load */

```

```

00304 \
DOCTEST_MSVC_SUPPRESS_WARNING(5105) /* macro producing 'defined' has undefined behavior */
00305 \
DOCTEST_MSVC_SUPPRESS_WARNING(4738) /* storing float result in memory, loss of performance */
00306 \
DOCTEST_MSVC_SUPPRESS_WARNING(5262) /* implicit fall-through */
00307
00308 #define DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END DOCTEST_MSVC_SUPPRESS_WARNING_POP
00309
00310 #endif // DOCTEST_PARTS_PUBLIC_WARNINGS
00311
00312 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
00313
00314 // =====
00315 // == FEATURE DETECTION ==
00316 // =====
00317
00318 #ifndef DOCTEST_PARTS_PUBLIC_CONFIG
00319 #define DOCTEST_PARTS_PUBLIC_CONFIG
00320
00321 #ifndef DOCTEST_PARTS_PUBLIC_PLATFORM
00322 #define DOCTEST_PARTS_PUBLIC_PLATFORM
00323
00324 #if defined(__APPLE__)
00325 // Apple detection taken from Catch2 codebase
00326 // For <TargetConditionals.h> information:
00327 //
https://github.com/swiftlang/swift-corelibs-foundation/blob/release/5.10/CoreFoundation/Base.subproj/SwiftRuntime/Target
00328 #include <TargetConditionals.h>
00329 #if (defined(TARGET_OS_MAC) && TARGET_OS_MAC == 1) || (defined(TARGET_OS_OSX) && TARGET_OS_OSX == 1)
00330 #define DOCTEST_PLATFORM_MAC
00331
00332 #elif defined(TARGET_OS_IPHONE) && TARGET_OS_IPHONE == 1
00333 #define DOCTEST_PLATFORM_IPHONE
00334 #endif
00335
00336 #elif defined(WIN32) || defined(_WIN32)
00337 #define DOCTEST_PLATFORM_WINDOWS
00338
00339 #elif defined(__wasi__)
00340 #define DOCTEST_PLATFORM_WASI
00341
00342 #else // defined(linux) || defined(__linux) // defined(__linux__)
00343 #define DOCTEST_PLATFORM_LINUX
00344 #endif // DOCTEST_PLATFORM
00345
00346 #endif // DOCTEST_PARTS_PUBLIC_PLATFORM
00347
00348 // general compiler feature support table: https://en.cppreference.com/w/cpp/compiler\_support
00349 // MSVC C++11 feature support table: https://msdn.microsoft.com/en-us/library/hh567368.aspx
00350 // GCC C++11 feature support table: https://gcc.gnu.org/projects/cxx-status.html
00351 // MSVC version table:
00352 // https://en.wikipedia.org/wiki/Microsoft\_Visual\_C%2B%2B#Internal\_version\_numbering
00353 // MSVC++ 14.3 (17) _MSC_VER == 1930 (Visual Studio 2022)
00354 // MSVC++ 14.2 (16) _MSC_VER == 1920 (Visual Studio 2019)
00355 // MSVC++ 14.1 (15) _MSC_VER == 1910 (Visual Studio 2017)
00356 // MSVC++ 14.0 _MSC_VER == 1900 (Visual Studio 2015)
00357 // MSVC++ 12.0 _MSC_VER == 1800 (Visual Studio 2013)
00358 // MSVC++ 11.0 _MSC_VER == 1700 (Visual Studio 2012)
00359 // MSVC++ 10.0 _MSC_VER == 1600 (Visual Studio 2010)
00360 // MSVC++ 9.0 _MSC_VER == 1500 (Visual Studio 2008)
00361 // MSVC++ 8.0 _MSC_VER == 1400 (Visual Studio 2005)
00362
00363 // Universal Windows Platform support
00364 #if defined(WINAPI_FAMILY) && (WINAPI_FAMILY == WINAPI_FAMILY_APP)
00365 #define DOCTEST_CONFIG_NO_WINDOWS_SEH
00366 #endif // WINAPI_FAMILY
00367 #if DOCTEST_MSVC && !defined(DOCTEST_CONFIG_WINDOWS_SEH)
00368 #define DOCTEST_CONFIG_WINDOWS_SEH
00369 #endif // MSVC
00370 #if defined(DOCTEST_CONFIG_NO_WINDOWS_SEH) && defined(DOCTEST_CONFIG_WINDOWS_SEH)
00371 #undef DOCTEST_CONFIG_WINDOWS_SEH
00372 #endif // DOCTEST_CONFIG_NO_WINDOWS_SEH
00373
00374 #if !defined(_WIN32) && !defined(__QNX__) && !defined(DOCTEST_CONFIG_POSIX_SIGNALS) &&
!defined(__EMSCRIPTEN__) && \
00375     !defined(__wasi__)
00376 #define DOCTEST_CONFIG_POSIX_SIGNALS
00377 #endif // _WIN32
00378 #if defined(DOCTEST_CONFIG_NO_POSIX_SIGNALS) && defined(DOCTEST_CONFIG_POSIX_SIGNALS)
00379 #undef DOCTEST_CONFIG_POSIX_SIGNALS
00380 #endif // DOCTEST_CONFIG_NO_POSIX_SIGNALS
00381
00382 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
00383 #if !defined(__cpp_exceptions) && !defined(__EXCEPTIONS) && !defined(_CPPUNWIND) || defined(__wasi__)
00384 #define DOCTEST_CONFIG_NO_EXCEPTIONS
00385 #endif // no exceptions

```

```

00386 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
00387
00388 #ifdef DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
00389 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
00390 #define DOCTEST_CONFIG_NO_EXCEPTIONS
00391 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
00392 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
00393
00394 #if defined(DOCTEST_CONFIG_NO_EXCEPTIONS) && !defined(DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS)
00395 #define DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
00396 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS && !DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
00397
00398 #ifdef __wasi__
00399 #define DOCTEST_CONFIG_NO_MULTITHREADING
00400 #endif
00401
00402 #if defined(DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN) && !defined(DOCTEST_CONFIG_IMPLEMENT)
00403 #define DOCTEST_CONFIG_IMPLEMENT
00404 #endif // DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
00405
00406 #if defined(_WIN32) || defined(__CYGWIN__)
00407 #if DOCTEST_MSVC
00408 #define DOCTEST_SYMBOL_EXPORT __declspec(dllexport)
00409 #define DOCTEST_SYMBOL_IMPORT __declspec(dllimport)
00410 #else // MSVC
00411 #define DOCTEST_SYMBOL_EXPORT __attribute__((dllexport))
00412 #define DOCTEST_SYMBOL_IMPORT __attribute__((dllimport))
00413 #endif // MSVC
00414 #else // _WIN32
00415 #define DOCTEST_SYMBOL_EXPORT __attribute__((visibility("default")))
00416 #define DOCTEST_SYMBOL_IMPORT
00417 #endif // _WIN32
00418
00419 #ifdef DOCTEST_CONFIG_IMPLEMENTATION_IN_DLL
00420 #ifdef DOCTEST_CONFIG_IMPLEMENT
00421 #define DOCTEST_INTERFACE DOCTEST_SYMBOL_EXPORT
00422 #else // DOCTEST_CONFIG_IMPLEMENT
00423 #define DOCTEST_INTERFACE DOCTEST_SYMBOL_IMPORT
00424 #endif // DOCTEST_CONFIG_IMPLEMENT
00425 #else // DOCTEST_CONFIG_IMPLEMENTATION_IN_DLL
00426 #define DOCTEST_INTERFACE
00427 #endif // DOCTEST_CONFIG_IMPLEMENTATION_IN_DLL
00428
00429 // needed for extern template instantiations
00430 // see https://github.com/fmtlib/fmt/issues/2228
00431 #if DOCTEST_MSVC
00432 #define DOCTEST_INTERFACE_DECL
00433 #define DOCTEST_INTERFACE_DEF DOCTEST_INTERFACE
00434 #else // DOCTEST_MSVC
00435 #define DOCTEST_INTERFACE_DECL DOCTEST_INTERFACE
00436 #define DOCTEST_INTERFACE_DEF
00437 #endif // DOCTEST_MSVC
00438
00439 #define DOCTEST_EMPTY
00440
00441 #if DOCTEST_MSVC
00442 #define DOCTEST_NOINLINE __declspec(noinline)
00443 #define DOCTEST_UNUSED
00444 #define DOCTEST_ALIGNMENT(x)
00445 #elif DOCTEST_CLANG && DOCTEST_CLANG < DOCTEST_COMPILER(3, 5, 0)
00446 #define DOCTEST_NOINLINE
00447 #define DOCTEST_UNUSED
00448 #define DOCTEST_ALIGNMENT(x)
00449 #else
00450 #define DOCTEST_NOINLINE __attribute__((noinline))
00451 #define DOCTEST_UNUSED __attribute__((unused))
00452 #define DOCTEST_ALIGNMENT(x) __attribute__((aligned(x)))
00453 #endif
00454
00455 #ifdef DOCTEST_CONFIG_NO_CONTRADICTING_INLINE
00456 #define DOCTEST_INLINE_NOINLINE inline
00457 #else
00458 #define DOCTEST_INLINE_NOINLINE inline DOCTEST_NOINLINE
00459 #endif
00460
00461 #ifndef DOCTEST_NORETURN
00462 #if DOCTEST_MSVC && (DOCTEST_MSVC < DOCTEST_COMPILER(19, 0, 0))
00463 #define DOCTEST_NORETURN
00464 #else // DOCTEST_MSVC
00465 #define DOCTEST_NORETURN [[noreturn]]
00466 #endif // DOCTEST_MSVC
00467 #endif // DOCTEST_NORETURN
00468
00469 #ifndef DOCTEST_NOEXCEPT
00470 #if DOCTEST_MSVC && (DOCTEST_MSVC < DOCTEST_COMPILER(19, 0, 0))
00471 #define DOCTEST_NOEXCEPT
00472 #else // DOCTEST_MSVC

```

```

00473 #define DOCTEST_NOEXCEPT noexcept
00474 #endif // DOCTEST_MSVC
00475 #endif // DOCTEST_NOEXCEPT
00476
00477 #ifndef DOCTEST_CONSTEXPR
00478 #if DOCTEST_MSVC && (DOCTEST_MSVC < DOCTEST_COMPILER(19, 0, 0))
00479 #define DOCTEST_CONSTEXPR const
00480 #define DOCTEST_CONSTEXPR_FUNC inline
00481 #else // DOCTEST_MSVC
00482 #define DOCTEST_CONSTEXPR constexpr
00483 #define DOCTEST_CONSTEXPR_FUNC constexpr
00484 #endif // DOCTEST_MSVC
00485 #endif // DOCTEST_CONSTEXPR
00486
00487 #ifndef DOCTEST_NO_SANITIZE_INTEGER
00488 #if DOCTEST_CLANG >= DOCTEST_COMPILER(3, 7, 0)
00489 #define DOCTEST_NO_SANITIZE_INTEGER __attribute__((no_sanitize("integer")))
00490 #else
00491 #define DOCTEST_NO_SANITIZE_INTEGER
00492 #endif
00493 #endif // DOCTEST_NO_SANITIZE_INTEGER
00494
00495 // this is kept here for backwards compatibility since the config option was changed
00496 #ifdef DOCTEST_CONFIG_USE_IOSFWD
00497 #ifndef DOCTEST_CONFIG_USE_STD_HEADERS
00498 #define DOCTEST_CONFIG_USE_STD_HEADERS
00499 #endif
00500 #endif // DOCTEST_CONFIG_USE_IOSFWD
00501
00502 // for clang - always include <version> or <ciso646> (which drags some std stuff)
00503 // because we want to check if we are using libc++ with the _LIBCPP_VERSION macro in
00504 // which case we don't want to forward declare stuff from std - for reference:
00505 // https://github.com/doctest/doctest/issues/126
00506 // https://github.com/doctest/doctest/issues/356
00507 #if DOCTEST_CLANG
00508 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
00509 #if DOCTEST_CPLUSPLUS >= 201703L && __has_include(<version>)
00510 #include <version>
00511 #else
00512 #include <ciso646>
00513 #endif
00514 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END
00515 #endif // clang
00516
00517 #ifdef _LIBCPP_VERSION
00518 #ifndef DOCTEST_CONFIG_USE_STD_HEADERS
00519 #define DOCTEST_CONFIG_USE_STD_HEADERS
00520 #endif
00521 #endif // _LIBCPP_VERSION
00522
00523 #ifdef DOCTEST_CONFIG_USE_STD_HEADERS
00524 #ifndef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00525 #define DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00526 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00527 #endif // DOCTEST_CONFIG_USE_STD_HEADERS
00528
00529 #if defined(__has_builtin)
00530 #define DOCTEST_HAS_BUILTIN(x) __has_builtin(x)
00531 #else
00532 #define DOCTEST_HAS_BUILTIN(x) 0
00533 #endif // __has_builtin
00534
00535 #endif // DOCTEST_PARTS_PUBLIC_CONFIG
00536
00537 // =====
00538 // == FEATURE DETECTION END ==
00539 // =====
00540 #ifndef DOCTEST_PARTS_PUBLIC_UTILITY
00541 #define DOCTEST_PARTS_PUBLIC_UTILITY
00542
00543
00544 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
00545
00546 #define DOCTEST_DECLARE_INTERFACE(name)
00547 \
00548 \
00549 \
00550 \
00551 \
00552 \
00553

```

```

00554 #define DOCTEST_DEFINE_INTERFACE(name) name::~name() = default;
00555
00556 #if !defined(DOCTEST_COUNTER)
00557 #if DOCTEST_CLANG >= DOCTEST_COMPILER(22, 0, 0)
00558 #define DOCTEST_COUNTER __LINE__
00559 #elif defined(__COUNTER__)
00560 #define DOCTEST_COUNTER __COUNTER__
00561 #else
00562 #define DOCTEST_COUNTER __LINE__
00563 #endif
00564 #endif // defined(DOCTEST_COUNTER)
00565
00566 // internal macros for string concatenation and anonymous variable name generation
00567 #define DOCTEST_CAT_IMPL(s1, s2) s1##s2
00568 #define DOCTEST_CAT(s1, s2) DOCTEST_CAT_IMPL(s1, s2)
00569 #define DOCTEST_ANONYMOUS(x) DOCTEST_CAT(x, DOCTEST_COUNTER)
00570
00571 #ifndef DOCTEST_CONFIG_ASSERTION_PARAMETERS_BY_VALUE
00572 #define DOCTEST_REF_WRAP(x) x &
00573 #else // DOCTEST_CONFIG_ASSERTION_PARAMETERS_BY_VALUE
00574 #define DOCTEST_REF_WRAP(x) x
00575 #endif // DOCTEST_CONFIG_ASSERTION_PARAMETERS_BY_VALUE
00576
00577 namespace doctest {
00578 namespace detail {
00579 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wunused-function")
00580 static DOCTEST_CONSTEXPR int consume(const int *, int) noexcept {
00581     return 0;
00582 }
00583 DOCTEST_CLANG_SUPPRESS_WARNING_POP
00584 } // namespace detail
00585 } // namespace doctest
00586
00587 #define DOCTEST_GLOBAL_NO_WARNINGS(var, ...)
00588 \
00589 \
00590 \
00591 \
00592 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
00593
00594 #endif // DOCTEST_PARTS_PUBLIC_UTILITY
00595 #ifndef DOCTEST_PARTS_PUBLIC_DEBUGGER
00596 #define DOCTEST_PARTS_PUBLIC_DEBUGGER
00597
00598
00599 #ifndef DOCTEST_BREAK_INTO_DEBUGGER
00600
00601 // should probably take a look at https://github.com/scottt/debugbreak
00602 #if DOCTEST_CLANG && DOCTEST_HAS_BUILTIN(__builtin_debugtrap)
00603 #define DOCTEST_BREAK_INTO_DEBUGGER() __builtin_debugtrap()
00604
00605 #elif defined(DOCTEST_PLATFORM_LINUX)
00606 #if defined(__GNUC__) && (defined(__i386) || defined(__x86_64))
00607 // Break at the location of the failing check if possible
00608 #define DOCTEST_BREAK_INTO_DEBUGGER() __asm__("int $3\n" ::) // NOLINT(hicpp-no-assembler)
00609 #else
00610 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
00611 #include <signal.h>
00612 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END
00613 #define DOCTEST_BREAK_INTO_DEBUGGER() raise(SIGTRAP)
00614 #endif
00615
00616 #elif defined(DOCTEST_PLATFORM_MAC)
00617 #if defined(__x86_64) || defined(__x86_64__) || defined(__amd64__) || defined(__i386)
00618 #define DOCTEST_BREAK_INTO_DEBUGGER() __asm__("int $3\n" ::) // NOLINT(hicpp-no-assembler)
00619 #elif defined(__ppc__) || defined(__ppc64__)
00620 // https://www.cocoawithlove.com/2008/03/break-into-debugger.html
00621 #define DOCTEST_BREAK_INTO_DEBUGGER() /* NOLINTNEXTLINE(hicpp-no-assembler) */
00622 \
00623 \
00624 \
00625 \
00626 \
00627 \
00628 \
00629 \
00630 \
00631 \
00632 \
00633 \
00634 \
00635 \
00636 \
00637 \
00638 \
00639 \
00640 \
00641 \
00642 \
00643 \
00644 \
00645 \
00646 \
00647 \
00648 \
00649 \
00650 \
00651 \
00652 \
00653 \
00654 \
00655 \
00656 \
00657 \
00658 \
00659 \
00660 \
00661 \
00662 \
00663 \
00664 \
00665 \
00666 \
00667 \
00668 \
00669 \
00670 \
00671 \
00672 \
00673 \
00674 \
00675 \
00676 \
00677 \
00678 \
00679 \
00680 \
00681 \
00682 \
00683 \
00684 \
00685 \
00686 \
00687 \
00688 \
00689 \
00690 \
00691 \
00692 \
00693 \
00694 \
00695 \
00696 \
00697 \
00698 \
00699 \
00700 \
00701 \
00702 \
00703 \
00704 \
00705 \
00706 \
00707 \
00708 \
00709 \
00710 \
00711 \
00712 \
00713 \
00714 \
00715 \
00716 \
00717 \
00718 \
00719 \
00720 \
00721 \
00722 \
00723 \
00724 \
00725 \
00726 \
00727 \
00728 \
00729 \
00730 \
00731 \
00732 \
00733 \
00734 \
00735 \
00736 \
00737 \
00738 \
00739 \
00740 \
00741 \
00742 \
00743 \
00744 \
00745 \
00746 \
00747 \
00748 \
00749 \
00750 \
00751 \
00752 \
00753 \
00754 \
00755 \
00756 \
00757 \
00758 \
00759 \
00760 \
00761 \
00762 \
00763 \
00764 \
00765 \
00766 \
00767 \
00768 \
00769 \
00770 \
00771 \
00772 \
00773 \
00774 \
00775 \
00776 \
00777 \
00778 \
00779 \
00780 \
00781 \
00782 \
00783 \
00784 \
00785 \
00786 \
00787 \
00788 \
00789 \
00790 \
00791 \
00792 \
00793 \
00794 \
00795 \
00796 \
00797 \
00798 \
00799 \
00800 \
00801 \
00802 \
00803 \
00804 \
00805 \
00806 \
00807 \
00808 \
00809 \
00810 \
00811 \
00812 \
00813 \
00814 \
00815 \
00816 \
00817 \
00818 \
00819 \
00820 \
00821 \
00822 \
00823 \
00824 \
00825 \
00826 \
00827 \
00828 \
00829 \
00830 \
00831 \
00832 \
00833 \
00834 \
00835 \
00836 \
00837 \
00838 \
00839 \
00840 \
00841 \
00842 \
00843 \
00844 \
00845 \
00846 \
00847 \
00848 \
00849 \
00850 \
00851 \
00852 \
00853 \
00854 \
00855 \
00856 \
00857 \
00858 \
00859 \
00860 \
00861 \
00862 \
00863 \
00864 \
00865 \
00866 \
00867 \
00868 \
00869 \
00870 \
00871 \
00872 \
00873 \
00874 \
00875 \
00876 \
00877 \
00878 \
00879 \
00880 \
00881 \
00882 \
00883 \
00884 \
00885 \
00886 \
00887 \
00888 \
00889 \
00890 \
00891 \
00892 \
00893 \
00894 \
00895 \
00896 \
00897 \
00898 \
00899 \
00900 \
00901 \
00902 \
00903 \
00904 \
00905 \
00906 \
00907 \
00908 \
00909 \
00910 \
00911 \
00912 \
00913 \
00914 \
00915 \
00916 \
00917 \
00918 \
00919 \
00920 \
00921 \
00922 \
00923 \
00924 \
00925 \
00926 \
00927 \
00928 \
00929 \
00930 \
00931 \
00932 \
00933 \
00934 \
00935 \
00936 \
00937 \
00938 \
00939 \
00940 \
00941 \
00942 \
00943 \
00944 \
00945 \
00946 \
00947 \
00948 \
00949 \
00950 \
00951 \
00952 \
00953 \
00954 \
00955 \
00956 \
00957 \
00958 \
00959 \
00960 \
00961 \
00962 \
00963 \
00964 \
00965 \
00966 \
00967 \
00968 \
00969 \
00970 \
00971 \
00972 \
00973 \
00974 \
00975 \
00976 \
00977 \
00978 \
00979 \
00980 \
00981 \
00982 \
00983 \
00984 \
00985 \
00986 \
00987 \
00988 \
00989 \
00990 \
00991 \
00992 \
00993 \
00994 \
00995 \
00996 \
00997 \
00998 \
00999 \

```

```

00637 #endif // linux
00638 #endif // DOCTEST_BREAK_INTO_DEBUGGER
00639
00640 #ifndef DOCTEST_CONFIG_DISABLE
00641
00642 namespace doctest {
00643 namespace detail {
00644 DOCTEST_INTERFACE bool isDebuggerActive();
00645 } // namespace detail
00646 } // namespace doctest
00647
00648 #endif
00649
00650 #endif // DOCTEST_PARTS_PUBLIC_DEBUGGER
00651 #ifndef DOCTEST_PARTS_PUBLIC_STD_FWD
00652 #define DOCTEST_PARTS_PUBLIC_STD_FWD
00653
00654
00655 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
00656
00657 #ifdef DOCTEST_CONFIG_USE_STD_HEADERS
00658 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
00659 #include <cstdlib>
00660 #include <ostream>
00661 #include <istream>
00662 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END
00663 #else // DOCTEST_CONFIG_USE_STD_HEADERS
00664
00665 // Forward declaring 'X' in namespace std is not permitted by the C++ Standard.
00666 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4643)
00667
00668 // NOLINTBEGIN(bugprone-std-namespace-modification, cert-dcl58-cpp)
00669 namespace std {
00670 typedef decltype(nullptr) nullptr_t; // NOLINT(modernize-use-using)
00671 typedef decltype(sizeof(void*)) size_t; // NOLINT(modernize-use-using)
00672 template <class charT>
00673 struct char_traits;
00674 template <>
00675 struct char_traits<char>;
00676 template <class charT, class traits>
00677 class basic_ostream; // NOLINT(fuchsia-virtual-inheritance)
00678 typedef basic_ostream<char, char_traits<char>> ostream; // NOLINT(modernize-use-using)
00679 template <class traits>
00680 // NOLINTNEXTLINE
00681 basic_ostream<char, traits> &operator<<(basic_ostream<char, traits> &, const char *);
00682 template <class charT, class traits>
00683 class basic_istream;
00684 typedef basic_istream<char, char_traits<char>> istream; // NOLINT(modernize-use-using)
00685 template <class... Types>
00686 class tuple;
00687 #if DOCTEST_MSVC >= DOCTEST_COMPILER(19, 20, 0)
00688 // see this issue on why this is needed: https://github.com/doctest/doctest/issues/183
00689 template <class Ty>
00690 class allocator;
00691 template <class Elem, class Traits, class Alloc>
00692 class basic_string;
00693 using string = basic_string<char, char_traits<char>, allocator<char>;
00694 #endif // VS 2019
00695 } // namespace std
00696 // NOLINTEND(bugprone-std-namespace-modification, cert-dcl58-cpp)
00697
00698 DOCTEST_MSVC_SUPPRESS_WARNING_POP
00699
00700 #endif // DOCTEST_CONFIG_USE_STD_HEADERS
00701
00702 namespace doctest {
00703 using std::size_t;
00704 } // namespace doctest
00705
00706 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
00707
00708 #endif // DOCTEST_PARTS_PUBLIC_STD_FWD
00709 #ifndef DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS
00710 #define DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS
00711
00712
00713 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
00714
00715 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00716 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
00717 #include <type_traits>
00718 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END
00719 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00720
00721 namespace doctest {
00722 namespace detail {
00723 namespace types {

```

```

00724
00725 #ifndef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
00726 using namespace std;
00727 #else
00728 template <bool COND, typename T = void>
00729 struct enable_if {};
00730
00731 template <typename T>
00732 struct enable_if<true, T> {
00733     using type = T;
00734 };
00735
00736 struct true_type {
00737     static DOCTEST_CONSTEXPR bool value = true;
00738 };
00739
00740 struct false_type {
00741     static DOCTEST_CONSTEXPR bool value = false;
00742 };
00743
00744 template <typename T>
00745 struct remove_reference {
00746     using type = T;
00747 };
00748
00749 template <typename T>
00750 struct remove_reference<T &> {
00751     using type = T;
00752 };
00753
00754 template <typename T>
00755 struct remove_reference<T &&> {
00756     using type = T;
00757 };
00758
00759 template <typename T, typename U>
00760 struct is_same : false_type {};
00761
00762 template <typename T>
00763 struct is_same<T, T> : true_type {};
00764
00765 template <typename T>
00766 struct is_rvalue_reference : false_type {};
00767
00768 template <typename T>
00769 struct is_rvalue_reference<T &&> : true_type {};
00770
00771 template <typename T>
00772 struct remove_const {
00773     using type = T;
00774 };
00775
00776 template <typename T>
00777 struct remove_const<const T> {
00778     using type = T;
00779 };
00780
00781 // Compiler intrinsics
00782 template <typename T>
00783 struct is_enum {
00784     static DOCTEST_CONSTEXPR bool value = __is_enum(T);
00785 };
00786
00787 template <typename T>
00788 struct underlying_type {
00789     using type = __underlying_type(T);
00790 };
00791
00792 template <typename T>
00793 struct is_pointer : false_type {};
00794
00795 template <typename T>
00796 struct is_pointer<T *> : true_type {};
00797
00798 template <typename T>
00799 struct is_array : false_type {};
00800 // NOLINTNEXTLINE(*-avoid-c-arrays)
00801 template <typename T, size_t SIZE>
00802 struct is_array<T[SIZE]> : true_type {};
00803 #endif
00804
00805 #if !(defined(DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS) && (DOCTEST_CPLUSPLUS >= 201703L))
00806 template <typename... Unused>
00807 using void_t = void;
00808 #endif
00809
00810 } // namespace types

```

```

00811 } // namespace detail
00812 } // namespace doctest
00813
00814 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
00815
00816 #endif // DOCTEST_PARTS_PUBLIC_STD_TYPE_TRAITS
00817 #ifndef DOCTEST_PARTS_PUBLIC_STD_UTILITY
00818 #define DOCTEST_PARTS_PUBLIC_STD_UTILITY
00819
00820
00821 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
00822
00823 namespace doctest {
00824 namespace detail {
00825
00826 // <utility>
00827 template <typename T>
00828 T &&declval();
00829
00830 template <class T>
00831 DOCTEST_CONSTEXPR_FUNC T &&forward(typename types::remove_reference<T>::type &t) DOCTEST_NOEXCEPT {
00832     return static_cast<T &&>(t);
00833 }
00834
00835 template <class T>
00836 // NOLINTNEXTLINE(cppcoreguidelines-rvalue-reference-param-not-moved)
00837 DOCTEST_CONSTEXPR_FUNC T &&forward(typename types::remove_reference<T>::type &&t) DOCTEST_NOEXCEPT {
00838     return static_cast<T &&>(t);
00839 }
00840
00841 template <typename T>
00842 struct deferred_false : types::false_type {};
00843
00844 } // namespace detail
00845 } // namespace doctest
00846
00847 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
00848
00849 #endif // DOCTEST_PARTS_PUBLIC_STD_UTILITY
00850 #ifndef DOCTEST_PARTS_PUBLIC_STRING
00851 #define DOCTEST_PARTS_PUBLIC_STRING
00852
00853
00854 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
00855
00856 namespace doctest {
00857 #ifndef DOCTEST_CONFIG_STRING_SIZE_TYPE
00858 #define DOCTEST_CONFIG_STRING_SIZE_TYPE unsigned
00859 #endif
00860
00861 namespace detail {
00862
00863 template <typename T, typename Enable = void>
00864 struct is_std_string : types::false_type {};
00865
00866 template <typename T>
00867 struct is_std_string<
00868     T,
00869     typename types::enable_if<
00870         types::is_same<decltype(declval<const T &>().c_str()), const char *>::value &&
00871         types::is_same<decltype(declval<const T &>().size()), size_t>::value>::type> :
00872         types::true_type {};
00873 } // namespace detail
00874
00875 // A 24 byte string class (can be as small as 17 for x64 and 13 for x86) that can hold strings
00876 // with length of up to 23 chars on the stack before going on the heap -
00877 // the last byte of the buffer is used for:
00878 // - "is small" bit - the highest bit - if "0" then it is small - otherwise its "1" (128)
00879 // - if small - capacity left before going on the heap - using the lowest 5 bits
00880 // - if small - 2 bits are left unused - the second and third highest ones
00881 // - if small - acts as a null terminator if strlen() is 23 (24 including the null terminator)
00882 // and the "is small" bit remains "0" ("as well as the capacity left") so its OK
00883 // Idea taken from this lecture about the string implementation of facebook/folly - fbstring
00884 // https://www.youtube.com/watch?v=kPR8h4-qZdk
00885 // TODO:
00886 // - optimizations - like not deleting memory unnecessarily in operator= and etc.
00887 // - resize/reserve/clear
00888 // - replace
00889 // - back/front
00890 // - iterator stuff
00891 // - find & friends
00892 // - push_back/pop_back
00893 // - assign/insert/erase
00894 // - relational operators as free functions - taking const char* as one of the params
00895 class DOCTEST_INTERFACE String {
00896 public:

```



```

00897     using size_type = DOCTEST_CONFIG_STRING_SIZE_TYPE;
00898
00899 private:
00900     static DOCTEST_CONSTEXPR size_type len = 24;
00901     static DOCTEST_CONSTEXPR size_type last = len - 1;
00902
00903     struct view // len should be more than sizeof(view) - because of the final byte for flags
00904     {
00905         char *ptr;
00906         size_type size;
00907         size_type capacity;
00908     };
00909
00910     union {
00911         char buf[len]; // NOLINT(*-avoid-c-arrays)
00912         view data;
00913     };
00914
00915     char *allocate(size_type sz);
00916
00917     bool isOnStack() const noexcept {
00918         // NOLINTNEXTLINE(cppcoreguidelines-pro-type-union-access)
00919         return (buf[last] & 128) == 0;
00920     }
00921
00922     void setOnHeap() noexcept;
00923     void setLast(size_type in = last) noexcept;
00924     void setSize(size_type sz) noexcept;
00925
00926     void copy(const String &other);
00927
00928 public:
00929     static DOCTEST_CONSTEXPR size_type npos = static_cast<size_type>(-1);
00930
00931     String() noexcept;
00932     ~String();
00933
00934     String(const char *in);
00935     String(const char *in, size_type in_size);
00936
00937     String(std::istream &in, size_type in_size);
00938
00939     template <typename T, typename detail::types::enable_if<detail::is_std_string<T>::value,
00940 bool>::type = true>
00941     String(const T &in)
00942         : String(in.c_str(), static_cast<size_type>(in.size())) {}
00943
00944     String(const String &other);
00945     String &operator=(const String &other);
00946
00947     String &operator+=(const String &other);
00948
00949     String(String &&other) noexcept;
00950     String &operator=(String &&other) noexcept;
00951
00952     char operator[](size_type i) const;
00953     char &operator[](size_type i);
00954
00955     // the only functions I'm willing to leave in the interface - available for inlining
00956     const char *c_str() const {
00957         return const_cast<String *>(this)->c_str(); // NOLINT
00958     }
00959
00960     // NOLINTBEGIN(cppcoreguidelines-pro-type-union-access)
00961     char *c_str() {
00962         if (isOnStack()) {
00963             // NOLINTNEXTLINE(cppcoreguidelines-pro-type-reinterpret-cast)
00964             return reinterpret_cast<char *>(buf);
00965         }
00966         return data.ptr;
00967     }
00968     // NOLINTEND(cppcoreguidelines-pro-type-union-access)
00969
00970     size_type size() const;
00971     size_type capacity() const;
00972
00973     String substr(size_type pos, size_type cnt = npos) &&;
00974     String substr(size_type pos, size_type cnt = npos) const &;
00975
00976     size_type find(char ch, size_type pos = 0) const;
00977     size_type rfind(char ch, size_type pos = npos) const;
00978
00979     int compare(const char *other, bool no_case = false) const;
00980     int compare(const String &other, bool no_case = false) const;
00981
00982     friend DOCTEST_INTERFACE std::ostream &operator<<(std::ostream &s, const String &in);
00983 };

```

```

00983
00984 DOCTEST_INTERFACE String operator+(const String &lhs, const String &rhs);
00985
00986 DOCTEST_INTERFACE bool operator==(const String &lhs, const String &rhs);
00987 DOCTEST_INTERFACE bool operator!=(const String &lhs, const String &rhs);
00988 DOCTEST_INTERFACE bool operator<(const String &lhs, const String &rhs);
00989 DOCTEST_INTERFACE bool operator>(const String &lhs, const String &rhs);
00990 DOCTEST_INTERFACE bool operator<=(const String &lhs, const String &rhs);
00991 DOCTEST_INTERFACE bool operator>=(const String &lhs, const String &rhs);
00992
00993 namespace detail {
00994
00995 // MSVS 2015 :(
00996 #if !DOCTEST_CLANG && defined(_MSC_VER) && _MSC_VER <= 1900
00997 template <typename T, typename = void>
00998 struct has_global_insertion_operator : types::false_type {};
00999
01000 template <typename T>
01001 struct has_global_insertion_operator<T, decltype(::operator<<(declval<std::ostream &>()), declval<const
T &>()), void())>
01002     : types::true_type {};
01003
01004 template <typename T, typename = void>
01005 struct has_insertion_operator {
01006     static DOCTEST_CONSTEXPR bool value = has_global_insertion_operator<T>::value;
01007 };
01008
01009 template <typename T, bool global>
01010 struct insert_hack;
01011
01012 template <typename T>
01013 struct insert_hack<T, true> {
01014     static void insert(std::ostream &os, const T &t) {
01015         ::operator<<(os, t);
01016     }
01017 };
01018
01019 template <typename T>
01020 struct insert_hack<T, false> {
01021     static void insert(std::ostream &os, const T &t) {
01022         operator<<(os, t);
01023     }
01024 };
01025
01026 template <typename T>
01027 using insert_hack_t = insert_hack<T, has_global_insertion_operator<T>::value>;
01028 #else
01029 template <typename T, typename = void>
01030 struct has_insertion_operator : types::false_type {};
01031 #endif
01032
01033 template <typename T>
01034 struct has_insertion_operator<T, decltype(operator<<(declval<std::ostream &>()), declval<const T &>()),
void())>
01035     : types::true_type {};
01036
01037 template <typename T>
01038 struct should_stringify_as_underlying_type {
01039     static DOCTEST_CONSTEXPR bool value =
01040         detail::types::is_enum<T>::value && !doctest::detail::has_insertion_operator<T>::value;
01041 };
01042
01043 template <typename T, typename = void>
01044 struct is_pair : types::false_type {};
01045
01046 template <typename T>
01047 struct is_pair<
01048     T,
01049     types::void_t<
01050         typename T::first_type,
01051         typename T::second_type,
01052         decltype(declval<T>().first),
01053         decltype(declval<T>().second)> > : types::true_type {};
01054
01055 template <typename T, typename = void>
01056 struct is_container : types::false_type {};
01057
01058 template <typename T>
01059 struct is_container<
01060     T,
01061     types::void_t<
01062         typename T::value_type,
01063         typename T::iterator,
01064         decltype(declval<T>().begin()),
01065         decltype(declval<T>().end())> > : types::true_type {};
01066
01067 DOCTEST_INTERFACE std::ostream *tlssPush();

```

```

01068 DOCTEST_INTERFACE String tlssPop();
01069
01070 template <bool C>
01071 struct StringMakerBase {
01072     template <typename T>
01073     static String convert(const DOCTEST_REF_WRAP(T)) {
01074 #ifdef DOCTEST_CONFIG_REQUIRE_STRINGIFICATION_FOR_ALL_USED_TYPES
01075         static_assert(deferred_false<T>::value, "No stringification detected for type T. See string
conversion manual");
01076 #endif
01077         return "{?}";
01078     }
01079 };
01080
01081 template <typename T, typename Enable = void>
01082 struct filldata;
01083
01084 template <typename T>
01085 void filloss(std::ostream *stream, const T &in) {
01086     filldata<T>::fill(stream, in);
01087 }
01088
01089 template <typename T, size_t N>
01090 void filloss(std::ostream *stream, const T (&in)[N]) { // NOLINT(*-avoid-c-arrays)
01091     // T[N], T(&)[N], T(&&)[N] have same behaviour.
01092     // Hence remove reference.
01093     filloss<typename types::remove_reference<decltype(in)>::type>(stream, in);
01094 }
01095
01096 template <typename T>
01097 String toStream(const T &in) {
01098     std::ostream *stream = tlssPush();
01099     filloss(stream, in);
01100     return tlssPop();
01101 }
01102
01103 template <>
01104 struct StringMakerBase<true> {
01105     template <typename T>
01106     static String convert(const DOCTEST_REF_WRAP(T) in) {
01107         return toStream(in);
01108     }
01109 };
01110
01111 } // namespace detail
01112
01113 template <typename T>
01114 struct StringMaker
01115     : public detail::StringMakerBase<
01116         detail::has_insertion_operator<T>::value || detail::types::is_pointer<T>::value ||
01117         detail::types::is_array<T>::value || detail::is_pair<T>::value ||
01118         detail::is_container<T>::value> {}
01119
01119 #ifndef DOCTEST_STRINGIFY
01120 #ifdef DOCTEST_CONFIG_DOUBLE_STRINGIFY
01121 #define DOCTEST_STRINGIFY(...) toString(toString(__VA_ARGS__))
01122 #else
01123 #define DOCTEST_STRINGIFY(...) toString(__VA_ARGS__)
01124 #endif
01125 #endif
01126
01127 template <typename T>
01128 String toString() {
01129     #if DOCTEST_CLANG == 0 && DOCTEST_GCC == 0 && DOCTEST_ICC == 0
01130         String ret = __FUNCSIG__; // class doctest::String __cdecl doctest::toString<TYPE>(void)
01131         String::size_type beginPos = ret.find('<');
01132         return ret.substr(beginPos + 1, ret.size() - beginPos -
static_cast<String::size_type>(sizeof(">(void)")));
01133     #else
01134         const String ret = __PRETTY_FUNCTION__; // doctest::String toString() [with T = TYPE]
01135         const String::size_type begin = ret.find('=') + 2;
01136         return ret.substr(begin, ret.size() - begin - 1);
01137     #endif // Compiler
01138 }
01139
01140 template <
01141     typename T,
01142     typename detail::types::enable_if<!detail::should_stringify_as_underlying_type<T>::value,
bool>::type = true>
01143 String toString(const DOCTEST_REF_WRAP(T) value) {
01144     return StringMaker<T>::convert(value);
01145 }
01146
01147 // NOLINTNEXTLINE(cppcoreguidelines-rvalue-reference-param-not-moved)
01148 inline String &&toString(String &&in) {
01149     return static_cast<String &&>(in);
01150 }

```

```

01151
01152 #ifndef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01153 DOCTEST_INTERFACE String toString(const char *in);
01154 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01155
01156 #if DOCTEST_MSVC >= DOCTEST_COMPILER(19, 20, 0)
01157 // see this issue on why this is needed: https://github.com/doctest/doctest/issues/183
01158 DOCTEST_INTERFACE String toString(const std::string &in);
01159 #endif // VS 2019
01160
01161 DOCTEST_INTERFACE String toString(const String &in);
01162
01163 DOCTEST_INTERFACE String toString(std::nullptr_t);
01164
01165 DOCTEST_INTERFACE String toString(bool in);
01166
01167 DOCTEST_INTERFACE String toString(float in);
01168 DOCTEST_INTERFACE String toString(double in);
01169 DOCTEST_INTERFACE String toString(double long in);
01170
01171 DOCTEST_INTERFACE String toString(char in);
01172 DOCTEST_INTERFACE String toString(char signed in);
01173 DOCTEST_INTERFACE String toString(char unsigned in);
01174 DOCTEST_INTERFACE String toString(short in);
01175 DOCTEST_INTERFACE String toString(short unsigned in);
01176 DOCTEST_INTERFACE String toString(signed in);
01177 DOCTEST_INTERFACE String toString(unsigned in);
01178 DOCTEST_INTERFACE String toString(long in);
01179 DOCTEST_INTERFACE String toString(long unsigned in);
01180 DOCTEST_INTERFACE String toString(long long in);
01181 DOCTEST_INTERFACE String toString(long long unsigned in);
01182
01183 template <
01184     typename T,
01185     typename detail::types::enable_if<detail::should_stringify_as_underlying_type<T>::value,
01186     bool>::type = true>
01187 String toString(const DOCTEST_REF_WRAP(T) value) {
01188     using UT = typename detail::types::underlying_type<T>::type;
01189     return (DOCTEST_STRINGIFY(static_cast<UT>(value)));
01190 }
01191
01192 namespace detail {
01193 template <typename T, typename Enable>
01194 struct filldata {
01195     static void fill(std::ostream *stream, const T &in) {
01196         #if defined(_MSC_VER) && _MSC_VER <= 1900
01197             insert_hack_t<T>::insert(*stream, in);
01198         #else
01199             operator<<(*stream, in);
01200         #endif // _MSV_VER
01201     };
01202
01203     DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4866)
01204     // NOLINTBEGIN(*-avoid-c-arrays)
01205     template <typename T, size_t N>
01206     struct filldata<T[N]> {
01207         static void fill(std::ostream *stream, const T (&in)[N]) {
01208             *stream << "[";
01209             for (size_t i = 0; i < N; i++) {
01210                 if (i != 0) {
01211                     *stream << ", ";
01212                 }
01213                 *stream << (DOCTEST_STRINGIFY(in[i]));
01214             }
01215             *stream << "]";
01216         }
01217     };
01218     // NOLINTEND(*-avoid-c-arrays)
01219     DOCTEST_MSVC_SUPPRESS_WARNING_POP
01220
01221     // Specialized since we don't want the terminating null byte!
01222     // NOLINTBEGIN(*-avoid-c-arrays)
01223     template <size_t N>
01224     struct filldata<const char[N]> {
01225         static void fill(std::ostream *stream, const char (&in)[N]) {
01226             *stream << String(in, in[N - 1] ? N : N - 1);
01227         } // NOLINT(clang-analyzer-cplusplus.NewDeleteLeaks)
01228     };
01229     // NOLINTEND(*-avoid-c-arrays)
01230
01231     template <>
01232     struct filldata<const void *> {
01233         DOCTEST_INTERFACE static void fill(std::ostream *stream, const void *in);
01234     };
01235
01236     template <>

```

```

01237 struct filldata<const volatile void *> {
01238     DOCTEST_INTERFACE static void fill(std::ostream *stream, const volatile void *in);
01239 };
01240
01241 template <typename T>
01242 struct filldata<T *> {
01243     DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4180)
01244     static void fill(std::ostream *stream, const T *in) {
01245         DOCTEST_MSVC_SUPPRESS_WARNING_POP
01246         DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wmicrosoft-cast")
01247         filldata<const volatile void *>::fill(
01248             stream,
01249             #if DOCTEST_GCC == 0 || DOCTEST_GCC >= DOCTEST_COMPILER(4, 9, 0)
01250                 // NOLINTNEXTLINE(cppcoreguidelines-pro-type-reinterpret-cast)
01251                 reinterpret_cast<const volatile void *>(in)
01252             #else
01253                 // NOLINTNEXTLINE(cppcoreguidelines-pro-type-reinterpret-cast)
01254                 reinterpret_cast<const volatile void *const *>(&in)
01255             #endif // DOCTEST_GCC
01256         );
01257         DOCTEST_CLANG_SUPPRESS_WARNING_POP
01258     }
01259 };
01260
01261 template <typename T>
01262 struct filldata<
01263     T,
01264     typename detail::types::enable_if<!detail::has_insertion_operator<T>::value &&
01265     detail::is_pair<T>::value>::type> {
01266     static void fill(std::ostream *stream, const T &in) {
01267         *stream << "{" << DOCTEST_STRINGIFY(in.first) << ", " << DOCTEST_STRINGIFY(in.second) << "}";
01268     };
01269 };
01270
01271 template <typename T>
01272 struct filldata<
01273     T,
01274     typename detail::types::enable_if<
01275         !detail::has_insertion_operator<T>::value && detail::is_container<T>::value>::type> {
01276     static void fill(std::ostream *stream, const DOCTEST_REF_WRAP(T) in) {
01277         *stream << "{";
01278         for (auto it = in.begin(); it != in.end(); ++it) {
01279             if (it != in.begin()) {
01280                 *stream << ", ";
01281             }
01282             *stream << DOCTEST_STRINGIFY(*it);
01283         }
01284         *stream << "}";
01285     };
01286 };
01287 #ifndef DOCTEST_CONFIG_DISABLE
01288 template <typename L, typename R>
01289 String stringifyBinaryExpr(const DOCTEST_REF_WRAP(L) lhs, const char *op, const DOCTEST_REF_WRAP(R)
01290     rhs) {
01291     return (DOCTEST_STRINGIFY(lhs)) + op + (DOCTEST_STRINGIFY(rhs));
01292 }
01293 #endif // DOCTEST_CONFIG_DISABLE
01294 } // namespace detail
01295 } // namespace doctest
01296
01297 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01298
01299 #endif // DOCTEST_PARTS_PUBLIC_STRING
01300 #ifndef DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS
01301 #define DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS
01302
01303
01304 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01305
01306 namespace doctest {
01307
01308 class DOCTEST_INTERFACE Contains {
01309 public:
01310     explicit Contains(const String &string);
01311
01312     bool checkWith(const String &other) const;
01313
01314     String string;
01315 };
01316
01317 DOCTEST_INTERFACE String toString(const Contains &in);
01318
01319 DOCTEST_INTERFACE bool operator==(const String &lhs, const Contains &rhs);
01320 DOCTEST_INTERFACE bool operator==(const Contains &lhs, const String &rhs);
01321 DOCTEST_INTERFACE bool operator!=(const String &lhs, const Contains &rhs);

```

```

01322 DOCTEST_INTERFACE bool operator!=(const Contains &lhs, const String &rhs);
01323
01324 } // namespace doctest
01325
01326 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01327
01328 #endif // DOCTEST_PARTS_PUBLIC_MATCHERS_CONTAINS
01329 #ifndef DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX
01330 #define DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX
01331
01332 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01333
01334 namespace doctest {
01335
01336 struct DOCTEST_INTERFACE Approx {
01337     Approx(double value);
01338
01339     Approx operator()(double value) const;
01340
01341 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01342     template <typename T>
01343     explicit Approx(
01344         const T &value,
01345         typename detail::types::enable_if<std::is_constructible<double, T::value>::type * =
01346             static_cast<T *>(nullptr)
01347     ) {
01348         *this = static_cast<double>(value);
01349     }
01350 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01351
01352     Approx &epsilon(double newEpsilon);
01353
01354 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01355     template <typename T>
01356     typename std::enable_if<std::is_constructible<double, T::value, Approx &>::type epsilon(const T
01357         &newEpsilon) {
01358         m_epsilon = static_cast<double>(newEpsilon);
01359         return *this;
01360     }
01361 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01362
01363     Approx &scale(double newScale);
01364
01365 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01366     template <typename T>
01367     typename std::enable_if<std::is_constructible<double, T::value, Approx &>::type scale(const T
01368         &newScale) {
01369         m_scale = static_cast<double>(newScale);
01370         return *this;
01371     }
01372 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01373
01374 // clang-format off
01375 DOCTEST_INTERFACE friend bool operator==(double lhs, const Approx &rhs);
01376 DOCTEST_INTERFACE friend bool operator==(const Approx &lhs, double rhs);
01377 DOCTEST_INTERFACE friend bool operator!=(double lhs, const Approx &rhs);
01378 DOCTEST_INTERFACE friend bool operator!=(const Approx &lhs, double rhs);
01379 DOCTEST_INTERFACE friend bool operator<=(double lhs, const Approx &rhs);
01380 DOCTEST_INTERFACE friend bool operator<=(const Approx &lhs, double rhs);
01381 DOCTEST_INTERFACE friend bool operator>=(double lhs, const Approx &rhs);
01382 DOCTEST_INTERFACE friend bool operator>=(const Approx &lhs, double rhs);
01383 DOCTEST_INTERFACE friend bool operator<(double lhs, const Approx &rhs);
01384 DOCTEST_INTERFACE friend bool operator<(const Approx &lhs, double rhs);
01385 DOCTEST_INTERFACE friend bool operator>(double lhs, const Approx &rhs);
01386 DOCTEST_INTERFACE friend bool operator>(const Approx &lhs, double rhs);
01387 // clang-format on
01388
01389 #ifdef DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01390 #define DOCTEST_APPROX_PREFIX
01391
01392     template <typename T>
01393
01394     friend typename std::enable_if<std::is_constructible<double, T::value, bool>::type
01395
01396     // clang-format off
01397     DOCTEST_APPROX_PREFIX operator==(const T &lhs, const Approx &rhs) { return
01398 operator==(static_cast<double>(lhs), rhs); }
01399 DOCTEST_APPROX_PREFIX operator==(const Approx &lhs, const T &rhs) { return operator==(rhs, lhs); }
01400 DOCTEST_APPROX_PREFIX operator!=(const T &lhs, const Approx &rhs) { return !operator==(lhs, rhs); }
01401 DOCTEST_APPROX_PREFIX operator!=(const Approx &lhs, const T &rhs) { return !operator==(rhs, lhs); }
01402
01403 DOCTEST_APPROX_PREFIX operator<=(const T &lhs, const Approx &rhs) { return
01404 static_cast<double>(lhs) < rhs.m_value || lhs == rhs; }
01405 DOCTEST_APPROX_PREFIX operator<=(const Approx &lhs, const T &rhs) { return lhs.m_value <
01406 static_cast<double>(rhs) || lhs == rhs; }

```

```

01399     DOCTEST_APPROX_PREFIX operator>=(const T &lhs, const Approx &rhs) { return
static_cast<double>(lhs) > rhs.m_value || lhs == rhs; }
01400     DOCTEST_APPROX_PREFIX operator>=(const Approx &lhs, const T &rhs) { return lhs.m_value >
static_cast<double>(rhs) || lhs == rhs; }
01401     DOCTEST_APPROX_PREFIX operator<(const T &lhs, const Approx &rhs) { return
static_cast<double>(lhs) < rhs.m_value && lhs != rhs; }
01402     DOCTEST_APPROX_PREFIX operator<(const Approx &lhs, const T &rhs) { return lhs.m_value <
static_cast<double>(rhs) && lhs != rhs; }
01403     DOCTEST_APPROX_PREFIX operator>(const T &lhs, const Approx &rhs) { return
static_cast<double>(lhs) > rhs.m_value && lhs != rhs; }
01404     DOCTEST_APPROX_PREFIX operator>(const Approx &lhs, const T &rhs) { return lhs.m_value >
static_cast<double>(rhs) && lhs != rhs; }
01405     // clang-format off
01406 #undef DOCTEST_APPROX_PREFIX
01407 #endif // DOCTEST_CONFIG_INCLUDE_TYPE_TRAITS
01408
01409     double m_epsilon;
01410     double m_scale;
01411     double m_value;
01412 };
01413
01414 DOCTEST_INTERFACE String toString(const Approx &in);
01415
01416 } // namespace doctest
01417
01418 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01419
01420 #endif // DOCTEST_PARTS_PUBLIC_MATCHERS_APPROX
01421 #ifndef DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN
01422 #define DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN
01423
01424 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01425 namespace doctest {
01426
01427 template <typename F>
01428 struct DOCTEST_INTERFACE_DECL IsNaN {
01429     F value;
01430     bool flipped;
01431     IsNaN(F f, bool flip = false)
01432         : value(f), flipped(flip) {}
01433
01434     IsNaN<F> operator!() const {
01435         return {value, !flipped};
01436     }
01437
01438     operator bool() const;
01439 };
01440
01441 #ifndef __MINGW32__
01442 extern template struct DOCTEST_INTERFACE_DECL IsNaN<float>;
01443 extern template struct DOCTEST_INTERFACE_DECL IsNaN<double>;
01444 extern template struct DOCTEST_INTERFACE_DECL IsNaN<long double>;
01445 #endif
01446
01447 DOCTEST_INTERFACE String toString(IsNaN<float> in);
01448 DOCTEST_INTERFACE String toString(IsNaN<double> in);
01449 DOCTEST_INTERFACE String toString(IsNaN<double long> in);
01450
01451 } // namespace doctest
01452
01453 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01454
01455 #endif // DOCTEST_PARTS_PUBLIC_MATCHERS_IS_NAN
01456 #ifndef DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS
01457 #define DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS
01458
01459 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01460 namespace doctest {
01461 namespace detail {
01462 struct DOCTEST_INTERFACE TestCase;
01463 } // namespace detail
01464
01465 struct ContextOptions {
01466     std::ostream *cout = nullptr; // stdout stream
01467     String binary_name; // the test binary name
01468
01469     const detail::TestCase *currentTest = nullptr;
01470
01471     // == parameters from the command line
01472     String out; // output filename
01473     String order_by; // how tests should be ordered
01474     unsigned rand_seed; // the seed for rand ordering
01475
01476

```

```

01480     unsigned first; // the first (matching) test to be executed
01481     unsigned last;  // the last (matching) test to be executed
01482
01483     int abort_after; // stop tests after this many failed assertions
01484     int subcase_filter_levels; // apply the subcase filters for the first N levels
01485
01486     bool success; // include successful assertions in output
01487     bool case_sensitive; // if filtering should be case sensitive
01488     bool exit; // if the program should be exited after the tests are ran/whatever
01489     bool duration; // print the time duration of each test case
01490     bool minimal; // minimal console output (only test failures)
01491     bool quiet; // no console output
01492     bool no_throw; // to skip exceptions-related assertion macros
01493     bool no_exitcode; // if the framework should return 0 as the exitcode
01494     bool no_run; // to not run the tests at all (can be done with an "*" exclude)
01495     bool no_intro; // to not print the intro of the framework
01496     bool no_version; // to not print the version of the framework
01497     bool no_colors; // if output to the console should be colorized
01498     bool force_colors; // forces the use of colors even when a tty cannot be detected
01499     bool no_breaks; // to not break into the debugger
01500     bool no_skip; // don't skip test cases which are marked to be skipped
01501     bool gnu_file_line; // if line numbers should be surrounded with :x: and not (x):
01502     bool no_path_in_filenames; // if the path to files should be removed from the output
01503     String strip_file_prefixes; // remove the longest matching one of these prefixes from any file
    paths in the output
01504     bool no_line_numbers; // if source code line numbers should be omitted from the output
01505     bool no_debug_output; // no output in the debug console when a debugger is attached
01506     bool no_skipped_summary; // don't print "skipped" in the summary !!! UNDOCUMENTED !!!
01507     bool no_time_in_output; // omit any time/timestamps from output !!! UNDOCUMENTED !!!
01508
01509     bool help; // to print the help
01510     bool version; // to print the version
01511     bool count; // if only the count of matching tests is to be retrieved
01512     bool list_test_cases; // to list all tests matching the filters
01513     bool list_test_suites; // to list all suites matching the filters
01514     bool list_reporters; // lists all registered reporters
01515 };
01516
01517 DOCTEST_INTERFACE const ContextOptions *getContextOptions();
01518
01519 } // namespace doctest
01520
01521 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01522
01523 #endif // DOCTEST_PARTS_PUBLIC_CONTEXT_OPTIONS
01524 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_TYPE
01525 #define DOCTEST_PARTS_PUBLIC_ASSERT_TYPE
01526
01527 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01528
01529 namespace doctest {
01530 namespace assertType {
01531 enum Enum {
01532     // macro traits
01533
01534     is_warn = 1,
01535     is_check = 2 * is_warn,
01536     is_require = 2 * is_check,
01537
01538     is_normal = 2 * is_require,
01539     is_throws = 2 * is_normal,
01540     is_throws_as = 2 * is_throws,
01541     is_throws_with = 2 * is_throws_as,
01542     is_nothrow = 2 * is_throws_with,
01543
01544     is_false = 2 * is_nothrow,
01545     is_unary = 2 * is_false, // not checked anywhere - used just to distinguish the types
01546
01547     is_eq = 2 * is_unary,
01548     is_ne = 2 * is_eq,
01549
01550     is_lt = 2 * is_ne,
01551     is_gt = 2 * is_lt,
01552
01553     is_ge = 2 * is_gt,
01554     is_le = 2 * is_ge,
01555
01556     // macro types
01557
01558     DT_WARN = is_normal | is_warn,
01559     DT_CHECK = is_normal | is_check,
01560     DT_REQUIRE = is_normal | is_require,
01561
01562     DT_WARN_FALSE = is_normal | is_false | is_warn,
01563     DT_CHECK_FALSE = is_normal | is_false | is_check,
01564     DT_REQUIRE_FALSE = is_normal | is_false | is_require,

```



```

01566
01567     DT_WARN_THROWS = is_throws | is_warn,
01568     DT_CHECK_THROWS = is_throws | is_check,
01569     DT_REQUIRE_THROWS = is_throws | is_require,
01570
01571     DT_WARN_THROWS_AS = is_throws_as | is_warn,
01572     DT_CHECK_THROWS_AS = is_throws_as | is_check,
01573     DT_REQUIRE_THROWS_AS = is_throws_as | is_require,
01574
01575     DT_WARN_THROWS_WITH = is_throws_with | is_warn,
01576     DT_CHECK_THROWS_WITH = is_throws_with | is_check,
01577     DT_REQUIRE_THROWS_WITH = is_throws_with | is_require,
01578
01579     DT_WARN_THROWS_WITH_AS = is_throws_with | is_throws_as | is_warn,
01580     DT_CHECK_THROWS_WITH_AS = is_throws_with | is_throws_as | is_check,
01581     DT_REQUIRE_THROWS_WITH_AS = is_throws_with | is_throws_as | is_require,
01582
01583     DT_WARN_NOTHROW = is_nothrow | is_warn,
01584     DT_CHECK_NOTHROW = is_nothrow | is_check,
01585     DT_REQUIRE_NOTHROW = is_nothrow | is_require,
01586
01587     DT_WARN_EQ = is_normal | is_eq | is_warn,
01588     DT_CHECK_EQ = is_normal | is_eq | is_check,
01589     DT_REQUIRE_EQ = is_normal | is_eq | is_require,
01590
01591     DT_WARN_NE = is_normal | is_ne | is_warn,
01592     DT_CHECK_NE = is_normal | is_ne | is_check,
01593     DT_REQUIRE_NE = is_normal | is_ne | is_require,
01594
01595     DT_WARN_GT = is_normal | is_gt | is_warn,
01596     DT_CHECK_GT = is_normal | is_gt | is_check,
01597     DT_REQUIRE_GT = is_normal | is_gt | is_require,
01598
01599     DT_WARN_LT = is_normal | is_lt | is_warn,
01600     DT_CHECK_LT = is_normal | is_lt | is_check,
01601     DT_REQUIRE_LT = is_normal | is_lt | is_require,
01602
01603     DT_WARN_GE = is_normal | is_ge | is_warn,
01604     DT_CHECK_GE = is_normal | is_ge | is_check,
01605     DT_REQUIRE_GE = is_normal | is_ge | is_require,
01606
01607     DT_WARN_LE = is_normal | is_le | is_warn,
01608     DT_CHECK_LE = is_normal | is_le | is_check,
01609     DT_REQUIRE_LE = is_normal | is_le | is_require,
01610
01611     DT_WARN_UNARY = is_normal | is_unary | is_warn,
01612     DT_CHECK_UNARY = is_normal | is_unary | is_check,
01613     DT_REQUIRE_UNARY = is_normal | is_unary | is_require,
01614
01615     DT_WARN_UNARY_FALSE = is_normal | is_false | is_unary | is_warn,
01616     DT_CHECK_UNARY_FALSE = is_normal | is_false | is_unary | is_check,
01617     DT_REQUIRE_UNARY_FALSE = is_normal | is_false | is_unary | is_require,
01618 };
01619 } // namespace assertType
01620
01621 DOCTEST_INTERFACE const char *assertString(assertType::Enum at);
01622 DOCTEST_INTERFACE const char *failureString(assertType::Enum at);
01623
01624 } // namespace doctest
01625
01626 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01627
01628 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_TYPE
01629 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_DATA
01630 #define DOCTEST_PARTS_PUBLIC_ASSERT_DATA
01631
01632 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01633
01634 namespace doctest {
01635
01636     struct DOCTEST_INTERFACE TestCaseData;
01637
01638     struct DOCTEST_INTERFACE AssertData {
01639         // common - for all asserts
01640         const TestCaseData *m_test_case;
01641         assertType::Enum m_at;
01642         const char *m_file;
01643         int m_line;
01644         const char *m_expr;
01645         bool m_failed;
01646
01647         // exception-related - for all asserts
01648         bool m_threw;
01649         String m_exception;
01650
01651         // for normal asserts

```

```

01653     String m_decomp;
01654
01655     // for specific exception-related asserts
01656     bool m_threw_as;
01657     const char *m_exception_type;
01658
01659     class DOCTEST_INTERFACE StringContains {
01660     private:
01661         Contains content;
01662         bool isContains;
01663
01664     public:
01665         StringContains(const String &str)
01666             : content(str), isContains(false) {}
01667
01668         StringContains(Contains cntn)
01669             : content(static_cast<Contains &&>(cntn)), isContains(true) {}
01670
01671         bool check(const String &str) {
01672             return isContains ? (content == str) : (content.string == str);
01673         }
01674
01675         operator const String &() const {
01676             return content.string;
01677         }
01678
01679         const char *c_str() const {
01680             return content.string.c_str();
01681         }
01682     } m_exception_string;
01683
01684     AssertData(
01685         assertType::Enum at,
01686         const char *file,
01687         int line,
01688         const char *expr,
01689         const char *exception_type,
01690         const StringContains &exception_string
01691     );
01692 };
01693
01694 } // namespace doctest
01695
01696 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01697
01698 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_DATA
01699 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR
01700 #define DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR
01701
01702 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01703
01704 #ifndef DOCTEST_CONFIG_DISABLE
01705
01706 namespace doctest {
01707 namespace detail {
01708
01709 #ifndef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01710 // clang-format off
01711 template<class T> struct decay_array { using type = T; };
01712 template<class T, unsigned N> struct decay_array<T[N]> { using type = T *; };
01713 template<class T> struct decay_array<T[]> { using type = T *; };
01714
01715 template<class T> struct not_char_pointer { static DOCTEST_CONSTEXPR int value = 1; };
01716 template<> struct not_char_pointer<char *> { static DOCTEST_CONSTEXPR int value = 0; };
01717 template<> struct not_char_pointer<const char *> { static DOCTEST_CONSTEXPR int value = 0; };
01718
01719 template<class T> struct can_use_op : public not_char_pointer<typename decay_array<T>::type> {};
01720 // clang-format on
01721 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01722
01723 #ifndef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01724 #define DOCTEST_COMPARISON_RETURN_TYPE bool
01725 #else // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01726 // clang-format off
01727 #define DOCTEST_COMPARISON_RETURN_TYPE typename types::enable_if<can_use_op<L>::value ||
can_use_op<R>::value, bool>::type
01728
01729 inline bool eq(const char *lhs, const char *rhs) { return String(lhs) == String(rhs); }
01730 inline bool ne(const char *lhs, const char *rhs) { return String(lhs) != String(rhs); }
01731 inline bool lt(const char *lhs, const char *rhs) { return String(lhs) < String(rhs); }
01732 inline bool gt(const char *lhs, const char *rhs) { return String(lhs) > String(rhs); }
01733 inline bool le(const char *lhs, const char *rhs) { return String(lhs) <= String(rhs); }
01734 inline bool ge(const char *lhs, const char *rhs) { return String(lhs) >= String(rhs); }
01735 // clang-format on
01736 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01737
01738 #define DOCTEST_RELATIONAL_OP(name, op)

```

```

\
01739     template <typename L, typename R>
\
01740     DOCTEST_COMPARISON_RETURN_TYPE name(const DOCTEST_REF_WRAP(L) lhs, const DOCTEST_REF_WRAP(R) rhs)
{
\
01741     return lhs op rhs;
\
01742 }
01743
01744 DOCTEST_RELATIONAL_OP(eq, ==)
01745 DOCTEST_RELATIONAL_OP(ne, !=)
01746 DOCTEST_RELATIONAL_OP(lt, <)
01747 DOCTEST_RELATIONAL_OP(gt, >)
01748 DOCTEST_RELATIONAL_OP(le, <=)
01749 DOCTEST_RELATIONAL_OP(ge, >=)
01750
01751 #ifndef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01752 #define DOCTEST_CMP_EQ(l, r) l == r
01753 #define DOCTEST_CMP_NE(l, r) l != r
01754 #define DOCTEST_CMP_GT(l, r) l > r
01755 #define DOCTEST_CMP_LT(l, r) l < r
01756 #define DOCTEST_CMP_GE(l, r) l >= r
01757 #define DOCTEST_CMP_LE(l, r) l <= r
01758 #else // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01759 #define DOCTEST_CMP_EQ(l, r) eq(l, r)
01760 #define DOCTEST_CMP_NE(l, r) ne(l, r)
01761 #define DOCTEST_CMP_GT(l, r) gt(l, r)
01762 #define DOCTEST_CMP_LT(l, r) lt(l, r)
01763 #define DOCTEST_CMP_GE(l, r) ge(l, r)
01764 #define DOCTEST_CMP_LE(l, r) le(l, r)
01765 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
01766
01767 namespace binaryAssertComparison {
01768 enum Enum { eq = 0, ne, gt, lt, ge, le };
01769 } // namespace binaryAssertComparison
01770
01771 // clang-format off
01772 template <int, class L, class R> struct RelationalComparator { bool operator()(const
DOCTEST_REF_WRAP(L), const DOCTEST_REF_WRAP(R) ) const { return false; } };
01773
01774 #define DOCTEST_BINARY_RELATIONAL_OP(n, op) \
01775     template <class L, class R> struct RelationalComparator<n, L, R> { bool operator()(const
DOCTEST_REF_WRAP(L) lhs, const DOCTEST_REF_WRAP(R) rhs) const { return op(lhs, rhs); } };
01776 // clang-format on
01777
01778 DOCTEST_BINARY_RELATIONAL_OP(0, doctest::detail::eq)
01779 DOCTEST_BINARY_RELATIONAL_OP(1, doctest::detail::ne)
01780 DOCTEST_BINARY_RELATIONAL_OP(2, doctest::detail::gt)
01781 DOCTEST_BINARY_RELATIONAL_OP(3, doctest::detail::lt)
01782 DOCTEST_BINARY_RELATIONAL_OP(4, doctest::detail::ge)
01783 DOCTEST_BINARY_RELATIONAL_OP(5, doctest::detail::le)
01784
01785 } // namespace detail
01786 } // namespace doctest
01787
01788 #endif // DOCTEST_CONFIG_DISABLE
01789
01790 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01791
01792 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_COMPARATOR
01793 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_RESULT
01794 #define DOCTEST_PARTS_PUBLIC_ASSERT_RESULT
01795
01796 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01797
01798 #ifndef DOCTEST_CONFIG_DISABLE
01800
01801 namespace doctest {
01802 namespace detail {
01803
01804 // more checks could be added - like in Catch:
01805 // https://github.com/catchorg/Catch2/pull/1480/files
01806 // https://github.com/catchorg/Catch2/pull/1481/files
01807 #define DOCTEST_FORBIT_EXPRESSION(rt, op)
\
01808     template <typename R>
\
01809     rt &operator op(const R &) {
\
01810         static_assert(deferred_false<R>::value, "Expression Too Complex Please Rewrite As Binary
Comparison!");
\
01811         return *this;
\
01812     }
01813
01814 struct DOCTEST_INTERFACE Result { // NOLINT(*-member-init)

```

```

01815     bool m_passed;
01816     String m_decomp;
01817
01818     Result() = default; // TODO: Why do we need this? (To remove NOLINT)
01819     Result(bool passed, const String &decomposition = String());
01820
01821     // forbidding some expressions based on this table:
01822     // https://en.cppreference.com/w/cpp/language/operator_precedence
01823     DOCTEST_FORBIT_EXPRESSION(Result, &)
01824     DOCTEST_FORBIT_EXPRESSION(Result, ^)
01825     DOCTEST_FORBIT_EXPRESSION(Result, |)
01826     DOCTEST_FORBIT_EXPRESSION(Result, &&)
01827     DOCTEST_FORBIT_EXPRESSION(Result, ||)
01828     DOCTEST_FORBIT_EXPRESSION(Result, ==)
01829     DOCTEST_FORBIT_EXPRESSION(Result, !=)
01830     DOCTEST_FORBIT_EXPRESSION(Result, <)
01831     DOCTEST_FORBIT_EXPRESSION(Result, >)
01832     DOCTEST_FORBIT_EXPRESSION(Result, <=)
01833     DOCTEST_FORBIT_EXPRESSION(Result, >=)
01834     DOCTEST_FORBIT_EXPRESSION(Result, =)
01835     DOCTEST_FORBIT_EXPRESSION(Result, +=)
01836     DOCTEST_FORBIT_EXPRESSION(Result, -=)
01837     DOCTEST_FORBIT_EXPRESSION(Result, *=)
01838     DOCTEST_FORBIT_EXPRESSION(Result, /=)
01839     DOCTEST_FORBIT_EXPRESSION(Result, %=)
01840     DOCTEST_FORBIT_EXPRESSION(Result, <=)
01841     DOCTEST_FORBIT_EXPRESSION(Result, >=)
01842     DOCTEST_FORBIT_EXPRESSION(Result, &=)
01843     DOCTEST_FORBIT_EXPRESSION(Result, ^=)
01844     DOCTEST_FORBIT_EXPRESSION(Result, |=)
01845 };
01846
01847 struct DOCTEST_INTERFACE ResultBuilder : public AssertData {
01848     ResultBuilder(
01849         assertType::Enum at,
01850         const char *file,
01851         int line,
01852         const char *expr,
01853         const char *exception_type = "",
01854         const String &exception_string = ""
01855     );
01856
01857     ResultBuilder(
01858         assertType::Enum at,
01859         const char *file,
01860         int line,
01861         const char *expr,
01862         const char *exception_type,
01863         const Contains &exception_string
01864     );
01865
01866     void setResult(const Result &res);
01867
01868     template <int comparison, typename L, typename R>
01869     DOCTEST_NOINLINE bool binary_assert(const DOCTEST_REF_WRAP(L) lhs, const DOCTEST_REF_WRAP(R) rhs)
01870     {
01871         m_failed = !RelationalComparator<comparison, L, R>()(lhs, rhs);
01872         if (m_failed || getContextOptions()->success) {
01873             m_decomp = stringifyBinaryExpr(lhs, " ", rhs);
01874         }
01875         return !m_failed;
01876     }
01877
01878     template <typename L>
01879     DOCTEST_NOINLINE bool unary_assert(const DOCTEST_REF_WRAP(L) val) {
01880         m_failed = !val;
01881
01882         if (m_at & assertType::is_false) {
01883             m_failed = !m_failed;
01884         }
01885
01886         if (m_failed || getContextOptions()->success) {
01887             m_decomp = (DOCTEST_STRINGIFY(val));
01888         }
01889         return !m_failed;
01890     }
01891
01892     void translateException();
01893
01894     bool log();
01895     void react() const;
01896 };
01897
01898 } // namespace detail
01899 } // namespace doctest
01900

```

```

01901 #endif // DOCTEST_CONFIG_DISABLE
01902
01903 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
01904
01905 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_RESULT
01906 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION
01907 #define DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION
01908
01909
01910 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
01911
01912 #ifndef DOCTEST_CONFIG_DISABLE
01913
01914 namespace doctest {
01915 namespace detail {
01916
01917 #if DOCTEST_CLANG && DOCTEST_CLANG < DOCTEST_COMPILER(3, 6, 0)
01918 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wunused-comparison")
01919 #endif
01920
01921 // This will check if there is any way it could find a operator like member or friend and uses it.
01922 // If not it doesn't find the operator or if the operator at global scope is defined after
01923 // this template, the template won't be instantiated due to SFINAE. Once the template is not
01924 // instantiated it can look for global operator using normal conversions.
01925 #ifdef __NVCC__
01926 #define SFINAE_OP(ret, op) ret
01927 #else
01928 #define SFINAE_OP(ret, op) decltype((void)(doctest::detail::declval<L>() op
doctest::detail::declval<R>()), ret{})
01929 #endif
01930
01931 #define DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(op, op_str, op_macro)
01932 \
01933 \
01934 \
01935 \
01936 \
01937 \
01938 \
01939 \
01940 \
01941 \
01942 \
01943 \
01944 #ifndef DOCTEST_CONFIG_NO_COMPARISON_WARNING_SUPPRESSION
01945
01946 DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
01947 DOCTEST_CLANG_SUPPRESS_WARNING("-Wsign-conversion")
01948 DOCTEST_CLANG_SUPPRESS_WARNING("-Wsign-compare")
01949 // DOCTEST_CLANG_SUPPRESS_WARNING("-Wdouble-promotion")
01950 // DOCTEST_CLANG_SUPPRESS_WARNING("-Wconversion")
01951 // DOCTEST_CLANG_SUPPRESS_WARNING("-Wfloat-equal")
01952
01953 DOCTEST_GCC_SUPPRESS_WARNING_PUSH
01954 DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-conversion")
01955 DOCTEST_GCC_SUPPRESS_WARNING("-Wsign-compare")
01956 // DOCTEST_GCC_SUPPRESS_WARNING("-Wdouble-promotion")
01957 // DOCTEST_GCC_SUPPRESS_WARNING("-Wconversion")
01958 // DOCTEST_GCC_SUPPRESS_WARNING("-Wfloat-equal")
01959
01960 DOCTEST_MSVC_SUPPRESS_WARNING_PUSH
01961 // https://stackoverflow.com/questions/39479163 what's the difference between 4018 and 4389
01962 DOCTEST_MSVC_SUPPRESS_WARNING(4388) // signed/unsigned mismatch
01963 DOCTEST_MSVC_SUPPRESS_WARNING(4389) // 'operator' : signed/unsigned mismatch
01964 DOCTEST_MSVC_SUPPRESS_WARNING(4018) // 'expression' : signed/unsigned mismatch
01965 // DOCTEST_MSVC_SUPPRESS_WARNING(4805) // 'operation' : unsafe mix of type 'type' and type 'type' in
01966 // operation
01967
01968 #endif // DOCTEST_CONFIG_NO_COMPARISON_WARNING_SUPPRESSION
01969
01970 template <typename L>
01971 struct Expression_lhs {
01972     L lhs;
01973     assertType::Enum m_at;
01974
01975     // NOLINTNEXTLINE(cppcoreguidelines-rvalue-reference-param-not-moved)

```

```

01976     explicit Expression_lhs(L &&in, assertType::Enum at)
01977     : lhs(static_cast<L &&>(in)), m_at(at) {}
01978
01979     DOCTEST_NOINLINE operator Result() {
01980         // this is needed only for MSVC 2015
01981         DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4800) // 'int': forcing value to bool
01982         bool res = static_cast<bool>(lhs); //
01983     NOLINT(bugprone-non-zero-enum-to-bool-conversion)
01984         DOCTEST_MSVC_SUPPRESS_WARNING_POP
01985         if (m_at & assertType::is_false) {
01986             res = !res;
01987         }
01988         if (!res || getContextOptions()->success) {
01989             return {res, (DOCTEST_STRINGIFY(lhs))};
01990         }
01991         return {res};
01992     }
01993
01994     /* This is required for user-defined conversions from Expression_lhs to L */
01995     operator L() const {
01996         return lhs;
01997     }
01998
01999     // clang-format off
02000     DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(==, " == ", DOCTEST_CMP_EQ)
02001     DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(!=, " != ", DOCTEST_CMP_NE)
02002     DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(>, " > ", DOCTEST_CMP_GT)
02003     DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(<, " < ", DOCTEST_CMP_LT)
02004     DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(>=, " >= ", DOCTEST_CMP_GE)
02005     DOCTEST_DO_BINARY_EXPRESSION_COMPARISON(<=, " <= ", DOCTEST_CMP_LE)
02006     // clang-format on
02007
02008     // forbidding some expressions based on this table:
02009     // https://en.cppreference.com/w/cpp/language/operator_precedence
02010     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, &)
02011     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, ^)
02012     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, |)
02013     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, &&)
02014     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, ||)
02015     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, =)
02016     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, +=)
02017     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, -=)
02018     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, *=)
02019     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, /=)
02020     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, %=)
02021     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, <=)
02022     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, >=)
02023     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, &)
02024     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, ^)
02025     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, |=)
02026     // these 2 are unfortunate because they should be allowed - they have higher precedence over the
02027     // comparisons, but the ExpressionDecomposer class uses the left shift operator to capture the
02028     // left operand of the binary expression...
02029     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, <<)
02030     DOCTEST_FORBIT_EXPRESSION(Expression_lhs, >>)
02031 };
02032
02033 #ifndef DOCTEST_CONFIG_NO_COMPARISON_WARNING_SUPPRESSION
02034
02035 DOCTEST_CLANG_SUPPRESS_WARNING_POP
02036 DOCTEST_MSVC_SUPPRESS_WARNING_POP
02037 DOCTEST_GCC_SUPPRESS_WARNING_POP
02038
02039 #endif // DOCTEST_CONFIG_NO_COMPARISON_WARNING_SUPPRESSION
02040
02041 #if DOCTEST_CLANG && DOCTEST_CLANG < DOCTEST_COMPILER(3, 6, 0)
02042 DOCTEST_CLANG_SUPPRESS_WARNING_POP
02043 #endif
02044
02045 struct DOCTEST_INTERFACE ExpressionDecomposer {
02046     assertType::Enum m_at;
02047
02048     ExpressionDecomposer(assertType::Enum at);
02049
02050     // The right operator for capturing expressions is "<=" instead of "<<" (based on the operator
02051     // precedence table) but then there will be warnings from GCC about "-Wparentheses" and since
02052     // "Pragma()" is problematic this will stay for now...
02053     // https://github.com/catchorg/Catch2/issues/870
02054     // https://github.com/catchorg/Catch2/issues/565
02055     template <typename L>
02056     Expression_lhs<const L &&>
02057     operator<(const L &&operand) { // bitfields bind to universal ref but not const rvalue ref
02058         return Expression_lhs<const L &&>(static_cast<const L &&>(operand), m_at);
02059     }
02060
02061     template <

```

```

02062         typename L,
02063         typename types::enable_if<!doctest::detail::types::is_rvalue_reference<L>::value, void>::type
* = nullptr>
02064     Expression_lhs<const L &> operator<>(const L &operand) {
02065         return Expression_lhs<const L &>(operand, m_at);
02066     }
02067 };
02068
02069 } // namespace detail
02070 } // namespace doctest
02071
02072 #endif // DOCTEST_CONFIG_DISABLE
02073
02074 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02075
02076 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_EXPRESSION
02077 #ifndef DOCTEST_PARTS_PUBLIC_COLOR
02078 #define DOCTEST_PARTS_PUBLIC_COLOR
02079
02080 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02081
02082 namespace doctest {
02083     namespace Color {
02084         enum Enum { // NOLINT(cert-int09-c, readability-enum-initial-value)
02085             None = 0,
02086             White,
02087             Red,
02088             Green,
02089             Blue,
02090             Cyan,
02091             Yellow,
02092             Grey,
02093
02094             Bright = 0x10,
02095
02096             BrightRed = Bright | Red,
02097             BrightGreen = Bright | Green,
02098             LightGrey = Bright | Grey,
02099             BrightWhite = Bright | White
02100         };
02101     };
02102
02103     DOCTEST_INTERFACE std::ostream &operator<>(std::ostream &s, Color::Enum code);
02104 } // namespace Color
02105 } // namespace doctest
02106
02107 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02108
02109 #endif // DOCTEST_PARTS_PUBLIC_COLOR
02110 #ifndef DOCTEST_PARTS_PUBLIC_SUBCASE
02111 #define DOCTEST_PARTS_PUBLIC_SUBCASE
02112
02113 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02114
02115 namespace doctest {
02116     struct DOCTEST_INTERFACE SubcaseSignature {
02117         String m_name;
02118         const char *m_file;
02119         int m_line;
02120
02121         bool operator==(const SubcaseSignature &other) const;
02122         bool operator<(const SubcaseSignature &other) const;
02123     };
02124
02125 #ifndef DOCTEST_CONFIG_DISABLE
02126     namespace detail {
02127         struct DOCTEST_INTERFACE Subcase {
02128             SubcaseSignature m_signature;
02129             bool m_entered = false;
02130
02131             Subcase(const String &name, const char *file, int line);
02132             Subcase(const Subcase &) = delete;
02133             Subcase(Subcase &&) = delete;
02134             Subcase &operator=(const Subcase &) = delete;
02135             Subcase &operator=(Subcase &&) = delete;
02136             ~Subcase();
02137
02138             operator bool() const;
02139
02140         private:
02141             bool checkFilters();
02142         };
02143     } // namespace detail
02144 #endif // DOCTEST_CONFIG_DISABLE
02145 }
02146
02147

```

```

02148 } // namespace doctest
02149
02150 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02151
02152 #endif // DOCTEST_PARTS_PUBLIC_SUBCASE
02153 #ifndef DOCTEST_PARTS_PUBLIC_TEST_SUITE
02154 #define DOCTEST_PARTS_PUBLIC_TEST_SUITE
02155
02156
02157 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02158
02159 #ifndef DOCTEST_CONFIG_DISABLE
02160
02161 namespace doctest {
02162 namespace detail {
02163
02164 struct DOCTEST_INTERFACE TestSuite {
02165     const char *m_test_suite = nullptr;
02166     const char *m_description = nullptr;
02167     bool m_skip = false;
02168     bool m_no_breaks = false;
02169     bool m_no_output = false;
02170     bool m_may_fail = false;
02171     bool m_should_fail = false;
02172     int m_expected_failures = 0;
02173     double m_timeout = 0;
02174
02175     TestSuite &operator*(const char *in) noexcept;
02176
02177     template <typename T>
02178     TestSuite &operator*(const T &in) noexcept {
02179         in.fill(*this);
02180         return *this;
02181     }
02182 };
02183
02184 // forward declarations of functions used by the macros
02185 DOCTEST_INTERFACE int setTestSuite(const TestSuite &ts) noexcept;
02186
02187 } // namespace detail
02188
02189 } // namespace doctest
02190
02191 // in a separate namespace outside of doctest because the DOCTEST_TEST_SUITE macro
02192 // introduces an anonymous namespace in which getCurrentTestSuite gets overridden
02193 namespace doctest_detail_test_suite_ns {
02194 DOCTEST_INTERFACE doctest::detail::TestSuite &getCurrentTestSuite() noexcept;
02195
02196 // this is here to clear the 'current test suite' for the current translation unit - at the top
02197 DOCTEST_GLOBAL_NO_WARNINGS(/* NOLINT(cert-err58-cpp) */
02198     DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_),
02199     doctest::detail::setTestSuite(doctest::detail::TestSuite() * ""))
02200 )
02201
02202 } // namespace doctest_detail_test_suite_ns
02203
02204 #endif // DOCTEST_CONFIG_DISABLE
02205
02206 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02207
02208 #endif // DOCTEST_PARTS_PUBLIC_TEST_SUITE
02209 #ifndef DOCTEST_PARTS_PUBLIC_TEST_CASE
02210 #define DOCTEST_PARTS_PUBLIC_TEST_CASE
02211
02212
02213 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02214
02215 namespace doctest {
02216
02217 struct DOCTEST_INTERFACE TestCaseData {
02218     String m_file; // the file in which the test was registered (using String - see #350)
02219     unsigned m_line; // the line where the test was registered
02220     const char *m_name; // name of the test case
02221     const char *m_test_suite; // the test suite in which the test was added
02222     const char *m_description;
02223     bool m_skip;
02224     bool m_no_breaks;
02225     bool m_no_output;
02226     bool m_may_fail;
02227     bool m_should_fail;
02228     int m_expected_failures;
02229     double m_timeout;
02230 };
02231
02232 #ifndef DOCTEST_CONFIG_DISABLE
02233 namespace detail {
02234

```



```

02235 using funcType = void (* )();
02236
02237 struct DOCTEST_INTERFACE TestCase : public TestCaseData {
02238     funcType m_test; // a function pointer to the test case
02239
02240     String m_type; // for templated test cases - gets appended to the real name
02241     int m_template_id; // an ID used to distinguish between the different versions of a templated
02242                       // test case
02243     String m_full_name; // contains the name (only for templated test cases!) + the template type
02244
02245     TestCase(
02246         funcType test,
02247         const char *file,
02248         unsigned line,
02249         const TestSuite &test_suite,
02250         const String &type = String(),
02251         int template_id = -1
02252     ) noexcept;
02253
02254     TestCase(const TestCase &other) noexcept;
02255     TestCase(TestCase &&) = delete;
02256
02257     DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(26434) // hides a non-virtual function
02258     TestCase &operator=(const TestCase &other) noexcept;
02259     DOCTEST_MSVC_SUPPRESS_WARNING_POP
02260
02261     TestCase &operator=(TestCase &&) = delete;
02262
02263     TestCase &operator*(const char *in) noexcept;
02264
02265     template <typename T>
02266     TestCase &operator*(const T &in) noexcept {
02267         in.fill(*this);
02268         return *this;
02269     }
02270
02271     bool operator<(const TestCase &other) const noexcept;
02272
02273     ~TestCase() = default;
02274 };
02275
02276 // forward declarations of functions used by the macros
02277 DOCTEST_INTERFACE int regTest(const TestCase &tc) noexcept;
02278
02279 } // namespace detail
02280 #endif // DOCTEST_CONFIG_DISABLE
02281
02282 } // namespace doctest
02283
02284 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02285
02286 #endif // DOCTEST_PARTS_PUBLIC_TEST_CASE
02287 #ifndef DOCTEST_PARTS_PUBLIC_DECORATORS
02288 #define DOCTEST_PARTS_PUBLIC_DECORATORS
02289
02290 #ifndef DOCTEST_CONFIG_DISABLE
02291 namespace doctest {
02292
02293 #define DOCTEST_DEFINE_DECORATOR(name, type, def)
02294
02295     struct name {
02296         type data;
02297
02298         name(type in = def) noexcept
02299             : data(in) {}
02300
02301         void fill(detail::TestCase &state) const noexcept {
02302             state.DOCTEST_CAT(m_, name) = data;
02303         }
02304
02305         void fill(detail::TestSuite &state) const noexcept {
02306             state.DOCTEST_CAT(m_, name) = data;
02307         }
02308     }
02309
02310 #define DOCTEST_DEFINE_DECORATOR(test_suite, const char *, "");
02311 #define DOCTEST_DEFINE_DECORATOR(description, const char *, "");
02312 #define DOCTEST_DEFINE_DECORATOR(skip, bool, true);

```

```

02311 DOCTEST_DEFINE_DECORATOR(no_breaks, bool, true);
02312 DOCTEST_DEFINE_DECORATOR(no_output, bool, true);
02313 DOCTEST_DEFINE_DECORATOR(timeout, double, 0);
02314 DOCTEST_DEFINE_DECORATOR(may_fail, bool, true);
02315 DOCTEST_DEFINE_DECORATOR(should_fail, bool, true);
02316 DOCTEST_DEFINE_DECORATOR(expected_failures, int, 0);
02317
02318 } // namespace doctest
02319
02320 #endif // DOCTEST_CONFIG_DISABLE
02321
02322 #endif // DOCTEST_PARTS_PUBLIC_DECORATORS
02323 #ifndef DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR
02324 #define DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR
02325
02326
02327 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02328
02329 namespace doctest {
02330 namespace detail {
02331
02332 #ifndef DOCTEST_CONFIG_DISABLE
02333
02334 struct DOCTEST_INTERFACE IExceptionTranslator {
02335     DOCTEST_DECLARE_INTERFACE(IExceptionTranslator)
02336     virtual bool translate(String &) const = 0;
02337 };
02338
02339 template <typename T>
02340 class ExceptionTranslator : public IExceptionTranslator {
02341 public:
02342     explicit ExceptionTranslator(String (*translateFunction)(T)) noexcept
02343         : m_translateFunction(translateFunction) {}
02344
02345     bool translate(String &res) const override {
02346 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
02347         try {
02348             throw;
02349         } catch (const T &ex) {
02350             res = m_translateFunction(ex);
02351             return true;
02352         } catch (...) {} // NOLINT (bugprone-empty-catch)
02353 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
02354         static_cast<void>(res); // to silence -Wunused-parameter
02355         return false;
02356     }
02357
02358 private:
02359     String (*m_translateFunction)(T);
02360 };
02361
02362 DOCTEST_INTERFACE void registerExceptionTranslatorImpl(const IExceptionTranslator *et) noexcept;
02363
02364 #endif // DOCTEST_CONFIG_DISABLE
02365
02366 } // namespace detail
02367
02368 #ifndef DOCTEST_CONFIG_DISABLE
02369
02370 template <typename T>
02371 int registerExceptionTranslator(String (*translateFunction)(T)) noexcept {
02372     DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wexit-time-destructors")
02373     static detail::ExceptionTranslator<T> exceptionTranslator(translateFunction);
02374     DOCTEST_CLANG_SUPPRESS_WARNING_POP
02375     detail::registerExceptionTranslatorImpl(&exceptionTranslator);
02376     return 0;
02377 }
02378
02379 #else // DOCTEST_CONFIG_DISABLE
02380
02381 template <typename T>
02382 int registerExceptionTranslator(String (*)(T)) noexcept {
02383     return 0;
02384 }
02385
02386 #endif // DOCTEST_CONFIG_DISABLE
02387
02388 } // namespace doctest
02389
02390 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02391
02392 #endif // DOCTEST_PARTS_PUBLIC_EXCEPTION_TRANSLATOR
02393 #ifndef DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE
02394 #define DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE
02395
02396
02397 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH

```

```

02398
02399 namespace doctest {
02400
02401 struct DOCTEST_INTERFACE IContextScope {
02402     DOCTEST_DECLARE_INTERFACE(IContextScope)
02403     virtual void stringify(std::ostream *) const = 0;
02404 };
02405
02406 #ifndef DOCTEST_CONFIG_DISABLE
02407 namespace detail {
02408
02409 // ContextScope base class used to allow implementing methods of ContextScope
02410 // that don't depend on the template parameter in doctest.cpp.
02411 struct DOCTEST_INTERFACE ContextScopeBase : public IContextScope {
02412     ContextScopeBase(const ContextScopeBase &) = delete;
02413
02414     ContextScopeBase &operator=(const ContextScopeBase &) = delete;
02415     ContextScopeBase &operator=(ContextScopeBase &&) = delete;
02416
02417     ~ContextScopeBase() override = default;
02418
02419 protected:
02420     ContextScopeBase();
02421     ContextScopeBase(ContextScopeBase &&other) noexcept;
02422
02423     void destroy();
02424     bool need_to_destroy{true};
02425 };
02426
02427 template <typename L>
02428 class ContextScope : public ContextScopeBase {
02429     L lambda_;
02430
02431 public:
02432     explicit ContextScope(const L &lambda)
02433         : lambda_(lambda) {}
02434
02435     // NOLINTNEXTLINE(cppcoreguidelines-rvalue-reference-param-not-moved)
02436     explicit ContextScope(L &&lambda)
02437         : lambda_(static_cast<L &&>(lambda)) {}
02438
02439     ContextScope(const ContextScope &) = delete;
02440     ContextScope(ContextScope &&) noexcept = default;
02441
02442     ContextScope &operator=(const ContextScope &) = delete;
02443     ContextScope &operator=(ContextScope &&) = delete;
02444
02445     void stringify(std::ostream *s) const override {
02446         lambda_(s);
02447     }
02448
02449     ~ContextScope() override {
02450         if (need_to_destroy) {
02451             destroy();
02452         }
02453     }
02454 };
02455
02456 template <typename L>
02457 ContextScope<L> MakeContextScope(const L &lambda) {
02458     return ContextScope<L>(lambda);
02459 }
02460
02461 } // namespace detail
02462 #endif // DOCTEST_CONFIG_DISABLE
02463
02464 } // namespace doctest
02465
02466 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02467
02468 #endif // DOCTEST_PARTS_PUBLIC_CONTEXT_SCOPE
02469 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE
02470 #define DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE
02471
02472 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02473
02474 namespace doctest {
02475
02476 struct DOCTEST_INTERFACE MessageData {
02477     String m_string;
02478     const char *m_file;
02479     int m_line;
02480     assertType::Enum m_severity;
02481 };
02482
02483 #ifndef DOCTEST_CONFIG_DISABLE

```

```

02485 namespace detail {
02486
02487 struct DOCTEST_INTERFACE MessageBuilder : public MessageData {
02488     std::ostream *m_stream;
02489     bool logged = false;
02490
02491     MessageBuilder(const char *file, int line, assertType::Enum severity);
02492
02493     MessageBuilder(const MessageBuilder &) = delete;
02494     MessageBuilder(MessageBuilder &&) = delete;
02495
02496     MessageBuilder &operator=(const MessageBuilder &) = delete;
02497     MessageBuilder &operator=(MessageBuilder &&) = delete;
02498
02499     ~MessageBuilder() noexcept(false);
02500
02501     // the preferred way of chaining parameters for stringification
02502     DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4866)
02503     template <typename T>
02504     MessageBuilder &operator,(const T &in) {
02505         *m_stream << (DOCTEST_STRINGIFY(in));
02506         return *this;
02507     }
02508     DOCTEST_MSVC_SUPPRESS_WARNING_POP
02509
02510     // kept here just for backwards-compatibility - the comma operator should be preferred now
02511     template <typename T>
02512     MessageBuilder &operator<<(const T &in) {
02513         return this->operator,(in);
02514     }
02515
02516     // the `,' operator has the lowest operator precedence - if `«' is used by the user then
02517     // the `,' operator will be called last which is not what we want and thus the `*' operator
02518     // is used first (has higher operator precedence compared to `«') so that we guarantee that
02519     // an operator of the MessageBuilder class is called first before the rest of the parameters
02520     template <typename T>
02521     MessageBuilder &operator*(const T &in) {
02522         return this->operator,(in);
02523     }
02524
02525     bool log();
02526     void react();
02527 };
02528
02529 } // namespace detail
02530 #endif // DOCTEST_CONFIG_DISABLE
02531
02532 } // namespace doctest
02533
02534 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02535
02536 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_MESSAGE
02537 #ifndef DOCTEST_PARTS_PUBLIC_PATH
02538 #define DOCTEST_PARTS_PUBLIC_PATH
02539
02540 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02541
02542 namespace doctest {
02543
02544     DOCTEST_INTERFACE const char *skipPathFromFilename(const char *file);
02545
02546 } // namespace doctest
02547
02548 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02549
02550 #endif // DOCTEST_PARTS_PUBLIC_PATH
02551 #ifndef DOCTEST_PARTS_PUBLIC_EXCEPTIONS
02552 #define DOCTEST_PARTS_PUBLIC_EXCEPTIONS
02553
02554 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02555
02556 #ifndef DOCTEST_CONFIG_DISABLE
02557
02558 namespace doctest {
02559     namespace detail {
02560
02561         struct DOCTEST_INTERFACE TestFailureException {};
02562
02563         DOCTEST_INTERFACE bool checkIfShouldThrow(assertType::Enum at);
02564
02565         #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
02566             DOCTEST_NORETURN
02567             #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
02568         DOCTEST_INTERFACE void throwException();
02569
02570     }
02571

```

```

02572 } // namespace detail
02573 } // namespace doctest
02574
02575 #endif // DOCTEST_CONFIG_DISABLE
02576
02577 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02578
02579 #endif // DOCTEST_PARTS_PUBLIC_EXCEPTIONS
02580 #ifndef DOCTEST_PARTS_PUBLIC_CONTEXT
02581 #define DOCTEST_PARTS_PUBLIC_CONTEXT
02582
02583
02584 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02585
02586 namespace doctest {
02587
02588 DOCTEST_INTERFACE extern bool is_running_in_test;
02589
02590 namespace detail {
02591 using assert_handler = void (*) (const AssertData &);
02592 struct ContextState;
02593 } // namespace detail
02594
02595 class DOCTEST_INTERFACE Context {
02596     detail::ContextState *p;
02597
02598     void parseArgs(int argc, const char *const *argv, bool withDefaults = false);
02599
02600 public:
02601     explicit Context(int argc = 0, const char *const *argv = nullptr);
02602
02603     Context(const Context &) = delete;
02604     Context(Context &&) = delete;
02605
02606     Context &operator=(const Context &) = delete;
02607     Context &operator=(Context &&) = delete;
02608
02609     ~Context(); // NOLINT(performance-trivially-destructible)
02610
02611     void applyCommandLine(int argc, const char *const *argv);
02612
02613     void addFilter(const char *filter, const char *value);
02614     void clearFilters();
02615     void setOption(const char *option, bool value);
02616     void setOption(const char *option, int value);
02617     void setOption(const char *option, const char *value);
02618
02619     bool shouldExit();
02620
02621     void setAsDefaultForAssertsOutOfTestCases();
02622
02623     void setAssertHandler(detail::assert_handler ah);
02624
02625     void setCout(std::ostream *out);
02626
02627     int run();
02628 };
02629 } // namespace doctest
02630
02631 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02632
02633 #endif // DOCTEST_PARTS_PUBLIC_CONTEXT
02634 #ifndef DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER
02635 #define DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER
02636
02637
02638 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02639
02640 #ifndef DOCTEST_CONFIG_DISABLE
02641
02642 namespace doctest {
02643 namespace detail {
02644
02645 DOCTEST_INTERFACE void failed_out_of_a_testing_context(const AssertData &ad);
02646
02647 DOCTEST_INTERFACE bool
02648 decomp_assert(assertType::Enum at, const char *file, int line, const char *expr, const Result
02649 &result);
02650 #define DOCTEST_ASSERT_OUT_OF_TESTS(decomp)
02651 \
02652     do { /* NOLINT(cppcoreguidelines-avoid-do-while) */
02653 \
02654         if (!is_running_in_test) {
02655 \
02656             if (failed) {

```

```

02654         ResultBuilder rb(at, file, line, expr);
02655     \
02656         rb.m_failed = failed;
02657     \
02658         rb.m_decomp = decomp;
02659     \
02660         failed_out_of_a_testing_context(rb);
02661     \
02662         if (isDebuggerActive() && !getContextOptions()->no_breaks)
02663     \
02664             DOCTEST_BREAK_INTO_DEBUGGER();
02665     \
02666         if (checkIfShouldThrow(at))
02667     \
02668             throwException();
02669     \
02670     }
02671     \
02672     return !failed;
02673     \
02674 }
02675 } while (false)
02676
02677 #define DOCTEST_ASSERT_IN_TESTS(decomp)
02678
02679     ResultBuilder rb(at, file, line, expr);
02680
02681     rb.m_failed = failed;
02682
02683     if (rb.m_failed || getContextOptions()->success)
02684     \
02685         rb.m_decomp = decomp;
02686     \
02687     if (rb.log())
02688     \
02689         DOCTEST_BREAK_INTO_DEBUGGER();
02690     \
02691     if (rb.m_failed && checkIfShouldThrow(at))
02692     \
02693         throwException()
02694
02695 template <int comparison, typename L, typename R>
02696 DOCTEST_NOINLINE bool binary_assert(
02697     assertType::Enum at,
02698     const char *file,
02699     int line,
02700     const char *expr,
02701     const DOCTEST_REF_WRAP(L) lhs,
02702     const DOCTEST_REF_WRAP(R) rhs
02703 ) {
02704     const bool failed = !RelationalComparator<comparison, L, R>()(lhs, rhs);
02705
02706     // #####
02707     // IF THE DEBUGGER BREAKS HERE - GO 1 LEVEL UP IN THE CALLSTACK FOR THE FAILING ASSERT
02708     // THIS IS THE EFFECT OF HAVING 'DOCTEST_CONFIG_SUPER_FAST_ASSERTS' DEFINED
02709     // #####
02710     DOCTEST_ASSERT_OUT_OF_TESTS(stringifyBinaryExpr(lhs, " ", rhs));
02711     DOCTEST_ASSERT_IN_TESTS(stringifyBinaryExpr(lhs, " ", rhs));
02712     return !failed;
02713 }
02714
02715 template <typename L>
02716 DOCTEST_NOINLINE bool
02717 unary_assert(assertType::Enum at, const char *file, int line, const char *expr, const
02718 DOCTEST_REF_WRAP(L) val) {
02719     bool failed = !val;
02720
02721     if (at & assertType::is_false)
02722         failed = !failed;
02723
02724     // #####
02725     // IF THE DEBUGGER BREAKS HERE - GO 1 LEVEL UP IN THE CALLSTACK FOR THE FAILING ASSERT
02726     // THIS IS THE EFFECT OF HAVING 'DOCTEST_CONFIG_SUPER_FAST_ASSERTS' DEFINED
02727     // #####
02728     DOCTEST_ASSERT_OUT_OF_TESTS((DOCTEST_STRINGIFY(val)));
02729     DOCTEST_ASSERT_IN_TESTS((DOCTEST_STRINGIFY(val)));
02730     return !failed;
02731 }
02732 }
02733
02734 } // namespace detail
02735 } // namespace doctest
02736
02737 #endif // DOCTEST_CONFIG_DISABLE
02738
02739 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02740

```

```

02721 #endif // DOCTEST_PARTS_PUBLIC_ASSERT_HANDLER
02722 #ifndef DOCTEST_PARTS_PUBLIC_REPORTER
02723 #define DOCTEST_PARTS_PUBLIC_REPORTER
02724
02725 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02726
02727 namespace doctest {
02728
02729 namespace TestCaseFailureReason {
02730 enum Enum {
02731     None = 0,
02732     AssertFailure = 1,           // an assertion has failed in the test case
02733     Exception = 2,              // test case threw an exception
02734     Crash = 4,                  // a crash...
02735     TooManyFailedAsserts = 8,    // the abort-after option
02736     Timeout = 16,               // see the timeout decorator
02737     ShouldHaveFailedButDidnt = 32, // see the should_fail decorator
02738     ShouldHaveFailedAndDid = 64,  // see the should_fail decorator
02739     DidntFailExactlyNumTimes = 128, // see the expected_failures decorator
02740     FailedExactlyNumTimes = 256,  // see the expected_failures decorator
02741     CouldHaveFailedAndDid = 512   // see the may_fail decorator
02742 };
02743 } // namespace TestCaseFailureReason
02744
02745 struct DOCTEST_INTERFACE CurrentTestStats {
02746     int numAssertsCurrentTest;
02747     int numAssertsFailedCurrentTest;
02748     double seconds;
02749     int failure_flags; // use TestCaseFailureReason::Enum
02750     bool testCaseSuccess;
02751 };
02752
02753 struct DOCTEST_INTERFACE TestCaseException {
02754     String error_string;
02755     bool is_crash;
02756 };
02757
02758 struct DOCTEST_INTERFACE TestRunStats {
02759     unsigned numTestCases;
02760     unsigned numTestCasesPassingFilters;
02761     unsigned numTestSuitesPassingFilters;
02762     unsigned numTestCasesFailed;
02763     int numAsserts;
02764     int numAssertsFailed;
02765 };
02766
02767 struct QueryData {
02768     const TestRunStats *run_stats = nullptr;
02769     const TestCaseData **data = nullptr;
02770     unsigned num_data = 0;
02771 };
02772
02773 struct DOCTEST_INTERFACE IReporter {
02774     // The constructor has to accept "const ContextOptions&" as a single argument
02775     // which has most of the options for the run + a pointer to the stdout stream
02776     // Reporter(const ContextOptions& in)
02777
02778     // called when a query should be reported (listing test cases, printing the version, etc.)
02779     virtual void report_query(const QueryData &) = 0;
02780
02781     // called when the whole test run starts
02782     virtual void test_run_start() = 0;
02783     // called when the whole test run ends (caching a pointer to the input doesn't make sense here)
02784     virtual void test_run_end(const TestRunStats &) = 0;
02785
02786     // called when a test case is started (safe to cache a pointer to the input)
02787     virtual void test_case_start(const TestCaseData &) = 0;
02788     // called when a test case is reentered because of unfinished subcases (safe to cache a pointer to
02789     the input)
02790     virtual void test_case_reenter(const TestCaseData &) = 0;
02791     // called when a test case has ended
02792     virtual void test_case_end(const CurrentTestStats &) = 0;
02793
02794     // called when an exception is thrown from the test case (or it crashes)
02795     virtual void test_case_exception(const TestCaseException &) = 0;
02796
02797     // called whenever a subcase is entered (don't cache pointers to the input)
02798     virtual void subcase_start(const SubcaseSignature &) = 0;
02799     // called whenever a subcase is exited (don't cache pointers to the input)
02800     virtual void subcase_end() = 0;
02801
02802     // called for each assert (don't cache pointers to the input)
02803     virtual void log_assert(const AssertData &) = 0;
02804     // called for each message (don't cache pointers to the input)
02805     virtual void log_message(const MessageData &) = 0;
02806

```

```

02807 // called when a test case is skipped either because it doesn't pass the filters,
02808 // has a skip decorator or isn't in the execution range (between first and last)
02809 // (safe to cache a pointer to the input)
02810 virtual void test_case_skipped(const TestCaseData &) = 0;
02811
02812 DOCTEST_DECLARE_INTERFACE(IReporter)
02813
02814 // can obtain all currently active contexts and stringify them if one wishes to do so
02815 static int get_num_active_contexts();
02816 static const IContextScope *const *get_active_contexts();
02817
02818 // can iterate through contexts which have been stringified automatically
02819 // in their destructors when an exception has been thrown
02820 static int get_num_stringified_contexts();
02821 static const String *get_stringified_contexts();
02822 };
02823
02824 namespace detail {
02825 using reporterCreatorFunc = IReporter *(*)(const ContextOptions &);
02826
02827 DOCTEST_INTERFACE void
02828 registerReporterImpl(const char *name, int prio, reporterCreatorFunc c, bool isReporter) noexcept;
02829
02830 template <typename Reporter>
02831 IReporter *reporterCreator(const ContextOptions &o) {
02832     return new Reporter(o);
02833 }
02834 } // namespace detail
02835
02836 template <typename Reporter>
02837 int registerReporter(const char *name, int priority, bool isReporter) noexcept {
02838     detail::registerReporterImpl(name, priority, detail::reporterCreator<Reporter>, isReporter);
02839     return 0;
02840 }
02841 } // namespace doctest
02842
02843 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02844
02845 #endif // DOCTEST_PARTS_PUBLIC_REPORTER
02846 #ifndef DOCTEST_PARTS_PUBLIC_GENERATOR
02847 #define DOCTEST_PARTS_PUBLIC_GENERATOR
02848
02849 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02850
02851 #ifndef DOCTEST_CONFIG_DISABLE
02852 namespace doctest {
02853 namespace detail {
02854
02855 DOCTEST_INTERFACE size_t acquireGeneratorDecisionIndex(size_t count);
02856
02857 template <typename T, typename... Rest>
02858 T acquireGeneratorValue(T first, Rest... rest) {
02859     const T values[] = {first, static_cast<T>(rest)...};
02860     const size_t idx = acquireGeneratorDecisionIndex(1 + sizeof...(Rest));
02861     return values[idx];
02862 }
02863 }
02864 } // namespace detail
02865 } // namespace doctest
02866 #endif // DOCTEST_CONFIG_DISABLE
02867
02868 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
02869
02870 #endif // DOCTEST_PARTS_PUBLIC_GENERATOR
02871 #ifndef DOCTEST_PARTS_PUBLIC_MACROS
02872 #define DOCTEST_PARTS_PUBLIC_MACROS
02873
02874 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_PUSH
02875
02876 #ifndef DOCTEST_CONFIG_DISABLE
02877 namespace doctest {
02878 namespace detail {
02879
02880 template <typename T>
02881 int instantiationHelper(const T &) noexcept {
02882     return 0;
02883 }
02884 }
02885 } // namespace detail
02886 } // namespace doctest
02887 #endif // DOCTEST_CONFIG_DISABLE
02888
02889 #ifndef DOCTEST_CONFIG_ASSERTS_RETURN_VALUES
02890 #define DOCTEST_FUNC_EMPTY [] { return false; }()
02891 #else
02892 #define DOCTEST_FUNC_EMPTY (void)0
02893

```



```

02894 #endif
02895
02896 // if registering is not disabled
02897 #ifndef DOCTEST_CONFIG_DISABLE
02898
02899 #ifdef DOCTEST_CONFIG_ASSERTS_RETURN_VALUES
02900 #define DOCTEST_FUNC_SCOPE_BEGIN [&]
02901 #define DOCTEST_FUNC_SCOPE_END ()
02902 #define DOCTEST_FUNC_SCOPE_RET(v) return v
02903 #else
02904 #define DOCTEST_FUNC_SCOPE_BEGIN do /* NOLINT(cppcoreguidelines-avoid-do-while) */
02905 #define DOCTEST_FUNC_SCOPE_END while (false)
02906 #define DOCTEST_FUNC_SCOPE_RET(v) (void)0
02907 #endif
02908
02909 // common code in asserts - for convenience
02910 #define DOCTEST_ASSERT_LOG_REACT_RETURN(b)
02911 \
02912 \
02913 \
02914 \
02915 \
02916 #ifndef DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
02917 #define DOCTEST_WRAP_IN_TRY(x) x;
02918 #else // DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
02919 #define DOCTEST_WRAP_IN_TRY(x)
02920 \
02921 \
02922 \
02923 #endif // DOCTEST_CONFIG_NO_TRY_CATCH_IN_ASSERTS
02924
02925 #ifdef DOCTEST_CONFIG_VOID_CAST_EXPRESSIONS
02926 #define DOCTEST_CAST_TO_VOID(...)
02927 \
02928 \
02929 \
02930 #else // DOCTEST_CONFIG_VOID_CAST_EXPRESSIONS
02931 #define DOCTEST_CAST_TO_VOID(...) __VA_ARGS__;
02932 #endif // DOCTEST_CONFIG_VOID_CAST_EXPRESSIONS
02933
02934 // registers the test by initializing a dummy var with a function
02935 #define DOCTEST_REGISTER_FUNCTION(global_prefix, f, decorators)
02936 \
02937 \
02938 \
02939 \
02940 \
02941 \
02942 \
02943
02944 #define DOCTEST_IMPLEMENT_FIXTURE(der, base, func, decorators)
02945 \
02946 \
02947 \
02948 \
02949 \
02950 \
02951 \
02952 \
02953 \

```

```

02954     }
02955     \
02956     DOCTEST_INLINE_NOINLINE void der::f() // NOLINT(misc-definitions-in-headers)
02957 #define DOCTEST_CREATE_AND_REGISTER_FUNCTION(f, decorators)
02958     \
02959     static void f();
02960     \
02961     DOCTEST_REGISTER_FUNCTION(DOCTEST_EMPTY, f, decorators)
02962     \
02963     static void f()
02964 #define DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS(f, proxy, decorators)
02965     \
02966     static doctest::detail::funcType proxy() {
02967     \
02968     return f;
02969     \
02970     }
02971     \
02972     DOCTEST_REGISTER_FUNCTION(inline, proxy(), decorators)
02973     \
02974     static void f()
02975 // for registering tests
02976 #define DOCTEST_TEST_CASE(decorators)
02977     \
02978     DOCTEST_CREATE_AND_REGISTER_FUNCTION(DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), decorators)
02979 // for registering tests in classes - requires C++17 for inline variables!
02980 #if DOCTEST_CPLUSPLUS >= 201703L
02981 #define DOCTEST_TEST_CASE_CLASS(decorators)
02982     \
02983     DOCTEST_CREATE_AND_REGISTER_FUNCTION_IN_CLASS(
02984     \
02985     DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), DOCTEST_ANONYMOUS(DOCTEST_ANON_PROXY_), decorators
02986     \
02987     )
02988 #else // DOCTEST_TEST_CASE_CLASS
02989 #define DOCTEST_TEST_CASE_CLASS(...)
02990 TEST_CASES_CAN_BE_REGISTERED_IN_CLASSES_ONLY_IN_CPP17_MODE_OR_WITH_VS_2017_OR_NEWER
02991 #endif // DOCTEST_TEST_CASE_CLASS
02992 // for registering tests with a fixture
02993 #define DOCTEST_TEST_CASE_FIXTURE(c, decorators)
02994     \
02995     DOCTEST_IMPLEMENT_FIXTURE(
02996     \
02997     DOCTEST_ANONYMOUS(DOCTEST_ANON_CLASS_), c, DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), decorators
02998     \
02999     )
03000 // for converting types to strings without the <typeinfo> header and demangling
03001 #define DOCTEST_TYPE_TO_STRING_AS(str, ...)
03002     \
03003     namespace doctest {
03004     \
03005     template <>
03006     \
03007     inline String toString<__VA_ARGS__>() {
03008     \
03009     return str;
03010     \
03011     }
03012     \
03013     }
03014     \
03015     static_assert(true, "")
03016 #define DOCTEST_TYPE_TO_STRING(...) DOCTEST_TYPE_TO_STRING_AS(#__VA_ARGS__, __VA_ARGS__)
03017 #define DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, iter, func)
03018     \
03019     template <typename T>
03020     \
03021     static void func();
03022     \
03023     namespace { /* NOLINT */
03024     \
03025     template <typename Tuple>
03026     \
03027     struct iter;
03028     \
03029     template <typename Type, typename... Rest>
03030     \
03031     struct iter<std::tuple<Type, Rest...> > {

```

```

03009         iter(const char *file, unsigned line, int index) noexcept {
03010             doctest::detail::regTest(
03011                 doctest::detail::TestCase(
03012                     func<Type>,
03013                     file,
03014                     line,
03015                     doctest_detail_test_suite_ns::getCurrentTestSuite(),
03016                     doctest::toString<Type>(),
03017                     int(line) * 1000 + index
03018                 ) *
03019                 dec
03020             );
03021             iter<std::tuple<Rest...>>(file, line, index + 1);
03022         }
03023     };
03024     template <>
03025     struct iter<std::tuple<>> {
03026         iter(const char *, unsigned, int) {}
03027     };
03028 }
03029 template <typename T>
03030 static void func()
03031
03032 #define DOCTEST_TEST_CASE_TEMPLATE_DEFINE(dec, T, id)
03033     DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, DOCTEST_CAT(id, ITERATOR),
03034     DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_))
03035 #define DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, anon, ...)
03036     DOCTEST_GLOBAL_NO_WARNINGS(
03037         DOCTEST_CAT(anon, DUMMY), /* NOLINT(cert-err58-cpp, fuchsia-statically-constructed-objects) */
03038         doctest::detail::instantiationHelper(DOCTEST_CAT(id, ITERATOR) < __VA_ARGS__ > (__FILE__,
03039         __LINE__, 0))
03040 )
03041 #define DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, ...)
03042     DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_),
03043     std::tuple<__VA_ARGS__>)
03043     static_assert(true, "")
03044
03045 #define DOCTEST_TEST_CASE_TEMPLATE_APPLY(id, ...)
03046     DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(id, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_), __VA_ARGS__)
03047     static_assert(true, "")
03048
03049 #define DOCTEST_TEST_CASE_TEMPLATE_IMPL(dec, T, anon, ...)
03050     DOCTEST_TEST_CASE_TEMPLATE_DEFINE_IMPL(dec, T, DOCTEST_CAT(anon, ITERATOR), anon);
03051     DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE_IMPL(anon, anon, std::tuple<__VA_ARGS__>)
03052     template <typename T>
03053     static void anon()
03054
03055 #define DOCTEST_TEST_CASE_TEMPLATE(dec, T, ...)
03056     DOCTEST_TEST_CASE_TEMPLATE_IMPL(dec, T, DOCTEST_ANONYMOUS(DOCTEST_ANON_TMP_), __VA_ARGS__)
03057
03058 // for subcases
03059 #define DOCTEST_SUBCASE(name)

```

```

03060 \
03061 \         if (const doctest::detail::Subcase &DOCTEST_ANONYMOUS(DOCTEST_ANON_SUBCASE_) DOCTEST_UNUSED =
03062 \             doctest::detail::Subcase(name, __FILE__, __LINE__))
03063 // for generating value-parameterized test inputs
03064 #define DOCTEST_GENERATE(...) doctest::detail::acquireGeneratorValue(__VA_ARGS__)
03065
03066 // for grouping tests in test suites by using code blocks
03067 #define DOCTEST_TEST_SUITE_IMPL(decorators, ns_name)
03068 \
03069 \         namespace ns_name {
03070 \
03071 \             namespace doctest_detail_test_suite_ns {
03072 \
03073 \                 static DOCTEST_NOINLINE doctest::detail::TestSuite &getCurrentTestSuite() noexcept {
03074 \
03075 \                     DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4640)
03076 \                     DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wexit-time-destructors")
03077 \                     DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wmissing-field-initializers")
03078 \
03079 \                     static doctest::detail::TestSuite data{};
03080 \
03081 \                     static bool initied = false;
03082 \
03083 \                     DOCTEST_MSVC_SUPPRESS_WARNING_POP
03084 \                     DOCTEST_CLANG_SUPPRESS_WARNING_POP
03085 \                     DOCTEST_GCC_SUPPRESS_WARNING_POP
03086 \
03087 \                     if (!initied) {
03088 \
03089 \                         data *decorators;
03090 \
03091 \                         initied = true;
03092 \
03093 \                     }
03094 \
03095 \                     return data;
03096 \
03097 \                 }
03098 \
03099 \             }
03100 \
03101 \         namespace ns_name
03102 #define DOCTEST_TEST_SUITE(decorators) DOCTEST_TEST_SUITE_IMPL(decorators,
03103 \
03104 \         DOCTEST_ANONYMOUS(DOCTEST_ANON_SUITE_))
03105 // for starting a testsuite block
03106 #define DOCTEST_TEST_SUITE_BEGIN(decorators)
03107 \
03108 \         DOCTEST_GLOBAL_NO_WARNINGS(
03109 \
03110 \             DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT(cert-err58-cpp) */
03111 \
03112 \             doctest::detail::setTestSuite(doctest::detail::TestSuite() * decorators)
03113 \
03114 \         )
03115 \
03116 \         static_assert(true, "")
03117 // for ending a testsuite block
03118 #define DOCTEST_TEST_SUITE_END
03119 \
03120 \         DOCTEST_GLOBAL_NO_WARNINGS(
03121 \
03122 \             DOCTEST_ANONYMOUS(DOCTEST_ANON_VAR_), /* NOLINT(cert-err58-cpp) */
03123 \
03124 \             doctest::detail::setTestSuite(doctest::detail::TestSuite() * "")
03125 \
03126 \         )
03127 \
03128 \         using DOCTEST_ANONYMOUS(DOCTEST_ANON_FOR_SEMICOLON_) = int
03129 // for registering exception translators
03130 #define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL(translatorName, signature)
03131 \
03132 \         inline doctest::String translatorName(signature);
03133 \
03134 \         DOCTEST_GLOBAL_NO_WARNINGS(

```

```

03111         DOCTEST_ANONYMOUS(DOCTEST_ANON_TRANSLATOR_VAR_), /* NOLINT(cert-err58-cpp) */
03112         doctest::registerExceptionTranslator(translatorName)
03113     )
03114     doctest::String translatorName(signature)
03115 #define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR(signature)
03116     DOCTEST_REGISTER_EXCEPTION_TRANSLATOR_IMPL(DOCTEST_ANONYMOUS(DOCTEST_ANON_TRANSLATOR_), signature)
03117 // for registering reporters
03118 #define DOCTEST_REGISTER_REPORTER(name, priority, reporter)
03119     DOCTEST_GLOBAL_NO_WARNINGS(
03120         DOCTEST_ANONYMOUS(DOCTEST_ANON_REPORTER_VAR_), /* NOLINT(cert-err58-cpp) */
03121         doctest::registerReporter<reporter>(name, priority, true)
03122     )
03123     static_assert(true, "")
03124 // for registering listeners
03125 #define DOCTEST_REGISTER_LISTENER(name, priority, reporter)
03126     DOCTEST_GLOBAL_NO_WARNINGS(
03127         DOCTEST_ANONYMOUS(DOCTEST_ANON_REPORTER_VAR_), /* NOLINT(cert-err58-cpp) */
03128         doctest::registerReporter<reporter>(name, priority, false)
03129     )
03130     static_assert(true, "")
03131 #define DOCTEST_INFO(...)
03132     DOCTEST_INFO_IMPL(DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_MB_),
03133         DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_OTHER_), __VA_ARGS__)
03134 #define DOCTEST_INFO_IMPL(mb_name, s_name, ...)
03135     auto DOCTEST_ANONYMOUS(DOCTEST_CAPTURE_) = doctest::detail::MakeContextScope([&](std::ostream
03136 *s_name) {
03137     doctest::detail::MessageBuilder mb_name(__FILE__, __LINE__, doctest::assertType::is_warn);
03138     mb_name.m_stream = s_name;
03139     mb_name *__VA_ARGS__;
03140 })
03141 #define DOCTEST_CAPTURE(x) DOCTEST_INFO(#x " := ", x)
03142 #define DOCTEST_ADD_AT_IMPL(type, file, line, mb, ...)
03143     DOCTEST_FUNC_SCOPE_BEGIN {
03144         doctest::detail::MessageBuilder mb(file, line, doctest::assertType::type);
03145         mb *__VA_ARGS__;
03146         if (mb.log())
03147             DOCTEST_BREAK_INTO_DEBUGGER();
03148         mb.react();
03149     }
03150     DOCTEST_FUNC_SCOPE_END
03151 #define DOCTEST_ADD_MESSAGE_AT(file, line, ...)
03152     DOCTEST_ADD_AT_IMPL(is_warn, file, line, DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)
03153 #define DOCTEST_ADD_FAIL_CHECK_AT(file, line, ...)
03154     DOCTEST_ADD_AT_IMPL(is_check, file, line, DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)
03155 #define DOCTEST_ADD_FAIL_AT(file, line, ...)
03156     DOCTEST_ADD_AT_IMPL(is_require, file, line, DOCTEST_ANONYMOUS(DOCTEST_MESSAGE_), __VA_ARGS__)
03157 #define DOCTEST_MESSAGE(...) DOCTEST_ADD_MESSAGE_AT(__FILE__, __LINE__, __VA_ARGS__)
03158 #define DOCTEST_FAIL_CHECK(...) DOCTEST_ADD_FAIL_CHECK_AT(__FILE__, __LINE__, __VA_ARGS__)

```

```

03166 #define DOCTEST_FAIL(...) DOCTEST_ADD_FAIL_AT(__FILE__, __LINE__, __VA_ARGS__)
03167
03168 #define DOCTEST_TO_LVALUE(...) __VA_ARGS__ // Not removed to keep backwards compatibility.
03169
03170 #ifndef DOCTEST_CONFIG_SUPER_FAST_ASSERTS
03171
03172 #define DOCTEST_ASSERT_IMPLEMENT_2(assert_type, ...)
03173 \
03174 \
03175 \
03176 \
03177 \
03178 \
03179 \
03180 \
03181 \
03182 #define DOCTEST_ASSERT_IMPLEMENT_1(assert_type, ...)
03183 \
03184 \
03185 \
03186 \
03187 \
03188 #define DOCTEST_BINARY_ASSERT(assert_type, comp, ...)
03189 \
03190 \
03191 \
03192 \
03193 \
03194 \
03195 \
03196 #define DOCTEST_UNARY_ASSERT(assert_type, ...)
03197 \
03198 \
03199 \
03200 \
03201 \
03202 \
03203 \
03204 #else // DOCTEST_CONFIG_SUPER_FAST_ASSERTS
03205
03206 // necessary for <ASSERT>_MESSAGE
03207 #define DOCTEST_ASSERT_IMPLEMENT_2 DOCTEST_ASSERT_IMPLEMENT_1
03208
03209 #define DOCTEST_ASSERT_IMPLEMENT_1(assert_type, ...)
03210 \
03211 \
03212 \
03213 \
03214 \
03215 \
03216 \
03217 \
03218 \
03219 #define DOCTEST_BINARY_ASSERT(assert_type, comparison, ...)

```

```

03220     doctest::detail::binary_assert<doctest::detail::binaryAssertComparison::comparison>(
03221         doctest::assertType::assert_type, __FILE__, __LINE__, #__VA_ARGS__, __VA_ARGS__
03222     )
03223
03224 #define DOCTEST_UNARY_ASSERT(assert_type, ...)
03225     doctest::detail::unary_assert(doctest::assertType::assert_type, __FILE__, __LINE__, #__VA_ARGS__,
    __VA_ARGS__)
03226
03227 #endif // DOCTEST_CONFIG_SUPER_FAST_ASSERTS
03228
03229 #define DOCTEST_WARN(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_WARN, __VA_ARGS__)
03230 #define DOCTEST_CHECK(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_CHECK, __VA_ARGS__)
03231 #define DOCTEST_REQUIRE(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_REQUIRE, __VA_ARGS__)
03232 #define DOCTEST_WARN_FALSE(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_WARN_FALSE, __VA_ARGS__)
03233 #define DOCTEST_CHECK_FALSE(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_CHECK_FALSE, __VA_ARGS__)
03234 #define DOCTEST_REQUIRE_FALSE(...) DOCTEST_ASSERT_IMPLEMENT_1(DT_REQUIRE_FALSE, __VA_ARGS__)
03235
03236 // clang-format off
03237 #define DOCTEST_WARN_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
    DOCTEST_ASSERT_IMPLEMENT_2(DT_WARN, cond); } DOCTEST_FUNC_SCOPE_END
03238 #define DOCTEST_CHECK_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
    DOCTEST_ASSERT_IMPLEMENT_2(DT_CHECK, cond); } DOCTEST_FUNC_SCOPE_END
03239 #define DOCTEST_REQUIRE_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
    DOCTEST_ASSERT_IMPLEMENT_2(DT_REQUIRE, cond); } DOCTEST_FUNC_SCOPE_END
03240 #define DOCTEST_WARN_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
    DOCTEST_ASSERT_IMPLEMENT_2(DT_WARN_FALSE, cond); } DOCTEST_FUNC_SCOPE_END
03241 #define DOCTEST_CHECK_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
    DOCTEST_ASSERT_IMPLEMENT_2(DT_CHECK_FALSE, cond); } DOCTEST_FUNC_SCOPE_END
03242 #define DOCTEST_REQUIRE_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
    DOCTEST_ASSERT_IMPLEMENT_2(DT_REQUIRE_FALSE, cond); } DOCTEST_FUNC_SCOPE_END
03243 // clang-format on
03244
03245 #define DOCTEST_WARN_EQ(...) DOCTEST_BINARY_ASSERT(DT_WARN_EQ, eq, __VA_ARGS__)
03246 #define DOCTEST_CHECK_EQ(...) DOCTEST_BINARY_ASSERT(DT_CHECK_EQ, eq, __VA_ARGS__)
03247 #define DOCTEST_REQUIRE_EQ(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_EQ, eq, __VA_ARGS__)
03248 #define DOCTEST_WARN_NE(...) DOCTEST_BINARY_ASSERT(DT_WARN_NE, ne, __VA_ARGS__)
03249 #define DOCTEST_CHECK_NE(...) DOCTEST_BINARY_ASSERT(DT_CHECK_NE, ne, __VA_ARGS__)
03250 #define DOCTEST_REQUIRE_NE(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_NE, ne, __VA_ARGS__)
03251 #define DOCTEST_WARN_GT(...) DOCTEST_BINARY_ASSERT(DT_WARN_GT, gt, __VA_ARGS__)
03252 #define DOCTEST_CHECK_GT(...) DOCTEST_BINARY_ASSERT(DT_CHECK_GT, gt, __VA_ARGS__)
03253 #define DOCTEST_REQUIRE_GT(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_GT, gt, __VA_ARGS__)
03254 #define DOCTEST_WARN_LT(...) DOCTEST_BINARY_ASSERT(DT_WARN_LT, lt, __VA_ARGS__)
03255 #define DOCTEST_CHECK_LT(...) DOCTEST_BINARY_ASSERT(DT_CHECK_LT, lt, __VA_ARGS__)
03256 #define DOCTEST_REQUIRE_LT(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_LT, lt, __VA_ARGS__)
03257 #define DOCTEST_WARN_GE(...) DOCTEST_BINARY_ASSERT(DT_WARN_GE, ge, __VA_ARGS__)
03258 #define DOCTEST_CHECK_GE(...) DOCTEST_BINARY_ASSERT(DT_CHECK_GE, ge, __VA_ARGS__)
03259 #define DOCTEST_REQUIRE_GE(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_GE, ge, __VA_ARGS__)
03260 #define DOCTEST_WARN_LE(...) DOCTEST_BINARY_ASSERT(DT_WARN_LE, le, __VA_ARGS__)
03261 #define DOCTEST_CHECK_LE(...) DOCTEST_BINARY_ASSERT(DT_CHECK_LE, le, __VA_ARGS__)
03262 #define DOCTEST_REQUIRE_LE(...) DOCTEST_BINARY_ASSERT(DT_REQUIRE_LE, le, __VA_ARGS__)
03263
03264 #define DOCTEST_WARN_UNARY(...) DOCTEST_UNARY_ASSERT(DT_WARN_UNARY, __VA_ARGS__)
03265 #define DOCTEST_CHECK_UNARY(...) DOCTEST_UNARY_ASSERT(DT_CHECK_UNARY, __VA_ARGS__)
03266 #define DOCTEST_REQUIRE_UNARY(...) DOCTEST_UNARY_ASSERT(DT_REQUIRE_UNARY, __VA_ARGS__)
03267 #define DOCTEST_WARN_UNARY_FALSE(...) DOCTEST_UNARY_ASSERT(DT_WARN_UNARY_FALSE, __VA_ARGS__)
03268 #define DOCTEST_CHECK_UNARY_FALSE(...) DOCTEST_UNARY_ASSERT(DT_CHECK_UNARY_FALSE, __VA_ARGS__)
03269 #define DOCTEST_REQUIRE_UNARY_FALSE(...) DOCTEST_UNARY_ASSERT(DT_REQUIRE_UNARY_FALSE, __VA_ARGS__)
03270
03271 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
03272
03273 #define DOCTEST_ASSERT_THROWS_AS(expr, assert_type, message, ...)
03274     DOCTEST_FUNC_SCOPE_BEGIN {
03275         if (!doctest::getContextOptions().no_throw) {
03276             doctest::detail::ResultBuilder DOCTEST_RB(
03277                 doctest::assertType::assert_type, __FILE__, __LINE__, #expr, #__VA_ARGS__, message
03278             );
03279             try {
03280                 DOCTEST_CAST_TO_VOID(expr)
03281             } catch (const typename doctest::detail::types::remove_const<
03282                 typename doctest::detail::types::remove_reference<__VA_ARGS__>::type>::type &) {
03283                 DOCTEST_RB.translateException();
03284                 DOCTEST_RB.m_threw_as = true;

```

```

03285         } catch (...) { DOCTEST_RB.translateException(); }
03286         DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);
03287     } else { /* NOLINT(*-else-after-return) */
03288         DOCTEST_FUNC_SCOPE_RET(false);
03289     }
03290 }
03291 DOCTEST_FUNC_SCOPE_END
03292
03293 #define DOCTEST_ASSERT_THROWS_WITH(expr, expr_str, assert_type, ...)
03294     DOCTEST_FUNC_SCOPE_BEGIN {
03295         if (!doctest::getContextOptions().no_throw) {
03296             doctest::detail::ResultBuilder DOCTEST_RB(
03297                 doctest::assertType::assert_type, __FILE__, __LINE__, expr_str, "", __VA_ARGS__
03298             );
03299             try {
03300                 DOCTEST_CAST_TO_VOID(expr)
03301             } catch (...) { DOCTEST_RB.translateException(); }
03302             DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);
03303         } else { /* NOLINT(*-else-after-return) */
03304             DOCTEST_FUNC_SCOPE_RET(false);
03305         }
03306     }
03307 DOCTEST_FUNC_SCOPE_END
03308
03309 #define DOCTEST_ASSERT_NOTHROW(assert_type, ...)
03310     DOCTEST_FUNC_SCOPE_BEGIN {
03311         doctest::detail::ResultBuilder DOCTEST_RB(doctest::assertType::assert_type, __FILE__,
03312     __LINE__, #__VA_ARGS__); \
03313         try {
03314             DOCTEST_CAST_TO_VOID(__VA_ARGS__)
03315         } catch (...) { DOCTEST_RB.translateException(); }
03316         DOCTEST_ASSERT_LOG_REACT_RETURN(DOCTEST_RB);
03317     }
03318 DOCTEST_FUNC_SCOPE_END
03319 // clang-format off
03320 #define DOCTEST_WARN_THROWS(...) DOCTEST_ASSERT_THROWS_WITH((__VA_ARGS__), #__VA_ARGS__,
DT_WARN_THROWS, "")
03321 #define DOCTEST_CHECK_THROWS(...) DOCTEST_ASSERT_THROWS_WITH((__VA_ARGS__), #__VA_ARGS__,
DT_CHECK_THROWS, "")
03322 #define DOCTEST_REQUIRE_THROWS(...) DOCTEST_ASSERT_THROWS_WITH((__VA_ARGS__), #__VA_ARGS__,
DT_REQUIRE_THROWS, "")
03323
03324 #define DOCTEST_WARN_THROWS_AS(expr, ...) DOCTEST_ASSERT_THROWS_AS(expr, DT_WARN_THROWS_AS, "",
__VA_ARGS__)
03325 #define DOCTEST_CHECK_THROWS_AS(expr, ...) DOCTEST_ASSERT_THROWS_AS(expr, DT_CHECK_THROWS_AS, "",
__VA_ARGS__)
03326 #define DOCTEST_REQUIRE_THROWS_AS(expr, ...) DOCTEST_ASSERT_THROWS_AS(expr, DT_REQUIRE_THROWS_AS, "",
__VA_ARGS__)
03327
03328 #define DOCTEST_WARN_THROWS_WITH(expr, ...) DOCTEST_ASSERT_THROWS_WITH(expr, #expr,
DT_WARN_THROWS_WITH, __VA_ARGS__)
03329 #define DOCTEST_CHECK_THROWS_WITH(expr, ...) DOCTEST_ASSERT_THROWS_WITH(expr, #expr,
DT_CHECK_THROWS_WITH, __VA_ARGS__)
03330 #define DOCTEST_REQUIRE_THROWS_WITH(expr, ...) DOCTEST_ASSERT_THROWS_WITH(expr, #expr,
DT_REQUIRE_THROWS_WITH, __VA_ARGS__)
03331
03332 #define DOCTEST_WARN_THROWS_WITH_AS(expr, message, ...) DOCTEST_ASSERT_THROWS_AS(expr,
DT_WARN_THROWS_WITH_AS, message, __VA_ARGS__)

```



```

03333 #define DOCTEST_CHECK_THROWS_WITH_AS(expr, message, ...) DOCTEST_ASSERT_THROWS_AS(expr,
    DT_CHECK_THROWS_WITH_AS, message, __VA_ARGS__)
03334 #define DOCTEST_REQUIRE_THROWS_WITH_AS(expr, message, ...) DOCTEST_ASSERT_THROWS_AS(expr,
    DT_REQUIRE_THROWS_WITH_AS, message, __VA_ARGS__)
03335
03336 #define DOCTEST_WARN_NOTHROW(...) DOCTEST_ASSERT_NOTHROW(DT_WARN_NOTHROW, __VA_ARGS__)
03337 #define DOCTEST_CHECK_NOTHROW(...) DOCTEST_ASSERT_NOTHROW(DT_CHECK_NOTHROW, __VA_ARGS__)
03338 #define DOCTEST_REQUIRE_NOTHROW(...) DOCTEST_ASSERT_NOTHROW(DT_REQUIRE_NOTHROW, __VA_ARGS__)
03339
03340 #define DOCTEST_WARN_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
    DOCTEST_WARN_THROWS(expr); } DOCTEST_FUNC_SCOPE_END
03341 #define DOCTEST_CHECK_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
    DOCTEST_CHECK_THROWS(expr); } DOCTEST_FUNC_SCOPE_END
03342 #define DOCTEST_REQUIRE_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN {
    DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_THROWS(expr); } DOCTEST_FUNC_SCOPE_END
03343 #define DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
    DOCTEST_INFO(__VA_ARGS__); DOCTEST_WARN_THROWS_AS(expr, ex); } DOCTEST_FUNC_SCOPE_END
03344 #define DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
    DOCTEST_INFO(__VA_ARGS__); DOCTEST_CHECK_THROWS_AS(expr, ex); } DOCTEST_FUNC_SCOPE_END
03345 #define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
    DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_THROWS_AS(expr, ex); } DOCTEST_FUNC_SCOPE_END
03346 #define DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_SCOPE_BEGIN {
    DOCTEST_INFO(__VA_ARGS__); DOCTEST_WARN_THROWS_WITH(expr, with); } DOCTEST_FUNC_SCOPE_END
03347 #define DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_SCOPE_BEGIN {
    DOCTEST_INFO(__VA_ARGS__); DOCTEST_CHECK_THROWS_WITH(expr, with); } DOCTEST_FUNC_SCOPE_END
03348 #define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_SCOPE_BEGIN {
    DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_THROWS_WITH(expr, with); } DOCTEST_FUNC_SCOPE_END
03349 #define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
    DOCTEST_INFO(__VA_ARGS__); DOCTEST_WARN_THROWS_WITH_AS(expr, with, ex); } DOCTEST_FUNC_SCOPE_END
03350 #define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
    DOCTEST_INFO(__VA_ARGS__); DOCTEST_CHECK_THROWS_WITH_AS(expr, with, ex); } DOCTEST_FUNC_SCOPE_END
03351 #define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_SCOPE_BEGIN {
    DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, ex); } DOCTEST_FUNC_SCOPE_END
03352 #define DOCTEST_WARN_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
    DOCTEST_WARN_NOTHROW(expr); } DOCTEST_FUNC_SCOPE_END
03353 #define DOCTEST_CHECK_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN { DOCTEST_INFO(__VA_ARGS__);
    DOCTEST_CHECK_NOTHROW(expr); } DOCTEST_FUNC_SCOPE_END
03354 #define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_SCOPE_BEGIN {
    DOCTEST_INFO(__VA_ARGS__); DOCTEST_REQUIRE_NOTHROW(expr); } DOCTEST_FUNC_SCOPE_END
03355 // clang-format on
03356
03357 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
03358
03359 // =====
03360 // == WHAT FOLLOWS IS VERSIONS OF THE MACROS THAT DO NOT DO ANY REGISTERING! ==
03361 // == THIS CAN BE ENABLED BY DEFINING DOCTEST_CONFIG_DISABLE GLOBALLY! ==
03362 // =====
03363 #else // DOCTEST_CONFIG_DISABLE
03364
03365 #define DOCTEST_IMPLEMENT_FIXTURE(der, base, func, name)
03366 \
    namespace /* NOLINT */ {
03367 \
    template <typename DOCTEST_UNUSED_TEMPLATE_TYPE>
03368 \
    struct der : public base {
03369 \
        void f();
03370 \
    };
03371 \
    }
03372 \
    template <typename DOCTEST_UNUSED_TEMPLATE_TYPE>
03373 \
    inline void der<DOCTEST_UNUSED_TEMPLATE_TYPE>::f()
03374
03375 #define DOCTEST_CREATE_AND_REGISTER_FUNCTION(f, name)
03376 \
    template <typename DOCTEST_UNUSED_TEMPLATE_TYPE>
03377 \
    static inline void f()
03378
03379 // for registering tests
03380 #define DOCTEST_TEST_CASE(name)
    DOCTEST_CREATE_AND_REGISTER_FUNCTION(DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), name)
03381
03382 // for registering tests in classes
03383 #define DOCTEST_TEST_CASE_CLASS(name)
    DOCTEST_CREATE_AND_REGISTER_FUNCTION(DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), name)
03384
03385 // for registering tests with a fixture
03386 #define DOCTEST_TEST_CASE_FIXTURE(x, name)
    \
    DOCTEST_IMPLEMENT_FIXTURE(DOCTEST_ANONYMOUS(DOCTEST_ANON_CLASS_), x,
    DOCTEST_ANONYMOUS(DOCTEST_ANON_FUNC_), name)
03388

```

```

03389 // for converting types to strings without the <typeinfo> header and demangling
03390 #define DOCTEST_TYPE_TO_STRING_AS(str, ...) static_assert(true, "")
03391 #define DOCTEST_TYPE_TO_STRING(...) static_assert(true, "")
03392
03393 // for typed tests
03394 #define DOCTEST_TEST_CASE_TEMPLATE(name, type, ...)
03395 \
03396 \
03397 \
03398 \
03399 \
03400 \
03401 \
03402 #define DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, ...) static_assert(true, "")
03403 #define DOCTEST_TEST_CASE_TEMPLATE_APPLY(id, ...) static_assert(true, "")
03404
03405 // for subcases
03406 #define DOCTEST_SUBCASE(name)
03407
03408 // for generating value-parameterized test inputs
03409 #define DOCTEST_GENERATE_IMPL(first, ...) (first)
03410 #define DOCTEST_GENERATE(...) DOCTEST_GENERATE_IMPL(__VA_ARGS__, DOCTEST_EMPTY)
03411
03412 // for a testsuite block
03413 #define DOCTEST_TEST_SUITE(name) namespace // NOLINT
03414
03415 // for starting a testsuite block
03416 #define DOCTEST_TEST_SUITE_BEGIN(name) static_assert(true, "")
03417
03418 // for ending a testsuite block
03419 #define DOCTEST_TEST_SUITE_END using DOCTEST_ANONYMOUS(DOCTEST_ANON_FOR_SEMICOLON_) = int
03420
03421 #define DOCTEST_REGISTER_EXCEPTION_TRANSLATOR(signature)
03422 \
03423 \
03424 \
03425 \
03426 \
03427 \
03428 \
03429 \
03430 \
03431 \
03432 \
03433 \
03434 \
03435 \
03436 \
03437 #if defined(DOCTEST_CONFIG_EVALUATE_ASSERTS_EVEN_WHEN_DISABLED) &&
    defined(DOCTEST_CONFIG_ASSERTS_RETURN_VALUES)
03438
03439 #define DOCTEST_WARN(...) [&] { return __VA_ARGS__; }()
03440 #define DOCTEST_CHECK(...) [&] { return __VA_ARGS__; }()
03441 #define DOCTEST_REQUIRE(...) [&] { return __VA_ARGS__; }()
03442 #define DOCTEST_WARN_FALSE(...) [&] { return !(__VA_ARGS__); }()
03443 #define DOCTEST_CHECK_FALSE(...) [&] { return !(__VA_ARGS__); }()
03444 #define DOCTEST_REQUIRE_FALSE(...) [&] { return !(__VA_ARGS__); }()
03445
03446 #define DOCTEST_WARN_MESSAGE(cond, ...) [&] { return cond; }()
03447 #define DOCTEST_CHECK_MESSAGE(cond, ...) [&] { return cond; }()
03448 #define DOCTEST_REQUIRE_MESSAGE(cond, ...) [&] { return cond; }()
03449 #define DOCTEST_WARN_FALSE_MESSAGE(cond, ...) [&] { return !(cond); }()
03450 #define DOCTEST_CHECK_FALSE_MESSAGE(cond, ...) [&] { return !(cond); }()
03451 #define DOCTEST_REQUIRE_FALSE_MESSAGE(cond, ...) [&] { return !(cond); }()
03452
03453 namespace doctest {
03454 namespace detail {
03455 #define DOCTEST_RELATIONAL_OP(name, op)
03456 \
03457 \
03458 \
03459 \
03460 \
03461 DOCTEST_RELATIONAL_OP(eq, ==)
03462 DOCTEST_RELATIONAL_OP(ne, !=)
03463 DOCTEST_RELATIONAL_OP(lt, <)
03464 DOCTEST_RELATIONAL_OP(gt, >)

```

```

03465 DOCTEST_RELATIONAL_OP(<le, <=)
03466 DOCTEST_RELATIONAL_OP(>ge, >=)
03467 } // namespace detail
03468 } // namespace doctest
03469
03470 #define DOCTEST_WARN_EQ(...) [&] { return doctest::detail::eq(__VA_ARGS__); }()
03471 #define DOCTEST_CHECK_EQ(...) [&] { return doctest::detail::eq(__VA_ARGS__); }()
03472 #define DOCTEST_REQUIRE_EQ(...) [&] { return doctest::detail::eq(__VA_ARGS__); }()
03473 #define DOCTEST_WARN_NE(...) [&] { return doctest::detail::ne(__VA_ARGS__); }()
03474 #define DOCTEST_CHECK_NE(...) [&] { return doctest::detail::ne(__VA_ARGS__); }()
03475 #define DOCTEST_REQUIRE_NE(...) [&] { return doctest::detail::ne(__VA_ARGS__); }()
03476 #define DOCTEST_WARN_LT(...) [&] { return doctest::detail::lt(__VA_ARGS__); }()
03477 #define DOCTEST_CHECK_LT(...) [&] { return doctest::detail::lt(__VA_ARGS__); }()
03478 #define DOCTEST_REQUIRE_LT(...) [&] { return doctest::detail::lt(__VA_ARGS__); }()
03479 #define DOCTEST_WARN_GT(...) [&] { return doctest::detail::gt(__VA_ARGS__); }()
03480 #define DOCTEST_CHECK_GT(...) [&] { return doctest::detail::gt(__VA_ARGS__); }()
03481 #define DOCTEST_REQUIRE_GT(...) [&] { return doctest::detail::gt(__VA_ARGS__); }()
03482 #define DOCTEST_WARN_LE(...) [&] { return doctest::detail::le(__VA_ARGS__); }()
03483 #define DOCTEST_CHECK_LE(...) [&] { return doctest::detail::le(__VA_ARGS__); }()
03484 #define DOCTEST_REQUIRE_LE(...) [&] { return doctest::detail::le(__VA_ARGS__); }()
03485 #define DOCTEST_WARN_GE(...) [&] { return doctest::detail::ge(__VA_ARGS__); }()
03486 #define DOCTEST_CHECK_GE(...) [&] { return doctest::detail::ge(__VA_ARGS__); }()
03487 #define DOCTEST_REQUIRE_GE(...) [&] { return doctest::detail::ge(__VA_ARGS__); }()
03488 #define DOCTEST_WARN_UNARY(...) [&] { return __VA_ARGS__; }()
03489 #define DOCTEST_CHECK_UNARY(...) [&] { return __VA_ARGS__; }()
03490 #define DOCTEST_REQUIRE_UNARY(...) [&] { return __VA_ARGS__; }()
03491 #define DOCTEST_WARN_UNARY_FALSE(...) [&] { return !(__VA_ARGS__); }()
03492 #define DOCTEST_CHECK_UNARY_FALSE(...) [&] { return !(__VA_ARGS__); }()
03493 #define DOCTEST_REQUIRE_UNARY_FALSE(...) [&] { return !(__VA_ARGS__); }()
03494
03495 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
03496
03497 #define DOCTEST_WARN_THROWS_WITH(expr, with, ...)
03498 \
03499 \
03500 \
03501 \
03502 \
03503 \
03504 \
03505 \
03506 \
03507 \
03508 \
03509 \
03510 \
03511 \
03512 \
03513 \
03514 \
03515 // clang-format off
03516 #define DOCTEST_WARN_THROWS(...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
03517 #define DOCTEST_CHECK_THROWS(...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
03518 #define DOCTEST_REQUIRE_THROWS(...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
03519 #define DOCTEST_WARN_THROWS_AS(expr, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
03520 #define DOCTEST_CHECK_THROWS_AS(expr, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
03521 #define DOCTEST_REQUIRE_THROWS_AS(expr, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
03522 #define DOCTEST_WARN_NOTHROW(...) [&] { try { __VA_ARGS__; return true; } catch (...) { return false; } }()
03523 #define DOCTEST_CHECK_NOTHROW(...) [&] { try { __VA_ARGS__; return true; } catch (...) { return false; } }()
03524 #define DOCTEST_REQUIRE_NOTHROW(...) [&] { try { __VA_ARGS__; return true; } catch (...) { return false; } }()
03525
03526 #define DOCTEST_WARN_THROWS_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
03527 #define DOCTEST_CHECK_THROWS_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
03528 #define DOCTEST_REQUIRE_THROWS_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return false; } catch (...) { return true; } }()
03529 #define DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
03530 #define DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
03531 #define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) [&] { try { expr; } catch (__VA_ARGS__) { return true; } catch (...) { } return false; }()
03532 #define DOCTEST_WARN_NOTHROW_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return true; } catch (...) { return false; } }()

```

```

        return false; } }())
03533 #define DOCTEST_CHECK_NOTHROW_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return true; } catch (...) {
        return false; } }())
03534 #define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...) [&] { try { __VA_ARGS__; return true; } catch (...)
        { return false; } }()
03535 // clang-format on
03536
03537 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
03538
03539 #else // DOCTEST_CONFIG_EVALUATE_ASSERTS_EVEN_WHEN_DISABLED
03540
03541 #define DOCTEST_WARN(...) DOCTEST_FUNC_EMPTY
03542 #define DOCTEST_CHECK(...) DOCTEST_FUNC_EMPTY
03543 #define DOCTEST_REQUIRE(...) DOCTEST_FUNC_EMPTY
03544 #define DOCTEST_WARN_FALSE(...) DOCTEST_FUNC_EMPTY
03545 #define DOCTEST_CHECK_FALSE(...) DOCTEST_FUNC_EMPTY
03546 #define DOCTEST_REQUIRE_FALSE(...) DOCTEST_FUNC_EMPTY
03547
03548 #define DOCTEST_WARN_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
03549 #define DOCTEST_CHECK_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
03550 #define DOCTEST_REQUIRE_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
03551 #define DOCTEST_WARN_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
03552 #define DOCTEST_CHECK_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
03553 #define DOCTEST_REQUIRE_FALSE_MESSAGE(cond, ...) DOCTEST_FUNC_EMPTY
03554
03555 #define DOCTEST_WARN_EQ(...) DOCTEST_FUNC_EMPTY
03556 #define DOCTEST_CHECK_EQ(...) DOCTEST_FUNC_EMPTY
03557 #define DOCTEST_REQUIRE_EQ(...) DOCTEST_FUNC_EMPTY
03558 #define DOCTEST_WARN_NE(...) DOCTEST_FUNC_EMPTY
03559 #define DOCTEST_CHECK_NE(...) DOCTEST_FUNC_EMPTY
03560 #define DOCTEST_REQUIRE_NE(...) DOCTEST_FUNC_EMPTY
03561 #define DOCTEST_WARN_GT(...) DOCTEST_FUNC_EMPTY
03562 #define DOCTEST_CHECK_GT(...) DOCTEST_FUNC_EMPTY
03563 #define DOCTEST_REQUIRE_GT(...) DOCTEST_FUNC_EMPTY
03564 #define DOCTEST_WARN_LT(...) DOCTEST_FUNC_EMPTY
03565 #define DOCTEST_CHECK_LT(...) DOCTEST_FUNC_EMPTY
03566 #define DOCTEST_REQUIRE_LT(...) DOCTEST_FUNC_EMPTY
03567 #define DOCTEST_WARN_GE(...) DOCTEST_FUNC_EMPTY
03568 #define DOCTEST_CHECK_GE(...) DOCTEST_FUNC_EMPTY
03569 #define DOCTEST_REQUIRE_GE(...) DOCTEST_FUNC_EMPTY
03570 #define DOCTEST_WARN_LE(...) DOCTEST_FUNC_EMPTY
03571 #define DOCTEST_CHECK_LE(...) DOCTEST_FUNC_EMPTY
03572 #define DOCTEST_REQUIRE_LE(...) DOCTEST_FUNC_EMPTY
03573
03574 #define DOCTEST_WARN_UNARY(...) DOCTEST_FUNC_EMPTY
03575 #define DOCTEST_CHECK_UNARY(...) DOCTEST_FUNC_EMPTY
03576 #define DOCTEST_REQUIRE_UNARY(...) DOCTEST_FUNC_EMPTY
03577 #define DOCTEST_WARN_UNARY_FALSE(...) DOCTEST_FUNC_EMPTY
03578 #define DOCTEST_CHECK_UNARY_FALSE(...) DOCTEST_FUNC_EMPTY
03579 #define DOCTEST_REQUIRE_UNARY_FALSE(...) DOCTEST_FUNC_EMPTY
03580
03581 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
03582
03583 #define DOCTEST_WARN_THROWS(...) DOCTEST_FUNC_EMPTY
03584 #define DOCTEST_CHECK_THROWS(...) DOCTEST_FUNC_EMPTY
03585 #define DOCTEST_REQUIRE_THROWS(...) DOCTEST_FUNC_EMPTY
03586 #define DOCTEST_WARN_THROWS_AS(expr, ...) DOCTEST_FUNC_EMPTY
03587 #define DOCTEST_CHECK_THROWS_AS(expr, ...) DOCTEST_FUNC_EMPTY
03588 #define DOCTEST_REQUIRE_THROWS_AS(expr, ...) DOCTEST_FUNC_EMPTY
03589 #define DOCTEST_WARN_THROWS_WITH(expr, ...) DOCTEST_FUNC_EMPTY
03590 #define DOCTEST_CHECK_THROWS_WITH(expr, ...) DOCTEST_FUNC_EMPTY
03591 #define DOCTEST_REQUIRE_THROWS_WITH(expr, ...) DOCTEST_FUNC_EMPTY
03592 #define DOCTEST_WARN_THROWS_WITH_AS(expr, with, ...) DOCTEST_FUNC_EMPTY
03593 #define DOCTEST_CHECK_THROWS_WITH_AS(expr, with, ...) DOCTEST_FUNC_EMPTY
03594 #define DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, ...) DOCTEST_FUNC_EMPTY
03595 #define DOCTEST_WARN_NOTHROW(...) DOCTEST_FUNC_EMPTY
03596 #define DOCTEST_CHECK_NOTHROW(...) DOCTEST_FUNC_EMPTY
03597 #define DOCTEST_REQUIRE_NOTHROW(...) DOCTEST_FUNC_EMPTY
03598
03599 #define DOCTEST_WARN_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
03600 #define DOCTEST_CHECK_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
03601 #define DOCTEST_REQUIRE_THROWS_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
03602 #define DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_EMPTY
03603 #define DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_EMPTY
03604 #define DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_FUNC_EMPTY
03605 #define DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_EMPTY
03606 #define DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_EMPTY
03607 #define DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_FUNC_EMPTY
03608 #define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_EMPTY
03609 #define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_EMPTY
03610 #define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_FUNC_EMPTY
03611 #define DOCTEST_WARN_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
03612 #define DOCTEST_CHECK_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
03613 #define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...) DOCTEST_FUNC_EMPTY
03614
03615 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
03616

```

```

03617 #endif // DOCTEST_CONFIG_EVALUATE_ASSERTS_EVEN_WHEN_DISABLED
03618
03619 #endif // DOCTEST_CONFIG_DISABLE
03620
03621 #ifdef DOCTEST_CONFIG_NO_EXCEPTIONS
03622
03623 #ifdef DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
03624 #define DOCTEST_EXCEPTION_EMPTY_FUNC DOCTEST_FUNC_EMPTY
03625 #else // DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
03626 #define DOCTEST_EXCEPTION_EMPTY_FUNC
03627
03628     [] {
03629
03630         static_assert(
03631
03632             false,
03633
03634             "Exceptions are disabled! "
03635
03636             "Use DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS if you want "
03637
03638             "to compile with exceptions disabled."
03639
03640         );
03641
03642         return false;
03643     }()
03644
03645 #undef DOCTEST_REQUIRE
03646 #undef DOCTEST_REQUIRE_FALSE
03647 #undef DOCTEST_REQUIRE_MESSAGE
03648 #undef DOCTEST_REQUIRE_FALSE_MESSAGE
03649 #undef DOCTEST_REQUIRE_EQ
03650 #undef DOCTEST_REQUIRE_NE
03651 #undef DOCTEST_REQUIRE_GT
03652 #undef DOCTEST_REQUIRE_LT
03653 #undef DOCTEST_REQUIRE_GE
03654 #undef DOCTEST_REQUIRE_LE
03655 #undef DOCTEST_REQUIRE_UNARY
03656 #undef DOCTEST_REQUIRE_UNARY_FALSE
03657
03658 #define DOCTEST_REQUIRE DOCTEST_EXCEPTION_EMPTY_FUNC
03659 #define DOCTEST_REQUIRE_FALSE DOCTEST_EXCEPTION_EMPTY_FUNC
03660 #define DOCTEST_REQUIRE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC
03661 #define DOCTEST_REQUIRE_FALSE_MESSAGE DOCTEST_EXCEPTION_EMPTY_FUNC
03662 #define DOCTEST_REQUIRE_EQ DOCTEST_EXCEPTION_EMPTY_FUNC
03663 #define DOCTEST_REQUIRE_NE DOCTEST_EXCEPTION_EMPTY_FUNC
03664 #define DOCTEST_REQUIRE_GT DOCTEST_EXCEPTION_EMPTY_FUNC
03665 #define DOCTEST_REQUIRE_LT DOCTEST_EXCEPTION_EMPTY_FUNC
03666 #define DOCTEST_REQUIRE_GE DOCTEST_EXCEPTION_EMPTY_FUNC
03667 #define DOCTEST_REQUIRE_LE DOCTEST_EXCEPTION_EMPTY_FUNC
03668 #define DOCTEST_REQUIRE_UNARY DOCTEST_EXCEPTION_EMPTY_FUNC
03669 #define DOCTEST_REQUIRE_UNARY_FALSE DOCTEST_EXCEPTION_EMPTY_FUNC
03670
03671 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS_BUT_WITH_ALL_ASSERTS
03672
03673 #define DOCTEST_WARN_THROWS(...) DOCTEST_EXCEPTION_EMPTY_FUNC
03674 #define DOCTEST_CHECK_THROWS(...) DOCTEST_EXCEPTION_EMPTY_FUNC
03675 #define DOCTEST_REQUIRE_THROWS(...) DOCTEST_EXCEPTION_EMPTY_FUNC
03676 #define DOCTEST_WARN_THROWS_AS(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03677 #define DOCTEST_CHECK_THROWS_AS(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03678 #define DOCTEST_REQUIRE_THROWS_AS(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03679 #define DOCTEST_WARN_THROWS_WITH(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03680 #define DOCTEST_CHECK_THROWS_WITH(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03681 #define DOCTEST_REQUIRE_THROWS_WITH(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03682 #define DOCTEST_WARN_THROWS_WITH_AS(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03683 #define DOCTEST_CHECK_THROWS_WITH_AS(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03684 #define DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03685 #define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03686 #define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03687 #define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03688 #define DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03689 #define DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03690 #define DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03691 #define DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03692 #define DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03693 #define DOCTEST_WARN_NO_THROW(...) DOCTEST_EXCEPTION_EMPTY_FUNC
03694 #define DOCTEST_CHECK_NO_THROW(...) DOCTEST_EXCEPTION_EMPTY_FUNC
03695 #define DOCTEST_REQUIRE_NO_THROW(...) DOCTEST_EXCEPTION_EMPTY_FUNC

```

```

03695 #define DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, ...) DOCTEST_EXCEPTION_EMPTY_FUNC
03696
03697 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
03698
03699 // KEPT FOR BACKWARDS COMPATIBILITY - FORWARDING TO THE RIGHT MACROS
03700 #define DOCTEST_FAST_WARN_EQ DOCTEST_WARN_EQ
03701 #define DOCTEST_FAST_CHECK_EQ DOCTEST_CHECK_EQ
03702 #define DOCTEST_FAST_REQUIRE_EQ DOCTEST_REQUIRE_EQ
03703 #define DOCTEST_FAST_WARN_NE DOCTEST_WARN_NE
03704 #define DOCTEST_FAST_CHECK_NE DOCTEST_CHECK_NE
03705 #define DOCTEST_FAST_REQUIRE_NE DOCTEST_REQUIRE_NE
03706 #define DOCTEST_FAST_WARN_GT DOCTEST_WARN_GT
03707 #define DOCTEST_FAST_CHECK_GT DOCTEST_CHECK_GT
03708 #define DOCTEST_FAST_REQUIRE_GT DOCTEST_REQUIRE_GT
03709 #define DOCTEST_FAST_WARN_LT DOCTEST_WARN_LT
03710 #define DOCTEST_FAST_CHECK_LT DOCTEST_CHECK_LT
03711 #define DOCTEST_FAST_REQUIRE_LT DOCTEST_REQUIRE_LT
03712 #define DOCTEST_FAST_WARN_GE DOCTEST_WARN_GE
03713 #define DOCTEST_FAST_CHECK_GE DOCTEST_CHECK_GE
03714 #define DOCTEST_FAST_REQUIRE_GE DOCTEST_REQUIRE_GE
03715 #define DOCTEST_FAST_WARN_LE DOCTEST_WARN_LE
03716 #define DOCTEST_FAST_CHECK_LE DOCTEST_CHECK_LE
03717 #define DOCTEST_FAST_REQUIRE_LE DOCTEST_REQUIRE_LE
03718
03719 #define DOCTEST_FAST_WARN_UNARY DOCTEST_WARN_UNARY
03720 #define DOCTEST_FAST_CHECK_UNARY DOCTEST_CHECK_UNARY
03721 #define DOCTEST_FAST_REQUIRE_UNARY DOCTEST_REQUIRE_UNARY
03722 #define DOCTEST_FAST_WARN_UNARY_FALSE DOCTEST_WARN_UNARY_FALSE
03723 #define DOCTEST_FAST_CHECK_UNARY_FALSE DOCTEST_CHECK_UNARY_FALSE
03724 #define DOCTEST_FAST_REQUIRE_UNARY_FALSE DOCTEST_REQUIRE_UNARY_FALSE
03725
03726 #define DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE(id, ...) DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id,
__VA_ARGS__)
03727
03728 // BDD style macros
03729 // clang-format off
03730 #define DOCTEST_SCENARIO(name) DOCTEST_TEST_CASE(" Scenario: "
name)
03731 #define DOCTEST_SCENARIO_CLASS(name) DOCTEST_TEST_CASE_CLASS(" Scenario: "
name)
03732 #define DOCTEST_SCENARIO_METHOD(x, name) DOCTEST_TEST_CASE_FIXTURE(x, " Scenario: "
name)
03733 #define DOCTEST_SCENARIO_TEMPLATE(name, T, ...) DOCTEST_TEST_CASE_TEMPLATE(" Scenario: "
name, T, __VA_ARGS__)
03734 #define DOCTEST_SCENARIO_TEMPLATE_DEFINE(name, T, id) DOCTEST_TEST_CASE_TEMPLATE_DEFINE(" Scenario: "
name, T, id)
03735
03736 #define DOCTEST_GIVEN(name) DOCTEST_SUBCASE(" Given: " name)
03737 #define DOCTEST_AND_GIVEN(name) DOCTEST_SUBCASE(" And: " name)
03738 #define DOCTEST_WHEN(name) DOCTEST_SUBCASE(" When: " name)
03739 #define DOCTEST_AND_WHEN(name) DOCTEST_SUBCASE(" And: " name)
03740 #define DOCTEST_THEN(name) DOCTEST_SUBCASE(" Then: " name)
03741 #define DOCTEST_AND_THEN(name) DOCTEST_SUBCASE(" And: " name)
03742 // clang-format on
03743
03744 // == SHORT VERSIONS OF THE MACROS
03745 #ifndef DOCTEST_CONFIG_NO_SHORT_MACRO_NAMES
03746
03747 #define TEST_CASE(name) DOCTEST_TEST_CASE(name)
03748 #define TEST_CASE_CLASS(name) DOCTEST_TEST_CASE_CLASS(name)
03749 #define TEST_CASE_FIXTURE(x, name) DOCTEST_TEST_CASE_FIXTURE(x, name)
03750 #define TYPE_TO_STRING_AS(str, ...) DOCTEST_TYPE_TO_STRING_AS(str, __VA_ARGS__)
03751 #define TYPE_TO_STRING(...) DOCTEST_TYPE_TO_STRING(__VA_ARGS__)
03752 #define TEST_CASE_TEMPLATE(name, T, ...) DOCTEST_TEST_CASE_TEMPLATE(name, T, __VA_ARGS__)
03753 #define TEST_CASE_TEMPLATE_DEFINE(name, T, id) DOCTEST_TEST_CASE_TEMPLATE_DEFINE(name, T, id)
03754 #define TEST_CASE_TEMPLATE_INVOKE(id, ...) DOCTEST_TEST_CASE_TEMPLATE_INVOKE(id, __VA_ARGS__)
03755 #define TEST_CASE_TEMPLATE_APPLY(id, ...) DOCTEST_TEST_CASE_TEMPLATE_APPLY(id, __VA_ARGS__)
03756 #define SUBCASE(name) DOCTEST_SUBCASE(name)
03757 #define GENERATE(...) DOCTEST_GENERATE(__VA_ARGS__)
03758 #define TEST_SUITE(decorators) DOCTEST_TEST_SUITE(decorators)
03759 #define TEST_SUITE_BEGIN(name) DOCTEST_TEST_SUITE_BEGIN(name)
03760 #define TEST_SUITE_END DOCTEST_TEST_SUITE_END
03761 #define REGISTER_EXCEPTION_TRANSLATOR(signature) DOCTEST_REGISTER_EXCEPTION_TRANSLATOR(signature)
03762 #define REGISTER_REPORTER(name, priority, reporter) DOCTEST_REGISTER_REPORTER(name, priority,
reporter)
03763 #define REGISTER_LISTENER(name, priority, reporter) DOCTEST_REGISTER_LISTENER(name, priority,
reporter)
03764 #define INFO(...) DOCTEST_INFO(__VA_ARGS__)
03765 #define CAPTURE(x) DOCTEST_CAPTURE(x)
03766 #define ADD_MESSAGE_AT(file, line, ...) DOCTEST_ADD_MESSAGE_AT(file, line, __VA_ARGS__)
03767 #define ADD_FAIL_CHECK_AT(file, line, ...) DOCTEST_ADD_FAIL_CHECK_AT(file, line, __VA_ARGS__)
03768 #define ADD_FAIL_AT(file, line, ...) DOCTEST_ADD_FAIL_AT(file, line, __VA_ARGS__)
03769 #define MESSAGE(...) DOCTEST_MESSAGE(__VA_ARGS__)
03770 #define FAIL_CHECK(...) DOCTEST_FAIL_CHECK(__VA_ARGS__)
03771 #define FAIL(...) DOCTEST_FAIL(__VA_ARGS__)
03772 #define TO_LVALUE(...) DOCTEST_TO_LVALUE(__VA_ARGS__)
03773

```



```

03774 #define WARN(...) DOCTEST_WARN(__VA_ARGS__)
03775 #define WARN_FALSE(...) DOCTEST_WARN_FALSE(__VA_ARGS__)
03776 #define WARN_THROWS(...) DOCTEST_WARN_THROWS(__VA_ARGS__)
03777 #define WARN_THROWS_AS(expr, ...) DOCTEST_WARN_THROWS_AS(expr, __VA_ARGS__)
03778 #define WARN_THROWS_WITH(expr, ...) DOCTEST_WARN_THROWS_WITH(expr, __VA_ARGS__)
03779 #define WARN_THROWS_WITH_AS(expr, with, ...) DOCTEST_WARN_THROWS_WITH_AS(expr, with, __VA_ARGS__)
03780 #define WARN_NOTHROW(...) DOCTEST_WARN_NOTHROW(__VA_ARGS__)
03781 #define CHECK(...) DOCTEST_CHECK(__VA_ARGS__)
03782 #define CHECK_FALSE(...) DOCTEST_CHECK_FALSE(__VA_ARGS__)
03783 #define CHECK_THROWS(...) DOCTEST_CHECK_THROWS(__VA_ARGS__)
03784 #define CHECK_THROWS_AS(expr, ...) DOCTEST_CHECK_THROWS_AS(expr, __VA_ARGS__)
03785 #define CHECK_THROWS_WITH(expr, ...) DOCTEST_CHECK_THROWS_WITH(expr, __VA_ARGS__)
03786 #define CHECK_THROWS_WITH_AS(expr, with, ...) DOCTEST_CHECK_THROWS_WITH_AS(expr, with, __VA_ARGS__)
03787 #define CHECK_NOTHROW(...) DOCTEST_CHECK_NOTHROW(__VA_ARGS__)
03788 #define REQUIRE(...) DOCTEST_REQUIRE(__VA_ARGS__)
03789 #define REQUIRE_FALSE(...) DOCTEST_REQUIRE_FALSE(__VA_ARGS__)
03790 #define REQUIRE_THROWS(...) DOCTEST_REQUIRE_THROWS(__VA_ARGS__)
03791 #define REQUIRE_THROWS_AS(expr, ...) DOCTEST_REQUIRE_THROWS_AS(expr, __VA_ARGS__)
03792 #define REQUIRE_THROWS_WITH(expr, ...) DOCTEST_REQUIRE_THROWS_WITH(expr, __VA_ARGS__)
03793 #define REQUIRE_THROWS_WITH_AS(expr, with, ...) DOCTEST_REQUIRE_THROWS_WITH_AS(expr, with,
__VA_ARGS__)
03794 #define REQUIRE_NOTHROW(...) DOCTEST_REQUIRE_NOTHROW(__VA_ARGS__)
03795
03796 // clang-format off
03797 #define WARN_MESSAGE(cond, ...) DOCTEST_WARN_MESSAGE(cond, __VA_ARGS__)
03798 #define WARN_FALSE_MESSAGE(cond, ...) DOCTEST_WARN_FALSE_MESSAGE(cond, __VA_ARGS__)
03799 #define WARN_THROWS_MESSAGE(expr, ...) DOCTEST_WARN_THROWS_MESSAGE(expr, __VA_ARGS__)
03800 #define WARN_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_WARN_THROWS_AS_MESSAGE(expr, ex, __VA_ARGS__)
03801 #define WARN_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_WARN_THROWS_WITH_MESSAGE(expr, with,
__VA_ARGS__)
03802 #define WARN_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_WARN_THROWS_WITH_AS_MESSAGE(expr,
with, ex, __VA_ARGS__)
03803 #define WARN_NOTHROW_MESSAGE(expr, ...) DOCTEST_WARN_NOTHROW_MESSAGE(expr, __VA_ARGS__)
03804 #define CHECK_MESSAGE(cond, ...) DOCTEST_CHECK_MESSAGE(cond, __VA_ARGS__)
03805 #define CHECK_FALSE_MESSAGE(cond, ...) DOCTEST_CHECK_FALSE_MESSAGE(cond, __VA_ARGS__)
03806 #define CHECK_THROWS_MESSAGE(expr, ...) DOCTEST_CHECK_THROWS_MESSAGE(expr, __VA_ARGS__)
03807 #define CHECK_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_CHECK_THROWS_AS_MESSAGE(expr, ex, __VA_ARGS__)
03808 #define CHECK_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_CHECK_THROWS_WITH_MESSAGE(expr, with,
__VA_ARGS__)
03809 #define CHECK_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...) DOCTEST_CHECK_THROWS_WITH_AS_MESSAGE(expr,
with, ex, __VA_ARGS__)
03810 #define CHECK_NOTHROW_MESSAGE(expr, ...) DOCTEST_CHECK_NOTHROW_MESSAGE(expr, __VA_ARGS__)
03811 #define REQUIRE_MESSAGE(cond, ...) DOCTEST_REQUIRE_MESSAGE(cond, __VA_ARGS__)
03812 #define REQUIRE_FALSE_MESSAGE(cond, ...) DOCTEST_REQUIRE_FALSE_MESSAGE(cond, __VA_ARGS__)
03813 #define REQUIRE_THROWS_MESSAGE(expr, ...) DOCTEST_REQUIRE_THROWS_MESSAGE(expr, __VA_ARGS__)
03814 #define REQUIRE_THROWS_AS_MESSAGE(expr, ex, ...) DOCTEST_REQUIRE_THROWS_AS_MESSAGE(expr, ex,
__VA_ARGS__)
03815 #define REQUIRE_THROWS_WITH_MESSAGE(expr, with, ...) DOCTEST_REQUIRE_THROWS_WITH_MESSAGE(expr, with,
__VA_ARGS__)
03816 #define REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, ...)
DOCTEST_REQUIRE_THROWS_WITH_AS_MESSAGE(expr, with, ex, __VA_ARGS__)
03817 #define REQUIRE_NOTHROW_MESSAGE(expr, ...) DOCTEST_REQUIRE_NOTHROW_MESSAGE(expr, __VA_ARGS__)
03818 // clang-format on
03819
03820 #define SCENARIO(name) DOCTEST_SCENARIO(name)
03821 #define SCENARIO_METHOD(x, name) DOCTEST_SCENARIO_METHOD(x, name)
03822 #define SCENARIO_CLASS(name) DOCTEST_SCENARIO_CLASS(name)
03823 #define SCENARIO_TEMPLATE(name, T, ...) DOCTEST_SCENARIO_TEMPLATE(name, T, __VA_ARGS__)
03824 #define SCENARIO_TEMPLATE_DEFINE(name, T, id) DOCTEST_SCENARIO_TEMPLATE_DEFINE(name, T, id)
03825 #define GIVEN(name) DOCTEST_GIVEN(name)
03826 #define AND_GIVEN(name) DOCTEST_AND_GIVEN(name)
03827 #define WHEN(name) DOCTEST_WHEN(name)
03828 #define AND_WHEN(name) DOCTEST_AND_WHEN(name)
03829 #define THEN(name) DOCTEST_THEN(name)
03830 #define AND_THEN(name) DOCTEST_AND_THEN(name)
03831
03832 #define WARN_EQ(...) DOCTEST_WARN_EQ(__VA_ARGS__)
03833 #define CHECK_EQ(...) DOCTEST_CHECK_EQ(__VA_ARGS__)
03834 #define REQUIRE_EQ(...) DOCTEST_REQUIRE_EQ(__VA_ARGS__)
03835 #define WARN_NE(...) DOCTEST_WARN_NE(__VA_ARGS__)
03836 #define CHECK_NE(...) DOCTEST_CHECK_NE(__VA_ARGS__)
03837 #define REQUIRE_NE(...) DOCTEST_REQUIRE_NE(__VA_ARGS__)
03838 #define WARN_GT(...) DOCTEST_WARN_GT(__VA_ARGS__)
03839 #define CHECK_GT(...) DOCTEST_CHECK_GT(__VA_ARGS__)
03840 #define REQUIRE_GT(...) DOCTEST_REQUIRE_GT(__VA_ARGS__)
03841 #define WARN_LT(...) DOCTEST_WARN_LT(__VA_ARGS__)
03842 #define CHECK_LT(...) DOCTEST_CHECK_LT(__VA_ARGS__)
03843 #define REQUIRE_LT(...) DOCTEST_REQUIRE_LT(__VA_ARGS__)
03844 #define WARN_GE(...) DOCTEST_WARN_GE(__VA_ARGS__)
03845 #define CHECK_GE(...) DOCTEST_CHECK_GE(__VA_ARGS__)
03846 #define REQUIRE_GE(...) DOCTEST_REQUIRE_GE(__VA_ARGS__)
03847 #define WARN_LE(...) DOCTEST_WARN_LE(__VA_ARGS__)
03848 #define CHECK_LE(...) DOCTEST_CHECK_LE(__VA_ARGS__)
03849 #define REQUIRE_LE(...) DOCTEST_REQUIRE_LE(__VA_ARGS__)
03850 #define WARN_UNARY(...) DOCTEST_WARN_UNARY(__VA_ARGS__)
03851 #define CHECK_UNARY(...) DOCTEST_CHECK_UNARY(__VA_ARGS__)
03852 #define REQUIRE_UNARY(...) DOCTEST_REQUIRE_UNARY(__VA_ARGS__)

```

```

03853 #define WARN_UNARY_FALSE(...) DOCTEST_WARN_UNARY_FALSE(__VA_ARGS__)
03854 #define CHECK_UNARY_FALSE(...) DOCTEST_CHECK_UNARY_FALSE(__VA_ARGS__)
03855 #define REQUIRE_UNARY_FALSE(...) DOCTEST_REQUIRE_UNARY_FALSE(__VA_ARGS__)
03856
03857 // KEPT FOR BACKWARDS COMPATIBILITY
03858 #define FAST_WARN_EQ(...) DOCTEST_FAST_WARN_EQ(__VA_ARGS__)
03859 #define FAST_CHECK_EQ(...) DOCTEST_FAST_CHECK_EQ(__VA_ARGS__)
03860 #define FAST_REQUIRE_EQ(...) DOCTEST_FAST_REQUIRE_EQ(__VA_ARGS__)
03861 #define FAST_WARN_NE(...) DOCTEST_FAST_WARN_NE(__VA_ARGS__)
03862 #define FAST_CHECK_NE(...) DOCTEST_FAST_CHECK_NE(__VA_ARGS__)
03863 #define FAST_REQUIRE_NE(...) DOCTEST_FAST_REQUIRE_NE(__VA_ARGS__)
03864 #define FAST_WARN_GT(...) DOCTEST_FAST_WARN_GT(__VA_ARGS__)
03865 #define FAST_CHECK_GT(...) DOCTEST_FAST_CHECK_GT(__VA_ARGS__)
03866 #define FAST_REQUIRE_GT(...) DOCTEST_FAST_REQUIRE_GT(__VA_ARGS__)
03867 #define FAST_WARN_LT(...) DOCTEST_FAST_WARN_LT(__VA_ARGS__)
03868 #define FAST_CHECK_LT(...) DOCTEST_FAST_CHECK_LT(__VA_ARGS__)
03869 #define FAST_REQUIRE_LT(...) DOCTEST_FAST_REQUIRE_LT(__VA_ARGS__)
03870 #define FAST_WARN_GE(...) DOCTEST_FAST_WARN_GE(__VA_ARGS__)
03871 #define FAST_CHECK_GE(...) DOCTEST_FAST_CHECK_GE(__VA_ARGS__)
03872 #define FAST_REQUIRE_GE(...) DOCTEST_FAST_REQUIRE_GE(__VA_ARGS__)
03873 #define FAST_WARN_LE(...) DOCTEST_FAST_WARN_LE(__VA_ARGS__)
03874 #define FAST_CHECK_LE(...) DOCTEST_FAST_CHECK_LE(__VA_ARGS__)
03875 #define FAST_REQUIRE_LE(...) DOCTEST_FAST_REQUIRE_LE(__VA_ARGS__)
03876
03877 #define FAST_WARN_UNARY(...) DOCTEST_FAST_WARN_UNARY(__VA_ARGS__)
03878 #define FAST_CHECK_UNARY(...) DOCTEST_FAST_CHECK_UNARY(__VA_ARGS__)
03879 #define FAST_REQUIRE_UNARY(...) DOCTEST_FAST_REQUIRE_UNARY(__VA_ARGS__)
03880 #define FAST_WARN_UNARY_FALSE(...) DOCTEST_FAST_WARN_UNARY_FALSE(__VA_ARGS__)
03881 #define FAST_CHECK_UNARY_FALSE(...) DOCTEST_FAST_CHECK_UNARY_FALSE(__VA_ARGS__)
03882 #define FAST_REQUIRE_UNARY_FALSE(...) DOCTEST_FAST_REQUIRE_UNARY_FALSE(__VA_ARGS__)
03883
03884 #define TEST_CASE_TEMPLATE_INSTANTIATE(id, ...) DOCTEST_TEST_CASE_TEMPLATE_INSTANTIATE(id,
__VA_ARGS__)
03885
03886 #endif // DOCTEST_CONFIG_NO_SHORT_MACRO_NAMES
03887
03888 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
03889
03890 #endif // DOCTEST_PARTS_PUBLIC_MACROS
03891
03892 DOCTEST_SUPPRESS_PUBLIC_WARNINGS_POP
03893
03894 #endif // DOCTEST_LIBRARY_INCLUDED
03895
03896 #if defined(DOCTEST_CONFIG_IMPLEMENT) && !defined(DOCTEST_LIBRARY_IMPLEMENTATION)
03897
03898 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wunused-macros")
03899 #define DOCTEST_LIBRARY_IMPLEMENTATION
03900 DOCTEST_CLANG_SUPPRESS_WARNING_POP
03901
03902 #ifndef DOCTEST_PARTS_PRIVATE_PRELUDE
03903 #define DOCTEST_PARTS_PRIVATE_PRELUDE
03904
03905
03906 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
03907
03908 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
03909
03910 // required includes - will go only in one translation unit!
03911 #include <ctime>
03912 #include <cmath>
03913 #include <climits>
03914 // borland (Embarcadero) compiler requires math.h and not cmath -
03915 // https://github.com/doctest/doctest/pull/37
03916 #ifdef __BORLANDC__
03917 #include <math.h>
03918 #endif // __BORLANDC__
03919 #include <new>
03920 #include <cstdio>
03921 #include <cstdlib>
03922 #include <cstring>
03923 #include <limits>
03924 #include <utility>
03925 #include <fstream>
03926 #include <sstream>
03927 #ifndef DOCTEST_CONFIG_NO_INCLUDE_Iostream
03928 #include <iostream>
03929 #endif // DOCTEST_CONFIG_NO_INCLUDE_Iostream
03930 #include <algorithm>
03931 #include <iomanip>
03932 #include <vector>
03933 #ifndef DOCTEST_CONFIG_NO_MULTITHREADING
03934 #include <atomic>
03935 #include <mutex>
03936 #define DOCTEST_DECLARE_MUTEX(name) std::mutex name;
03937 #define DOCTEST_DECLARE_STATIC_MUTEX(name) static DOCTEST_DECLARE_MUTEX(name)
03938 #define DOCTEST_LOCK_MUTEX(name) const std::lock_guard<std::mutex>

```



```

        DOCTEST_ANONYMOUS(DOCTEST_ANON_LOCK_) (name);
03939 #else // DOCTEST_CONFIG_NO_MULTITHREADING
03940 #define DOCTEST_DECLARE_MUTEX(name)
03941 #define DOCTEST_DECLARE_STATIC_MUTEX(name)
03942 #define DOCTEST_LOCK_MUTEX(name)
03943 #endif // DOCTEST_CONFIG_NO_MULTITHREADING
03944 #include <set>
03945 #include <map>
03946 #include <unordered_set>
03947 #include <exception>
03948 #include <stdexcept>
03949 #if defined(DOCTEST_CONFIG_POSIX_SIGNALS) || defined(DOCTEST_CONFIG_WINDOWS_SEH)
03950 #include <csignal>
03951 #endif // DOCTEST_CONFIG_POSIX_SIGNALS
03952 #include <cfloat>
03953 #include <cctype>
03954 #include <cstdint>
03955 #include <string>
03956
03957 #ifdef DOCTEST_PLATFORM_MAC
03958 #include <sys/types.h>
03959 #include <unistd.h>
03960 #include <sys/sysctl.h>
03961 #endif // DOCTEST_PLATFORM_MAC
03962
03963 #ifdef DOCTEST_PLATFORM_WINDOWS
03964
03965 // defines for a leaner windows.h
03966 #ifndef WIN32_LEAN_AND_MEAN
03967 #define WIN32_LEAN_AND_MEAN
03968 #define DOCTEST_UNDEF_WIN32_LEAN_AND_MEAN
03969 #endif // WIN32_LEAN_AND_MEAN
03970 #ifndef NOMINMAX
03971 #define NOMINMAX
03972 #define DOCTEST_UNDEF_NOMINMAX
03973 #endif // NOMINMAX
03974
03975 // not sure what AfxWin.h is for - here I do what Catch does
03976 #ifdef __AFXDLL
03977 #include <AfxWin.h>
03978 #else
03979 #include <windows.h>
03980 #endif
03981 #include <io.h>
03982
03983 #ifdef DOCTEST_UNDEF_WIN32_LEAN_AND_MEAN
03984 #undef WIN32_LEAN_AND_MEAN
03985 #undef DOCTEST_UNDEF_WIN32_LEAN_AND_MEAN
03986 #endif // DOCTEST_UNDEF_WIN32_LEAN_AND_MEAN
03987 #ifdef DOCTEST_UNDEF_NOMINMAX
03988 #undef NOMINMAX
03989 #undef DOCTEST_UNDEF_NOMINMAX
03990 #endif // DOCTEST_UNDEF_NOMINMAX
03991
03992 #else // DOCTEST_PLATFORM_WINDOWS
03993
03994 #include <sys/time.h>
03995 #include <unistd.h>
03996
03997 #endif // DOCTEST_PLATFORM_WINDOWS
03998
03999 // this is a fix for https://github.com/doctest/doctest/issues/348
04000 // https://mail.gnome.org/archives/xml/2012-January/msg00000.html
04001 #if !defined(HAVE_UNISTD_H) && !defined(STDOUT_FILENO)
04002 #define STDOUT_FILENO fileno(stdout)
04003 #endif // HAVE_UNISTD_H
04004
04005 DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END
04006
04007 // counts the number of elements in a C array
04008 #define DOCTEST_COUNTOF(x) (sizeof(x) / sizeof(x[0]))
04009
04010 #ifdef DOCTEST_CONFIG_DISABLE
04011 #define DOCTEST_BRANCH_ON_DISABLED(if_disabled, if_not_disabled) if_disabled
04012 #else // DOCTEST_CONFIG_DISABLE
04013 #define DOCTEST_BRANCH_ON_DISABLED(if_disabled, if_not_disabled) if_not_disabled
04014 #endif // DOCTEST_CONFIG_DISABLE
04015
04016 #ifndef DOCTEST_THREAD_LOCAL
04017 #if defined(DOCTEST_CONFIG_NO_MULTITHREADING) || DOCTEST_MSVC && (DOCTEST_MSVC < DOCTEST_COMPILER(19,
0, 0))
04018 #define DOCTEST_THREAD_LOCAL
04019 #else // DOCTEST_MSVC
04020 #define DOCTEST_THREAD_LOCAL thread_local
04021 #endif // DOCTEST_MSVC
04022 #endif // DOCTEST_THREAD_LOCAL
04023

```

```

04024 #ifndef DOCTEST_MULTI_LANE_ATOMICS_THREAD_LANES
04025 #define DOCTEST_MULTI_LANE_ATOMICS_THREAD_LANES 32
04026 #endif
04027
04028 #ifndef DOCTEST_MULTI_LANE_ATOMICS_CACHE_LINE_SIZE
04029 #define DOCTEST_MULTI_LANE_ATOMICS_CACHE_LINE_SIZE 64
04030 #endif
04031
04032 #if defined(WINAPI_FAMILY) && (WINAPI_FAMILY == WINAPI_FAMILY_APP)
04033 #define DOCTEST_CONFIG_NO_MULTI_LANE_ATOMICS
04034 #endif
04035
04036 #ifndef DOCTEST_CDECL
04037 #define DOCTEST_CDECL __cdecl
04038 #endif
04039
04040 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04041
04042 #endif // DOCTEST_PARTS_PRIVATE_PRELUDE
04043 #ifndef DOCTEST_PARTS_PRIVATE_CONTEXT_STATE
04044 #define DOCTEST_PARTS_PRIVATE_CONTEXT_STATE
04045
04046 #ifndef DOCTEST_PARTS_PRIVATE_TIMER
04047 #define DOCTEST_PARTS_PRIVATE_TIMER
04048
04049 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04050
04051 #ifndef DOCTEST_CONFIG_DISABLE
04052
04053 namespace doctest {
04054 namespace detail {
04055
04056 namespace timer_large_integer {
04057
04058 #if defined(DOCTEST_PLATFORM_WINDOWS)
04059 using type = ULONGLONG;
04060 #else // DOCTEST_PLATFORM_WINDOWS
04061 using type = std::uint64_t;
04062 #endif // DOCTEST_PLATFORM_WINDOWS
04063 } // namespace timer_large_integer
04064
04065 using ticks_t = timer_large_integer::type;
04066
04067 ticks_t getCurrentTicks();
04068
04069 struct Timer {
04070     void start();
04071     unsigned int getElapsedMicroseconds() const;
04072     double getElapsedSeconds() const;
04073 private:
04074     ticks_t m_ticks = 0;
04075 };
04076
04077 } // namespace detail
04078 } // namespace doctest
04079
04080 #endif // DOCTEST_CONFIG_DISABLE
04081
04082 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04083
04084 #endif // DOCTEST_PARTS_PRIVATE_TIMER
04085 #ifndef DOCTEST_PARTS_PRIVATE_ATOMIC
04086 #define DOCTEST_PARTS_PRIVATE_ATOMIC
04087
04088 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04089
04090 #ifndef DOCTEST_CONFIG_DISABLE
04091
04092 namespace doctest {
04093 namespace detail {
04094
04095 #ifdef DOCTEST_CONFIG_NO_MULTITHREADING
04096 template <typename T>
04097 using Atomic = T;
04098 #else // DOCTEST_CONFIG_NO_MULTITHREADING
04099 template <typename T>
04100 using Atomic = std::atomic<T>;
04101 #endif // DOCTEST_CONFIG_NO_MULTITHREADING
04102
04103 #if defined(DOCTEST_CONFIG_NO_MULTI_LANE_ATOMICS) || defined(DOCTEST_CONFIG_NO_MULTITHREADING)
04104 template <typename T>
04105 using MultiLaneAtomic = Atomic<T>;
04106 #else // DOCTEST_CONFIG_NO_MULTI_LANE_ATOMICS
04107 // Provides a multilane implementation of an atomic variable that supports add, sub, load,

```

```

04111 // store. Instead of using a single atomic variable, this splits up into multiple ones,
04112 // each sitting on a separate cache line. The goal is to provide a speedup when most
04113 // operations are modifying. It achieves this with two properties:
04114 //
04115 // * Multiple atomics are used, so chance of congestion from the same atomic is reduced.
04116 // * Each atomic sits on a separate cache line, so false sharing is reduced.
04117 //
04118 // The disadvantage is that there is a small overhead due to the use of TLS, and load/store
04119 // is slower because all atomics have to be accessed.
04120 template <typename T>
04121 class MultiLaneAtomic {
04122     struct CacheLineAlignedAtomic {
04123         Atomic<T> atomic{};
04124         char padding[DOCTEST_MULTI_LANE_ATOMICS_CACHE_LINE_SIZE - sizeof(Atomic<T>)];
04125     };
04126     CacheLineAlignedAtomic m_atomics[DOCTEST_MULTI_LANE_ATOMICS_THREAD_LANES];
04127
04128     static_assert(
04129         sizeof(CacheLineAlignedAtomic) == DOCTEST_MULTI_LANE_ATOMICS_CACHE_LINE_SIZE,
04130         "guarantee one atomic takes exactly one cache line"
04131     );
04132
04133 public:
04134     T operator++() DOCTEST_NOEXCEPT {
04135         return fetch_add(1) + 1;
04136     }
04137
04138     T operator++(int) DOCTEST_NOEXCEPT { // NOLINT(cert-dcl21-cpp)
04139         return fetch_add(1);
04140     }
04141
04142     T fetch_add(T arg, std::memory_order order = std::memory_order_seq_cst) DOCTEST_NOEXCEPT {
04143         return myAtomic().fetch_add(arg, order);
04144     }
04145
04146     T fetch_sub(T arg, std::memory_order order = std::memory_order_seq_cst) DOCTEST_NOEXCEPT {
04147         return myAtomic().fetch_sub(arg, order);
04148     }
04149
04150     operator T() const DOCTEST_NOEXCEPT {
04151         return load();
04152     }
04153
04154     T load(std::memory_order order = std::memory_order_seq_cst) const DOCTEST_NOEXCEPT {
04155         auto result = T();
04156         for (const auto &c: m_atomics) {
04157             result += c.atomic.load(order);
04158         }
04159         return result;
04160     }
04161
04162     // NOLINTNEXTLINE(cppcoreguidelines-c-copy-assignment-signature,
04163     // misc-unconventional-assign-operator)
04164     T operator=(T desired) DOCTEST_NOEXCEPT {
04165         store(desired);
04166         return desired;
04167     }
04168
04169     void store(T desired, std::memory_order order = std::memory_order_seq_cst) DOCTEST_NOEXCEPT {
04170         // first value becomes desired", all others become 0.
04171         for (auto &c: m_atomics) {
04172             c.atomic.store(desired, order);
04173             desired = {};
04174         }
04175
04176 private:
04177     // Each thread has a different atomic that it operates on. If more than NumLanes threads
04178     // use this, some will use the same atomic. So performance will degrade a bit, but still
04179     // everything will work.
04180     //
04181     // The logic here is a bit tricky. The call should be as fast as possible, so that there
04182     // is minimal to no overhead in determining the correct atomic for the current thread.
04183     //
04184     // 1. A global static counter laneCounter counts continuously up.
04185     // 2. Each successive thread will use modulo operation of that counter so it gets an atomic
04186     //    assigned in a round-robin fashion.
04187     // 3. This tlsLaneIdx is stored in the thread local data, so it is directly available with
04188     //    little overhead.
04189     Atomic<T> &myAtomic() DOCTEST_NOEXCEPT {
04190         static Atomic<size_t> laneCounter;
04191         DOCTEST_THREAD_LOCAL size_t tlsLaneIdx = laneCounter++ %
04192             DOCTEST_MULTI_LANE_ATOMICS_THREAD_LANES;
04193
04194         return m_atomics[tlsLaneIdx].atomic;
04195     };

```

```

04196 #endif // DOCTEST_CONFIG_NO_MULTI_LANE_ATOMICS
04197
04198 } // namespace detail
04199 } // namespace doctest
04200
04201 #endif // DOCTEST_CONFIG_DISABLE
04202
04203 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04204
04205 #endif // DOCTEST_PARTS_PRIVATE_ATOMIC
04206 #ifndef DOCTEST_PARTS_PRIVATE_TRAVERSAL
04207 #define DOCTEST_PARTS_PRIVATE_TRAVERSAL
04208
04209 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04210
04211 #ifndef DOCTEST_CONFIG_DISABLE
04212
04213 namespace doctest {
04214 namespace detail {
04215
04216 struct DecisionPoint {
04217     // Number of branches available at this depth for the current traversal path.
04218     size_t branch_count = 0;
04219     // Encountered sibling subcases in source order for subcase decision points.
04220     std::vector<SubcaseSignature> subcases;
04221 };
04222
04223 class TraversalState {
04224 public:
04225     size_t activeSubcaseDepth() const {
04226         return m_activeSubcaseDepth;
04227     }
04228
04229     void resetForTestCase();
04230     void resetForRun();
04231     bool advance();
04232     bool tryEnterSubcase(const SubcaseSignature &signature);
04233     void leaveSubcase();
04234     size_t unwindActiveSubcases();
04235     size_t acquireGeneratorIndex(size_t count);
04236 private:
04237     // decisionPath is the selected traversal prefix; discoveredDecisionPath is rebuilt
04238     // on each rerun to describe the branches encountered at each depth.
04239     std::vector<DecisionPoint> m_discoveredDecisionPath;
04240     std::vector<size_t> m_decisionPath;
04241     size_t m_decisionDepth = 0;
04242     std::vector<size_t> m_enteredSubcaseDepths;
04243     size_t m_activeSubcaseDepth = 0;
04244
04245     DecisionPoint &ensureDecisionPointAtCurrentDepth();
04246 };
04247
04248 } // namespace detail
04249 } // namespace doctest
04250
04251 #endif // DOCTEST_CONFIG_DISABLE
04252
04253 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04254
04255 #endif // DOCTEST_PARTS_PRIVATE_TRAVERSAL
04256
04257 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04258
04259 #ifndef DOCTEST_CONFIG_DISABLE
04260
04261 namespace doctest {
04262 namespace detail {
04263
04264 // this holds both parameters from the command line and runtime data for tests
04265 struct ContextState : ContextOptions, TestRunStats, CurrentTestCaseStats {
04266     MultiLaneAtomic<int> numAssertsCurrentTest_atomic;
04267     MultiLaneAtomic<int> numAssertsFailedCurrentTest_atomic;
04268
04269     std::vector<std::vector<String>> filters = decltype(filters)(9); // 9 different filters
04270
04271     std::vector<IReporter *> reporters_currently_used;
04272
04273     assert_handler ah = nullptr;
04274
04275     Timer timer;
04276
04277     std::vector<String> stringifiedContexts; // logging from INFO() due to an exception
04278
04279     // Backtrack traversal state shared by SUBCASE and GENERATE.
04280     TraversalState traversal;

```

```

04283     Atomic<bool> shouldLogCurrentException;
04284
04285     void resetRunData();
04286
04287     void finalizeTestCaseData();
04288 };
04289
04290 extern ContextState *g_cs;
04291
04292 // used to avoid locks for the debug output
04293 // TODO: figure out if this is indeed necessary/correct - seems like either there still
04294 // could be a race or that there wouldn't be a race even if using the context directly
04295 extern DOCTEST_THREAD_LOCAL bool g_no_colors;
04296
04297 } // namespace detail
04298 } // namespace doctest
04299
04300 #endif // DOCTEST_CONFIG_DISABLE
04301
04302 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04303
04304 #endif // DOCTEST_PARTS_PRIVATE_CONTEXT_STATE
04305
04306 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04307
04308 #ifndef DOCTEST_CONFIG_DISABLE
04309
04310 namespace doctest {
04311
04312     using detail::g_cs;
04313
04314     AssertData::AssertData(
04315         assertType::Enum at,
04316         const char *file,
04317         int line,
04318         const char *expr,
04319         const char *exception_type,
04320         const StringContains &exception_string
04321     )
04322     : m_test_case(g_cs->currentTest),
04323       m_at(at),
04324       m_file(file),
04325       m_line(line),
04326       m_expr(expr),
04327       m_failed(true),
04328       m_threw(false),
04329       m_threw_as(false),
04330       m_exception_type(exception_type),
04331       m_exception_string(exception_string) {
04332     #if DOCTEST_MSVC
04333         if (m_expr[0] == ' ') // this happens when variadic macros are disabled under MSVC
04334             ++m_expr;
04335     #endif // MSVC
04336     }
04337
04338 } // namespace doctest
04339
04340 #endif // DOCTEST_CONFIG_DISABLE
04341
04342 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04343
04344 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04345
04346 #ifndef DOCTEST_CONFIG_DISABLE
04347
04348 namespace doctest {
04349     namespace detail {
04350
04351         ExpressionComposer::ExpressionComposer(assertType::Enum at)
04352             : m_at(at) {}
04353
04354     } // namespace detail
04355 } // namespace doctest
04356
04357 #endif // DOCTEST_CONFIG_DISABLE
04358
04359 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04360 #ifndef DOCTEST_PARTS_PRIVATE_REPORTER
04361 #define DOCTEST_PARTS_PRIVATE_REPORTER
04362
04363 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04364
04365 #ifndef DOCTEST_CONFIG_DISABLE
04366
04367 namespace doctest {
04368     namespace detail {

```

```

04370 // the int (priority) is part of the key for automatic sorting - sadly one can register a
04371 // reporter with a duplicate name and a different priority but hopefully that won't happen often :|
04372 using reporterMap = std::map<std::pair<int, String>, detail::reporterCreatorFunc>;
04373
04374 reporterMap &getReporters() noexcept;
04375 reporterMap &getListeners() noexcept;
04376 } // namespace detail
04377
04378 #define DOCTEST_ITERATE_THROUGH_REPORTERS(function, ...)
04379 \
04380 \
04381 \
04382 } // namespace doctest
04383
04384 #endif // DOCTEST_CONFIG_DISABLE
04385
04386 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04387
04388 #endif // DOCTEST_PARTS_PRIVATE_REPORTER
04389 #ifndef DOCTEST_PARTS_PRIVATE_ASSERT_HANDLER
04390 #define DOCTEST_PARTS_PRIVATE_ASSERT_HANDLER
04391
04392
04393 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04394
04395 #ifndef DOCTEST_CONFIG_DISABLE
04396
04397 namespace doctest {
04398 namespace detail {
04399
04400 void addAssert(assertType::Enum at);
04401
04402 void addFailedAssert(assertType::Enum at);
04403
04404 #if defined(DOCTEST_CONFIG_POSIX_SIGNALS) || defined(DOCTEST_CONFIG_WINDOWS_SEH)
04405 void reportFatal(const std::string &message);
04406 #endif // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
04407
04408 } // namespace detail
04409 } // namespace doctest
04410
04411 #endif // DOCTEST_CONFIG_DISABLE
04412
04413 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04414
04415 #endif // DOCTEST_PARTS_PRIVATE_ASSERT_HANDLER
04416
04417 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04418
04419 #ifndef DOCTEST_CONFIG_DISABLE
04420
04421 namespace doctest {
04422 namespace detail {
04423
04424 void addAssert(assertType::Enum at) {
04425     if ((at & assertType::is_warn) == 0)
04426         g_cs->numAssertsCurrentTest_atomic++;
04427 }
04428
04429 void addFailedAssert(assertType::Enum at) {
04430     if ((at & assertType::is_warn) == 0)
04431         g_cs->numAssertsFailedCurrentTest_atomic++;
04432 }
04433
04434 #if defined(DOCTEST_CONFIG_POSIX_SIGNALS) || defined(DOCTEST_CONFIG_WINDOWS_SEH)
04435 void reportFatal(const std::string &message) {
04436     g_cs->failure_flags |= TestCaseFailureReason::Crash;
04437
04438     DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_exception, {message.c_str(), true});
04439
04440     for (size_t i = g_cs->traversal.unwindActiveSubcases(); i > 0; --i)
04441         DOCTEST_ITERATE_THROUGH_REPORTERS(subcase_end, DOCTEST_EMPTY);
04442     g_cs->finalizeTestCaseData();
04443
04444     DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_end, *g_cs);
04445
04446     DOCTEST_ITERATE_THROUGH_REPORTERS(test_run_end, *g_cs);
04447 }
04448 #endif // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
04449
04450 void failed_out_of_a_testing_context(const AssertData &ad) {
04451     if (g_cs->ah)
04452         g_cs->ah(ad);
04453     else
04454         std::abort();

```

```

04455 }
04456
04457 bool decomp_assert(assertType::Enum at, const char *file, int line, const char *expr, const Result
    &result) {
04458     const bool failed = !result.m_passed;
04459
04460     // #####
04461     // IF THE DEBUGGER BREAKS HERE - GO 1 LEVEL UP IN THE CALLSTACK FOR THE FAILING ASSERT
04462     // THIS IS THE EFFECT OF HAVING 'DOCTEST_CONFIG_SUPER_FAST_ASSERTS' DEFINED
04463     // #####
04464     DOCTEST_ASSERT_OUT_OF_TESTS(result.m_decomp);
04465     DOCTEST_ASSERT_IN_TESTS(result.m_decomp);
04466     return !failed;
04467 }
04468
04469 } // namespace detail
04470 } // namespace doctest
04471
04472 #endif // DOCTEST_CONFIG_DISABLE
04473
04474 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04475
04476 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04477
04478 #ifndef DOCTEST_CONFIG_DISABLE
04479
04480 namespace doctest {
04481 namespace detail {
04482
04483     MessageBuilder::MessageBuilder(const char *file, int line, assertType::Enum severity) {
04484         m_stream = tlssPush();
04485         m_file = file;
04486         m_line = line;
04487         m_severity = severity;
04488     }
04489
04490     MessageBuilder::~MessageBuilder() noexcept(false) {
04491         if (!logged)
04492             tlssPop();
04493     }
04494
04495     bool MessageBuilder::log() {
04496         if (!logged) {
04497             m_string = tlssPop();
04498             logged = true;
04499         }
04500
04501         DOCTEST_ITERATE_THROUGH_REPORTERS(log_message, *this);
04502
04503         const bool isWarn = m_severity & assertType::is_warn;
04504
04505         // warn is just a message in this context so we don't treat it as an assert
04506         if (!isWarn) {
04507             addAssert(m_severity);
04508             addFailedAssert(m_severity);
04509         }
04510
04511         return isDebuggerActive() && !getContextOptions()->no_breaks && !isWarn &&
            (g_cs->currentTest == nullptr || !g_cs->currentTest->m_no_breaks); // break into debugger
04512     }
04513 }
04514
04515 void MessageBuilder::react() {
04516     if (m_severity & assertType::is_require)
04517         throwException();
04518 }
04519
04520 } // namespace detail
04521 } // namespace doctest
04522
04523 #endif // DOCTEST_CONFIG_DISABLE
04524
04525 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04526 #ifndef DOCTEST_PARTS_PRIVATE_EXCEPTION_TRANSLATOR
04527 #define DOCTEST_PARTS_PRIVATE_EXCEPTION_TRANSLATOR
04528
04529 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04530
04531 #ifndef DOCTEST_CONFIG_DISABLE
04532
04533 namespace doctest {
04534 namespace detail {
04535
04536     std::vector<const IExceptionTranslator *> &getExceptionTranslators() noexcept;
04537     String translateActiveException() noexcept;
04538
04539 } // namespace detail
04540 }

```

```

04541 } // namespace doctest
04542
04543 #endif // DOCTEST_CONFIG_DISABLE
04544
04545 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04546
04547 #endif // DOCTEST_PARTS_PRIVATE_EXCEPTION_TRANSLATOR
04548
04549 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04550
04551 #ifndef DOCTEST_CONFIG_DISABLE
04552
04553 namespace doctest {
04554 namespace detail {
04555
04556 Result::Result(bool passed, const String &decomposition)
04557     : m_passed(passed), m_decomp(decomposition) {}
04558
04559 ResultBuilder::ResultBuilder(
04560     assertType::Enum at,
04561     const char *file,
04562     int line,
04563     const char *expr,
04564     const char *exception_type,
04565     const String &exception_string
04566 )
04567     : AssertData(at, file, line, expr, exception_type, exception_string) {}
04568 // NOLINT(clang-analyzer-cplusplus.NewDeleteLeaks)
04569
04570 ResultBuilder::ResultBuilder(
04571     assertType::Enum at,
04572     const char *file,
04573     int line,
04574     const char *expr,
04575     const char *exception_type,
04576     const Contains &exception_string
04577 )
04578     : AssertData(at, file, line, expr, exception_type, exception_string) {}
04579
04580 void ResultBuilder::setResult(const Result &res) {
04581     m_decomp = res.m_decomp;
04582     m_failed = !res.m_passed;
04583 }
04584
04585 void ResultBuilder::translateException() {
04586     m_threw = true;
04587     m_exception = translateActiveException();
04588 }
04589
04590 bool ResultBuilder::log() {
04591     if (m_at & assertType::is_throws) {
04592         m_failed = !m_threw;
04593     } else if (m_at & assertType::is_throws_as) && (m_at & assertType::is_throws_with)) {
04594         m_failed = !m_threw_as || !m_exception_string.check(m_exception);
04595     } else if (m_at & assertType::is_throws_as) {
04596         m_failed = !m_threw_as;
04597     } else if (m_at & assertType::is_throws_with) {
04598         m_failed = !m_exception_string.check(m_exception);
04599     } else if (m_at & assertType::is_nothrow) {
04600         m_failed = m_threw;
04601     }
04602
04603     if (m_exception.size())
04604         m_exception = "\"" + m_exception + "\"";
04605
04606     if (is_running_in_test) {
04607         addAssert(m_at);
04608         DOCTEST_ITERATE_THROUGH_REPORTERS(log_assert, *this);
04609
04610         if (m_failed)
04611             addFailedAssert(m_at);
04612     } else if (m_failed) {
04613         failed_out_of_a_testing_context(*this);
04614     }
04615
04616     return m_failed && isDebuggerActive() && !getContextOptions()->no_breaks &&
04617         (g_cs->currentTest == nullptr || !g_cs->currentTest->m_no_breaks); // break into debugger
04618 }
04619
04620 void ResultBuilder::react() const {
04621     if (m_failed && checkIfShouldThrow(m_at))
04622         throwException();
04623 }
04624
04625 } // namespace detail
04626 } // namespace doctest
04627

```



```

04628 #endif // DOCTEST_CONFIG_DISABLE
04629
04630 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04631 #ifndef DOCTEST_PARTS_PRIVATE_EXCEPTIONS
04632 #define DOCTEST_PARTS_PRIVATE_EXCEPTIONS
04633
04634
04635 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04636
04637 namespace doctest {
04638 namespace detail {
04639
04640 template <typename Ex>
04641 DOCTEST_NORETURN void throw_exception(const Ex &e) {
04642 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
04643     throw e;
04644 #else // DOCTEST_CONFIG_NO_EXCEPTIONS
04645 #ifdef DOCTEST_CONFIG_HANDLE_EXCEPTION
04646     DOCTEST_CONFIG_HANDLE_EXCEPTION(e);
04647 #else // DOCTEST_CONFIG_HANDLE_EXCEPTION
04648 #ifndef DOCTEST_CONFIG_NO_INCLUDE_IOSTREAM
04649         std::cerr << "doctest will terminate because it needed to throw an exception.\n"
04650             << "The message was: " << e.what() << '\n';
04651 #endif // DOCTEST_CONFIG_NO_INCLUDE_IOSTREAM
04652 #endif // DOCTEST_CONFIG_HANDLE_EXCEPTION
04653         std::terminate();
04654 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
04655 }
04656
04657 #ifndef DOCTEST_INTERNAL_ERROR
04658 #define DOCTEST_INTERNAL_ERROR(msg)
04659 \
04660     detail::throw_exception(std::logic_error(__FILE__ ":" DOCTEST_TOSTR(__LINE__) ": Internal doctest
04661     error: " msg))
04662 #endif // DOCTEST_INTERNAL_ERROR
04663 } // namespace detail
04664 } // namespace doctest
04665
04666 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04667 #endif // DOCTEST_PARTS_PRIVATE_EXCEPTIONS
04668
04669 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04670
04671 namespace doctest {
04672
04673 const char *assertString(assertType::Enum at) {
04674     DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4061) // enum 'x' in switch of enum 'y' is not explicitly
04675     handled
04676 #define DOCTEST_GENERATE_ASSERT_TYPE_CASE(assert_type)
04677 \
04678     case assertType::DT_##assert_type: return #assert_type
04679 #define DOCTEST_GENERATE_ASSERT_TYPE_CASES(assert_type)
04680 \
04681     DOCTEST_GENERATE_ASSERT_TYPE_CASE(WARN_##assert_type);
04682 \
04683     DOCTEST_GENERATE_ASSERT_TYPE_CASE(CHECK_##assert_type);
04684 \
04685     DOCTEST_GENERATE_ASSERT_TYPE_CASE(REQUIRE_##assert_type)
04686 DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
04687 DOCTEST_CLANG_SUPPRESS_WARNING("-Wswitch-enum")
04688 DOCTEST_GCC_SUPPRESS_WARNING_PUSH
04689 DOCTEST_GCC_SUPPRESS_WARNING("-Wswitch-enum")
04690 switch (at) {
04691     DOCTEST_GENERATE_ASSERT_TYPE_CASE(WARN);
04692     DOCTEST_GENERATE_ASSERT_TYPE_CASE(CHECK);
04693     DOCTEST_GENERATE_ASSERT_TYPE_CASE(REQUIRE);
04694     DOCTEST_GENERATE_ASSERT_TYPE_CASES(FALSE);
04695     DOCTEST_GENERATE_ASSERT_TYPE_CASES(THROWS);
04696     DOCTEST_GENERATE_ASSERT_TYPE_CASES(THROWS_AS);
04697     DOCTEST_GENERATE_ASSERT_TYPE_CASES(THROWS_WITH);
04698     DOCTEST_GENERATE_ASSERT_TYPE_CASES(THROWS_WITH_AS);
04699     DOCTEST_GENERATE_ASSERT_TYPE_CASES(NOTHROW);
04700     DOCTEST_GENERATE_ASSERT_TYPE_CASES(EQ);
04701     DOCTEST_GENERATE_ASSERT_TYPE_CASES(NE);
04702     DOCTEST_GENERATE_ASSERT_TYPE_CASES(GT);
04703     DOCTEST_GENERATE_ASSERT_TYPE_CASES(LT);
04704     DOCTEST_GENERATE_ASSERT_TYPE_CASES(GE);
04705     DOCTEST_GENERATE_ASSERT_TYPE_CASES(LE);

```

```

04708
04709     DOCTEST_GENERATE_ASSERT_TYPE_CASES(UNARY);
04710     DOCTEST_GENERATE_ASSERT_TYPE_CASES(UNARY_FALSE);
04711
04712     default: DOCTEST_INTERNAL_ERROR("Tried stringifying invalid assert type!");
04713 }
04714 DOCTEST_CLANG_SUPPRESS_WARNING_POP
04715 DOCTEST_GCC_SUPPRESS_WARNING_POP
04716 DOCTEST_MSVC_SUPPRESS_WARNING_POP
04717 }
04718
04719 const char *failureString(assertType::Enum at) {
04720     if (at & assertType::is_warn)
04721         return "WARNING";
04722     if (at & assertType::is_check)
04723         return "ERROR";
04724     if (at & assertType::is_require)
04725         return "FATAL ERROR";
04726     return "";
04727 }
04728
04729 } // namespace doctest
04730
04731 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04732
04733 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04734
04735 #if !defined(DOCTEST_CONFIG_COLORS_NONE)
04736 #if !defined(DOCTEST_CONFIG_COLORS_WINDOWS) && !defined(DOCTEST_CONFIG_COLORS_ANSI)
04737 #ifdef DOCTEST_PLATFORM_WINDOWS
04738 #define DOCTEST_CONFIG_COLORS_WINDOWS
04739 #else // linux
04740 #define DOCTEST_CONFIG_COLORS_ANSI
04741 #endif // platform
04742 #endif // DOCTEST_CONFIG_COLORS_WINDOWS && DOCTEST_CONFIG_COLORS_ANSI
04743 #endif // DOCTEST_CONFIG_COLORS_NONE
04744
04745 namespace doctest {
04746
04747     namespace detail {
04748         void color_to_stream(std::ostream &s, Color::Enum) DOCTEST_BRANCH_ON_DISABLED({}, ; )
04749     } // namespace detail
04750
04751     namespace Color {
04752         std::ostream &operator<<(std::ostream &s, Color::Enum code) {
04753             detail::color_to_stream(s, code);
04754             return s;
04755         }
04756     } // namespace Color
04757
04758 #ifndef DOCTEST_CONFIG_DISABLE
04759     namespace detail {
04760         DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
04761         void color_to_stream(std::ostream &s, Color::Enum code) {
04762             static_cast<void>(s); // for DOCTEST_CONFIG_COLORS_NONE or DOCTEST_CONFIG_COLORS_WINDOWS
04763             static_cast<void>(code); // for DOCTEST_CONFIG_COLORS_NONE
04764 #ifdef DOCTEST_CONFIG_COLORS_ANSI
04765             if (g_no_colors || (isatty(STDOUT_FILENO) == false && getContextOptions()->force_colors == false))
04766                 return;
04767
04768             auto col = ""; // NOLINT(clang-analyzer-deadcode.DeadStores)
04769             DOCTEST_CLANG_SUPPRESS_WARNING_PUSH
04770             DOCTEST_CLANG_SUPPRESS_WARNING("-Wcovered-switch-default")
04771             switch (code) {
04772                 case Color::Red: col = "[0;31m"; break;
04773                 case Color::Green: col = "[0;32m"; break;
04774                 case Color::Blue: col = "[0;34m"; break;
04775                 case Color::Cyan: col = "[0;36m"; break;
04776                 case Color::Yellow: col = "[0;33m"; break;
04777                 case Color::Grey: col = "[1;30m"; break;
04778                 case Color::LightGrey: col = "[0;37m"; break;
04779                 case Color::BrightRed: col = "[1;31m"; break;
04780                 case Color::BrightGreen: col = "[1;32m"; break;
04781                 case Color::BrightWhite: col = "[1;37m"; break;
04782                 case Color::Bright: // invalid
04783                 case Color::None:
04784                 case Color::White:
04785                 default: col = "[0m";
04786             }
04787             DOCTEST_CLANG_SUPPRESS_WARNING_POP
04788             s << "\033" << col;
04789 #endif // DOCTEST_CONFIG_COLORS_ANSI
04790
04791 #ifdef DOCTEST_CONFIG_COLORS_WINDOWS
04792             if (g_no_colors || (isatty(_fileno(stdout)) == false && getContextOptions()->force_colors ==
04793 false))
04794                 return;

```

```

04794
04795     static struct ConsoleHelper {
04796         HANDLE stdoutHandle;
04797         WORD origFgAttrs;
04798         WORD origBgAttrs;
04799
04800         ConsoleHelper() {
04801             stdoutHandle = GetStdHandle(STD_OUTPUT_HANDLE);
04802             CONSOLE_SCREEN_BUFFER_INFO csbiInfo;
04803             GetConsoleScreenBufferInfo(stdoutHandle, &csbiInfo);
04804             origFgAttrs =
04805                 csbiInfo.wAttributes & ~(BACKGROUND_GREEN | BACKGROUND_RED | BACKGROUND_BLUE |
BACKGROUND_INTENSITY);
04806             origBgAttrs =
04807                 csbiInfo.wAttributes & ~(FOREGROUND_GREEN | FOREGROUND_RED | FOREGROUND_BLUE |
FOREGROUND_INTENSITY);
04808         }
04809     } ch;
04810
04811     #define DOCTEST_SET_ATTR(x) SetConsoleTextAttribute(ch.stdoutHandle, x | ch.origBgAttrs)
04812
04813     // clang-format off
04814     switch (code) {
04815         case Color::White:      DOCTEST_SET_ATTR(FOREGROUND_GREEN | FOREGROUND_RED |
FOREGROUND_BLUE); break;
04816         case Color::Red:        DOCTEST_SET_ATTR(FOREGROUND_RED);
break;
04817         case Color::Green:      DOCTEST_SET_ATTR(FOREGROUND_GREEN);
break;
04818         case Color::Blue:       DOCTEST_SET_ATTR(FOREGROUND_BLUE);
break;
04819         case Color::Cyan:       DOCTEST_SET_ATTR(FOREGROUND_BLUE | FOREGROUND_GREEN);
break;
04820         case Color::Yellow:     DOCTEST_SET_ATTR(FOREGROUND_RED | FOREGROUND_GREEN);
break;
04821         case Color::Grey:       DOCTEST_SET_ATTR(0);
break;
04822         case Color::LightGrey:  DOCTEST_SET_ATTR(FOREGROUND_INTENSITY);
break;
04823         case Color::BrightRed:  DOCTEST_SET_ATTR(FOREGROUND_INTENSITY | FOREGROUND_RED);
break;
04824         case Color::BrightGreen: DOCTEST_SET_ATTR(FOREGROUND_INTENSITY | FOREGROUND_GREEN);
break;
04825         case Color::BrightWhite: DOCTEST_SET_ATTR(FOREGROUND_INTENSITY | FOREGROUND_GREEN |
FOREGROUND_RED | FOREGROUND_BLUE); break;
04826         case Color::None:
04827         case Color::Bright:      // invalid
04828         default:                 DOCTEST_SET_ATTR(ch.origFgAttrs);
04829     }
04830     // clang-format on
04831 #endif // DOCTEST_CONFIG_COLORS_WINDOWS
04832 }
04833 DOCTEST_CLANG_SUPPRESS_WARNING_POP
04834 } // namespace detail
04835 #endif // DOCTEST_CONFIG_DISABLED
04836 } // namespace doctest
04837
04838 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04839
04840 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04841
04842 namespace doctest {
04843
04844     const ContextOptions *getContextOptions() {
04845         return DOCTEST_BRANCH_ON_DISABLED(nullptr, detail::g_cs);
04846     }
04847
04848 } // namespace doctest
04849
04850 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04851 #ifndef DOCTEST_PARTS_PRIVATE_REPORTERS_COMMON
04852 #define DOCTEST_PARTS_PRIVATE_REPORTERS_COMMON
04853
04854
04855 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04856
04857 #ifndef DOCTEST_CONFIG_OPTIONS_PREFIX
04858 #define DOCTEST_CONFIG_OPTIONS_PREFIX "dt-"
04859 #endif
04860
04861 #ifndef DOCTEST_CONFIG_DISABLE
04862
04863     namespace doctest {
04864
04865         void fulltext_log_assert_to_stream(std::ostream &s, const AssertData &rb);
04866
04867     } // namespace doctest

```

```

04868
04869 #endif // DOCTEST_CONFIG_DISABLE
04870
04871 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04872
04873 #endif // DOCTEST_PARTS_PRIVATE_REPORTERS_COMMON
04874 #ifndef DOCTEST_PARTS_PRIVATE_REPORTERS_DEBUG_OUTPUT_WINDOW
04875 #define DOCTEST_PARTS_PRIVATE_REPORTERS_DEBUG_OUTPUT_WINDOW
04876
04877 #ifndef DOCTEST_PARTS_PRIVATE_REPORTERS_CONSOLE
04878 #define DOCTEST_PARTS_PRIVATE_REPORTERS_CONSOLE
04879
04880
04881 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04882
04883 #ifdef DOCTEST_CONFIG_NO_UNPREFIXED_OPTIONS
04884 #define DOCTEST_OPTIONS_PREFIX_DISPLAY DOCTEST_CONFIG_OPTIONS_PREFIX
04885 #else
04886 #define DOCTEST_OPTIONS_PREFIX_DISPLAY ""
04887 #endif
04888
04889 #ifndef DOCTEST_CONFIG_DISABLE
04890
04891 namespace doctest {
04892
04893 struct Whitespace {
04894     int nrSpaces;
04895     explicit Whitespace(int nr);
04896 };
04897
04898 std::ostream &operator<<(std::ostream &out, const Whitespace &ws);
04899
04900 struct ConsoleReporter : public IReporter {
04901     std::ostream &s;
04902     bool hasLoggedCurrentTestStart;
04903     std::vector<SubcaseSignature> subcasesStack;
04904     size_t currentSubcaseLevel;
04905     DOCTEST_DECLARE_MUTEX(mutex)
04906
04907     // caching pointers/references to objects of these types - safe to do
04908     const ContextOptions &opt;
04909     const TestCaseData *tc;
04910
04911     ConsoleReporter(const ContextOptions &co);
04912
04913     ConsoleReporter(const ContextOptions &co, std::ostream &ostr);
04914
04915     // =====
04916     // WHAT FOLLOWS ARE HELPERS USED BY THE OVERRIDES OF THE VIRTUAL METHODS OF THE INTERFACE
04917     // =====
04918
04919     void separator_to_stream();
04920
04921     static const char *getSuccessOrFailString(bool success, assertType::Enum at, const char
    *success_str);
04922
04923     static Color::Enum getSuccessOrFailColor(bool success, assertType::Enum at);
04924
04925     void successOrFailColoredStringToStream(bool success, assertType::Enum at, const char *success_str
    = "SUCCESS");
04926
04927     void log_contexts();
04928
04929     // this was requested to be made virtual so users could override it
04930     virtual void file_line_to_stream(const char *file, int line, const char *tail = "");
04931
04932     void logTestStart();
04933
04934     void printVersion();
04935
04936     void printIntro();
04937
04938     void printHelp();
04939
04940     void printRegisteredReporters();
04941
04942     // =====
04943     // WHAT FOLLOWS ARE OVERRIDES OF THE VIRTUAL METHODS OF THE REPORTER INTERFACE
04944     // =====
04945
04946     void report_query(const QueryData &in) override;
04947
04948     void test_run_start() override;
04949
04950     void test_run_end(const TestRunStats &p) override;
04951
04952     void test_case_start(const TestCaseData &in) override;

```

```

04953
04954     void test_case_reenter(const TestCaseData &) override;
04955
04956     void test_case_end(const CurrentTestCaseStats &st) override;
04957
04958     void test_case_exception(const TestCaseException &e) override;
04959
04960     void subcase_start(const SubcaseSignature &subc) override;
04961
04962     void subcase_end() override;
04963
04964     void log_assert(const AssertData &rb) override;
04965
04966     void log_message(const MessageData &mb) override;
04967
04968     void test_case_skipped(const TestCaseData &) override;
04969 };
04970
04971 DOCTEST_REGISTER_REPORTER("console", 0, ConsoleReporter);
04972
04973 } // namespace doctest
04974
04975 #endif // DOCTEST_CONFIG_DISABLE
04976
04977 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
04978
04979 #endif // DOCTEST_PARTS_PRIVATE_REPORTERS_CONSOLE
04980
04981 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
04982
04983 #ifndef DOCTEST_CONFIG_DISABLE
04984
04985 namespace doctest {
04986 namespace detail {
04987
04988 #ifndef DOCTEST_PLATFORM_WINDOWS
04989 struct DebugOutputWindowReporter : public ConsoleReporter {
04990     DOCTEST_THREAD_LOCAL static std::ostringstream oss;
04991
04992     DebugOutputWindowReporter(const ContextOptions &co);
04993
04994     void test_run_start() override;
04995     void test_run_end(const TestRunStats &in) override;
04996     void test_case_start(const TestCaseData &in) override;
04997     void test_case_reenter(const TestCaseData &in) override;
04998     void test_case_end(const CurrentTestCaseStats &in) override;
04999     void test_case_exception(const TestCaseException &in) override;
05000     void subcase_start(const SubcaseSignature &in) override;
05001     void subcase_end(DOCTEST_EMPTY DOCTEST_EMPTY) override;
05002     void log_assert(const AssertData &in) override;
05003     void log_message(const MessageData &in) override;
05004     void test_case_skipped(const TestCaseData &in) override;
05005 };
05006 #endif // DOCTEST_PLATFORM_WINDOWS
05007
05008 } // namespace detail
05009 } // namespace doctest
05010
05011 #endif // DOCTEST_CONFIG_DISABLE
05012
05013 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05014
05015 #endif // DOCTEST_PARTS_PRIVATE_REPORTERS_DEBUG_OUTPUT_WINDOW
05016 #ifndef DOCTEST_PARTS_PRIVATE_TEST_CASE
05017 #define DOCTEST_PARTS_PRIVATE_TEST_CASE
05018
05019
05020 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05021
05022 #ifndef DOCTEST_CONFIG_DISABLE
05023
05024 namespace doctest {
05025 namespace detail {
05026
05027 // all the registered tests
05028 std::set<TestCase> &getRegisteredTests();
05029
05030 } // namespace detail
05031 } // namespace doctest
05032
05033 #endif // DOCTEST_CONFIG_DISABLE
05034
05035 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05036
05037 #endif // DOCTEST_PARTS_PRIVATE_TEST_CASE
05038 #ifndef DOCTEST_PARTS_PRIVATE_FILTERS
05039 #define DOCTEST_PARTS_PRIVATE_FILTERS

```

```

05040
05041
05042 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05043
05044 #ifndef DOCTEST_CONFIG_DISABLE
05045
05046 namespace doctest {
05047 namespace detail {
05048
05049 // matching of a string against a wildcard mask (case sensitivity configurable) taken from
05050 // https://www.codeproject.com/Articles/1088/Wildcard-string-compare-globbing
05051 int wilddcmp(const char *str, const char *wild, bool caseSensitive);
05052
05053 // checks if the name matches any of the filters (and can be configured what to do when empty)
05054 bool matchesAny(const char *name, const std::vector<String> &filters, bool matchEmpty, bool
caseSensitive);
05055
05056 } // namespace detail
05057 } // namespace doctest
05058
05059 #endif // DOCTEST_CONFIG_DISABLE
05060
05061 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05062
05063 #endif // DOCTEST_PARTS_PRIVATE_FILTERS
05064 #ifndef DOCTEST_PARTS_PRIVATE_SIGNALS
05065 #define DOCTEST_PARTS_PRIVATE_SIGNALS
05066
05067 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05068
05069 #ifndef DOCTEST_CONFIG_DISABLE
05070
05071 namespace doctest {
05072 namespace detail {
05073
05074 #if !defined(DOCTEST_CONFIG_POSIX_SIGNALS) && !defined(DOCTEST_CONFIG_WINDOWS_SEH)
05075 struct FatalConditionHandler {
05076     static void reset();
05077     static void allocateAltStackMem();
05078     static void freeAltStackMem();
05079 };
05080 #else // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
05081 #ifdef DOCTEST_PLATFORM_WINDOWS
05082
05083 struct SignalDefs {
05084     DWORD id;
05085     const char *name;
05086 };
05087
05088 struct FatalConditionHandler {
05089     static LONG CALLBACK handleException(PEXCEPTION_POINTERS ExceptionInfo);
05090     static void allocateAltStackMem();
05091     static void freeAltStackMem();
05092
05093     FatalConditionHandler();
05094
05095     static void reset();
05096
05097     ~FatalConditionHandler();
05098 private:
05099     static UINT prev_error_mode_1;
05100     static int prev_error_mode_2;
05101     static unsigned int prev_abort_behavior;
05102     static int prev_report_mode;
05103     static _HFILE prev_report_file;
05104     static void(DOCTEST_CDECL *prev_sigabrt_handler)(int);
05105     static std::terminate_handler original_terminate_handler;
05106     static bool isSet;
05107     static ULONG guaranteeSize;
05108     static LPTOP_LEVEL_EXCEPTION_FILTER previousTop;
05109 };
05110 #else // DOCTEST_PLATFORM_WINDOWS
05111
05112 struct SignalDefs {
05113     int id;
05114     const char *name;
05115 };
05116
05117 struct FatalConditionHandler {
05118     static bool isSet;
05119     static struct sigaction oldSigActions[6];
05120     static stack_t oldSigStack;
05121     static size_t altStackSize;

```

```

05126     static char *altStackMem;
05127
05128     static void handleSignal(int sig);
05129
05130     static void allocateAltStackMem();
05131
05132     static void freeAltStackMem();
05133
05134     FatalConditionHandler();
05135
05136     ~FatalConditionHandler();
05137     static void reset();
05138 };
05139
05140 #endif // DOCTEST_PLATFORM_WINDOWS
05141 #endif // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
05142
05143 } // namespace detail
05144 } // namespace doctest
05145
05146 #endif // DOCTEST_CONFIG_DISABLE
05147
05148 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05149
05150 #endif // DOCTEST_PARTS_PRIVATE_SIGNALS
05151
05152 // Fix for #1035
05153 #ifndef DOCTEST_PARTS_PRIVATE_REPORTERS_JUNIT
05154 #define DOCTEST_PARTS_PRIVATE_REPORTERS_JUNIT
05155
05156 #ifndef DOCTEST_PARTS_PRIVATE_XML
05157 #define DOCTEST_PARTS_PRIVATE_XML
05158
05159 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05160
05161 #ifndef DOCTEST_CONFIG_DISABLE
05162
05163 namespace doctest {
05164 namespace detail {
05165
05166 // =====
05167 // The following code has been taken verbatim from Catch2/include/internal/catch_xmlwriter.h
05168 // This is done so cherry-picking bug fixes is trivial - even the style/formatting is untouched.
05169 // =====
05170 /* clang-format off */ /* NOLINTBEGIN */
05171
05172     class XmlEncode {
05173     public:
05174         enum ForWhat { ForTextNodes, ForAttributes };
05175
05176         XmlEncode( std::string const& str, ForWhat forWhat = ForTextNodes );
05177
05178         void encodeTo( std::ostream& os ) const;
05179
05180         friend std::ostream& operator < ( std::ostream& os, XmlEncode const& xmlEncode );
05181
05182     private:
05183         std::string m_str;
05184         ForWhat m_forWhat;
05185     };
05186
05187     class XmlWriter {
05188     public:
05189         class ScopedElement {
05190         public:
05191             ScopedElement( XmlWriter* writer );
05192
05193             ScopedElement( ScopedElement&& other ) DOCTEST_NOEXCEPT;
05194             ScopedElement& operator=( ScopedElement&& other ) DOCTEST_NOEXCEPT;
05195
05196             ~ScopedElement();
05197
05198             ScopedElement& writeText( std::string const& text, bool indent = true );
05199
05200             template<typename T>
05201             ScopedElement& writeAttribute( std::string const& name, T const& attribute ) {
05202                 m_writer->writeAttribute( name, attribute );
05203                 return *this;
05204             }
05205
05206         private:
05207             mutable XmlWriter* m_writer = nullptr;
05208         };
05209
05210 #ifndef DOCTEST_CONFIG_NO_INCLUDE_Iostream

```

```

05213     XmlWriter( std::ostream& os = std::cout );
05214 #else // DOCTEST_CONFIG_NO_INCLUDE_Iostream
05215     XmlWriter( std::ostream& os );
05216 #endif // DOCTEST_CONFIG_NO_INCLUDE_Iostream
05217     ~XmlWriter();
05218
05219     XmlWriter( XmlWriter const& ) = delete;
05220     XmlWriter& operator=( XmlWriter const& ) = delete;
05221
05222     XmlWriter& startElement( std::string const& name );
05223
05224     ScopedElement scopedElement( std::string const& name );
05225
05226     XmlWriter& endElement();
05227
05228     XmlWriter& writeAttribute( std::string const& name, std::string const& attribute );
05229
05230     XmlWriter& writeAttribute( std::string const& name, const char* attribute );
05231
05232     XmlWriter& writeAttribute( std::string const& name, bool attribute );
05233
05234     template<typename T>
05235     XmlWriter& writeAttribute( std::string const& name, T const& attribute ) {
05236         std::stringstream rss;
05237         rss << attribute;
05238         return writeAttribute( name, rss.str() );
05239     }
05240
05241     XmlWriter& writeText( std::string const& text, bool indent = true );
05242
05243     //XmlWriter& writeComment( std::string const& text );
05244
05245     //void writeStylesheetRef( std::string const& url );
05246
05247     //XmlWriter& writeBlankLine();
05248
05249     void ensureTagClosed();
05250
05251     void writeDeclaration();
05252
05253     private:
05254
05255         void newlineIfNecessary();
05256
05257         bool m_tagIsOpen = false;
05258         bool m_needsNewline = false;
05259         std::vector<std::string> m_tags;
05260         std::string m_indent;
05261         std::ostream& m_os;
05262     };
05263
05264     /* clang-format on */ /* NOLINTEND */
05265     // =====
05266     // End of copy-pasted code from Catch
05267     // =====
05268
05269 } // namespace detail
05270 } // namespace doctest
05271
05272 #endif // DOCTEST_CONFIG_DISABLE
05273
05274 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05275
05276 #endif // DOCTEST_PARTS_PRIVATE_XML
05277
05278 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05279
05280 #ifndef DOCTEST_CONFIG_DISABLE
05281 namespace doctest {
05282
05283     // TODO:
05284     // - log_message()
05285     // - respond to queries
05286     // - honor remaining options
05287     // - more attributes in tags
05288     struct JUnitReporter : public IReporter {
05289         detail::XmlWriter xml;
05290         DOCTEST_DECLARE_MUTEX(mutex)
05291         detail::Timer timer;
05292         std::vector<String> deepestSubcaseStackNames;
05293
05294         struct JUnitTestCaseData {
05295             static std::string getTimestamp();
05296
05297             struct JUnitTestMessage {
05298                 JUnitTestMessage(const std::string &_message, const std::string &_type, const std::string

```



```

        &_details);
05300
05301         JUnitTestMessage(const std::string &_message, const std::string &_details);
05302
05303         std::string message, type, details;
05304     };
05305
05306     struct JUnitTestCase {
05307         JUnitTestCase(const std::string &_classname, const std::string &_name);
05308
05309         std::string classname, name;
05310         double time;
05311         std::vector<JUnitTestMessage> failures, errors;
05312     };
05313
05314     void add(const std::string &classname, const std::string &name);
05315
05316     void appendSubcaseNamesToLastTestcase(std::vector<String> nameStack);
05317
05318     void addTime(double time);
05319
05320     void addFailure(const std::string &message, const std::string &type, const std::string
&details);
05321
05322     void addError(const std::string &message, const std::string &details);
05323
05324     std::vector<JUnitTestCase> testcases;
05325     double totalSeconds = 0;
05326     int totalErrors = 0, totalFailures = 0;
05327 };
05328
05329 JUnitTestCaseData testCaseData;
05330
05331 // caching pointers/references to objects of these types - safe to do
05332 const ContextOptions &opt;
05333 const TestCaseData *tc = nullptr;
05334
05335 JUnitReporter(const ContextOptions &co);
05336
05337 unsigned line(unsigned l) const;
05338
05339 // =====
05340 // WHAT FOLLOWS ARE OVERRIDES OF THE VIRTUAL METHODS OF THE REPORTER INTERFACE
05341 // =====
05342
05343 void report_query(const QueryData &) override;
05344
05345 void test_run_start() override;
05346
05347 void test_run_end(const TestRunStats &p) override;
05348
05349 void test_case_start(const TestCaseData &in) override;
05350
05351 void test_case_reenter(const TestCaseData &in) override;
05352
05353 void test_case_end(const CurrentTestCaseStats &) override;
05354
05355 void test_case_exception(const TestCaseException &e) override;
05356
05357 void subcase_start(const SubcaseSignature &in) override;
05358
05359 void subcase_end() override;
05360
05361 void log_assert(const AssertData &rb) override;
05362
05363 void log_message(const MessageData &mb) override;
05364
05365 void test_case_skipped(const TestCaseData &) override;
05366
05367 static void log_contexts(std::ostream &s);
05368 };
05369
05370 DOCTEST_REGISTER_REPORTER("junit", 0, JUnitReporter);
05371
05372 } // namespace doctest
05373
05374 #endif // DOCTEST_CONFIG_DISABLE
05375
05376 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05377
05378 #endif // DOCTEST_PARTS_PRIVATE_REPORTERS_JUNIT
05379 #ifndef DOCTEST_PARTS_PRIVATE_REPORTERS_XML
05380 #define DOCTEST_PARTS_PRIVATE_REPORTERS_XML
05381
05382 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05383
05384

```

```

05385 #ifndef DOCTEST_CONFIG_DISABLE
05386
05387 namespace doctest {
05388
05389 struct XmlReporter : public IReporter {
05390     detail::XmlWriter xml;
05391     DOCTEST_DECLARE_MUTEX(mutex)
05392
05393     // caching pointers/references to objects of these types - safe to do
05394     const ContextOptions &opt;
05395     const TestCaseData *tc = nullptr;
05396
05397     XmlReporter(const ContextOptions &co);
05398
05399     void log_contexts();
05400
05401     unsigned line(unsigned l) const;
05402
05403     void test_case_start_impl(const TestCaseData &in);
05404
05405     // =====
05406     // WHAT FOLLOWS ARE OVERRIDES OF THE VIRTUAL METHODS OF THE REPORTER INTERFACE
05407     // =====
05408
05409     void report_query(const QueryData &in) override;
05410
05411     void test_run_start() override;
05412
05413     void test_run_end(const TestRunStats &p) override;
05414
05415     void test_case_start(const TestCaseData &in) override;
05416
05417     void test_case_reenter(const TestCaseData &) override;
05418
05419     void test_case_end(const CurrentTestCaseStats &st) override;
05420
05421     void test_case_exception(const TestCaseException &e) override;
05422
05423     void subcase_start(const SubcaseSignature &in) override;
05424
05425     void subcase_end() override;
05426
05427     void log_assert(const AssertData &rb) override;
05428
05429     void log_message(const MessageData &mb) override;
05430
05431     void test_case_skipped(const TestCaseData &in) override;
05432 };
05433
05434 DOCTEST_REGISTER_REPORTER("xml", 0, XmlReporter);
05435
05436 } // namespace doctest
05437
05438 #endif // DOCTEST_CONFIG_DISABLE
05439
05440 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
05441
05442 #endif // DOCTEST_PARTS_PRIVATE_REPORTERS_XML
05443
05444 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
05445
05446 namespace doctest {
05447
05448 bool is_running_in_test = false;
05449
05450 #ifndef DOCTEST_CONFIG_DISABLE
05451
05452 // NOLINTBEGIN(readability-convert-member-functions-to-static)
05453 Context::Context(int, const char *const *) {}
05454 Context::~Context() = default;
05455 void Context::applyCommandLine(int, const char *const *) {}
05456 void Context::addFilter(const char *, const char *) {}
05457 void Context::clearFilters() {}
05458 void Context::setOption(const char *, bool) {}
05459 void Context::setOption(const char *, int) {}
05460 void Context::setOption(const char *, const char *) {}
05461 bool Context::shouldExit() {
05462     return false;
05463 }
05464 void Context::setAsDefaultForAssertsOutOfTestCases() {}
05465 void Context::setAssertHandler(detail::assert_handler) {}
05466 void Context::setCout(std::ostream *) {}
05467 int Context::run() {
05468     return 0;
05469 }
05470 // NOLINTEND(readability-convert-member-functions-to-static)
05471

```

```

05472 #else
05473
05474 namespace detail {
05475 // for sorting tests by file/line
05476 bool fileOrderComparator(const TestCase *lhs, const TestCase *rhs) {
05477     // this is needed because MSVC gives different case for drive letters
05478     // for __FILE__ when evaluated in a header and a source file
05479     const int res = lhs->m_file.compare(rhs->m_file, static_cast<bool>(DOCTEST_MSVC));
05480     if (res != 0)
05481         return res < 0;
05482     if (lhs->m_line != rhs->m_line)
05483         return lhs->m_line < rhs->m_line;
05484     return lhs->m_template_id < rhs->m_template_id;
05485 }
05486
05487 // for sorting tests by suite/file/line
05488 bool suiteOrderComparator(const TestCase *lhs, const TestCase *rhs) {
05489     const int res = std::strcmp(lhs->m_test_suite, rhs->m_test_suite);
05490     if (res != 0)
05491         return res < 0;
05492     return fileOrderComparator(lhs, rhs);
05493 }
05494
05495 // for sorting tests by name/suite/file/line
05496 bool nameOrderComparator(const TestCase *lhs, const TestCase *rhs) {
05497     const int res = std::strcmp(lhs->m_name, rhs->m_name);
05498     if (res != 0)
05499         return res < 0;
05500     return suiteOrderComparator(lhs, rhs);
05501 }
05502
05503 // the implementation of parseOption()
05504 bool parseOptionImpl(int argc, const char *const *argv, const char *pattern, String *value) {
05505     // going from the end to the beginning and stopping on the first occurrence from the end
05506     for (int i = argc; i > 0; --i) {
05507         auto index = i - 1;
05508         auto temp = std::strstr(argv[index], pattern);
05509         if (temp && (value || strlen(temp) == strlen(pattern))) {
05510             // eliminate matches in which the chars before the option are not '-'
05511             bool noBadCharsFound = true;
05512             auto curr = argv[index];
05513             while (curr != temp) {
05514                 if (*curr++ != '-') {
05515                     noBadCharsFound = false;
05516                     break;
05517                 }
05518             }
05519             if (noBadCharsFound && argv[index][0] == '-') {
05520                 if (value) {
05521                     // parsing the value of an option
05522                     temp += strlen(pattern);
05523                     const unsigned len = strlen(temp);
05524                     if (len) {
05525                         *value = temp;
05526                         return true;
05527                     }
05528                 } else {
05529                     // just a flag - no value
05530                     return true;
05531                 }
05532             }
05533         }
05534     }
05535     return false;
05536 }
05537
05538 // parses an option and returns the string after the '=' character
05539 bool parseOption(
05540     int argc, const char *const *argv, const char *pattern, String *value = nullptr, const String
    &defaultVal = String()
05541 ) {
05542     if (value)
05543         *value = defaultVal;
05544 #ifndef DOCTEST_CONFIG_NO_UNPREFIXED_OPTIONS
05545     // offset (normally 3 for "dt-") to skip prefix
05546     if (parseOptionImpl(argc, argv, pattern + strlen(DOCTEST_CONFIG_OPTIONS_PREFIX), value))
05547         return true;
05548 #endif // DOCTEST_CONFIG_NO_UNPREFIXED_OPTIONS
05549     return parseOptionImpl(argc, argv, pattern, value);
05550 }
05551
05552 // locates a flag on the command line
05553 bool parseFlag(int argc, const char *const *argv, const char *pattern) {
05554     return parseOption(argc, argv, pattern);
05555 }
05556
05557 // parses a comma separated list of words after a pattern in one of the arguments in argv

```

```

05558 bool parseCommaSepArgs(int argc, const char *const *argv, const char *pattern, std::vector<String>
05559 &res) {
05560     String filtersString;
05561     if (parseOption(argc, argv, pattern, &filtersString)) {
05562         // tokenize with "," as a separator, unless escaped with backslash
05563         std::ostream s;
05564         auto flush = [&s, &res]() {
05565             auto string = s.str();
05566             if (!string.empty()) {
05567                 res.emplace_back(string.c_str());
05568             }
05569             s.str("");
05570         };
05571         bool seenBackslash = false;
05572         const char *current = filtersString.c_str();
05573         const char *end = current + strlen(current);
05574         while (current != end) {
05575             const char character = *current++;
05576             if (seenBackslash) {
05577                 seenBackslash = false;
05578                 if (character == ',' || character == '\\') {
05579                     s.put(character);
05580                     continue;
05581                 }
05582                 s.put('\\');
05583             }
05584             if (character == '\\') {
05585                 seenBackslash = true;
05586             } else if (character == ',') {
05587                 flush();
05588             } else {
05589                 s.put(character);
05590             }
05591         }
05592         if (seenBackslash) {
05593             s.put('\\');
05594         }
05595         flush();
05596         return true;
05597     }
05598     return false;
05599 }
05600 }
05601
05602 enum optionType { option_bool, option_int };
05603
05604 // parses an int/bool option from the command line
05605 bool parseIntOption(int argc, const char *const *argv, const char *pattern, optionType type, int &res)
05606 {
05607     String parsedValue;
05608     if (!parseOption(argc, argv, pattern, &parsedValue))
05609         return false;
05610     if (type) {
05611         // integer
05612         // TODO: change this to use std::stoi or something else! currently it uses undefined
05613         // behavior - assumes '0' on failed parse...
05614         // NOLINTNEXTLINE(bugprone-unchecked-string-to-number-conversion, cert-err34-c)
05615         const int theInt = std::atoi(parsedValue.c_str());
05616         if (theInt != 0) {
05617             res = theInt;
05618             return true;
05619         }
05620     } else {
05621         // boolean
05622         const char positive[][5] = {"1", "true", "on", "yes"}; // 5 - strlen("true") + 1
05623         const char negative[][6] = {"0", "false", "off", "no"}; // 6 - strlen("false") + 1
05624         // if the value matches any of the positive/negative possibilities
05625         for (unsigned i = 0; i < 4; i++) {
05626             if (parsedValue.compare(positive[i], true) == 0) {
05627                 res = 1;
05628                 return true;
05629             }
05630             if (parsedValue.compare(negative[i], true) == 0) {
05631                 res = 0;
05632                 return true;
05633             }
05634         }
05635     }
05636     return false;
05637 }
05638 }
05639
05640 } // namespace detail
05641
05642 Context::Context(int argc, const char *const *argv)

```

```

05643     : p(new detail::ContextState) {
05644         parseArgs(argc, argv, true);
05645         if (argc)
05646             p->binary_name = argv[0];
05647     }
05648
05649 Context::~Context() {
05650     if (detail::g_cs == p)
05651         detail::g_cs = nullptr;
05652     delete p;
05653 }
05654
05655 void Context::applyCommandLine(int argc, const char *const *argv) {
05656     parseArgs(argc, argv);
05657     if (argc)
05658         p->binary_name = argv[0];
05659 }
05660
05661 // parses args
05662 void Context::parseArgs(int argc, const char *const *argv, bool withDefaults) {
05663     using namespace detail;
05664
05665     // clang-format off
05666     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "source-file=", p->filters[0]);
05667     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "sf=", p->filters[0]);
05668     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "source-file-exclude=", p->filters[1]);
05669     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "sfe=", p->filters[1]);
05670     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "test-suite=", p->filters[2]);
05671     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "ts=", p->filters[2]);
05672     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "test-suite-exclude=", p->filters[3]);
05673     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "tse=", p->filters[3]);
05674     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "test-case=", p->filters[4]);
05675     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "tc=", p->filters[4]);
05676     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "test-case-exclude=", p->filters[5]);
05677     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "tce=", p->filters[5]);
05678     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "subcase=", p->filters[6]);
05679     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "sc=", p->filters[6]);
05680     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "subcase-exclude=", p->filters[7]);
05681     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "sce=", p->filters[7]);
05682     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "reporters=", p->filters[8]);
05683     parseCommaSepArgs(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "r=", p->filters[8]);
05684     // clang-format on
05685
05686     int intRes = 0;
05687     String strRes;
05688
05689 #define DOCTEST_PARSE_AS_BOOL_OR_FLAG(name, sname, var, default)
05690     \
05691     \
05692     \
05693     \
05694     \
05695     \
05696     \
05697     \
05698     \
05699     \
05700
05701 #define DOCTEST_PARSE_INT_OPTION(name, sname, var, default)
05702     \
05703     \
05704     \
05705     \
05706     \
05707
05708 #define DOCTEST_PARSE_STR_OPTION(name, sname, var, default)
05709     \
05710     \
05711     \

```

```

05712
05713 DOCTEST_PARSE_STR_OPTION("out", "o", out, "");
05714 DOCTEST_PARSE_STR_OPTION("order-by", "ob", order_by, "file");
05715 DOCTEST_PARSE_INT_OPTION("rand-seed", "rs", rand_seed, 0);
05716
05717 DOCTEST_PARSE_INT_OPTION("first", "f", first, 0);
05718 DOCTEST_PARSE_INT_OPTION("last", "l", last, UINT_MAX);
05719
05720 DOCTEST_PARSE_INT_OPTION("abort-after", "aa", abort_after, 0);
05721 DOCTEST_PARSE_INT_OPTION("subcase-filter-levels", "scfl", subcase_filter_levels, INT_MAX);
05722
05723 DOCTEST_PARSE_AS_BOOL_OR_FLAG("success", "s", success, false);
05724 DOCTEST_PARSE_AS_BOOL_OR_FLAG("case-sensitive", "cs", case_sensitive, false);
05725 DOCTEST_PARSE_AS_BOOL_OR_FLAG("exit", "e", exit, false);
05726 DOCTEST_PARSE_AS_BOOL_OR_FLAG("duration", "d", duration, false);
05727 DOCTEST_PARSE_AS_BOOL_OR_FLAG("minimal", "m", minimal, false);
05728 DOCTEST_PARSE_AS_BOOL_OR_FLAG("quiet", "q", quiet, false);
05729 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-throw", "nt", no_throw, false);
05730 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-exitcode", "ne", no_exitcode, false);
05731 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-run", "nr", no_run, false);
05732 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-intro", "ni", no_intro, false);
05733 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-version", "nv", no_version, false);
05734 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-colors", "nc", no_colors, false);
05735 DOCTEST_PARSE_AS_BOOL_OR_FLAG("force-colors", "fc", force_colors, false);
05736 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-breaks", "nb", no_breaks, false);
05737 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-skip", "ns", no_skip, false);
05738 DOCTEST_PARSE_AS_BOOL_OR_FLAG("gnu-file-line", "gfl", gnu_file_line, !bool(DOCTEST_MSVC));
05739 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-path-filenames", "npf", no_path_in_filenames, false);
05740 DOCTEST_PARSE_STR_OPTION("strip-file-prefixes", "sfp", strip_file_prefixes, "");
05741 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-line-numbers", "nln", no_line_numbers, false);
05742 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-debug-output", "ndo", no_debug_output, false);
05743 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-skipped-summary", "nss", no_skipped_summary, false);
05744 DOCTEST_PARSE_AS_BOOL_OR_FLAG("no-time-in-output", "ntio", no_time_in_output, false);
05745
05746 if (withDefaults) {
05747     p->help = false;
05748     p->version = false;
05749     p->count = false;
05750     p->list_test_cases = false;
05751     p->list_test_suites = false;
05752     p->list_reporters = false;
05753 }
05754 if (parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "help") ||
05755     parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "h") ||
05756     parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "?")) {
05757     p->help = true;
05758     p->exit = true;
05759 }
05760 if (parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "version") ||
05761     parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "v")) {
05762     p->version = true;
05763     p->exit = true;
05764 }
05765 if (parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "count") ||
05766     parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "c")) {
05767     p->count = true;
05768     p->exit = true;
05769 }
05770 if (parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "list-test-cases") ||
05771     parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "ltc")) {
05772     p->list_test_cases = true;
05773     p->exit = true;
05774 }
05775 if (parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "list-test-suites") ||
05776     parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "lts")) {
05777     p->list_test_suites = true;
05778     p->exit = true;
05779 }
05780 if (parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "list-reporters") ||
05781     parseFlag(argc, argv, DOCTEST_CONFIG_OPTIONS_PREFIX "lr")) {
05782     p->list_reporters = true;
05783     p->exit = true;
05784 }
05785 }
05786
05787 // allows the user to add procedurally to the filters from the command line
05788 void Context::addFilter(const char *filter, const char *value) {
05789     setOption(filter, value);
05790 }
05791
05792 // allows the user to clear all filters from the command line
05793 void Context::clearFilters() {
05794     for (auto &curr: p->filters)
05795         curr.clear();
05796 }
05797
05798 // allows the user to override procedurally the bool options from the command line

```

```

05799 void Context::setOption(const char *option, bool value) {
05800     setOption(option, value ? "true" : "false");
05801 }
05802
05803 // allows the user to override procedurally the int options from the command line
05804 void Context::setOption(const char *option, int value) {
05805     setOption(option, toString(value).c_str());
05806 }
05807
05808 // allows the user to override procedurally the string options from the command line
05809 void Context::setOption(const char *option, const char *value) {
05810     auto argv = String("-") + option + "=" + value;
05811     auto lvalue = argv.c_str();
05812     parseArgs(1, &lvalue);
05813 }
05814
05815 // users should query this in their main() and exit the program if true
05816 bool Context::shouldExit() {
05817     return p->exit;
05818 }
05819
05820 void Context::setAsDefaultForAssertsOutOfTestCases() {
05821     detail::g_cs = p;
05822 }
05823
05824 void Context::setAssertHandler(detail::assert_handler ah) {
05825     p->ah = ah;
05826 }
05827
05828 void Context::setCout(std::ostream *out) {
05829     p->cout = out;
05830 }
05831
05832 static class DiscardOStream : public std::ostream {
05833 private:
05834     class : public std::streambuf {
05835     private:
05836         // allowing some buffering decreases the amount of calls to overflow
05837         char buf[1024];
05838     protected:
05839         std::streamsize xsputn(const char_type *, std::streamsize count) override {
05840             return count;
05841         }
05842     };
05843     int_type overflow(int_type ch) override {
05844         setp(std::begin(buf), std::end(buf));
05845         return traits_type::not_eof(ch);
05846     }
05847 } discardBuf;
05848
05849 public:
05850     DiscardOStream() noexcept
05851         : std::ostream(&discardBuf) {}
05852 } discardOut;
05853
05854 // the main function that does all the filtering and test running
05855 int Context::run() {
05856     using namespace detail;
05857
05858     // save the old context state in case such was setup - for using asserts out of a testing context
05859     auto old_cs = g_cs;
05860     // this is the current contest
05861     g_cs = p;
05862     is_running_in_test = true;
05863
05864     g_no_colors = p->no_colors;
05865     p->resetRunData();
05866
05867     std::fstream fstr;
05868     if (p->cout == nullptr) {
05869         if (p->quiet) {
05870             p->cout = &discardOut;
05871         } else if (p->out.size()) {
05872             // to a file if specified
05873             fstr.open(p->out.c_str(), std::fstream::out);
05874             p->cout = &fstr;
05875             if (!fstr.is_open()) {
05876                 // clang-format off
05877                 std::cerr << Color::Cyan << "[doctest] " << Color::None << "Could not open " << p->out << "
05878 for writing!" << std::endl;
05879                 std::cerr << Color::Cyan << "[doctest] " << Color::None << "Defaulting to std::cout
05880 instead" << std::endl;
05881                 p->cout = &std::cout;
05882                 // clang-format on
05883             }
05884         }
05885     }

```

```

05884     } else {
05885 #ifndef DOCTEST_CONFIG_NO_INCLUDE_Iostream
05886     // stdout by default
05887     p->cout = &std::cout;
05888 #else // DOCTEST_CONFIG_NO_INCLUDE_Iostream
05889     return EXIT_FAILURE;
05890 #endif // DOCTEST_CONFIG_NO_INCLUDE_Iostream
05891 }
05892 }
05893
05894 FatalConditionHandler::allocateAltStackMem();
05895
05896 auto cleanup_and_return = [&]() {
05897     FatalConditionHandler::freeAltStackMem();
05898
05899     if (fstr.is_open())
05900         fstr.close();
05901
05902     // restore context
05903     g_cs = old_cs;
05904     is_running_in_test = false;
05905
05906     // we have to free the reporters which were allocated when the run started
05907     for (auto &curr: p->reporters_currently_used)
05908         delete curr;
05909     p->reporters_currently_used.clear();
05910
05911     if (p->numTestCasesFailed && !p->no_exitcode)
05912         return EXIT_FAILURE;
05913     return EXIT_SUCCESS;
05914 };
05915
05916 // setup default reporter if none is given through the command line
05917 if (p->filters[8].empty())
05918     p->filters[8].emplace_back("console");
05919
05920 // check to see if any of the registered reporters has been selected
05921 for (auto &curr: getReporters()) {
05922     if (matchesAny(curr.first.second.c_str(), p->filters[8], false, p->case_sensitive))
05923         p->reporters_currently_used.push_back(curr.second(*g_cs));
05924 }
05925
05926 // TODO: check if there is nothing in reporters_currently_used
05927
05928 // prepend all listeners
05929 for (auto &curr: getListeners())
05930     p->reporters_currently_used.insert(p->reporters_currently_used.begin(), curr.second(*g_cs));
05931
05932 #ifdef DOCTEST_PLATFORM_WINDOWS
05933     if (isDebuggerActive() && p->no_debug_output == false)
05934         p->reporters_currently_used.push_back(new DebugOutputWindowReporter(*g_cs));
05935 #endif // DOCTEST_PLATFORM_WINDOWS
05936
05937 // handle version, help and no_run
05938 if (p->no_run || p->version || p->help || p->list_reporters) {
05939     DOCTEST_ITERATE_THROUGH_REPORTERS(report_query, QueryData());
05940
05941     return cleanup_and_return();
05942 }
05943
05944 std::vector<const TestCase *> testArray;
05945 for (auto &curr: getRegisteredTests())
05946     testArray.push_back(&curr);
05947 p->numTestCases = testArray.size();
05948
05949 // sort the collected records
05950 if (!testArray.empty()) {
05951     if (p->order_by.compare("file", true) == 0) {
05952         std::sort(testArray.begin(), testArray.end(), fileOrderComparator);
05953     } else if (p->order_by.compare("suite", true) == 0) {
05954         std::sort(testArray.begin(), testArray.end(), suiteOrderComparator);
05955     } else if (p->order_by.compare("name", true) == 0) {
05956         std::sort(testArray.begin(), testArray.end(), nameOrderComparator);
05957     } else if (p->order_by.compare("rand", true) == 0) {
05958         std::srand(p->rand_seed);
05959
05960         // random_shuffle implementation
05961         const auto first = testArray.data();
05962         for (size_t i = testArray.size() - 1; i > 0; --i) {
05963             // NOLINTNEXTLINE(cert-msc30-c, cert-msc50-cpp, concurrency-mt-unsafe,
misc-predictable-rand)
05964             const int idxToSwap = static_cast<int>(std::rand() % (i + 1));
05965
05966             const auto temp = first[i];
05967
05968             first[i] = first[idxToSwap];
05969             first[idxToSwap] = temp;

```



```

05970     }
05971     } else if (p->order_by.compare("none", true) == 0) {
05972         // means no sorting - beneficial for death tests which call into the executable
05973         // with a specific test case in mind - we don't want to slow down the startup times
05974     }
05975 }
05976
05977 std::set<String> testSuitesPassingFilt;
05978
05979 const bool query_mode = p->count || p->list_test_cases || p->list_test_suites;
05980 std::vector<const TestCaseData *> queryResults;
05981
05982 if (!query_mode)
05983     DOCTEST_ITERATE_THROUGH_REPORTERS(test_run_start, DOCTEST_EMPTY);
05984
05985 // invoke the registered functions if they match the filter criteria (or just count them)
05986 for (auto &curr: testArray) {
05987     const auto &tc = *curr;
05988
05989     bool skip_me = false;
05990     if (tc.m_skip && !p->no_skip)
05991         skip_me = true;
05992
05993     if (!matchesAny(tc.m_file.c_str(), p->filters[0], true, p->case_sensitive))
05994         skip_me = true;
05995     if (matchesAny(tc.m_file.c_str(), p->filters[1], false, p->case_sensitive))
05996         skip_me = true;
05997     if (!matchesAny(tc.m_test_suite, p->filters[2], true, p->case_sensitive))
05998         skip_me = true;
05999     if (matchesAny(tc.m_test_suite, p->filters[3], false, p->case_sensitive))
06000         skip_me = true;
06001     if (!matchesAny(tc.m_name, p->filters[4], true, p->case_sensitive))
06002         skip_me = true;
06003     if (matchesAny(tc.m_name, p->filters[5], false, p->case_sensitive))
06004         skip_me = true;
06005
06006     if (!skip_me)
06007         p->numTestCasesPassingFilters++;
06008
06009     // skip the test if it is not in the execution range
06010     if ((p->last < p->numTestCasesPassingFilters && p->first <= p->last) ||
06011         (p->first > p->numTestCasesPassingFilters))
06012         skip_me = true;
06013
06014     if (skip_me) {
06015         if (!query_mode)
06016             DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_skipped, tc);
06017         continue;
06018     }
06019
06020     // do not execute the test if we are to only count the number of filter passing tests
06021     if (p->count)
06022         continue;
06023
06024     // print the name of the test and don't execute it
06025     if (p->list_test_cases) {
06026         queryResults.push_back(&tc);
06027         continue;
06028     }
06029
06030     // print the name of the test suite if not done already and don't execute it
06031     if (p->list_test_suites) {
06032         if ((testSuitesPassingFilt.count(tc.m_test_suite) == 0) && tc.m_test_suite[0] != '\0') {
06033             queryResults.push_back(&tc);
06034             testSuitesPassingFilt.insert(tc.m_test_suite);
06035             p->numTestSuitesPassingFilters++;
06036         }
06037         continue;
06038     }
06039
06040     // execute the test if it passes all the filtering
06041     {
06042         p->currentTest = &tc;
06043
06044         p->failure_flags = TestCaseFailureReason::None;
06045         p->seconds = 0;
06046
06047         // reset atomic counters
06048         p->numAssertsFailedCurrentTest_atomic = 0;
06049         p->numAssertsCurrentTest_atomic = 0;
06050
06051         p->traversal.resetForTestCase();
06052
06053         DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_start, tc);
06054
06055         p->timer.start();
06056     }

```

```

06057         bool run_test = true;
06058
06059         do { // NOLINT(cppcoreguidelines-avoid-do-while)
06060             // Reset per-run traversal data while keeping the current decision path prefix.
06061             p->traversal.resetForRun();
06062
06063             p->shouldLogCurrentException = true;
06064
06065             // reset stuff for logging with INFO()
06066             p->stringifiedContexts.clear();
06067
06068             #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
06069                 try {
06070             #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
06071             // MSVC 2015 diagnoses fatalConditionHandler as unused (because reset() is a
06072             // static method)
06073                 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4101) // unreferenced local variable
06074                 const FatalConditionHandler fatalConditionHandler; // Handle signals
06075                 static_cast<void>(fatalConditionHandler);
06076                 // execute the test
06077                 tc.m_test();
06078                 FatalConditionHandler::reset();
06079                 DOCTEST_MSVC_SUPPRESS_WARNING_POP
06080             #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
06081                 } catch (const TestFailureException &) {
06082                     p->failure_flags |= TestCaseFailureReason::AssertFailure;
06083                 } catch (...) {
06084                     DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_exception,
06085 {translateActiveException(), false});
06086                     p->failure_flags |= TestCaseFailureReason::Exception;
06087             #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
06088
06089             // exit this loop if enough assertions have failed - even if there are more subcases
06090             if (p->abort_after > 0 &&
06091                 p->numAssertsFailed + p->numAssertsFailedCurrentTest_atomic >= p->abort_after) {
06092                 run_test = false;
06093                 p->failure_flags |= TestCaseFailureReason::TooManyFailedAsserts;
06094             }
06095
06096             const bool has_next_path = run_test ? p->traversal.advance() : false;
06097
06098             if (has_next_path && run_test)
06099                 DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_reenter, tc);
06100             if (!has_next_path)
06101                 run_test = false;
06102         } while (run_test);
06103
06104         p->finalizeTestCaseData();
06105
06106         DOCTEST_ITERATE_THROUGH_REPORTERS(test_case_end, *g_cs);
06107
06108         p->currentTest = nullptr;
06109
06110         // stop executing tests if enough assertions have failed
06111         if (p->abort_after > 0 && p->numAssertsFailed >= p->abort_after)
06112             break;
06113     }
06114 }
06115
06116 if (!query_mode) {
06117     DOCTEST_ITERATE_THROUGH_REPORTERS(test_run_end, *g_cs);
06118 } else {
06119     QueryData qdata;
06120     qdata.run_stats = g_cs;
06121     qdata.data = queryResults.data();
06122     qdata.num_data = static_cast<unsigned>(queryResults.size());
06123     DOCTEST_ITERATE_THROUGH_REPORTERS(report_query, qdata);
06124 }
06125
06126 return cleanup_and_return();
06127 }
06128
06129 #endif // DOCTEST_CONFIG_DISABLE
06130 } // namespace doctest
06131
06132 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06133 #ifndef DOCTEST_PARTS_PRIVATE_CONTEXT_SCOPE
06134 #define DOCTEST_PARTS_PRIVATE_CONTEXT_SCOPE
06135
06136 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06137
06138 #ifndef DOCTEST_CONFIG_DISABLE
06139 namespace doctest {

```

```

06143 namespace detail {
06144 extern DOCTEST_THREAD_LOCAL std::vector<IContextScope *> g_infoContexts; // for logging with INFO()
06145 } // namespace detail
06146 } // namespace doctest
06147
06148 #endif // DOCTEST_CONFIG_DISABLE
06149
06150 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06151
06152 #endif // DOCTEST_PARTS_PRIVATE_CONTEXT_SCOPE
06153
06154 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06155
06156 namespace doctest {
06157
06158 DOCTEST_DEFINE_INTERFACE(IContextScope)
06159
06160 #ifndef DOCTEST_CONFIG_DISABLE
06161 namespace detail {
06162 DOCTEST_THREAD_LOCAL std::vector<IContextScope *> g_infoContexts; // for logging with INFO()
06163
06164 ContextScopeBase::ContextScopeBase() {
06165     g_infoContexts.push_back(this);
06166 }
06167
06168 ContextScopeBase::ContextScopeBase(ContextScopeBase &&other) noexcept {
06169     if (other.need_to_destroy) {
06170         other.destroy();
06171     }
06172     other.need_to_destroy = false;
06173     g_infoContexts.push_back(this);
06174 }
06175
06176 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4996) // std::uncaught_exception is deprecated in C++17
06177 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
06178 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
06179 // destroy cannot be inlined into the destructor because that would mean calling stringify after
06180 // ContextScope has been destroyed (base class destructors run after derived class destructors).
06181 // Instead, ContextScope calls this method directly from its destructor.
06182 void ContextScopeBase::destroy() {
06183     #if defined(__cpp_lib_uncaught_exceptions) && __cpp_lib_uncaught_exceptions >= 201411L &&
06184         \
06185         (!defined(__MAC_OS_X_VERSION_MIN_REQUIRED) || __MAC_OS_X_VERSION_MIN_REQUIRED >= 101200)
06186     if (std::uncaught_exceptions() > 0) {
06187     #else
06188     if (std::uncaught_exception()) {
06189     #endif
06189         std::ostringstream s;
06190         this->stringify(&s);
06191         g_cs->stringifiedContexts.emplace_back(s.str().c_str());
06192     }
06193     g_infoContexts.pop_back();
06194 }
06195 DOCTEST_CLANG_SUPPRESS_WARNING_POP
06196 DOCTEST_GCC_SUPPRESS_WARNING_POP
06197 DOCTEST_MSVC_SUPPRESS_WARNING_POP
06198 } // namespace detail
06199 #endif // DOCTEST_CONFIG_DISABLE
06200
06201 } // namespace doctest
06202
06203 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06204
06205 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06206
06207 #ifndef DOCTEST_CONFIG_DISABLE
06208
06209 namespace doctest {
06210 namespace detail {
06211
06212 ContextState *g_cs = nullptr;
06213 DOCTEST_THREAD_LOCAL bool g_no_colors;
06214
06215 void ContextState::resetRunData() {
06216     numTestCases = 0;
06217     numTestCasesPassingFilters = 0;
06218     numTestSuitesPassingFilters = 0;
06219     numTestCasesFailed = 0;
06220     numAsserts = 0;
06221     numAssertsFailed = 0;
06222     numAssertsCurrentTest = 0;
06223     numAssertsFailedCurrentTest = 0;
06224 }
06225
06226 void ContextState::finalizeTestCaseData() {
06227     seconds = timer.getElapsedSeconds();
06228

```

```

06229 // update the non-atomic counters
06230 numAsserts += numAssertsCurrentTest_atomic;
06231 numAssertsFailed += numAssertsFailedCurrentTest_atomic;
06232 numAssertsCurrentTest = numAssertsCurrentTest_atomic;
06233 numAssertsFailedCurrentTest = numAssertsFailedCurrentTest_atomic;
06234
06235 if (numAssertsFailedCurrentTest)
06236     failure_flags |= TestCaseFailureReason::AssertFailure;
06237
06238 if (Approx(currentTest->m_timeout).epsilon(DBL_EPSILON) != 0 &&
06239     Approx(seconds).epsilon(DBL_EPSILON) > currentTest->m_timeout)
06240     failure_flags |= TestCaseFailureReason::Timeout;
06241
06242 if (currentTest->m_should_fail) {
06243     if (failure_flags) {
06244         failure_flags |= TestCaseFailureReason::ShouldHaveFailedAndDid;
06245     } else {
06246         failure_flags |= TestCaseFailureReason::ShouldHaveFailedButDidnt;
06247     }
06248 } else if (failure_flags && currentTest->m_may_fail) {
06249     failure_flags |= TestCaseFailureReason::CouldHaveFailedAndDid;
06250 } else if (currentTest->m_expected_failures > 0) {
06251     if (numAssertsFailedCurrentTest == currentTest->m_expected_failures) {
06252         failure_flags |= TestCaseFailureReason::FailedExactlyNumTimes;
06253     } else {
06254         failure_flags |= TestCaseFailureReason::DidntFailExactlyNumTimes;
06255     }
06256 }
06257
06258 const bool ok_to_fail = (TestCaseFailureReason::ShouldHaveFailedAndDid & failure_flags) ||
06259     (TestCaseFailureReason::CouldHaveFailedAndDid & failure_flags) ||
06260     (TestCaseFailureReason::FailedExactlyNumTimes & failure_flags);
06261
06262 // if any subcase has failed - the whole test case has failed
06263 testCaseSuccess = !(failure_flags && !ok_to_fail);
06264 if (!testCaseSuccess)
06265     numTestCasesFailed++;
06266 }
06267
06268 } // namespace detail
06269 } // namespace doctest
06270
06271 #endif // DOCTEST_CONFIG_DISABLE
06272
06273 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06274
06275 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06276
06277 #ifndef DOCTEST_CONFIG_DISABLE
06278
06279 namespace doctest {
06280 namespace detail {
06281 #ifdef DOCTEST_IS_DEBUGGER_ACTIVE
06282 bool isDebuggerActive() {
06283     return DOCTEST_IS_DEBUGGER_ACTIVE();
06284 }
06285 #else // DOCTEST_IS_DEBUGGER_ACTIVE
06286 #ifdef DOCTEST_PLATFORM_LINUX
06287 class ErrnoGuard {
06288 public:
06289     ErrnoGuard()
06290         : m_oldErrno(errno) {}
06291     ~ErrnoGuard() {
06292         errno = m_oldErrno;
06293     }
06294 private:
06295     int m_oldErrno;
06296 };
06297 #endif
06298 // See the comments in Catch2 for the reasoning behind this implementation:
06299 // https://github.com/Catchorg/Catch2/blob/v2.13.1/include/internal/catch_debugger.cpp#L79-L102
06300 bool isDebuggerActive() {
06301     const ErrnoGuard guard;
06302     std::ifstream in("/proc/self/status");
06303     for (std::string line; std::getline(in, line);) {
06304         static const int PREFIX_LEN = 11;
06305         if (line.compare(0, PREFIX_LEN, "TracerPid:") == 0) {
06306             return line.length() > PREFIX_LEN && line[PREFIX_LEN] != '0';
06307         }
06308     }
06309     return false;
06310 }
06311 #elif defined(DOCTEST_PLATFORM_MAC)
06312 // The following function is taken directly from the following technical note:
06313 // https://developer.apple.com/library/archive/qa/qa1361/_index.html
06314 // Returns true if the current process is being debugged (either
06315 // running under the debugger or has a debugger attached post facto).

```

```

06316 bool isDebuggerActive() {
06317     int mib[4];
06318     kinfo_proc info;
06319     size_t size;
06320     // Initialize the flags so that, if sysctl fails for some bizarre
06321     // reason, we get a predictable result.
06322     info.kp_proc.p_flag = 0;
06323     // Initialize mib, which tells sysctl the info we want, in this case
06324     // we're looking for information about a specific process ID.
06325     mib[0] = CTL_KERN;
06326     mib[1] = KERN_PROC;
06327     mib[2] = KERN_PROC_PID;
06328     mib[3] = getpid();
06329     // Call sysctl.
06330     size = sizeof(info);
06331     if (sysctl(mib, DOCTEST_COUNTOF(mib), &info, &size, nullptr, 0) != 0) {
06332         std::cerr << "\nCall to sysctl failed - unable to determine if debugger is active *\n";
06333         return false;
06334     }
06335     // We're being debugged if the P_TRACED flag is set.
06336     return ((info.kp_proc.p_flag & P_TRACED) != 0);
06337 }
06338 #elif DOCTEST_MSVC || defined(__MINGW32__) || defined(__MINGW64__)
06339 bool isDebuggerActive() {
06340     return ::IsDebuggerPresent() != 0;
06341 }
06342 #else
06343 bool isDebuggerActive() {
06344     return false;
06345 }
06346 #endif // Platform
06347 #endif // DOCTEST_IS_DEBUGGER_ACTIVE
06348 } // namespace detail
06349 } // namespace doctest
06350
06351 #endif // DOCTEST_CONFIG_DISABLE
06352
06353 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06354
06355 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06356
06357 #ifndef DOCTEST_CONFIG_DISABLE
06358
06359 namespace doctest {
06360 namespace detail {
06361
06362 DOCTEST_DEFINE_INTERFACE(IEExceptionTranslator)
06363
06364 void registerExceptionTranslatorImpl(const IEExceptionTranslator *et) noexcept {
06365     if (std::find(getExceptionTranslators().begin(), getExceptionTranslators().end(), et) ==
06366         getExceptionTranslators().end())
06367         getExceptionTranslators().push_back(et);
06368 }
06369
06370 std::vector<const IEExceptionTranslator *> &getExceptionTranslators() noexcept {
06371     static std::vector<const IEExceptionTranslator *> data;
06372     return data;
06373 }
06374
06375 String translateActiveException() noexcept {
06376 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
06377     String res;
06378     auto &translators = getExceptionTranslators();
06379     for (auto &curr: translators)
06380         if (curr->translate(res))
06381             return res;
06382     // clang-format off
06383     DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wcatch-value")
06384     try {
06385         throw;
06386     } catch (std::exception &ex) {
06387         return ex.what();
06388     } catch (std::string &msg) {
06389         return msg.c_str();
06390     } catch (const char *msg) {
06391         return msg;
06392     } catch (...) {
06393         return "unknown exception";
06394     }
06395     DOCTEST_GCC_SUPPRESS_WARNING_POP
06396     // clang-format on
06397 #else // DOCTEST_CONFIG_NO_EXCEPTIONS
06398     return "";
06399 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
06400 }
06401
06402 } // namespace detail

```

```

06403 } // namespace doctest
06404
06405 #endif // DOCTEST_CONFIG_DISABLE
06406
06407 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06408
06409 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06410
06411 #ifndef DOCTEST_CONFIG_DISABLE
06412
06413 namespace doctest {
06414 namespace detail {
06415
06416 bool checkIfShouldThrow(assertType::Enum at) {
06417     if (at & assertType::is_require)
06418         return true;
06419
06420     if ((at & assertType::is_check) && getContextOptions()->abort_after > 0 &&
06421         (g_cs->numAssertsFailed + g_cs->numAssertsFailedCurrentTest_atomic) >=
06422         getContextOptions()->abort_after)
06423         return true;
06424     return false;
06425 }
06426
06427 #ifndef DOCTEST_CONFIG_NO_EXCEPTIONS
06428 DOCTEST_NORETURN void throwException() {
06429     g_cs->shouldLogCurrentException = false;
06430     throw TestFailureException(); // NOLINT(hicpp-exception-baseclass)
06431 }
06432 #else // DOCTEST_CONFIG_NO_EXCEPTIONS
06433 void throwException() {}
06434 #endif // DOCTEST_CONFIG_NO_EXCEPTIONS
06435
06436 } // namespace detail
06437 } // namespace doctest
06438
06439 #endif // DOCTEST_CONFIG_DISABLE
06440
06441 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06442
06443 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06444
06445 namespace doctest {
06446 namespace detail {
06447
06448 int wilddcmp(const char *str, const char *wild, bool caseSensitive) {
06449     const char *cp = str;
06450     const char *mp = wild;
06451
06452     while ((*str) && (*wild != '*')) {
06453         if ((caseSensitive ? (*wild != *str) : (tolower(*wild) != tolower(*str))) && (*wild != '?')) {
06454             return 0;
06455         }
06456         wild++;
06457         str++;
06458     }
06459
06460     while (*str) {
06461         if (*wild == '*') {
06462             if (!++wild) {
06463                 return 1;
06464             }
06465             mp = wild;
06466             cp = str + 1;
06467         } else if ((caseSensitive ? (*wild == *str) : (tolower(*wild) == tolower(*str))) || (*wild ==
06468         '?')) {
06469             wild++;
06470             str++;
06471         } else {
06472             wild = mp;
06473             str = cp++;
06474         }
06475     }
06476     while (*wild == '*') {
06477         wild++;
06478     }
06479     return !*wild;
06480 }
06481
06482 bool matchesAny(const char *name, const std::vector<String> &filters, bool matchEmpty, bool
06483 caseSensitive) {
06484     if (filters.empty() && matchEmpty)
06485         return true;
06486     for (auto &curr: filters)
06487         if (wilddcmp(name, curr.c_str(), caseSensitive))

```

```

06487         return true;
06488     return false;
06489 }
06490
06491 } // namespace detail
06492 } // namespace doctest
06493
06494 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06495
06496 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06497
06498 #ifndef DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
06499 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4007) // 'function' : must be 'attribute' - see issue #182
06500 int main(int argc, char **argv) {
06501     return doctest::Context(argc, argv).run();
06502 }
06503 DOCTEST_MSVC_SUPPRESS_WARNING_POP
06504 #endif // DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
06505
06506 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06507
06508 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06509
06510 namespace doctest {
06511
06512     Approx::Approx(double value)
06513         : m_epsilon(static_cast<double>(std::numeric_limits<float>::epsilon()) * 100), m_scale(1.0),
06514           m_value(value) {}
06515
06516     Approx Approx::operator()(double value) const {
06517         Approx approx(value);
06518         approx.epsilon(m_epsilon);
06519         approx.scale(m_scale);
06520         return approx;
06521     }
06522
06523     Approx &Approx::epsilon(double newEpsilon) {
06524         m_epsilon = newEpsilon;
06525         return *this;
06526     }
06527
06528     Approx &Approx::scale(double newScale) {
06529         m_scale = newScale;
06530         return *this;
06531     }
06532
06533     bool operator==(double lhs, const Approx &rhs) {
06534         // Thanks to Richard Harris for his help refining this formula
06535         return std::fabs(lhs - rhs.m_value) <
06536            rhs.m_epsilon * (rhs.m_scale + std::max<double>(std::fabs(lhs), std::fabs(rhs.m_value)));
06537     }
06538
06539     bool operator==(const Approx &lhs, double rhs) {
06540         return operator==(rhs, lhs);
06541     }
06542
06543     bool operator!=(double lhs, const Approx &rhs) {
06544         return !operator==(lhs, rhs);
06545     }
06546
06547     bool operator!=(const Approx &lhs, double rhs) {
06548         return !operator==(rhs, lhs);
06549     }
06550
06551     bool operator<=(double lhs, const Approx &rhs) {
06552         return lhs < rhs.m_value || lhs == rhs;
06553     }
06554
06555     bool operator<=(const Approx &lhs, double rhs) {
06556         return lhs.m_value < rhs || lhs == rhs;
06557     }
06558
06559     bool operator>=(double lhs, const Approx &rhs) {
06560         return lhs > rhs.m_value || lhs == rhs;
06561     }
06562
06563     bool operator>=(const Approx &lhs, double rhs) {
06564         return lhs.m_value > rhs || lhs == rhs;
06565     }
06566
06567     bool operator<(double lhs, const Approx &rhs) {
06568         return lhs < rhs.m_value && lhs != rhs;
06569     }
06570
06571     bool operator<(const Approx &lhs, double rhs) {
06572         return lhs.m_value < rhs && lhs != rhs;
06573     }
06574
06575 }

```

```

06573 bool operator>(double lhs, const Approx &rhs) {
06574     return lhs > rhs.m_value && lhs != rhs;
06575 }
06576
06577 bool operator>(const Approx &lhs, double rhs) {
06578     return lhs.m_value > rhs && lhs != rhs;
06579 }
06580
06581 String toString(const Approx &in) {
06582     return "Approx( " + doctest::toString(in.m_value) + " )";
06583 }
06584
06585 } // namespace doctest
06586
06587 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06588
06589 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06590
06591 namespace doctest {
06592
06593 Contains::Contains(const String &str)
06594     : string(str) {}
06595
06596 bool Contains::checkWith(const String &other) const {
06597     return strstr(other.c_str(), string.c_str()) != nullptr;
06598 }
06599
06600 String toString(const Contains &in) {
06601     return "Contains( " + in.string + " )";
06602 }
06603
06604 bool operator==(const String &lhs, const Contains &rhs) {
06605     return rhs.checkWith(lhs);
06606 }
06607
06608 bool operator==(const Contains &lhs, const String &rhs) {
06609     return lhs.checkWith(rhs);
06610 }
06611
06612 bool operator!=(const String &lhs, const Contains &rhs) {
06613     return !rhs.checkWith(lhs);
06614 }
06615
06616 bool operator!=(const Contains &lhs, const String &rhs) {
06617     return !lhs.checkWith(rhs);
06618 }
06619
06620 } // namespace doctest
06621
06622 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06623
06624 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06625
06626 namespace doctest {
06627
06628 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4738)
06629 template <typename F>
06630 IsNaN<F>::operator bool() const {
06631     return std::isnan(value) ^ flipped;
06632 }
06633 DOCTEST_MSVC_SUPPRESS_WARNING_POP
06634 template struct DOCTEST_INTERFACE_DEF IsNaN<float>;
06635 template struct DOCTEST_INTERFACE_DEF IsNaN<double>;
06636 template struct DOCTEST_INTERFACE_DEF IsNaN<long double>;
06637
06638 template <typename F>
06639 String toString(IsNaN<F> in) {
06640     return String(in.flipped ? "! " : "") + "IsNaN( " + doctest::toString(in.value) + " )";
06641 }
06642
06643 String toString(IsNaN<float> in) {
06644     return toString<float>(in);
06645 }
06646
06647 String toString(IsNaN<double> in) {
06648     return toString<double>(in);
06649 }
06650
06651 String toString(IsNaN<double long> in) {
06652     return toString<double long>(in);
06653 }
06654
06655 } // namespace doctest
06656
06657 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06658
06659 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH

```



```

06660
06661 #ifndef DOCTEST_CONFIG_OPTIONS_FILE_PREFIX_SEPARATOR
06662 #define DOCTEST_CONFIG_OPTIONS_FILE_PREFIX_SEPARATOR ':'
06663 #endif
06664
06665 namespace doctest {
06666
06667 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wnull-dereference")
06668 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wnull-dereference")
06669 // depending on the current options this will remove the path of filenames
06670 const char *skipPathFromFilename(const char *file) {
06671 #ifndef DOCTEST_CONFIG_DISABLE
06672     if (getContextOptions()->no_path_in_filenames) {
06673         auto back = std::strrchr(file, '\\');
06674         auto forward = std::strrchr(file, '/');
06675         if (back || forward) {
06676             if (back > forward)
06677                 forward = back;
06678             return forward + 1;
06679         }
06680     } else {
06681         const auto prefixes = getContextOptions()->strip_file_prefixes;
06682         const char separator = DOCTEST_CONFIG_OPTIONS_FILE_PREFIX_SEPARATOR;
06683         String::size_type longest_match = 0U;
06684         for (String::size_type pos = 0U; pos < prefixes.size(); ++pos) {
06685             const auto prefix_start = pos;
06686             pos = std::min(prefixes.find(separator, prefix_start), prefixes.size());
06687
06688             const auto prefix_size = pos - prefix_start;
06689             if (prefix_size > longest_match) {
06690                 // TODO under DOCTEST_MSVC: does the comparison need strnicmp() to work with drive
06691                 // letter capitalization?
06692                 if (0 == std::strncmp(prefixes.c_str() + prefix_start, file, prefix_size)) {
06693                     longest_match = prefix_size;
06694                 }
06695             }
06696         }
06697         return &file[longest_match];
06698     }
06699 #endif // DOCTEST_CONFIG_DISABLE
06700     return file;
06701 }
06702 DOCTEST_CLANG_SUPPRESS_WARNING_POP
06703 DOCTEST_GCC_SUPPRESS_WARNING_POP
06704
06705 } // namespace doctest
06706
06707 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06708
06709 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06710
06711 namespace doctest {
06712 #ifndef DOCTEST_CONFIG_DISABLE
06713
06714 DOCTEST_DEFINE_INTERFACE(IReporter)
06715
06716 int IReporter::get_num_active_contexts() {
06717     return 0;
06718 }
06719
06720 const IContextScope *const *IReporter::get_active_contexts() {
06721     return nullptr;
06722 }
06723
06724 int IReporter::get_num_stringified_contexts() {
06725     return 0;
06726 }
06727
06728 const String *IReporter::get_stringified_contexts() {
06729     return nullptr;
06730 }
06731
06732 int registerReporter(const char *, int, IReporter *) {
06733     return 0;
06734 }
06735
06736 #else
06737
06738 namespace detail {
06739 reporterMap &getReporters() noexcept {
06740     static reporterMap data;
06741     return data;
06742 }
06743
06744 reporterMap &getListeners() noexcept {
06745     static reporterMap data;
06746     return data;

```

```

06747 }
06748 } // namespace detail
06749
06750 DOCTEST_DEFINE_INTERFACE(IReporter)
06751
06752 int IReporter::get_num_active_contexts() {
06753     return static_cast<int>(detail::g_infoContexts.size());
06754 }
06755
06756 const IContextScope *const *IReporter::get_active_contexts() {
06757     return get_num_active_contexts() ? detail::g_infoContexts.data() : nullptr;
06758 }
06759
06760 int IReporter::get_num_stringified_contexts() {
06761     return static_cast<int>(detail::g_cs->stringifiedContexts.size());
06762 }
06763
06764 const String *IReporter::get_stringified_contexts() {
06765     return get_num_stringified_contexts() ? detail::g_cs->stringifiedContexts.data() : nullptr;
06766 }
06767
06768 namespace detail {
06769 void registerReporterImpl(const char *name, int priority, reporterCreatorFunc c, bool isReporter)
06770     noexcept {
06771     if (isReporter)
06772         getReporters().insert(reporterMap::value_type(reporterMap::key_type(priority, name), c));
06773     else
06774         getListeners().insert(reporterMap::value_type(reporterMap::key_type(priority, name), c));
06775 } // namespace detail
06776
06777 #endif // DOCTEST_CONFIG_DISABLE
06778 } // namespace doctest
06779
06780 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06781
06782 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06783
06784 #ifndef DOCTEST_CONFIG_DISABLE
06785 namespace doctest {
06786
06787 void fulltext_log_assert_to_stream(std::ostream &s, const AssertData &rb) {
06788     // clang-format off
06789     if ((rb.m_at & (assertType::is_throws_as | assertType::is_throws_with)) == 0)
06790         s << Color::Cyan << assertString(rb.m_at) << "( " << rb.m_expr << " ) " << Color::None;
06791
06792     if (rb.m_at & assertType::is_throws) {
06793         s << (rb.m_threw ? "threw as expected!" : "did NOT throw at all!") << "\n";
06794     } else if ((rb.m_at & assertType::is_throws_as) && (rb.m_at & assertType::is_throws_with)) {
06795         s << Color::Cyan << assertString(rb.m_at) << "( "
06796         << rb.m_expr << ", \" " << rb.m_exception_string.c_str() << "\", " << rb.m_exception_type
06797         << " ) " << Color::None;
06798     }
06799
06800     if (rb.m_threw) {
06801         if (!rb.m_failed) {
06802             s << "threw as expected!\n";
06803         } else {
06804             s << "threw a DIFFERENT exception! (contents: " << rb.m_exception << ")\n";
06805         }
06806     } else {
06807         s << "did NOT throw at all!\n";
06808     }
06809 } else if (rb.m_at & assertType::is_throws_as) {
06810     s << Color::Cyan << assertString(rb.m_at) << "( "
06811     << rb.m_expr << ", " << rb.m_exception_type
06812     << " ) " << Color::None
06813     << (rb.m_threw ? (rb.m_threw_as ? "threw as expected!" : "threw a DIFFERENT exception: ") :
06814     "did NOT throw at all!")
06815     << Color::Cyan << rb.m_exception << "\n";
06816 } else if (rb.m_at & assertType::is_throws_with) {
06817     s << Color::Cyan << assertString(rb.m_at) << "( "
06818     << rb.m_expr << ", \" " << rb.m_exception_string.c_str()
06819     << "\" ) " << Color::None
06820     << (rb.m_threw ? (!rb.m_failed ? "threw as expected!" : "threw a DIFFERENT exception: ") :
06821     "did NOT throw at all!")
06822     << Color::Cyan << rb.m_exception << "\n";
06823 } else if (rb.m_at & assertType::is_nothrow) {
06824     s << (rb.m_threw ? "THREW exception: " : "didn't throw!") << Color::Cyan << rb.m_exception <<
06825     "\n";
06826 } else {
06827     s << (rb.m_threw ? "THREW exception: " : (!rb.m_failed ? "is correct!\n" : "is NOT
06828     correct!\n"));
06829     if (rb.m_threw)
06830         s << rb.m_exception << "\n";
06831     else
06832         s << " values: " << assertString(rb.m_at) << "( " << rb.m_decomp << " )\n";
06833 }
06834 }
06835 }
06836 }
06837 }
06838 }
06839 }
06840 }
06841 }
06842 }
06843 }
06844 }
06845 }
06846 }
06847 }
06848 }
06849 }
06850 }
06851 }
06852 }
06853 }
06854 }
06855 }
06856 }
06857 }
06858 }
06859 }
06860 }
06861 }
06862 }
06863 }
06864 }
06865 }
06866 }
06867 }
06868 }
06869 }
06870 }
06871 }
06872 }
06873 }
06874 }
06875 }
06876 }
06877 }
06878 }
06879 }
06880 }
06881 }
06882 }
06883 }
06884 }
06885 }
06886 }
06887 }
06888 }
06889 }
06890 }
06891 }
06892 }
06893 }
06894 }
06895 }
06896 }
06897 }
06898 }
06899 }
06900 }
06901 }
06902 }
06903 }
06904 }
06905 }
06906 }
06907 }
06908 }
06909 }
06910 }
06911 }
06912 }
06913 }
06914 }
06915 }
06916 }
06917 }
06918 }
06919 }
06920 }
06921 }
06922 }
06923 }
06924 }
06925 }
06926 }
06927 }
06928 }
06929 }
06930 }
06931 }
06932 }
06933 }
06934 }
06935 }
06936 }
06937 }
06938 }
06939 }
06940 }
06941 }
06942 }
06943 }
06944 }
06945 }
06946 }
06947 }
06948 }
06949 }
06950 }
06951 }
06952 }
06953 }
06954 }
06955 }
06956 }
06957 }
06958 }
06959 }
06960 }
06961 }
06962 }
06963 }
06964 }
06965 }
06966 }
06967 }
06968 }
06969 }
06970 }
06971 }
06972 }
06973 }
06974 }
06975 }
06976 }
06977 }
06978 }
06979 }
06980 }
06981 }
06982 }
06983 }
06984 }
06985 }
06986 }
06987 }
06988 }
06989 }
06990 }
06991 }
06992 }
06993 }
06994 }
06995 }
06996 }
06997 }
06998 }
06999 }
07000 }
07001 }
07002 }
07003 }
07004 }
07005 }
07006 }
07007 }
07008 }
07009 }
07010 }
07011 }
07012 }
07013 }
07014 }
07015 }
07016 }
07017 }
07018 }
07019 }
07020 }
07021 }
07022 }
07023 }
07024 }
07025 }
07026 }
07027 }
07028 }
07029 }
07030 }
07031 }
07032 }
07033 }
07034 }
07035 }
07036 }
07037 }
07038 }
07039 }
07040 }
07041 }
07042 }
07043 }
07044 }
07045 }
07046 }
07047 }
07048 }
07049 }
07050 }
07051 }
07052 }
07053 }
07054 }
07055 }
07056 }
07057 }
07058 }
07059 }
07060 }
07061 }
07062 }
07063 }
07064 }
07065 }
07066 }
07067 }
07068 }
07069 }
07070 }
07071 }
07072 }
07073 }
07074 }
07075 }
07076 }
07077 }
07078 }
07079 }
07080 }
07081 }
07082 }
07083 }
07084 }
07085 }
07086 }
07087 }
07088 }
07089 }
07090 }
07091 }
07092 }
07093 }
07094 }
07095 }
07096 }
07097 }
07098 }
07099 }
07100 }
07101 }
07102 }
07103 }
07104 }
07105 }
07106 }
07107 }
07108 }
07109 }
07110 }
07111 }
07112 }
07113 }
07114 }
07115 }
07116 }
07117 }
07118 }
07119 }
07120 }
07121 }
07122 }
07123 }
07124 }
07125 }
07126 }
07127 }
07128 }
07129 }
07130 }
07131 }
07132 }
07133 }
07134 }
07135 }
07136 }
07137 }
07138 }
07139 }
07140 }
07141 }
07142 }
07143 }
07144 }
07145 }
07146 }
07147 }
07148 }
07149 }
07150 }
07151 }
07152 }
07153 }
07154 }
07155 }
07156 }
07157 }
07158 }
07159 }
07160 }
07161 }
07162 }
07163 }
07164 }
07165 }
07166 }
07167 }
07168 }
07169 }
07170 }
07171 }
07172 }
07173 }
07174 }
07175 }
07176 }
07177 }
07178 }
07179 }
07180 }
07181 }
07182 }
07183 }
07184 }
07185 }
07186 }
07187 }
07188 }
07189 }
07190 }
07191 }
07192 }
07193 }
07194 }
07195 }
07196 }
07197 }
07198 }
07199 }
07200 }
07201 }
07202 }
07203 }
07204 }
07205 }
07206 }
07207 }
07208 }
07209 }
07210 }
07211 }
07212 }
07213 }
07214 }
07215 }
07216 }
07217 }
07218 }
07219 }
07220 }
07221 }
07222 }
07223 }
07224 }
07225 }
07226 }
07227 }
07228 }
07229 }
07230 }
07231 }
07232 }
07233 }
07234 }
07235 }
07236 }
07237 }
07238 }
07239 }
07240 }
07241 }
07242 }
07243 }
07244 }
07245 }
07246 }
07247 }
07248 }
07249 }
07250 }
07251 }
07252 }
07253 }
07254 }
07255 }
07256 }
07257 }
07258 }
07259 }
07260 }
07261 }
07262 }
07263 }
07264 }
07265 }
07266 }
07267 }
07268 }
07269 }
07270 }
07271 }
07272 }
07273 }
07274 }
07275 }
07276 }
07277 }
07278 }
07279 }
07280 }
07281 }
07282 }
07283 }
07284 }
07285 }
07286 }
07287 }
07288 }
07289 }
07290 }
07291 }
07292 }
07293 }
07294 }
07295 }
07296 }
07297 }
07298 }
07299 }
07300 }
07301 }
07302 }
07303 }
07304 }
07305 }
07306 }
07307 }
07308 }
07309 }
07310 }
07311 }
07312 }
07313 }
07314 }
07315 }
07316 }
07317 }
07318 }
07319 }
07320 }
07321 }
07322 }
07323 }
07324 }
07325 }
07326 }
07327 }
07328 }
07329 }
07330 }
07331 }
07332 }
07333 }
07334 }
07335 }
07336 }
07337 }
07338 }
07339 }
07340 }
07341 }
07342 }
07343 }
07344 }
07345 }
07346 }
07347 }
07348 }
07349 }
07350 }
07351 }
07352 }
07353 }
07354 }
07355 }
07356 }
07357 }
07358 }
07359 }
07360 }
07361 }
07362 }
07363 }
07364 }
07365 }
07366 }
07367 }
07368 }
07369 }
07370 }
07371 }
07372 }
07373 }
07374 }
07375 }
07376 }
07377 }
07378 }
07379 }
07380 }
07381 }
07382 }
07383 }
07384 }
07385 }
07386 }
07387 }
07388 }
07389 }
07390 }
07391 }
07392 }
07393 }
07394 }
07395 }
07396 }
07397 }
07398 }
07399 }
07400 }
07401 }
07402 }
07403 }
07404 }
07405 }
07406 }
07407 }
07408 }
07409 }
07410 }
07411 }
07412 }
07413 }
07414 }
07415 }
07416 }
07417 }
07418 }
07419 }
07420 }
07421 }
07422 }
07423 }
07424 }
07425 }
07426 }
07427 }
07428 }
07429 }
07430 }
07431 }
07432 }
07433 }
07434 }
07435 }
07436 }
07437 }
07438 }
07439 }
07440 }
07441 }
07442 }
07443 }
07444 }
07445 }
07446 }
07447 }
07448 }
07449 }
07450 }
07451 }
07452 }
07453 }
07454 }
07455 }
07456 }
07457 }
07458 }
07459 }
07460 }
07461 }
07462 }
07463 }
07464 }
07465 }
07466 }
07467 }
07468 }
07469 }
07470 }
07471 }
07472 }
07473 }
07474 }
07475 }
07476 }
07477 }
07478 }
07479 }
07480 }
07481 }
07482 }
07483 }
07484 }
07485 }
07486 }
07487 }
07488 }
07489 }
07490 }
07491 }
07492 }
07493 }
07494 }
07495 }
07496 }
07497 }
07498 }
07499 }
07500 }
07501 }
07502 }
07503 }
07504 }
07505 }
07506 }
07507 }
07508 }
07509 }
07510 }
07511 }
07512 }
07513 }
07514 }
07515 }
07516 }
07517 }
07518 }
07519 }
07520 }
07521 }
07522 }
07523 }
07524 }
07525 }
07526 }
07527 }
07528 }
07529 }
07530 }
07531 }
07532 }
07533 }
07534 }
07535 }
07536 }
07537 }
07538 }
07539 }
07540 }
07541 }
07542 }
07543 }
07544 }
07545 }
07546 }
07547 }
07548 }
07549 }
07550 }
07551 }
07552 }
07553 }
07554 }
07555 }
07556 }
07557 }
07558 }
07559 }
07560 }
07561 }
07562 }
07563 }
07564 }
07565 }
07566 }
07567 }
07568 }
07569 }
07570 }
07571 }
07572 }
07573 }
07574 }
07575 }
07576 }
07577 }
07578 }
07579 }
07580 }
07581 }
07582 }
07583 }
07584 }
07585 }
07586 }
07587 }
07588 }
07589 }
07590 }
07591 }
07592 }
07593 }
07594 }
07595 }
07596 }
07597 }
07598 }
07599 }
07600 }
07601 }
07602 }
07603 }
07604 }
07605 }
07606 }
07607 }
07608 }
07609 }
07610 }
07611 }
07612 }
07613 }
07614 }
07615 }
07616 }
07617 }
07618 }
07619 }
07620 }
07621 }
07622 }
07623 }
07624 }
07625 }
07626 }
07627 }
07628 }
07629 }
07630 }
07631 }
07632 }
07633 }
07634 }
07635 }
07636 }
07637 }
07638 }
07639 }
07640 }
07641 }
07642 }
07643 }
07644 }
07645 }
07646 }
07647 }
07648 }
07649 }
07650 }
07651 }
07652 }
07653 }
07654 }
07655 }
07656 }
07657 }
07658 }
07659 }
07660 }
07661 }
07662 }
07663 }
07664 }
07665 }
07666 }
07667 }
07668 }
07669 }
07670 }
07671 }
07672 }
07673 }
07674 }
07675 }
07676 }
07677 }
07678 }
07679 }
07680 }
07681 }
07682 }
07683 }
07684 }
07685 }
07686 }
07687 }
07688 }
07689 }
07690 }
07691 }
07692 }
07693 }
07694 }
07695 }
07696 }
07697 }
07698 }
07699 }
07700 }
07701 }
07702 }
07703 }
07704 }
07705 }
07706 }
07707 }
07708 }
07709 }
07710 }
07711 }
07712 }
07713 }
07714 }
07715 }
07716 }
07717 }
07718 }
07719 }
07720 }
07721 }
07722 }
07723 }
07724 }
07725 }
07726 }
07727 }
07728 }
07729 }
07730 }
07731 }
07732 }
07733 }
07734 }
07735 }
07736 }
07737 }
07738 }
07739 }
07740 }
07741 }
07742 }
07743 }
07744 }
07745 }
07746 }
07747 }
07748 }
07749 }
07750 }
07751 }
07752 }
07753 }
07754 }
07755 }
07756 }
07757 }
07758 }
07759 }
07760 }
07761 }
07762 }
07763 }
07764 }
07765 }
07766 }
07767 }
07768 }
07769 }
07770 }
07771 }
07772 }
07773 }
07774 }
07775 }
07776 }
07777 }
07778 }
07779 }
07780 }
07781 }
07782 }
07783 }
07784 }
07785 }
07786 }
07787 }
07788 }
07789 }
07790 }
07791 }
07792 }
07793 }
07794 }
07795 }
07796 }
07797 }
07798 }
07799 }
07800 }
07801 }
07802 }
07803 }
07804 }
07805 }
07806 }
07807 }
07808 }
07809 }
07810 }
07811 }
07812 }
07813 }
07814 }
07815 }
07816 }
07817 }
07818 }
07819 }
07820 }
07821 }
07822 }
07823 }
07824 }
07825 }
07826 }
07827 }
07828 }
07829 }
07830 }
07831 }
07832 }
07833 }
07834 }
07835 }
07836 }
07837 }
07838 }
07839 }
07840 }
07841 }
07842 }
07843 }
07844 }
07845 }
07846 }
07847 }
07848 }
07849 }
07850 }
07851 }
07852 }
07853 }
07854 }
07855 }
07856 }
07857 }
07858 }
07859 }
07860 }
07861 }
07862 }
07863 }
07864 }
07865 }
07866 }
07867 }
07868 }
07869 }
07870 }
07871 }
07872 }
07873 }
07874 }
07875 }
07876 }
07877 }
07878 }
07879 }
07880 }
07881 }
07882 }
07883 }
07884 }
07885 }
07886 }
07887 }
07888 }
07889 }
07890 }
07891 }
07892 }
07893 }
07894 }
07895 }
07896 }
07897 }
07898 }
07899 }
07900 }
07901 }
07902 }
07903 }
07904 }
07905 }
07906 }
07907 }
07908 }
07909 }
07910 }
07911 }
07912 }
07913 }
07914 }
07915 }
07916 }
07917 }
07918 }
07919 }
07920 }
07921 }
07922 }
07923 }
07924 }
07925 }
07926 }
07927 }
07928 }
07929 }
07930 }
07931 }
07932 }
07933 }
07934 }
07935 }
07936 }
07937 }
07938 }
07939 }
07940 }
07941 }
07942 }
07943 }
07944 }
07945 }
07946 }
07947 }
07948 }
07949 }
07950 }
07951 }
07952 }
07953 }
07954 }
07955 }
07956 }
07957 }
07958 }
07959 }
07960 }
07961 }
07962 }
07963 }
07964 }
07965 }
07966 }
07967 }
07968 }
07969 }
07970 }
07971 }
07972 }
07973 }
07974 }
07975 }
07976 }
07977 }
07978 }
07979 }
07980 }
07981 }
07982 }
07983 }
07984 }
07985 }
07986 }
07987 }
07988 }
07989 }
07990 }
07991 }
07992 }
07993 }
07994 }
07995 }
07996 }
07997 }
07998 }
07999 }
08000 }
08001 }
08002 }
08003 }
08004 }
08005 }
08006 }
08007 }
08008 }
08009 }
08010 }
08011 }
08012 }
08013 }
08014 }
08015 }
08016 }
08017 }
08018 }
08019 }
08020 }
08021 }
08022 }
08023 }
08024 }
08025 }
08026 }
08027 }
08028 }
08029 }
08030 }
08031 }
08032 }
08033 }
08034 }
08035 }
08036 }
08037 }
08038 }
08039 }
08040 }
08041 }
08042 }
08043 }
08044 }
08045 }
08046 }
08047 }
08048 }
08049 }
08050 }
08051 }
08052 }
08053 }
08054 }
08055 }
08056 }
08057 }
08058 }
08059 }
08060 }
08061 }
08062 }
08063 }
08064 }
08065 }
08066 }
08067 }
08068 }
08069 }
08070 }
08071 }
08072 }
08073 }
08074 }
08075 }
08076 }
08077 }
08078 }
08079 }
08080 }
08081 }
08082 }
08083 }
08084 }
08085 }
08086 }
08087 }
08088 }
08089 }
08090 }
08091 }
08092 }
08093 }
08094 }
08095 }
08096 }
08097 }
08098 }
08099 }
08100 }
08101 }
08102 }
08103 }
08104 }
08105 }
08106 }
08107 }
08108 }
08109 }
08110 }
08111 }
08112 }
08113 }
08114 }
08115 }
08116 }
08117 }
08118 }
08119 }
08120 }
08121 }
08122 }
08123 }
08124 }
08125 }
08126 }
08127 }
08128 }
08129 }
08130 }
08131 }
08132 }
08133 }
08134 }
08135 }
08136 }
08137 }
08138 }
08139 }
08140 }
08141 }
08142 }
08143 }
08144 }
08145 }
08146 }
08147 }
08148 }
08149 }
08150 }
08151 }
08152 }
08153 }
08154 }
08155 }
08156 }
08157 }
08158 }
08159 }
08160 }
08161 }
08162 }
08163 }
08164 }
08165 }
08166 }
08167 }
08168 }
08169 }
08170 }
08171 }
08172 }
08173 }
08174 }
08175 }
08176 }
08177 }
08178 }
08179 }
08180 }
08181 }
08182 }
08183 }
08184 }
08185 }
08186 }
08187 }
08188 }
08189 }
08190 }
08191 }
08192 }
08193 }
08194 }
08195 }
08196 }
08197 }
08198 }
08199 }
08200 }
08201 }
08202 }
08203 }
08204 }
08205 }
08206 }
08207 }
08208 }
08209 }
08210 }
08211 }
08212 }
08213 }
08214 }
08215 }
08216 }
08217 }
08218 }
08219 }
08220 }
08221 }
08222 }
08223 }
08224 }
08225 }
08226 }
08227 }
08228 }
08229 }
08230 }
08231 }
08232 }
08233 }
08234 }
08235 }
08236 }
08237 }
08238 }
08239 }
08240 }
08241 }
08242 }
08243 }
08244 }
08245 }
08246 }
08247 }
08248 }
08249 }
08250 }
08251 }
08252 }
08253 }
08254 }
08255 }
08256 }
08257 }
08258 }
08259 }
08260 }
08261 }
08262 }
08263 }
08264 }
08265 }
08266 }
08267 }
08268 }
08269 }
08270 }
08271 }
08272 }
08273 }
08274 }
08275 }
08276 }
08277 }
08278 }
08279 }
08280 }
08281 }
08282 }
08283 }
08284 }
08285 }
08286 }
08287 }
08288 }
08289 }
08290 }
08291 }
08292 }
08293 }
08294 }
08295 }
08296 }
08297 }
08298 }
08299 }
08300 }
08301 }
08302 }
08303 }
08304 }
08305 }
08306 }
08307 }
08308 }
08309 }
08310 }
08311 }
08312 }
08313 }
08314 }
08315 }
08316 }
08317 }
08318 }
08319 }
08320 }
08321 }
08322 }
08323 }
08324 }
08325 }
08326 }
08327 }
08328 }
08329 }
08330 }
08331 }
08332 }
08333 }
08334 }
08335 }
08336 }
08337 }
08338 }
08339 }
08340 }
08341 }
08342 }
08343 }
08344 }
083
```

```

06829     }
06830     // clang-format on
06831 }
06832
06833 } // namespace doctest
06834
06835 #endif // DOCTEST_CONFIG_DISABLE
06836
06837 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
06838
06839 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
06840
06841 #ifndef DOCTEST_CONFIG_DISABLE
06842
06843 namespace doctest {
06844
06845     using detail::g_cs;
06846
06847     Whitespace::Whitespace(int nr)
06848         : nrSpaces(nr) {}
06849
06850     std::ostream &operator<<(std::ostream &out, const Whitespace &ws) {
06851         if (ws.nrSpaces != 0)
06852             out << std::setw(ws.nrSpaces) << ' ';
06853         return out;
06854     }
06855
06856     ConsoleReporter::ConsoleReporter(const ContextOptions &co)
06857         : s(*co.cout), opt(co) {}
06858
06859     ConsoleReporter::ConsoleReporter(const ContextOptions &co, std::ostream &ostr)
06860         : s(ostr), opt(co) {}
06861
06862     void ConsoleReporter::separator_to_stream() {
06863         s << Color::Yellow
06864           << "=====
06865           "\n";
06866     }
06867
06868     const char *ConsoleReporter::getSuccessOrFailString(bool success, assertType::Enum at, const char
    *success_str) {
06869         if (success)
06870             return success_str;
06871         return failureString(at);
06872     }
06873
06874     Color::Enum ConsoleReporter::getSuccessOrFailColor(bool success, assertType::Enum at) {
06875         return success ? Color::BrightGreen : (at & assertType::is_warn) ? Color::Yellow : Color::Red;
06876     }
06877
06878     void ConsoleReporter::successOrFailColoredStringToStream(bool success, assertType::Enum at, const char
    *success_str) {
06879         s << getSuccessOrFailColor(success, at) << getSuccessOrFailString(success, at, success_str) << ": ";
06880     }
06881
06882     void ConsoleReporter::log_contexts() {
06883         const int num_contexts = get_num_active_contexts();
06884         if (num_contexts) {
06885             auto contexts = get_active_contexts();
06886
06887             s << Color::None << " logged: ";
06888             for (int i = 0; i < num_contexts; ++i) {
06889                 s << (i == 0 ? "" : " ");
06890                 contexts[i] -> stringify(&s);
06891                 s << "\n";
06892             }
06893         }
06894
06895         s << "\n";
06896     }
06897
06898     // this was requested to be made virtual so users could override it
06899     void ConsoleReporter::file_line_to_stream(const char *file, int line, const char *tail) {
06900         s << Color::LightGrey << skipPathFromFilename(file) << (opt.gnu_file_line ? ":" : "(")
06901           << (opt.no_line_numbers ? 0 : line) // 0 or the real num depending on the option
06902           << (opt.gnu_file_line ? ":" : ")") << tail;
06903     }
06904
06905     void ConsoleReporter::logTestStart() {
06906         if (hasLoggedCurrentTestStart)
06907             return;
06908
06909         separator_to_stream();
06910         file_line_to_stream(tc->m_file.c_str(), static_cast<int>(tc->m_line), "\n");
06911         if (tc->m_description)
06912             s << Color::Yellow << "DESCRIPTION: " << Color::None << tc->m_description << "\n";
06913         if (tc->m_test_suite && tc->m_test_suite[0] != '\0')

```

```

06914         s « Color::Yellow « "TEST SUITE: " « Color::None « tc->m_test_suite « "\n";
06915     if (strcmp(tc->m_name, " Scenario:", 11) != 0)
06916         s « Color::Yellow « "TEST CASE: ";
06917     s « Color::None « tc->m_name « "\n";
06918
06919     for (size_t i = 0; i < currentSubcaseLevel; ++i) {
06920         if (subcasesStack[i].m_name[0] != '\0')
06921             s « " " « subcasesStack[i].m_name « "\n";
06922     }
06923
06924     if (currentSubcaseLevel != subcasesStack.size()) {
06925         s « Color::Yellow « "\nDEEPEST SUBCASE STACK REACHED (DIFFERENT FROM THE CURRENT ONE):\n" «
Color::None;
06926         for (size_t i = 0; i < subcasesStack.size(); ++i) {
06927             if (subcasesStack[i].m_name[0] != '\0')
06928                 s « " " « subcasesStack[i].m_name « "\n";
06929         }
06930     }
06931
06932     s « "\n";
06933
06934     hasLoggedCurrentTestStart = true;
06935 }
06936
06937 void ConsoleReporter::printVersion() {
06938     if (opt.no_version == false)
06939         s « Color::Cyan « "[doctest] " « Color::None « "doctest version is \"" « DOCTEST_VERSION_STR «
"\n\n";
06940 }
06941
06942 void ConsoleReporter::printIntro() {
06943     if (opt.no_intro == false) {
06944         printVersion();
06945         s « Color::Cyan « "[doctest] " « Color::None
« "run with \"--\" DOCTEST_OPTIONS_PREFIX_DISPLAY \"help\" for options\n";
06946     }
06947 }
06948
06949 void ConsoleReporter::printHelp() {
06950     const int sizePrefixDisplay = static_cast<int>(strlen(DOCTEST_OPTIONS_PREFIX_DISPLAY));
06951     printVersion();
06952     // clang-format off
06953     s « Color::Cyan « "[doctest]\n" « Color::None;
06954     s « Color::Cyan « "[doctest] " « Color::None;
06955     s « "boolean values: \"1/on/yes/true\" or \"0/off/no/false\"\n";
06956     s « Color::Cyan « "[doctest] " « Color::None;
06957     s « "filter values: \"str1,str2,str3\" (comma separated strings)\n";
06958     s « Color::Cyan « "[doctest]\n" « Color::None;
06959     s « Color::Cyan « "[doctest] " « Color::None;
06960     s « "filters use wildcards for matching strings\n";
06961     s « Color::Cyan « "[doctest] " « Color::None;
06962     s « "something passes a filter if any of the strings in a filter matches\n";
06963     #ifndef DOCTEST_CONFIG_NO_UNPREFIXED_OPTIONS
06964     s « Color::Cyan « "[doctest]\n" « Color::None;
06965     s « Color::Cyan « "[doctest] " « Color::None;
06966     s « "ALL FLAGS, OPTIONS AND FILTERS ALSO AVAILABLE WITH A \"" DOCTEST_CONFIG_OPTIONS_PREFIX "\"
PREFIX!!\n";
06967     #endif
06968     s « Color::Cyan « "[doctest]\n" « Color::None;
06969     s « Color::Cyan « "[doctest] " « Color::None;
06970     s « "Query flags - the program quits after them. Available:\n\n";
06971     s « " " DOCTEST_OPTIONS_PREFIX_DISPLAY "?", --" DOCTEST_OPTIONS_PREFIX_DISPLAY "help, --"
DOCTEST_OPTIONS_PREFIX_DISPLAY "h "
06972     « Whitespace(sizePrefixDisplay * 0) « "prints this message\n";
06973     s « " " DOCTEST_OPTIONS_PREFIX_DISPLAY "v, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "version
"
06974     « Whitespace(sizePrefixDisplay * 1) « "prints the version\n";
06975     s « " " DOCTEST_OPTIONS_PREFIX_DISPLAY "c, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "count
"
06976     « Whitespace(sizePrefixDisplay * 1) « "prints the number of matching tests\n";
06977     s « " " DOCTEST_OPTIONS_PREFIX_DISPLAY "ltc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "list-test-cases
"
06978     « Whitespace(sizePrefixDisplay * 1) « "lists all matching tests by name\n";
06979     s « " " DOCTEST_OPTIONS_PREFIX_DISPLAY "lts, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "list-test-suites
"
06980     « Whitespace(sizePrefixDisplay * 1) « "lists all matching test suites\n";
06981     s « " " DOCTEST_OPTIONS_PREFIX_DISPLAY "lrr, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "list-reporters
"
06982     « Whitespace(sizePrefixDisplay * 1) « "lists all registered reporters\n\n";
06983     // ===== « 79
06984     s « Color::Cyan « "[doctest] " « Color::None;
06985     s « "The available <int>/<string> options/filters are:\n\n";
06986     s « " " DOCTEST_OPTIONS_PREFIX_DISPLAY "tc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"test-case=<filters> "
06987     « Whitespace(sizePrefixDisplay * 1) « "filters tests by their name\n";
06988     s « " " DOCTEST_OPTIONS_PREFIX_DISPLAY "tce, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"test-case-exclude=<filters> "

```

```

06990     « Whitespace(sizePrefixDisplay * 1) « "filters OUT tests by their name\n";
06991     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "sf, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"source-file=<filters> "
06992     « Whitespace(sizePrefixDisplay * 1) « "filters      tests by their file\n";
06993     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "sfe, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"source-file-exclude=<filters> "
06994     « Whitespace(sizePrefixDisplay * 1) « "filters OUT tests by their file\n";
06995     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ts, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"test-suite=<filters> "
06996     « Whitespace(sizePrefixDisplay * 1) « "filters      tests by their test suite\n";
06997     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "tse, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"test-suite-exclude=<filters> "
06998     « Whitespace(sizePrefixDisplay * 1) « "filters OUT tests by their test suite\n";
06999     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "sc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"subcase=<filters> "
07000     « Whitespace(sizePrefixDisplay * 1) « "filters      subcases by their name\n";
07001     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "sce, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"subcase-exclude=<filters> "
07002     « Whitespace(sizePrefixDisplay * 1) « "filters OUT subcases by their name\n";
07003     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "r, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"reporters=<filters> "
07004     « Whitespace(sizePrefixDisplay * 1) « "reporters to use (console is default)\n";
07005     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "o, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "out=<string>
"
07006     « Whitespace(sizePrefixDisplay * 1) « "output filename\n";
07007     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ob, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"order-by=<string> "
07008     « Whitespace(sizePrefixDisplay * 1) « "how the tests should be ordered\n";
07009     s « Whitespace(sizePrefixDisplay * 3) « "                                     <string> -
[file/suite/name/rand/none]\n";
07010     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "rs, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "rand-seed=<int>
"
07011     « Whitespace(sizePrefixDisplay * 1) « "seed for random ordering\n";
07012     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "f, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "first=<int>
"
07013     « Whitespace(sizePrefixDisplay * 1) « "the first test passing the filters to\n";
07014     s « Whitespace(sizePrefixDisplay * 3) « "                                     execute - for
range-based execution\n";
07015     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "l, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "last=<int>
"
07016     « Whitespace(sizePrefixDisplay * 1) « "the last test passing the filters to\n";
07017     s « Whitespace(sizePrefixDisplay * 3) « "                                     execute - for
range-based execution\n";
07018     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "aa, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"abort-after=<int> "
07019     « Whitespace(sizePrefixDisplay * 1) « "stop after <int> failed assertions\n";
07020     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "scfl, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"subcase-filter-levels=<int> "
07021     « Whitespace(sizePrefixDisplay * 1) « "apply filters for the first <int> levels\n";
07022     s « Color::Cyan « "\n[doctest] " « Color::None;
07023     s « "Bool options - can be used like flags and true is assumed. Available:\n\n";
07024     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "s, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "success=<bool>
"
07025     « Whitespace(sizePrefixDisplay * 1) « "include successful assertions in output\n";
07026     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "cs, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"case-sensitive=<bool> "
07027     « Whitespace(sizePrefixDisplay * 1) « "filters being treated as case sensitive\n";
07028     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "e, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "exit=<bool>
"
07029     « Whitespace(sizePrefixDisplay * 1) « "exits after the tests finish\n";
07030     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "d, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "duration=<bool>
"
07031     « Whitespace(sizePrefixDisplay * 1) « "prints the time duration of each test\n";
07032     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "m, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "minimal=<bool>
"
07033     « Whitespace(sizePrefixDisplay * 1) « "minimal console output (only failures)\n";
07034     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "q, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "quiet=<bool>
"
07035     « Whitespace(sizePrefixDisplay * 1) « "no console output\n";
07036     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nt, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "no-throw=<bool>
"
07037     « Whitespace(sizePrefixDisplay * 1) « "skips exceptions-related assert checks\n";
07038     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ne, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-exitcode=<bool> "
07039     « Whitespace(sizePrefixDisplay * 1) « "returns (or exits) always with success\n";
07040     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nr, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "no-run=<bool>
"
07041     « Whitespace(sizePrefixDisplay * 1) « "skips all runtime doctest operations\n";
07042     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ni, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "no-intro=<bool>
"
07043     « Whitespace(sizePrefixDisplay * 1) « "omit the framework intro in the output\n";
07044     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nv, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-version=<bool> "
07045     « Whitespace(sizePrefixDisplay * 1) « "omit the framework version in the output\n";
07046     s « " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "no-colors=<bool>
"
07047     « Whitespace(sizePrefixDisplay * 1) « "disables colors in output\n";

```

```

07048     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "fc, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"force-colors=<bool> "
07049     << Whitespace(sizePrefixDisplay * 1) << "use colors even when not in a tty\n";
07050     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nb, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "no-breaks=<bool>
"
07051     << Whitespace(sizePrefixDisplay * 1) << "disables breakpoints in debuggers\n";
07052     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "ns, --" DOCTEST_OPTIONS_PREFIX_DISPLAY "no-skip=<bool>
"
07053     << Whitespace(sizePrefixDisplay * 1) << "don't skip test cases marked as skip\n";
07054     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "gfl, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"gnu-file-line=<bool> "
07055     << Whitespace(sizePrefixDisplay * 1) << ":n: vs (n): for line numbers in output\n";
07056     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "npf, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-path-filenames=<bool> "
07057     << Whitespace(sizePrefixDisplay * 1) << "only filenames and no paths in output\n";
07058     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "sfp, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"strip-file-prefixes=<p1:p2> "
07059     << Whitespace(sizePrefixDisplay * 1) << "whenever file paths start with this prefix, remove it
from the output\n";
07060     s << " -" DOCTEST_OPTIONS_PREFIX_DISPLAY "nln, --" DOCTEST_OPTIONS_PREFIX_DISPLAY
"no-line-numbers=<bool> "
07061     << Whitespace(sizePrefixDisplay * 1) << "0 instead of real line numbers in output\n";
07062     // ===== « 79
07063     // clang-format on
07064
07065     s << Color::Cyan << "\n[doctest] " << Color::None;
07066     s << "for more information visit the project documentation\n\n";
07067 }
07068
07069 void ConsoleReporter::printRegisteredReporters() {
07070     printVersion();
07071     auto printReporters = [this](const detail::reporterMap &reporters, const char *type) {
07072         if (!reporters.empty()) {
07073             s << Color::Cyan << "[doctest] " << Color::None << "listing all registered " << type << "\n";
07074             for (auto &curr: reporters)
07075                 s << "priority: " << std::setw(5) << curr.first.first << " name: " << curr.first.second <<
"\n";
07076         }
07077     };
07078     printReporters(detail::getListeners(), "listeners");
07079     printReporters(detail::getReporters(), "reporters");
07080 }
07081
07082 void ConsoleReporter::report_query(const QueryData &in) {
07083     if (opt.version) {
07084         printVersion();
07085     } else if (opt.help) {
07086         printHelp();
07087     } else if (opt.list_reporters) {
07088         printRegisteredReporters();
07089     } else if (opt.count || opt.list_test_cases) {
07090         if (opt.list_test_cases) {
07091             s << Color::Cyan << "[doctest] " << Color::None << "listing all test case names\n";
07092             separator_to_stream();
07093         }
07094
07095         for (unsigned i = 0; i < in.num_data; ++i)
07096             s << Color::None << in.data[i]->m_name << "\n";
07097
07098         separator_to_stream();
07099
07100         s << Color::Cyan << "[doctest] " << Color::None
07101         << "unskipped test cases passing the current filters: " << g_cs->numTestCasesPassingFilters <<
"\n";
07102
07103     } else if (opt.list_test_suites) {
07104         s << Color::Cyan << "[doctest] " << Color::None << "listing all test suites\n";
07105         separator_to_stream();
07106
07107         for (unsigned i = 0; i < in.num_data; ++i)
07108             s << Color::None << in.data[i]->m_test_suite << "\n";
07109
07110         separator_to_stream();
07111
07112         s << Color::Cyan << "[doctest] " << Color::None
07113         << "unskipped test cases passing the current filters: " << g_cs->numTestCasesPassingFilters <<
"\n";
07114         s << Color::Cyan << "[doctest] " << Color::None
07115         << "test suites with unskipped test cases passing the current filters: " <<
g_cs->numTestSuitesPassingFilters
07116         << "\n";
07117     }
07118 }
07119
07120 void ConsoleReporter::test_run_start() {
07121     if (!opt.minimal)
07122         printIntro();

```

```

07123 }
07124
07125 void ConsoleReporter::test_run_end(const TestRunStats &p) {
07126     if (opt.minimal && p.numTestCasesFailed == 0)
07127         return;
07128
07129     separator_to_stream();
07130     s << std::dec;
07131
07132     auto totwidth = static_cast<int>(std::ceil(
07133         log10(static_cast<double>(std::max(p.numTestCasesPassingFilters,
07134             static_cast<unsigned>(p.numAsserts))) + 1)
07135     ));
07136     auto passwidth = static_cast<int>(std::ceil(log10(
07137         static_cast<double>(std::max(
07138             p.numTestCasesPassingFilters - p.numTestCasesFailed,
07139             static_cast<unsigned>(p.numAsserts - p.numAssertsFailed)
07140         )) + 1
07141     ));
07142     auto failwidth = static_cast<int>(std::ceil(
07143         log10(static_cast<double>(std::max(p.numTestCasesFailed,
07144             static_cast<unsigned>(p.numAssertsFailed))) + 1)
07145     ));
07146     const bool anythingFailed = p.numTestCasesFailed > 0 || p.numAssertsFailed > 0;
07147     s << Color::Cyan << "[doctest] " << Color::None << "test cases: " << std::setw(totwidth)
07148         << p.numTestCasesPassingFilters << " | "
07149         << ((p.numTestCasesPassingFilters == 0 || anythingFailed) ? Color::None : Color::Green) <<
07150         std::setw(passwidth)
07151         << p.numTestCasesPassingFilters - p.numTestCasesFailed << " passed" << Color::None << " | "
07152         << (p.numTestCasesFailed > 0 ? Color::Red : Color::None) << std::setw(failwidth) <<
07153         p.numTestCasesFailed
07154         << " failed" << Color::None << " |";
07155     if (opt.no_skipped_summary == false) {
07156         const unsigned int numSkipped = p.numTestCases - p.numTestCasesPassingFilters;
07157         s << " " << (numSkipped == 0 ? Color::None : Color::Yellow) << numSkipped << " skipped" <<
07158         Color::None;
07159     }
07160     s << "\n";
07161     s << Color::Cyan << "[doctest] " << Color::None << "assertions: " << std::setw(totwidth) << p.numAsserts
07162     << " | "
07163     << ((p.numAsserts == 0 || anythingFailed) ? Color::None : Color::Green) << std::setw(passwidth)
07164     << (p.numAsserts - p.numAssertsFailed) << " passed" << Color::None << " | "
07165     << (p.numAssertsFailed > 0 ? Color::Red : Color::None) << std::setw(failwidth) <<
07166     p.numAssertsFailed << " failed"
07167     << Color::None << " |\n";
07168     s << Color::Cyan << "[doctest] " << Color::None
07169     << "Status: " << (p.numTestCasesFailed > 0 ? Color::Red : Color::Green)
07170     << ((p.numTestCasesFailed > 0) ? "FAILURE!" : "SUCCESS!") << Color::None << std::endl;
07171 }
07172
07173 void ConsoleReporter::test_case_start(const TestCaseData &in) {
07174     DOCTEST_LOCK_MUTEX(mutex)
07175     hasLoggedCurrentTestStart = false;
07176     tc = &in;
07177     subcasesStack.clear();
07178     currentSubcaseLevel = 0;
07179 }
07180
07181 void ConsoleReporter::test_case_reenter(const TestCaseData &) {
07182     DOCTEST_LOCK_MUTEX(mutex)
07183     subcasesStack.clear();
07184 }
07185
07186 void ConsoleReporter::test_case_end(const CurrentTestCaseStats &st) {
07187     DOCTEST_LOCK_MUTEX(mutex)
07188     if (tc->m_no_output)
07189         return;
07190
07191     // log the preamble of the test case only if there is something
07192     // else to print - something other than that an assert has failed
07193     if (opt.duration ||
07194         (st.failure_flags && st.failure_flags !=
07195         static_cast<int>(TestCaseFailureReason::AssertFailure)))
07196         logTestStart();
07197
07198     if (opt.duration)
07199         s << Color::None << std::setprecision(6) << std::fixed << st.seconds << " s: " << tc->m_name << "\n";
07200
07201     if (st.failure_flags & TestCaseFailureReason::Timeout)
07202         s << Color::Red << "Test case exceeded time limit of " << std::setprecision(6) << std::fixed <<
07203         tc->m_timeout
07204         << "!\n";
07205
07206     if (st.failure_flags & TestCaseFailureReason::ShouldHaveFailedButDidnt) {
07207         s << Color::Red << "Should have failed but didn't! Marking it as failed!\n";
07208     } else if (st.failure_flags & TestCaseFailureReason::ShouldHaveFailedAndDid) {

```

```

07201         s « Color::Yellow « "Failed as expected so marking it as not failed\n";
07202     } else if (st.failure_flags & TestCaseFailureReason::CouldHaveFailedAndDid) {
07203         s « Color::Yellow « "Allowed to fail so marking it as not failed\n";
07204     } else if (st.failure_flags & TestCaseFailureReason::DidntFailExactlyNumTimes) {
07205         s « Color::Red « "Didn't fail exactly " « tc->m_expected_failures « " times so marking it as
failed!\n";
07206     } else if (st.failure_flags & TestCaseFailureReason::FailedExactlyNumTimes) {
07207         s « Color::Yellow « "Failed exactly " « tc->m_expected_failures
07208             « " times as expected so marking it as not failed!\n";
07209     }
07210     if (st.failure_flags & TestCaseFailureReason::TooManyFailedAsserts) {
07211         s « Color::Red « "Aborting - too many failed asserts!\n";
07212     }
07213     s « Color::None;
07214 }
07215
07216 void ConsoleReporter::test_case_exception(const TestCaseException &e) {
07217     DOCTEST_LOCK_MUTEX(mutex)
07218     if (tc->m_no_output)
07219         return;
07220
07221     logTestStart();
07222
07223     file_line_to_stream(tc->m_file.c_str(), static_cast<int>(tc->m_line), " ");
07224     successOrFailColoredStringToStream(false, e.is_crash ? assertType::is_require :
assertType::is_check);
07225     s « Color::Red « (e.is_crash ? "test case CRASHED: " : "test case THREW exception: ");
07226     s « Color::Cyan « e.error_string « "\n";
07227
07228     const int num_stringified_contexts = get_num_stringified_contexts();
07229     if (num_stringified_contexts) {
07230         auto stringified_contexts = get_stringified_contexts();
07231         s « Color::None « " logged: ";
07232         for (int i = num_stringified_contexts; i > 0; --i) {
07233             s « (i == num_stringified_contexts ? " " : " ") « stringified_contexts[i - 1] «
"\n";
07234         }
07235     }
07236     s « "\n" « Color::None;
07237 }
07238
07239 void ConsoleReporter::subcase_start(const SubcaseSignature &subc) {
07240     DOCTEST_LOCK_MUTEX(mutex)
07241     subcasesStack.push_back(subc);
07242     ++currentSubcaseLevel;
07243     hasLoggedCurrentTestStart = false;
07244 }
07245
07246 void ConsoleReporter::subcase_end() {
07247     DOCTEST_LOCK_MUTEX(mutex)
07248     --currentSubcaseLevel;
07249     hasLoggedCurrentTestStart = false;
07250 }
07251
07252 void ConsoleReporter::log_assert(const AssertData &rb) {
07253     if (!rb.m_failed && !opt.success) || tc->m_no_output)
07254         return;
07255
07256     DOCTEST_LOCK_MUTEX(mutex)
07257
07258     logTestStart();
07259
07260     file_line_to_stream(rb.m_file, rb.m_line, " ");
07261     successOrFailColoredStringToStream(!rb.m_failed, rb.m_at);
07262
07263     fulltext_log_assert_to_stream(s, rb);
07264
07265     log_contexts();
07266 }
07267
07268 void ConsoleReporter::log_message(const MessageData &mb) {
07269     if (tc->m_no_output)
07270         return;
07271
07272     DOCTEST_LOCK_MUTEX(mutex)
07273
07274     logTestStart();
07275
07276     file_line_to_stream(mb.m_file, mb.m_line, " ");
07277     s « getSuccessOrFailColor(false, mb.m_severity)
07278         « getSuccessOrFailString(mb.m_severity & assertType::is_warn, mb.m_severity, "MESSAGE") « ": ";
07279     s « Color::None « mb.m_string « "\n";
07280     log_contexts();
07281 }
07282
07283 void ConsoleReporter::test_case_skipped(const TestCaseData &) {}
07284

```



```

07285 } // namespace doctest
07286
07287 #endif // DOCTEST_CONFIG_DISABLE
07288
07289 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
07290
07291 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
07292
07293 #ifndef DOCTEST_CONFIG_DISABLE
07294
07295 namespace doctest {
07296 namespace detail {
07297
07298 #ifdef DOCTEST_PLATFORM_WINDOWS
07299 #define DOCTEST_OUTPUT_DEBUG_STRING(text) ::OutputDebugStringA(text)
07300 #endif // Platform
07301
07302 #ifdef DOCTEST_PLATFORM_WINDOWS
07303
07304 DOCTEST_THREAD_LOCAL std::ostringstream DebugOutputWindowReporter::oss;
07305
07306 DebugOutputWindowReporter::DebugOutputWindowReporter(const ContextOptions &co)
07307     : ConsoleReporter(co, oss) {}
07308
07309 #define DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(func, type, arg)
07310 \
07311 \
07312 \
07313 \
07314 \
07315 \
07316 \
07317 \
07318 \
07319 \
07320 \
07321
07322 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_run_start, DOCTEST_EMPTY, DOCTEST_EMPTY)
07323 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_run_end, const TestRunStats &, in)
07324 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_start, const TestCaseData &, in)
07325 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_reenter, const TestCaseData &, in)
07326 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_end, const CurrentTestCaseStats &, in)
07327 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_exception, const TestCaseException &, in)
07328 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(subcase_start, const SubcaseSignature &, in)
07329 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(subcase_end, DOCTEST_EMPTY, DOCTEST_EMPTY)
07330 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(log_assert, const AssertData &, in)
07331 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(log_message, const MessageData &, in)
07332 DOCTEST_DEBUG_OUTPUT_REPORTER_OVERRIDE(test_case_skipped, const TestCaseData &, in)
07333
07334 #endif // DOCTEST_PLATFORM_WINDOWS
07335
07336 } // namespace detail
07337 } // namespace doctest
07338
07339 #endif // DOCTEST_CONFIG_DISABLE
07340
07341 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
07342
07343 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
07344
07345 #ifndef DOCTEST_CONFIG_DISABLE
07346
07347 namespace doctest {
07348
07349 std::string JUnitReporter::JUnitTestCaseData::getCurrentTimestamp() {
07350     // Beware, this is not reentrant because of backward compatibility issues
07351     // Also, UTC only, again because of backward compatibility (%z is C++11)
07352     time_t rawtime{};
07353     static_cast<void>(std::time(&rawtime));
07354     const auto timeStampSize = sizeof("2017-01-16T17:06:45Z");
07355
07356     std::tm timeInfo;
07357 #if defined(DOCTEST_PLATFORM_WINDOWS)
07358     gmtime_s(&timeInfo, &rawtime);
07359 #elif defined(__STDC_LIB_EXT1__)
07360     gmtime_s(&rawtime, &timeInfo);

```

```

07361 #else // DOCTEST_PLATFORM_WINDOWS
07362     gmtime_r(&rawtime, &timeInfo);
07363 #endif // DOCTEST_PLATFORM_WINDOWS
07364
07365     char timeStamp[timeStampSize];
07366     const char *const fmt = "%Y-%m-%dT%H:%M:%SZ";
07367
07368     static_cast<void>(std::strftime(timeStamp, timeStampSize, fmt, &timeInfo));
07369     return std::string(timeStamp);
07370 }
07371
07372 JUnitReporter::JUnitTestCaseData::JUnitTestMessage::JUnitTestMessage(
07373     const std::string &_message, const std::string &_type, const std::string &_details
07374 )
07375     : message(_message), type(_type), details(_details) {}
07376
07377 JUnitReporter::JUnitTestCaseData::JUnitTestMessage::JUnitTestMessage(
07378     const std::string &_message, const std::string &_details
07379 )
07380     : message(_message), type(), details(_details) {}
07381
07382 JUnitReporter::JUnitTestCaseData::JUnitTestCase::JUnitTestCase(const std::string &_classname, const
std::string &_name)
07383     : classname(_classname), name(_name), time(0), failures(), errors() {}
07384
07385 void JUnitReporter::JUnitTestCaseData::add(const std::string &classname, const std::string &name) {
07386     testcases.emplace_back(classname, name);
07387 }
07388
07389 void JUnitReporter::JUnitTestCaseData::appendSubcaseNamesToLastTestcase(std::vector<String> nameStack)
{
07390     for (auto &curr: nameStack)
07391         if (curr.size())
07392             testcases.back().name += std::string("/") + curr.c_str();
07393 }
07394
07395 void JUnitReporter::JUnitTestCaseData::addTime(double time) {
07396     if (time < 1e-4)
07397         time = 0;
07398     testcases.back().time = time;
07399     totalSeconds += time;
07400 }
07401
07402 void JUnitReporter::JUnitTestCaseData::addFailure(
07403     const std::string &message, const std::string &type, const std::string &details
07404 ) {
07405     testcases.back().failures.emplace_back(message, type, details);
07406     ++totalFailures;
07407 }
07408
07409 void JUnitReporter::JUnitTestCaseData::addError(const std::string &message, const std::string
&details) {
07410     testcases.back().errors.emplace_back(message, details);
07411     ++totalErrors;
07412 }
07413
07414 JUnitReporter::JUnitReporter(const ContextOptions &co)
07415     : xml(*co.cout), opt(co) {}
07416
07417 unsigned JUnitReporter::line(unsigned l) const {
07418     return opt.no_line_numbers ? 0 : l;
07419 }
07420
07421 void JUnitReporter::report_query(const QueryData &) {
07422     xml.writeDeclaration();
07423 }
07424
07425 void JUnitReporter::test_run_start() {
07426     xml.writeDeclaration();
07427 }
07428
07429 void JUnitReporter::test_run_end(const TestRunStats &p) {
07430     // remove .exe extension - mainly to have the same output on UNIX and Windows
07431     // NOLINTNEXTLINE(misc-const-correctness)
07432     std::string binary_name = skipPathFromFilename(opt.binary_name.c_str());
07433     #ifndef DOCTEST_PLATFORM_WINDOWS
07434     if (binary_name.rfind(".exe") != std::string::npos)
07435         binary_name = binary_name.substr(0, binary_name.length() - 4);
07436     #endif // DOCTEST_PLATFORM_WINDOWS
07437
07438     xml.startElement("testsuites");
07439     xml.startElement("testsuite")
07440         .writeAttribute("name", binary_name)
07441         .writeAttribute("errors", testCaseData.totalErrors)
07442         .writeAttribute("failures", testCaseData.totalFailures)
07443         .writeAttribute("tests", p.numAsserts);
07444     if (opt.no_time_in_output == false) {

```

```

07445         xml.writeAttribute("time", testCaseData.totalSeconds);
07446         xml.writeAttribute("timestamp", JUnitTestCaseData::getCurrentTimestamp());
07447     }
07448     if (opt.no_version == false)
07449         xml.writeAttribute("doctest_version", DOCTEST_VERSION_STR);
07450
07451     for (const auto &testCase: testCaseData.testcases) {
07452         xml.startElement("testcase")
07453             .writeAttribute("classname", testCase.classname)
07454             .writeAttribute("name", testCase.name);
07455         if (opt.no_time_in_output == false)
07456             xml.writeAttribute("time", testCase.time);
07457         // This is not ideal, but it should be enough to mimic gtest's junit output.
07458         xml.writeAttribute("status", "run");
07459
07460         for (const auto &failure: testCase.failures) {
07461             xml.scopedElement("failure")
07462                 .writeAttribute("message", failure.message)
07463                 .writeAttribute("type", failure.type)
07464                 .writeText(failure.details, false);
07465         }
07466
07467         for (const auto &error: testCase.errors) {
07468             xml.scopedElement("error").writeAttribute("message",
07469 error.message).writeText(error.details);
07469         }
07470
07471         xml.endElement();
07472     }
07473     xml.endElement();
07474     xml.endElement();
07475 }
07476
07477 void JUnitReporter::test_case_start(const TestCaseData &in) {
07478     DOCTEST_LOCK_MUTEX(mutex)
07479     testCaseData.add(skipPathFromFilename(in.m_file.c_str(), in.m_name);
07480     timer.start();
07481     tc = &in;
07482 }
07483
07484 void JUnitReporter::test_case_reenter(const TestCaseData &in) {
07485     DOCTEST_LOCK_MUTEX(mutex)
07486     testCaseData.addTime(timer.getElapsedSeconds());
07487     testCaseData.appendSubcaseNamesToLastTestcase(deepestSubcaseStackNames);
07488     deepestSubcaseStackNames.clear();
07489
07490     timer.start();
07491     testCaseData.add(skipPathFromFilename(in.m_file.c_str(), in.m_name);
07492     tc = &in;
07493 }
07494
07495 void JUnitReporter::test_case_end(const CurrentTestCaseStats &st) {
07496     DOCTEST_LOCK_MUTEX(mutex)
07497     testCaseData.addTime(timer.getElapsedSeconds());
07498     testCaseData.appendSubcaseNamesToLastTestcase(deepestSubcaseStackNames);
07499     deepestSubcaseStackNames.clear();
07500
07501     if (st.failure_flags & TestCaseFailureReason::Timeout) {
07502         auto *stream = detail::tlssPush();
07503         *stream << "Test case exceeded time limit of " << std::setprecision(6) << std::fixed <<
07504         tc->m_timeout;
07505         testCaseData.addError("timeout", detail::tlssPop().c_str());
07506     }
07507
07508     if (st.failure_flags & TestCaseFailureReason::ShouldHaveFailedButDidnt) {
07509         testCaseData.addError("should_fail", "Should have failed, but didn't");
07510     } else if (st.failure_flags & TestCaseFailureReason::DidntFailExactlyNumTimes) {
07511         auto *stream = detail::tlssPush();
07512         *stream << "Should have failed exactly " << tc->m_expected_failures << " times, but didn't";
07513         testCaseData.addError("should_fail", detail::tlssPop().c_str());
07514     }
07515
07516     if (st.failure_flags & TestCaseFailureReason::TooManyFailedAsserts) {
07517         testCaseData.addError("abort_after", "Too many failed asserts");
07518     }
07519     tc = nullptr;
07520 }
07521
07522 void JUnitReporter::test_case_exception(const TestCaseException &e) {
07523     DOCTEST_LOCK_MUTEX(mutex)
07524     testCaseData.addError("exception", e.error_string.c_str());
07525 }
07526
07527 void JUnitReporter::subcase_start(const SubcaseSignature &in) {
07528     DOCTEST_LOCK_MUTEX(mutex)
07529     deepestSubcaseStackNames.push_back(in.m_name);

```

```

07530 }
07531
07532 void JUnitReporter::subcase_end() {}
07533
07534 void JUnitReporter::log_assert(const AssertData &rb) {
07535     if (!rb.m_failed) // report only failures & ignore the `success` option
07536         return;
07537
07538     DOCTEST_LOCK_MUTEX(mutex)
07539
07540     std::ostringstream os;
07541     os << skipPathFromFilename(rb.m_file) << (opt.gnu_file_line ? ":" : "(") << line(rb.m_line)
07542         << (opt.gnu_file_line ? ":" : ":") << std::endl;
07543
07544     fulltext_log_assert_to_stream(os, rb);
07545     log_contexts(os);
07546     testCaseData.addFailure(rb.m_decomp.c_str(), assertString(rb.m_at), os.str());
07547 }
07548
07549 void JUnitReporter::log_message(const MessageData &mb) {
07550     if (mb.m_severity & assertType::is_warn) // report only failures
07551         return;
07552
07553     DOCTEST_LOCK_MUTEX(mutex)
07554
07555     std::ostringstream os;
07556     os << skipPathFromFilename(mb.m_file) << (opt.gnu_file_line ? ":" : "(") << line(mb.m_line)
07557         << (opt.gnu_file_line ? ":" : ":") << std::endl;
07558
07559     os << mb.m_string.c_str() << "\n";
07560     log_contexts(os);
07561
07562     testCaseData.addFailure(
07563         mb.m_string.c_str(), mb.m_severity & assertType::is_check ? "FAIL_CHECK" : "FAIL", os.str()
07564     );
07565 }
07566
07567 void JUnitReporter::test_case_skipped(const TestCaseData &) {}
07568
07569 void JUnitReporter::log_contexts(std::ostringstream &s) {
07570     const int num_contexts = get_num_active_contexts();
07571     if (num_contexts) {
07572         auto contexts = get_active_contexts();
07573
07574         s << " logged: ";
07575         for (int i = 0; i < num_contexts; ++i) {
07576             s << (i == 0 ? "" : " ") << contexts[i] << " ";
07577             contexts[i] ->stringify(&s);
07578             s << std::endl;
07579         }
07580     }
07581 }
07582
07583 } // namespace doctest
07584
07585 #endif // DOCTEST_CONFIG_DISABLE
07586
07587 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
07588
07589 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
07590
07591 #ifndef DOCTEST_CONFIG_DISABLE
07592
07593 namespace doctest {
07594
07595 XmlReporter::XmlReporter(const ContextOptions &co)
07596     : xml(*co.cout), opt(co) {}
07597
07598 void XmlReporter::log_contexts() {
07599     const int num_contexts = get_num_active_contexts();
07600     if (num_contexts) {
07601         auto contexts = get_active_contexts();
07602         std::stringstream ss;
07603         for (int i = 0; i < num_contexts; ++i) {
07604             contexts[i] ->stringify(&ss);
07605             xml.scopedElement("Info").writeText(ss.str());
07606             ss.str("");
07607         }
07608     }
07609 }
07610
07611 unsigned XmlReporter::line(unsigned l) const {
07612     return opt.no_line_numbers ? 0 : l;
07613 }
07614
07615 void XmlReporter::test_case_start_impl(const TestCaseData &in) {
07616     bool open_ts_tag = false;

```

```

07617     if (tc != nullptr) { // we have already opened a test suite
07618         if (std::strcmp(tc->m_test_suite, in.m_test_suite) != 0) {
07619             xml.endElement();
07620             open_ts_tag = true;
07621         }
07622     } else {
07623         open_ts_tag = true; // first test case ==> first test suite
07624     }
07625
07626     if (open_ts_tag) {
07627         xml.startElement("TestSuite");
07628         xml.writeAttribute("name", in.m_test_suite);
07629     }
07630
07631     tc = &in;
07632     xml.startElement("TestCase")
07633         .writeAttribute("name", in.m_name)
07634         .writeAttribute("filename", skipPathFromFilename(in.m_file.c_str()))
07635         .writeAttribute("line", line(in.m_line))
07636         .writeAttribute("description", in.m_description);
07637
07638     if (Approx(in.m_timeout) != 0)
07639         xml.writeAttribute("timeout", in.m_timeout);
07640     if (in.m_may_fail)
07641         xml.writeAttribute("may_fail", true);
07642     if (in.m_should_fail)
07643         xml.writeAttribute("should_fail", true);
07644 }
07645
07646 void XmlReporter::report_query(const QueryData &in) {
07647     test_run_start();
07648     if (opt.list_reporters) {
07649         for (auto &curr: detail::getListeners())
07650             xml.scopedElement("Listener")
07651                 .writeAttribute("priority", curr.first.first)
07652                 .writeAttribute("name", curr.first.second);
07653         for (auto &curr: detail::getReporters())
07654             xml.scopedElement("Reporter")
07655                 .writeAttribute("priority", curr.first.first)
07656                 .writeAttribute("name", curr.first.second);
07657     } else if (opt.count || opt.list_test_cases) {
07658         for (unsigned i = 0; i < in.num_data; ++i) {
07659             xml.scopedElement("TestCase")
07660                 .writeAttribute("name", in.data[i]->m_name)
07661                 .writeAttribute("testsuite", in.data[i]->m_test_suite)
07662                 .writeAttribute("filename", skipPathFromFilename(in.data[i]->m_file.c_str()))
07663                 .writeAttribute("line", line(in.data[i]->m_line))
07664                 .writeAttribute("skipped", in.data[i]->m_skip);
07665         }
07666         xml.scopedElement("OverallResultsTestCases")
07667             .writeAttribute("unskipped", in.run_stats->numTestCasesPassingFilters);
07668     } else if (opt.list_test_suites) {
07669         for (unsigned i = 0; i < in.num_data; ++i)
07670             xml.scopedElement("TestSuite").writeAttribute("name", in.data[i]->m_test_suite);
07671         xml.scopedElement("OverallResultsTestCases")
07672             .writeAttribute("unskipped", in.run_stats->numTestCasesPassingFilters);
07673         xml.scopedElement("OverallResultsTestSuites")
07674             .writeAttribute("unskipped", in.run_stats->numTestSuitesPassingFilters);
07675     }
07676     xml.endElement();
07677 }
07678
07679 void XmlReporter::test_run_start() {
07680     xml.writeDeclaration();
07681
07682     // remove .exe extension - mainly to have the same output on UNIX and Windows
07683     // NOLINTNEXTLINE(misc-const-correctness)
07684     std::string binary_name = skipPathFromFilename(opt.binary_name.c_str());
07685 #ifdef DOCTEST_PLATFORM_WINDOWS
07686     if (binary_name.rfind(".exe") != std::string::npos)
07687         binary_name = binary_name.substr(0, binary_name.length() - 4);
07688 #endif // DOCTEST_PLATFORM_WINDOWS
07689
07690     xml.startElement("doctest").writeAttribute("binary", binary_name);
07691     if (opt.no_version == false)
07692         xml.writeAttribute("version", DOCTEST_VERSION_STR);
07693
07694     // only the consequential ones (TODO: filters)
07695     xml.scopedElement("Options")
07696         .writeAttribute("order_by", opt.order_by.c_str())
07697         .writeAttribute("rand_seed", opt.rand_seed)
07698         .writeAttribute("first", opt.first)
07699         .writeAttribute("last", opt.last)
07700         .writeAttribute("abort_after", opt.abort_after)
07701         .writeAttribute("subcase_filter_levels", opt.subcase_filter_levels)
07702         .writeAttribute("case_sensitive", opt.case_sensitive)
07703         .writeAttribute("no_throw", opt.no_throw)

```

```

07704         .writeAttribute("no_skip", opt.no_skip);
07705     }
07706
07707 void XmlReporter::test_run_end(const TestRunStats &p) {
07708     if (tc) // the TestSuite tag - only if there has been at least 1 test case
07709         xml.endElement();
07710
07711     xml.scopedElement("OverallResultsAsserts")
07712         .writeAttribute("successes", p.numAsserts - p.numAssertsFailed)
07713         .writeAttribute("failures", p.numAssertsFailed);
07714
07715     xml.startElement("OverallResultsTestCases")
07716         .writeAttribute("successes", p.numTestCasesPassingFilters - p.numTestCasesFailed)
07717         .writeAttribute("failures", p.numTestCasesFailed);
07718     if (opt.no_skipped_summary == false)
07719         xml.writeAttribute("skipped", p.numTestCases - p.numTestCasesPassingFilters);
07720     xml.endElement();
07721
07722     xml.endElement();
07723 }
07724
07725 void XmlReporter::test_case_start(const TestCaseData &in) {
07726     DOCTEST_LOCK_MUTEX(mutex)
07727     test_case_start_impl(in);
07728     xml.ensureTagClosed();
07729 }
07730
07731 void XmlReporter::test_case_reenter(const TestCaseData &) {}
07732
07733 void XmlReporter::test_case_end(const CurrentTestCaseStats &st) {
07734     DOCTEST_LOCK_MUTEX(mutex)
07735     xml.startElement("OverallResultsAsserts")
07736         .writeAttribute("successes", st.numAssertsCurrentTest - st.numAssertsFailedCurrentTest)
07737         .writeAttribute("failures", st.numAssertsFailedCurrentTest)
07738         .writeAttribute("test_case_success", st.testCaseSuccess);
07739     if (opt.duration)
07740         xml.writeAttribute("duration", st.seconds);
07741     if (tc->m_expected_failures)
07742         xml.writeAttribute("expected_failures", tc->m_expected_failures);
07743     xml.endElement();
07744
07745     xml.endElement();
07746 }
07747
07748 void XmlReporter::test_case_exception(const TestCaseException &e) {
07749     DOCTEST_LOCK_MUTEX(mutex)
07750
07751     xml.scopedElement("Exception").writeAttribute("crash",
07752         e.is_crash).writeText(e.error_string.c_str());
07753 }
07754
07755 void XmlReporter::subcase_start(const SubcaseSignature &in) {
07756     DOCTEST_LOCK_MUTEX(mutex)
07757     xml.startElement("SubCase")
07758         .writeAttribute("name", in.m_name)
07759         .writeAttribute("filename", skipPathFromFilename(in.m_file))
07760         .writeAttribute("line", line(in.m_line));
07761     xml.ensureTagClosed();
07762 }
07763
07764 void XmlReporter::subcase_end() {
07765     DOCTEST_LOCK_MUTEX(mutex)
07766     xml.endElement();
07767 }
07768
07769 void XmlReporter::log_assert(const AssertData &rb) {
07770     if (!rb.m_failed && !opt.success)
07771         return;
07772     DOCTEST_LOCK_MUTEX(mutex)
07773
07774     xml.startElement("Expression")
07775         .writeAttribute("success", !rb.m_failed)
07776         .writeAttribute("type", assertString(rb.m_at))
07777         .writeAttribute("filename", skipPathFromFilename(rb.m_file))
07778         .writeAttribute("line", line(rb.m_line));
07779
07780     xml.scopedElement("Original").writeText(rb.m_expr);
07781
07782     if (rb.m_threw)
07783         xml.scopedElement("Exception").writeText(rb.m_exception.c_str());
07784
07785     if (rb.m_at & assertType::is_throws_as)
07786         xml.scopedElement("ExpectedException").writeText(rb.m_exception_type);
07787     if (rb.m_at & assertType::is_throws_with)
07788         xml.scopedElement("ExpectedExceptionString").writeText(rb.m_exception_string.c_str());
07789     if ((rb.m_at & assertType::is_normal) && !rb.m_threw)

```

```

07790         xml.scopedElement("Expanded").writeText(rb.m_decomp.c_str());
07791
07792     log_contexts();
07793
07794     xml.endElement();
07795 }
07796
07797 void XmlReporter::log_message(const MessageData &mb) {
07798     DOCTEST_LOCK_MUTEX(mutex)
07799
07800     xml.startElement("Message")
07801         .writeAttribute("type", failureString(mb.m_severity))
07802         .writeAttribute("filename", skipPathFromFilename(mb.m_file))
07803         .writeAttribute("line", line(mb.m_line));
07804
07805     xml.scopedElement("Text").writeText(mb.m_string.c_str());
07806
07807     log_contexts();
07808
07809     xml.endElement();
07810 }
07811
07812 void XmlReporter::test_case_skipped(const TestCaseData &in) {
07813     if (opt.no_skipped_summary == false) {
07814         DOCTEST_LOCK_MUTEX(mutex)
07815         test_case_start_impl(in);
07816         xml.writeAttribute("skipped", "true");
07817         xml.endElement();
07818     }
07819 }
07820
07821 } // namespace doctest
07822
07823 #endif // DOCTEST_CONFIG_DISABLE
07824
07825 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
07826 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
07827
07828 #ifndef DOCTEST_CONFIG_DISABLE
07829
07830 namespace doctest {
07831 namespace detail {
07832
07833 #if !defined(DOCTEST_CONFIG_POSIX_SIGNALS) && !defined(DOCTEST_CONFIG_WINDOWS_SEH)
07834 void FatalConditionHandler::reset() {}
07835 void FatalConditionHandler::allocateAltStackMem() {}
07836 void FatalConditionHandler::freeAltStackMem() {}
07837 #else // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
07838 #ifdef DOCTEST_PLATFORM_WINDOWS
07839 // There is no 1-1 mapping between signals and windows exceptions.
07840 // Windows can easily distinguish between SO and SigSegV,
07841 // but SigInt, SigTerm, etc are handled differently.
07842 SignalDefs signalDefs[] = {
07843     {static_cast<DWORD>(EXCEPTION_ILLEGAL_INSTRUCTION), "SIGILL - Illegal instruction signal"},
07844     {static_cast<DWORD>(EXCEPTION_STACK_OVERFLOW), "SIGSEGV - Stack overflow"},
07845     {static_cast<DWORD>(EXCEPTION_ACCESS_VIOLATION), "SIGSEGV - Segmentation violation signal"},
07846     {static_cast<DWORD>(EXCEPTION_INT_DIVIDE_BY_ZERO), "Divide by zero error"},
07847 };
07848
07849 LONG CALLBACK FatalConditionHandler::handleException(PEXCEPTION_POINTERS ExceptionInfo) {
07850     // Multiple threads may enter this filter/handler at once. We want the error message to be
07851     // printed on the console just once no matter how many threads have crashed.
07852     DOCTEST_DECLARE_STATIC_MUTEX(mutex)
07853     static bool execute = true;
07854     {
07855         DOCTEST_LOCK_MUTEX(mutex)
07856         if (execute) {
07857             bool reported = false;
07858             for (size_t i = 0; i < DOCTEST_COUNTOF(signalDefs); ++i) {
07859                 if (ExceptionInfo->ExceptionRecord->ExceptionCode == signalDefs[i].id) {
07860                     reportFatal(signalDefs[i].name);
07861                     reported = true;
07862                     break;
07863                 }
07864             }
07865             if (reported == false)
07866                 reportFatal("Unhandled SEH exception caught");
07867             if (isDebuggerActive() && !g_cs->no_breaks)
07868                 DOCTEST_BREAK_INTO_DEBUGGER();
07869         }
07870         execute = false;
07871     }
07872     std::exit(EXIT_FAILURE);
07873 }
07874 }
07875 }
07876 }

```

```

07877
07878 void FatalConditionHandler::allocateAltStackMem() {}
07879 void FatalConditionHandler::freeAltStackMem() {}
07880
07881 FatalConditionHandler::FatalConditionHandler() {
07882     isSet = true;
07883     // 32k seems enough for doctest to handle stack overflow,
07884     // but the value was found experimentally, so there is no strong guarantee
07885     guaranteeSize = 32 * 1024;
07886     // Register an unhandled exception filter
07887     previousTop = SetUnhandledExceptionFilter(handleException);
07888     // Pass in guarantee size to be filled
07889     SetThreadStackGuarantee(&guaranteeSize);
07890
07891     // On Windows uncaught exceptions from another thread, exceptions from
07892     // destructors, or calls to std::terminate are not a SEH exception
07893
07894     // The terminal handler gets called when:
07895     // - std::terminate is called FROM THE TEST RUNNER THREAD
07896     // - an exception is thrown from a destructor FROM THE TEST RUNNER THREAD
07897     original_terminate_handler = std::get_terminate();
07898     std::set_terminate([]() DOCTEST_NOEXCEPT {
07899         reportFatal("Terminate handler called");
07900         if (isDebuggerActive() && !g_cs->no_breaks)
07901             DOCTEST_BREAK_INTO_DEBUGGER();
07902         std::exit(EXIT_FAILURE); // explicitly exit - otherwise the SIGABRT handler may be called as
07903         well
07904     });
07905
07906     // SIGABRT is raised when:
07907     // - std::terminate is called FROM A DIFFERENT THREAD
07908     // - an exception is thrown from a destructor FROM A DIFFERENT THREAD
07909     // - an uncaught exception is thrown FROM A DIFFERENT THREAD
07910     prev_sigabrt_handler = std::signal(SIGABRT, [](int signal) DOCTEST_NOEXCEPT {
07911         if (signal == SIGABRT) {
07912             reportFatal("SIGABRT - Abort (abnormal termination) signal");
07913             if (isDebuggerActive() && !g_cs->no_breaks)
07914                 DOCTEST_BREAK_INTO_DEBUGGER();
07915             std::exit(EXIT_FAILURE);
07916         }
07917     });
07918
07919     // The following settings are taken from google test, and more
07920     // specifically from UnitTest::Run() inside of gtest.cc
07921
07922     // the user does not want to see pop-up dialogs about crashes
07923     prev_error_mode_1 = SetErrorMode(
07924         SEM_FAILCRITICALERRORS | SEM_NOALIGNMENTFAULTEXCEPT | SEM_NOGPFAULTERRORBOX |
07925         SEM_NOOPENFILEERRORBOX
07926     );
07927     // This forces the abort message to go to stderr in all circumstances.
07928     prev_error_mode_2 = _set_error_mode(_OUT_TO_STDERR);
07929     // In the debug version, Visual Studio pops up a separate dialog
07930     // offering a choice to debug the aborted program - we want to disable that.
07931     prev_abort_behavior = _set_abort_behavior(0x0, _WRITE_ABORT_MSG | _CALL_REPORTFAULT);
07932     // In debug mode, the Windows CRT can crash with an assertion over invalid
07933     // input (e.g. passing an invalid file descriptor). The default handling
07934     // for these assertions is to pop up a dialog and wait for user input.
07935     // Instead ask the CRT to dump such assertions to stderr non-interactively.
07936     prev_report_mode = _CrtSetReportMode(_CRT_ASSERT, _CRTDBG_MODE_FILE | _CRTDBG_MODE_DEBUG);
07937     prev_report_file = _CrtSetReportFile(_CRT_ASSERT, _CRTDBG_FILE_STDERR);
07938 }
07939
07940 void FatalConditionHandler::reset() {
07941     if (isSet) {
07942         // Unregister handler and restore the old guarantee
07943         SetUnhandledExceptionFilter(previousTop);
07944         SetThreadStackGuarantee(&guaranteeSize);
07945         std::set_terminate(original_terminate_handler);
07946         std::signal(SIGABRT, prev_sigabrt_handler);
07947         SetErrorMode(prev_error_mode_1);
07948         _set_error_mode(prev_error_mode_2);
07949         _set_abort_behavior(prev_abort_behavior, _WRITE_ABORT_MSG | _CALL_REPORTFAULT);
07950         static_cast<void>(_CrtSetReportMode(_CRT_ASSERT, prev_report_mode));
07951         static_cast<void>(_CrtSetReportFile(_CRT_ASSERT, prev_report_file));
07952         isSet = false;
07953     }
07954 }
07955
07956 FatalConditionHandler::~FatalConditionHandler() {
07957     reset();
07958 }
07959
07960 UINT FatalConditionHandler::prev_error_mode_1;
07961 int FatalConditionHandler::prev_error_mode_2;
07962 unsigned int FatalConditionHandler::prev_abort_behavior;
07963 int FatalConditionHandler::prev_report_mode;

```



```

07962 _HFILE FatalConditionHandler::prev_report_file;
07963 void(DOCTEST_CDECL *FatalConditionHandler::prev_sigabrt_handler)(int);
07964 std::terminate_handler FatalConditionHandler::original_terminate_handler;
07965 bool FatalConditionHandler::isSet = false;
07966 ULONG FatalConditionHandler::guaranteeSize = 0;
07967 LPTOP_LEVEL_EXCEPTION_FILTER FatalConditionHandler::previousTop = nullptr;
07968
07969 #else // DOCTEST_PLATFORM_WINDOWS
07970
07971 SignalDefs signalDefs[] = {
07972     {SIGINT, "SIGINT - Terminal interrupt signal"},
07973     {SIGILL, "SIGILL - Illegal instruction signal"},
07974     {SIGFPE, "SIGFPE - Floating point error signal"},
07975     {SIGSEGV, "SIGSEGV - Segmentation violation signal"},
07976     {SIGTERM, "SIGTERM - Termination request signal"},
07977     {SIGABRT, "SIGABRT - Abort (abnormal termination) signal"}
07978 };
07979 static_assert(
07980     DOCTEST_COUNTOF(signalDefs) == DOCTEST_COUNTOF(FatalConditionHandler::oldSigActions), "arrays
    should match in size"
07981 );
07982
07983 void FatalConditionHandler::handleSignal(int sig) {
07984     const char *name = "<unknown signal>";
07985     for (std::size_t i = 0; i < DOCTEST_COUNTOF(signalDefs); ++i) {
07986         const SignalDefs &def = signalDefs[i];
07987         if (sig == def.id) {
07988             name = def.name;
07989             break;
07990         }
07991     }
07992     reset();
07993     reportFatal(name);
07994     static_cast<void>(raise(sig));
07995 }
07996
07997 void FatalConditionHandler::allocateAltStackMem() {
07998     altStackMem = new char[altStackSize];
07999 }
08000
08001 void FatalConditionHandler::freeAltStackMem() {
08002     delete[] altStackMem;
08003 }
08004
08005 FatalConditionHandler::FatalConditionHandler() {
08006     isSet = true;
08007     stack_t sigStack;
08008     sigStack.ss_sp = altStackMem;
08009     sigStack.ss_size = altStackSize;
08010     sigStack.ss_flags = 0;
08011     sigaltstack(&sigStack, &oldSigStack);
08012     struct sigaction sa = {};
08013     sa.sa_handler = handleSignal;
08014     sa.sa_flags = SA_ONSTACK;
08015     for (std::size_t i = 0; i < DOCTEST_COUNTOF(signalDefs); ++i) {
08016         sigaction(signalDefs[i].id, &sa, &oldSigActions[i]);
08017     }
08018 }
08019
08020 FatalConditionHandler::~FatalConditionHandler() {
08021     reset();
08022 }
08023
08024 void FatalConditionHandler::reset() {
08025     if (isSet) {
08026         // Set signals back to previous values -- hopefully nobody overwrote them in the meantime
08027         for (std::size_t i = 0; i < DOCTEST_COUNTOF(signalDefs); ++i) {
08028             sigaction(signalDefs[i].id, &oldSigActions[i], nullptr);
08029         }
08030         // Return the old stack
08031         sigaltstack(&oldSigStack, nullptr);
08032         isSet = false;
08033     }
08034 }
08035
08036 bool FatalConditionHandler::isSet = false;
08037 struct sigaction FatalConditionHandler::oldSigActions[DOCTEST_COUNTOF(signalDefs)] = {};
08038 stack_t FatalConditionHandler::oldSigStack = {};
08039 size_t FatalConditionHandler::altStackSize = 4 * SIGSTKSZ;
08040 char *FatalConditionHandler::altStackMem = nullptr;
08041
08042 #endif // DOCTEST_PLATFORM_WINDOWS
08043 #endif // DOCTEST_CONFIG_POSIX_SIGNALS || DOCTEST_CONFIG_WINDOWS_SEH
08044
08045 } // namespace detail
08046 } // namespace doctest
08047

```

```

08048 #endif // DOCTEST_CONFIG_DISABLE
08049
08050 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08051
08052 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08053
08054 namespace doctest {
08055     namespace detail {
08056
08057         DOCTEST_THREAD_LOCAL class oss {
08058             std::vector<std::streampos> stack;
08059             std::stringstream ss;
08060
08061         public:
08062             std::ostream *push() {
08063                 stack.push_back(ss.tellp());
08064                 return &ss;
08065             }
08066
08067             String pop() {
08068                 if (stack.empty())
08069                     DOCTEST_INTERNAL_ERROR("TLSS was empty when trying to pop!");
08070
08071                 const std::streampos pos = stack.back();
08072                 stack.pop_back();
08073                 const unsigned sz = static_cast<unsigned>(ss.tellp() - pos);
08074                 ss.rdbuf()->pubseekpos(pos, std::ios::in | std::ios::out);
08075                 return String(ss, sz);
08076             }
08077         } g_oss; // NOLINT(bugprone-throwing-static-initialization, cert-err58-cpp)
08078
08079         std::ostream *tlssPush() {
08080             return g_oss.push();
08081         }
08082
08083         String tlssPop() {
08084             return g_oss.pop();
08085         }
08086     } // namespace detail
08087 } // namespace doctest
08088
08089 // case insensitive strcmp
08090 static int stricmp(const char *a, const char *b) {
08091     for (; a++, b++) {
08092         const int d = tolower(*a) - tolower(*b);
08093         if (d != 0 || !*a)
08094             return d;
08095     }
08096 }
08097
08098 // NOLINTBEGIN(cppcoreguidelines-pro-type-union-access)
08099
08100 char *String::allocate(size_type sz) {
08101     if (sz <= last) {
08102         buf[sz] = '\0';
08103         setLast(last - sz);
08104         return buf;
08105     } else {
08106         setOnHeap();
08107         data.size = sz;
08108         data.capacity = data.size + 1;
08109         data.ptr = new char[data.capacity];
08110         data.ptr[sz] = '\0';
08111         return data.ptr;
08112     }
08113 }
08114
08115 void String::setOnHeap() noexcept {
08116     // NOLINTNEXTLINE(cppcoreguidelines-pro-type-reinterpret-cast)
08117     *reinterpret_cast<unsigned char *>(&buf[last]) = 128;
08118 }
08119
08120 void String::setLast(size_type in) noexcept {
08121     buf[last] = static_cast<char>(in);
08122 }
08123
08124 void String::setSize(size_type sz) noexcept {
08125     if (isOnStack()) {
08126         buf[sz] = '\0';
08127         setLast(last - sz);
08128     } else {
08129         data.ptr[sz] = '\0';
08130         data.size = sz;
08131     }
08132 }
08133
08134 void String::copy(const String &other) {

```

```

08135     if (other.isOnStack()) {
08136         memcpy(buf, other.buf, len);
08137     } else {
08138         memcpy(allocate(other.data.size), other.data.ptr, other.data.size);
08139     }
08140 } // NOLINT(clang-analyzer-cplusplus.NewDeleteLeaks)
08141
08142 String::String() noexcept {
08143     buf[0] = '\0';
08144     setLast();
08145 }
08146
08147 String::~String() {
08148     if (!isOnStack())
08149         delete[] data.ptr;
08150 } // NOLINT(clang-analyzer-cplusplus.NewDeleteLeaks)
08151
08152 String::String(const char *in)
08153     : String(in, strlen(in)) {}
08154
08155 String::String(const char *in, size_type in_size) {
08156     memcpy(allocate(in_size), in, in_size);
08157 }
08158
08159 String::String(std::istream &in, size_type in_size) {
08160     in.read(allocate(in_size), in_size);
08161 }
08162
08163 String::String(const String &other) {
08164     copy(other);
08165 }
08166
08167 String &String::operator=(const String &other) {
08168     if (this != &other) {
08169         if (!isOnStack())
08170             delete[] data.ptr;
08171
08172         copy(other);
08173     }
08174
08175     return *this;
08176 }
08177
08178 String &String::operator+=(const String &other) {
08179     const size_type my_old_size = size();
08180     const size_type other_size = other.size();
08181     const size_type total_size = my_old_size + other_size;
08182     if (isOnStack()) {
08183         if (total_size < len) {
08184             // append to the current stack space
08185             memcpy(buf + my_old_size, other.c_str(), other_size + 1);
08186             // NOLINTNEXTLINE(clang-analyzer-cplusplus.NewDeleteLeaks)
08187             setLast(last - total_size);
08188         } else {
08189             // alloc new chunk
08190             char *temp = new char[total_size + 1];
08191             // copy current data to new location before writing in the union
08192             memcpy(temp, buf, my_old_size); // skip the +1 ('\0') for speed
08193             // update data in union
08194             setOnHeap();
08195             data.size = total_size;
08196             data.capacity = data.size + 1;
08197             data.ptr = temp;
08198             // transfer the rest of the data
08199             memcpy(data.ptr + my_old_size, other.c_str(), other_size + 1);
08200         }
08201     } else {
08202         if (data.capacity > total_size) {
08203             // append to the current heap block
08204             data.size = total_size;
08205             memcpy(data.ptr + my_old_size, other.c_str(), other_size + 1);
08206         } else {
08207             // resize
08208             data.capacity *= 2;
08209             if (data.capacity <= total_size)
08210                 data.capacity = total_size + 1;
08211             // alloc new chunk
08212             char *temp = new char[data.capacity];
08213             // copy current data to new location before releasing it
08214             memcpy(temp, data.ptr, my_old_size); // skip the +1 ('\0') for speed
08215             // release old chunk
08216             delete[] data.ptr;
08217             // update the rest of the union members
08218             data.size = total_size;
08219             data.ptr = temp;
08220             // transfer the rest of the data
08221             memcpy(data.ptr + my_old_size, other.c_str(), other_size + 1);

```

```

08222     }
08223 }
08224
08225     return *this;
08226 }
08227
08228 String::String(String &&other) noexcept {
08229     memcpy(buf, other.buf, len);
08230     other.buf[0] = '\0';
08231     other.setLast();
08232 }
08233
08234 String &String::operator=(String &&other) noexcept {
08235     if (this != &other) {
08236         if (!isOnStack())
08237             delete[] data.ptr;
08238         memcpy(buf, other.buf, len);
08239         other.buf[0] = '\0';
08240         other.setLast();
08241     }
08242     return *this;
08243 }
08244
08245 char String::operator[](size_type i) const {
08246     // NOLINTNEXTLINE(cppcoreguidelines-pro-type-const-cast)
08247     return const_cast<String *>(this)->operator[](i);
08248 }
08249
08250 char &String::operator[](size_type i) {
08251     if (isOnStack())
08252         // NOLINTNEXTLINE(cppcoreguidelines-pro-type-reinterpret-cast)
08253         return reinterpret_cast<char *>(buf)[i];
08254     return data.ptr[i];
08255 }
08256
08257 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wmaybe-uninitialized")
08258 String::size_type String::size() const {
08259     if (isOnStack())
08260         return last - (static_cast<size_type>(buf[last]) & 31); // using "last" would work only if
08261         "len" is 32
08262     return data.size;
08263 }
08264 DOCTEST_GCC_SUPPRESS_WARNING_POP
08265
08266 String::size_type String::capacity() const {
08267     if (isOnStack())
08268         return len;
08269     return data.capacity;
08270 }
08271
08272 String String::substr(size_type pos, size_type cnt) && {
08273     cnt = std::min(cnt, size() - pos);
08274     char *cptr = c_str();
08275     memmove(cptr, cptr + pos, cnt);
08276     setSize(cnt);
08277     return std::move(*this);
08278 }
08279
08280 String String::substr(size_type pos, size_type cnt) const & {
08281     cnt = std::min(cnt, size() - pos);
08282     return String{c_str() + pos, cnt};
08283 }
08284
08285 String::size_type String::find(char ch, size_type pos) const {
08286     const char *begin = c_str();
08287     const char *end = begin + size();
08288     const char *it = begin + pos;
08289     for (; it < end && *it != ch; it++) {}
08290     if (it < end) {
08291         return static_cast<size_type>(it - begin);
08292     } else {
08293         return npos;
08294     }
08295 }
08296
08297 String::size_type String::rfind(char ch, size_type pos) const {
08298     if (size() == 0) {
08299         return npos;
08300     }
08301     const char *begin = c_str();
08302     const char *it = begin + std::min(pos, size() - 1);
08303     for (; it >= begin && *it != ch; it--) {}
08304     if (it >= begin) {
08305         return static_cast<size_type>(it - begin);
08306     } else {
08307         return npos;
08308     }
08309 }

```

```

08308     }
08309 }
08310
08311 int String::compare(const char *other, bool no_case) const {
08312     if (no_case)
08313         return doctest::stricmp(c_str(), other);
08314     return std::strcmp(c_str(), other);
08315 }
08316
08317 int String::compare(const String &other, bool no_case) const {
08318     return compare(other.c_str(), no_case);
08319 }
08320
08321 String operator+(const String &lhs, const String &rhs) {
08322     return String(lhs) += rhs;
08323 }
08324
08325 bool operator==(const String &lhs, const String &rhs) {
08326     return lhs.compare(rhs) == 0;
08327 }
08328
08329 bool operator!=(const String &lhs, const String &rhs) {
08330     return lhs.compare(rhs) != 0;
08331 }
08332
08333 bool operator<(const String &lhs, const String &rhs) {
08334     return lhs.compare(rhs) < 0;
08335 }
08336
08337 bool operator>(const String &lhs, const String &rhs) {
08338     return lhs.compare(rhs) > 0;
08339 }
08340
08341 bool operator<=(const String &lhs, const String &rhs) {
08342     return (lhs != rhs) ? lhs.compare(rhs) < 0 : true;
08343 }
08344
08345 bool operator>=(const String &lhs, const String &rhs) {
08346     return (lhs != rhs) ? lhs.compare(rhs) > 0 : true;
08347 }
08348
08349 std::ostream &operator<<(std::ostream &s, const String &in) {
08350     return s << in.c_str();
08351 }
08352
08353 namespace detail {
08354
08355 void filldata<const void *>::fill(std::ostream *stream, const void *in) {
08356     filldata<const volatile void *>::fill(stream, in);
08357 }
08358
08359 void filldata<const volatile void *>::fill(std::ostream *stream, const volatile void *in) {
08360     if (in) {
08361         *stream << in;
08362     } else {
08363         *stream << "nullptr";
08364     }
08365 }
08366
08367 template <typename T>
08368 String toStringLit(T t) {
08369     // NOLINTNEXTLINE(misc-const-correctness)
08370     std::ostream *os = tlssPush();
08371     os->operator<<(t);
08372     return tlssPop();
08373 }
08374 } // namespace detail
08375
08376 #ifdef DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
08377 String toString(const char *in) {
08378     return String("\") + (in ? in : "{null string}") + "\"";
08379 }
08380 #endif // DOCTEST_CONFIG_TREAT_CHAR_STAR_AS_STRING
08381
08382 #if DOCTEST_MSVC >= DOCTEST_COMPILER(19, 20, 0)
08383 // see this issue on why this is needed: https://github.com/doctest/doctest/issues/183
08384 String toString(const std::string &in) {
08385     return in.c_str();
08386 }
08387 #endif // VS 2019
08388
08389 String toString(const String &in) {
08390     return in;
08391 }
08392
08393 String toString(std::nullptr_t) {
08394     return "nullptr";

```

```

08395 }
08396
08397 String toString(bool in) {
08398     return in ? "true" : "false";
08399 }
08400
08401 String toString(float in) {
08402     return detail::toStreamLit(in);
08403 }
08404 String toString(double in) {
08405     return detail::toStreamLit(in);
08406 }
08407 String toString(double long in) {
08408     return detail::toStreamLit(in);
08409 }
08410
08411 String toString(char in) {
08412     return detail::toStreamLit(static_cast<signed>(in));
08413 }
08414 String toString(char signed in) {
08415     return detail::toStreamLit(static_cast<signed>(in));
08416 }
08417 String toString(char unsigned in) {
08418     return detail::toStreamLit(static_cast<unsigned>(in));
08419 }
08420 String toString(short in) {
08421     return detail::toStreamLit(in);
08422 }
08423 String toString(short unsigned in) {
08424     return detail::toStreamLit(in);
08425 }
08426 String toString(signed in) {
08427     return detail::toStreamLit(in);
08428 }
08429 String toString(unsigned in) {
08430     return detail::toStreamLit(in);
08431 }
08432 String toString(long in) {
08433     return detail::toStreamLit(in);
08434 }
08435 String toString(long unsigned in) {
08436     return detail::toStreamLit(in);
08437 }
08438 String toString(long long in) {
08439     return detail::toStreamLit(in);
08440 }
08441 String toString(long long unsigned in) {
08442     return detail::toStreamLit(in);
08443 }
08444
08445 // NOLINTEND(cppcoreguidelines-pro-type-union-access)
08446
08447 } // namespace doctest
08448
08449 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08450
08451 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08452
08453 namespace doctest {
08454
08455 bool SubcaseSignature::operator==(const SubcaseSignature &other) const {
08456     return m_line == other.m_line && std::strcmp(m_file, other.m_file) == 0 && m_name == other.m_name;
08457 }
08458
08459 bool SubcaseSignature::operator<(const SubcaseSignature &other) const {
08460     if (m_line != other.m_line)
08461         return m_line < other.m_line;
08462     if (std::strcmp(m_file, other.m_file) != 0)
08463         return std::strcmp(m_file, other.m_file) < 0;
08464     return m_name.compare(other.m_name) < 0;
08465 }
08466
08467 #ifndef DOCTEST_CONFIG_DISABLE
08468 namespace detail {
08469
08470 bool Subcase::checkFilters() {
08471     if (g_cs->traversal.activeSubcaseDepth() < static_cast<size_t>(g_cs->subcase_filter_levels)) {
08472         if (!matchesAny(m_signature.m_name.c_str(), g_cs->filters[6], true, g_cs->case_sensitive))
08473             return true;
08474         if (matchesAny(m_signature.m_name.c_str(), g_cs->filters[7], false, g_cs->case_sensitive))
08475             return true;
08476     }
08477     return false;
08478 }
08479
08480 Subcase::Subcase(const String &name, const char *file, int line)
08481     : m_signature({name, file, line}) {

```

```

08482     if (checkFilters())
08483         return;
08484
08485     if (!g_cs->traversal.tryEnterSubcase(m_signature))
08486         return;
08487
08488     m_entered = true;
08489     DOCTEST_ITERATE_THROUGH_REPORTERS(subcase_start, m_signature);
08490 }
08491
08492 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(4996) // std::uncaught_exception is deprecated in C++17
08493 DOCTEST_GCC_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
08494 DOCTEST_CLANG_SUPPRESS_WARNING_WITH_PUSH("-Wdeprecated-declarations")
08495
08496 Subcase::~Subcase() {
08497     if (m_entered) {
08498         g_cs->traversal.leaveSubcase();
08499
08500 #if defined(__cpp_lib_uncaught_exceptions) && __cpp_lib_uncaught_exceptions >= 201411L &&
08501     \
08502         (!defined(__MAC_OS_X_VERSION_MIN_REQUIRED) || __MAC_OS_X_VERSION_MIN_REQUIRED >= 101200)
08503 #else
08504         if (std::uncaught_exception() > 0
08505         if (std::uncaught_exception())
08506 #endif
08507         && g_cs->shouldLogCurrentException() {
08508             DOCTEST_ITERATE_THROUGH_REPORTERS(
08509                 test_case_exception,
08510                 {"exception thrown in subcase - will translate later "
08511                  "when the whole test case has been exited (cannot "
08512                  "translate while there is an active exception)",
08513                  false}
08514             );
08515             g_cs->shouldLogCurrentException = false;
08516         }
08517         DOCTEST_ITERATE_THROUGH_REPORTERS(subcase_end, DOCTEST_EMPTY);
08518     }
08519 }
08520
08521 DOCTEST_CLANG_SUPPRESS_WARNING_POP
08522 DOCTEST_GCC_SUPPRESS_WARNING_POP
08523 DOCTEST_MSVC_SUPPRESS_WARNING_POP
08524
08525 Subcase::operator bool() const {
08526     return m_entered;
08527 }
08528
08529 } // namespace detail
08530 #endif // DOCTEST_CONFIG_DISABLE
08531
08532 } // namespace doctest
08533
08534 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08535
08536 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08537
08538 #ifndef DOCTEST_CONFIG_DISABLE
08539
08540 namespace doctest {
08541 namespace detail {
08542
08543 std::set<TestCase> &getRegisteredTests() {
08544     static std::set<TestCase> data;
08545     return data;
08546 }
08547
08548 TestCase::TestCase(
08549     funcType test, const char *file, unsigned line, const TestSuite &test_suite, const String &type,
08550     int template_id
08551 ) noexcept {
08552     m_file = file;
08553     m_line = line;
08554     m_name = nullptr; // will be later overridden in operator*
08555     m_test_suite = test_suite.m_test_suite;
08556     m_description = test_suite.m_description;
08557     m_skip = test_suite.m_skip;
08558     m_no_breaks = test_suite.m_no_breaks;
08559     m_no_output = test_suite.m_no_output;
08560     m_may_fail = test_suite.m_may_fail;
08561     m_should_fail = test_suite.m_should_fail;
08562     m_expected_failures = test_suite.m_expected_failures;
08563     m_timeout = test_suite.m_timeout;
08564
08565     m_test = test;
08566     m_type = type;
08567     m_template_id = template_id;

```

```

08567 }
08568
08569 TestCase::TestCase(const TestCase &other) noexcept // NOLINT(bugprone-copy-constructor-init)
08570 : TestCaseData() {
08571     *this = other;
08572 }
08573
08574 DOCTEST_MSVC_SUPPRESS_WARNING_WITH_PUSH(26434) // hides a non-virtual function
08575 TestCase &TestCase::operator=(const TestCase &other) noexcept { // NOLINT(cert-oop54-cpp)
08576     TestCaseData::operator=(other);
08577     m_test = other.m_test;
08578     m_type = other.m_type;
08579     m_template_id = other.m_template_id;
08580     m_full_name = other.m_full_name;
08581
08582     if (m_template_id != -1)
08583         m_name = m_full_name.c_str();
08584     return *this;
08585 }
08586 DOCTEST_MSVC_SUPPRESS_WARNING_POP
08587
08588 TestCase &TestCase::operator*(const char *in) noexcept {
08589     m_name = in;
08590     // make a new name with an appended type for templated test case
08591     if (m_template_id != -1) {
08592         m_full_name = String(m_name) + "<" + m_type + ">";
08593         // redirect the name to point to the newly constructed full name
08594         m_name = m_full_name.c_str();
08595     }
08596     return *this;
08597 }
08598
08599 bool TestCase::operator<(const TestCase &other) const noexcept {
08600     // this will be used only to differentiate between test cases - not relevant for sorting
08601     if (m_line != other.m_line)
08602         return m_line < other.m_line;
08603     const int name_cmp = strcmp(m_name, other.m_name);
08604     if (name_cmp != 0)
08605         return name_cmp < 0;
08606     const int file_cmp = m_file.compare(other.m_file);
08607     if (file_cmp != 0)
08608         return file_cmp < 0;
08609     return m_template_id < other.m_template_id;
08610 }
08611
08612 // used by the macros for registering tests
08613 int regTest(const TestCase &tc) noexcept {
08614     getRegisteredTests().insert(tc);
08615     return 0;
08616 }
08617
08618 } // namespace detail
08619 } // namespace doctest
08620
08621 #endif // DOCTEST_CONFIG_DISABLE
08622
08623 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08624
08625 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08626
08627 #ifndef DOCTEST_CONFIG_DISABLE
08628
08629 namespace doctest {
08630 namespace detail {
08631
08632 TestSuite &TestSuite::operator*(const char *in) noexcept {
08633     m_test_suite = in;
08634     return *this;
08635 }
08636
08637 // sets the current test suite
08638 int setTestSuite(const TestSuite &ts) noexcept {
08639     doctest_detail_test_suite_ns::getCurrentTestSuite() = ts;
08640     return 0;
08641 }
08642
08643 } // namespace detail
08644 } // namespace doctest
08645
08646 namespace doctest_detail_test_suite_ns {
08647 // holds the current test suite
08648 doctest::detail::TestSuite &getCurrentTestSuite() noexcept {
08649     static doctest::detail::TestSuite data{};
08650     return data;
08651 }
08652 } // namespace doctest_detail_test_suite_ns
08653

```



```

08654 #endif // DOCTEST_CONFIG_DISABLE
08655
08656 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08657
08658 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08659
08660 #ifndef DOCTEST_CONFIG_DISABLE
08661
08662 namespace doctest {
08663 namespace detail {
08664
08665 #ifdef DOCTEST_CONFIG_GETCURRENTTICKS
08666 ticks_t getCurrentTicks() {
08667     return DOCTEST_CONFIG_GETCURRENTTICKS();
08668 }
08669 #elif defined(DOCTEST_PLATFORM_WINDOWS)
08670 ticks_t getCurrentTicks() {
08671     static LARGE_INTEGER hz = {{0}}, hzo = {{0}};
08672     if (!hz.QuadPart) {
08673         QueryPerformanceFrequency(&hz);
08674         QueryPerformanceCounter(&hzo);
08675     }
08676     LARGE_INTEGER t;
08677     QueryPerformanceCounter(&t);
08678     return ((t.QuadPart - hzo.QuadPart) * LONGLONG(1000000)) / hz.QuadPart;
08679 }
08680 #else // DOCTEST_PLATFORM_WINDOWS
08681 ticks_t getCurrentTicks() {
08682     timeval t;
08683     gettimeofday(&t, nullptr);
08684     return static_cast<ticks_t>(t.tv_sec) * 1000000 + static_cast<ticks_t>(t.tv_usec);
08685 }
08686 #endif // DOCTEST_PLATFORM_WINDOWS
08687
08688 void Timer::start() {
08689     m_ticks = getCurrentTicks();
08690 }
08691
08692 unsigned int Timer::getElapsedMicroseconds() const {
08693     return static_cast<unsigned int>(getCurrentTicks() - m_ticks);
08694 }
08695
08696 // unsigned int Timer::getElapsedMilliseconds() const {
08697 //     return static_cast<unsigned int>(getElapsedMicroseconds() / 1000);
08698 // }
08699
08700 double Timer::getElapsedSeconds() const {
08701     return static_cast<double>(getCurrentTicks() - m_ticks) / 1000000.0;
08702 }
08703 } // namespace detail
08704 } // namespace doctest
08705
08706 #endif // DOCTEST_CONFIG_DISABLE
08707 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08708
08709 #include <algorithm>
08710
08711 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08712
08713 #ifndef DOCTEST_CONFIG_DISABLE
08714 namespace doctest {
08715 namespace detail {
08716
08717 DOCTEST_NOINLINE DecisionPoint &TraversalState::ensureDecisionPointAtCurrentDepth() {
08718     const size_t depth = m_decisionDepth;
08719
08720     if (m_discoveredDecisionPath.size() == depth) {
08721         m_discoveredDecisionPath.emplace_back();
08722
08723         if (m_decisionPath.size() == depth)
08724             m_decisionPath.push_back(0);
08725     }
08726
08727     return m_discoveredDecisionPath[depth];
08728 }
08729
08730 void TraversalState::resetForTestCase() {
08731     m_decisionPath.clear();
08732     m_discoveredDecisionPath.clear();
08733     m_enteredSubcaseDepths.clear();
08734     m_activeSubcaseDepth = 0;
08735     m_decisionDepth = 0;
08736 }
08737
08738 } // namespace detail
08739 } // namespace doctest

```

```

08741
08742 void TraversalState::resetForRun() {
08743     m_activeSubcaseDepth = 0;
08744     m_discoveredDecisionPath.clear();
08745     m_decisionDepth = 0;
08746     m_enteredSubcaseDepths.clear();
08747 }
08748
08749 bool TraversalState::advance() {
08750     const size_t maxDepth = std::min(m_decisionPath.size(), m_discoveredDecisionPath.size());
08751     for (size_t depth = maxDepth; depth > 0; --depth) {
08752         const size_t index = depth - 1;
08753         if (m_decisionPath[index] + 1 < m_discoveredDecisionPath[index].branch_count) {
08754             ++m_decisionPath[index];
08755             m_decisionPath.resize(index + 1);
08756             return true;
08757         }
08758     }
08759     return false;
08760 }
08761
08762 bool TraversalState::tryEnterSubcase(const SubcaseSignature &signature) {
08763     DecisionPoint &point = ensureDecisionPointAtCurrentDepth();
08764     std::vector<SubcaseSignature> &subcases = point.subcases;
08765     size_t siblingIndex = 0;
08766
08767     for (; siblingIndex < subcases.size(); ++siblingIndex) {
08768         if (subcases[siblingIndex] == signature)
08769             break;
08770     }
08771
08772     if (siblingIndex == subcases.size())
08773         subcases.push_back(signature);
08774
08775     point.branch_count = subcases.size();
08776
08777     if (siblingIndex != m_decisionPath[m_decisionDepth])
08778         return false;
08779
08780     m_enteredSubcaseDepths.push_back(m_decisionDepth);
08781     m_activeSubcaseDepth++;
08782     m_decisionDepth++;
08783     return true;
08784 }
08785
08786 void TraversalState::leaveSubcase() {
08787     m_decisionDepth = m_enteredSubcaseDepths.back();
08788     m_enteredSubcaseDepths.pop_back();
08789     m_activeSubcaseDepth--;
08790 }
08791
08792 size_t TraversalState::unwindActiveSubcases() {
08793     const size_t activeSubcaseCount = m_activeSubcaseDepth;
08794
08795     while (m_activeSubcaseDepth > 0)
08796         leaveSubcase();
08797
08798     return activeSubcaseCount;
08799 }
08800
08801 size_t TraversalState::acquireGeneratorIndex(size_t count) {
08802     DecisionPoint &point = ensureDecisionPointAtCurrentDepth();
08803     point.branch_count = count;
08804
08805     const size_t index = m_decisionPath[m_decisionDepth];
08806     m_decisionDepth++;
08807     return index < count ? index : 0;
08808 }
08809
08810 size_t acquireGeneratorDecisionIndex(size_t count) {
08811     return g_cs->traversal.acquireGeneratorIndex(count);
08812 }
08813 } // namespace detail
08814 } // namespace doctest
08815
08816 #endif // DOCTEST_CONFIG_DISABLE
08817
08818 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
08819
08820 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_PUSH
08821
08822 #ifndef DOCTEST_CONFIG_DISABLE
08823 namespace doctest {
08824 namespace detail {
08825

```

```

08828 // =====
08829 // The following code has been taken verbatim from Catch2/include/internal/catch_xmlwriter.cpp
08830 // This is done so cherry-picking bug fixes is trivial - even the style/formatting is untouched.
08831 // =====
08832 /* clang-format off */ /* NOLINTBEGIN */
08833
08834 using uchar = unsigned char;
08835
08836 size_t trailingBytes(unsigned char c) {
08837     if ((c & 0xE0) == 0xC0) {
08838         return 2;
08839     }
08840     if ((c & 0xF0) == 0xE0) {
08841         return 3;
08842     }
08843     if ((c & 0xF8) == 0xF0) {
08844         return 4;
08845     }
08846     DOCTEST_INTERNAL_ERROR("Invalid multibyte utf-8 start byte encountered");
08847 }
08848
08849 uint32_t headerValue(unsigned char c) {
08850     if ((c & 0xE0) == 0xC0) {
08851         return c & 0x1F;
08852     }
08853     if ((c & 0xF0) == 0xE0) {
08854         return c & 0x0F;
08855     }
08856     if ((c & 0xF8) == 0xF0) {
08857         return c & 0x07;
08858     }
08859     DOCTEST_INTERNAL_ERROR("Invalid multibyte utf-8 start byte encountered");
08860 }
08861
08862 void hexEscapeChar(std::ostream& os, unsigned char c) {
08863     std::ios_base::fmtflags f(os.flags());
08864     os << "\\x"
08865         << std::uppercase << std::hex << std::setfill('0') << std::setw(2)
08866         << static_cast<int>(c);
08867     os.flags(f);
08868 }
08869
08870 XmlEncode::XmlEncode( std::string const& str, ForWhat forWhat )
08871 :   m_str( str ),
08872   m_forWhat( forWhat )
08873 {}
08874
08875 void XmlEncode::encodeTo( std::ostream& os ) const {
08876     // Apostrophe escaping not necessary if we always use " to write attributes
08877     // (see: https://www.w3.org/TR/xml/#syntax)
08878
08879     for( std::size_t idx = 0; idx < m_str.size(); ++ idx ) {
08880         uchar c = m_str[idx];
08881         switch (c) {
08882             case '<': os << "&lt;"; break;
08883             case '&': os << "&amp;"; break;
08884
08885             case '>':
08886                 // See: https://www.w3.org/TR/xml/#syntax
08887                 if (idx > 2 && m_str[idx - 1] == '[' && m_str[idx - 2] == '[')
08888                     os << "&gt;";
08889                 else
08890                     os << c;
08891                 break;
08892
08893             case '\"':
08894                 if (m_forWhat == ForAttributes)
08895                     os << "&quot;";
08896                 else
08897                     os << c;
08898                 break;
08899
08900             default:
08901                 // Check for control characters and invalid utf-8
08902
08903                 // Escape control characters in standard ascii
08904                 // see
08905                 https://stackoverflow.com/questions/404107/why-are-control-characters-illegal-in-xml-1-0
08906                 if (c < 0x09 || (c > 0x0D && c < 0x20) || c == 0x7F) {
08907                     hexEscapeChar(os, c);
08908                     break;
08909                 }
08910
08911                 // Plain ASCII: Write it to stream
08912                 if (c < 0x7F) {
08913                     os << c;
08914                     break;
08915                 }

```

```

08914         }
08915
08916         // UTF-8 territory
08917         // Check if the encoding is valid and if it is not, hex escape bytes.
08918         // Important: We do not check the exact decoded values for validity, only the encoding
format
08919         // First check that this bytes is a valid lead byte:
08920         // This means that it is not encoded as 1111 1XXX
08921         // Or as 10XX XXXX
08922         if (c < 0xC0 ||
08923             c >= 0xF8) {
08924             hexEscapeChar(os, c);
08925             break;
08926         }
08927
08928         auto encBytes = trailingBytes(c);
08929         // Are there enough bytes left to avoid accessing out-of-bounds memory?
08930         if (idx + encBytes - 1 >= m_str.size()) {
08931             hexEscapeChar(os, c);
08932             break;
08933         }
08934         // The header is valid, check data
08935         // The next encBytes bytes must together be a valid utf-8
08936         // This means: bitpattern 10XX XXXX and the extracted value is sane (ish)
08937         bool valid = true;
08938         uint32_t value = headerValue(c);
08939         for (std::size_t n = 1; n < encBytes; ++n) {
08940             uchar nc = m_str[idx + n];
08941             valid &= ((nc & 0xC0) == 0x80);
08942             value = (value << 6) | (nc & 0x3F);
08943         }
08944
08945         if (
08946             // Wrong bit pattern of following bytes
08947             (!valid) ||
08948             // Overlong encodings
08949             (value < 0x80) ||
08950             (
08951                 value < 0x800    && encBytes > 2) || // removed "0x80 <= value
&&" because redundant
08951                 (0x800 < value && value < 0x10000 && encBytes > 3) ||
08952                 // Encoded value out of range
08953                 (value >= 0x110000)
08954             ) {
08955             hexEscapeChar(os, c);
08956             break;
08957         }
08958
08959         // If we got here, this is in fact a valid(ish) utf-8 sequence
08960         for (std::size_t n = 0; n < encBytes; ++n) {
08961             os << m_str[idx + n];
08962         }
08963         idx += encBytes - 1;
08964         break;
08965     }
08966 }
08967 }
08968
08969 std::ostream& operator << ( std::ostream& os, XmlEncode const& xmlEncode ) {
08970     xmlEncode.encodeTo( os );
08971     return os;
08972 }
08973
08974 XmlWriter::ScopedElement::ScopedElement( XmlWriter* writer )
08975 :   m_writer( writer )
08976 {}
08977
08978 XmlWriter::ScopedElement::ScopedElement( ScopedElement&& other ) DOCTEST_NOEXCEPT
08979 :   m_writer( other.m_writer ){
08980     other.m_writer = nullptr;
08981 }
08982 XmlWriter::ScopedElement& XmlWriter::ScopedElement::operator=( ScopedElement&& other )
DOCTEST_NOEXCEPT {
08983     if ( m_writer ) {
08984         m_writer->endElement();
08985     }
08986     m_writer = other.m_writer;
08987     other.m_writer = nullptr;
08988     return *this;
08989 }
08990
08991
08992 XmlWriter::ScopedElement::~ScopedElement() {
08993     if( m_writer )
08994         m_writer->endElement();
08995 }
08996
08997 XmlWriter::ScopedElement& XmlWriter::ScopedElement::writeText( std::string const& text, bool

```

```

        indent ) {
08998         m_writer->writeText( text, indent );
08999         return *this;
09000     }
09001
09002     XmlWriter::XmlWriter( std::ostream& os ) : m_os( os )
09003     {
09004         // writeDeclaration(); // called explicitly by the reporters that use the writer class - see
issue #627
09005     }
09006
09007     XmlWriter::~XmlWriter() {
09008         while( !m_tags.empty() )
09009             endElement();
09010     }
09011
09012     XmlWriter& XmlWriter::startElement( std::string const& name ) {
09013         ensureTagClosed();
09014         newlineIfNecessary();
09015         m_os << m_indent << "<" << name;
09016         m_tags.push_back( name );
09017         m_indent += " ";
09018         m_tagIsOpen = true;
09019         return *this;
09020     }
09021
09022     XmlWriter::ScopedElement XmlWriter::scopedElement( std::string const& name ) {
09023         ScopedElement scoped( this );
09024         startElement( name );
09025         return scoped;
09026     }
09027
09028     XmlWriter& XmlWriter::endElement() {
09029         newlineIfNecessary();
09030         m_indent = m_indent.substr( 0, m_indent.size()-2 );
09031         if( m_tagIsOpen ) {
09032             m_os << ">";
09033             m_tagIsOpen = false;
09034         }
09035         else {
09036             m_os << m_indent << "</" << m_tags.back() << ">";
09037         }
09038         m_os << std::endl;
09039         m_tags.pop_back();
09040         return *this;
09041     }
09042
09043     XmlWriter& XmlWriter::writeAttribute( std::string const& name, std::string const& attribute ) {
09044         if( !name.empty() && !attribute.empty() )
09045             m_os << " " << name << "=" << XmlEncode( attribute, XmlEncode::ForAttributes ) << "'";
09046         return *this;
09047     }
09048
09049     XmlWriter& XmlWriter::writeAttribute( std::string const& name, const char* attribute ) {
09050         if( !name.empty() && attribute && attribute[0] != '\0' )
09051             m_os << " " << name << "=" << XmlEncode( attribute, XmlEncode::ForAttributes ) << "'";
09052         return *this;
09053     }
09054
09055     XmlWriter& XmlWriter::writeAttribute( std::string const& name, bool attribute ) {
09056         m_os << " " << name << "=" << ( attribute ? "true" : "false" ) << "'";
09057         return *this;
09058     }
09059
09060     XmlWriter& XmlWriter::writeText( std::string const& text, bool indent ) {
09061         if( !text.empty() ) {
09062             bool tagWasOpen = m_tagIsOpen;
09063             ensureTagClosed();
09064             if( tagWasOpen && indent )
09065                 m_os << m_indent;
09066             m_os << XmlEncode( text );
09067             m_needsNewline = true;
09068         }
09069         return *this;
09070     }
09071
09072     //XmlWriter& XmlWriter::writeComment( std::string const& text ) {
09073     //    ensureTagClosed();
09074     //    m_os << m_indent << "<!--" << text << "-->";
09075     //    m_needsNewline = true;
09076     //    return *this;
09077     //}
09078
09079     //void XmlWriter::writeStylesheetRef( std::string const& url ) {
09080     //    m_os << "<?xml-stylesheet type='text/xsl' href='" << url << "'?>\n";
09081     //}
09082

```

```

09083 //XmlWriter& XmlWriter::writeBlankLine() {
09084 //     ensureTagClosed();
09085 //     m_os << '\n';
09086 //     return *this;
09087 //}
09088
09089 void XmlWriter::ensureTagClosed() {
09090     if( m_tagIsOpen ) {
09091         m_os << ">" << std::endl;
09092         m_tagIsOpen = false;
09093     }
09094 }
09095
09096 void XmlWriter::writeDeclaration() {
09097     m_os << "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n";
09098 }
09099
09100 void XmlWriter::newlineIfNecessary() {
09101     if( m_needsNewline ) {
09102         m_os << std::endl;
09103         m_needsNewline = false;
09104     }
09105 }
09106
09107 /* clang-format on */ /* NOLINTEND */
09108 // =====
09109 // End of copy-pasted code from Catch
09110 // =====
09111
09112 } // namespace detail
09113 } // namespace doctest
09114
09115 #endif // DOCTEST_CONFIG_DISABLE
09116
09117 DOCTEST_SUPPRESS_PRIVATE_WARNINGS_POP
09118
09119 #endif // defined(DOCTEST_CONFIG_IMPLEMENT) && !defined(DOCTEST_LIBRARY_IMPLEMENTATION)

```

9.25 tests/test_studentas.cpp File Reference

```

#include "doctest.h"
#include "student.h"
#include <utility>

```

Macros

- `#define DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN`

Functions

- `TEST_CASE` ("Default konstruktorius")
- `TEST_CASE` ("Setteriai ir getteriai")
- `TEST_CASE` ("MedlVidSkaciavimas sy lyginiais pazymiais")
- `TEST_CASE` ("MedlVidSkaciavimas sy nelyginiais pazymiais")
- `TEST_CASE` ("MedlVidSkaciavimas be pazymiu")
- `TEST_CASE` ("ClearPaz istrina pazymius")
- `TEST_CASE` ("Konstruktoriaus kopijavimas")
- `TEST_CASE` ("Kopijavimo priskyrimo operatorius")
- `TEST_CASE` ("Perkelimo konstruktorius")
- `TEST_CASE` ("Perkelimo priskyrimo operatorius")
- `TEST_CASE` ("MedlVidSkaciavimas su lyginiais pazymiais bet kokia tvarka")
- `TEST_CASE` ("MedlVidSkaciavimas su nelyginiais pazymiais bet kokia tvarka")

9.25.1 Macro Definition Documentation

9.25.1.1 DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN

```
#define DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
```

9.25.2 Function Documentation

9.25.2.1 TEST_CASE() [1/12]

```
TEST_CASE (
    "ClearPaz istrina pazymius" )
```

9.25.2.2 TEST_CASE() [2/12]

```
TEST_CASE (
    "Default konstruktorius" )
```

9.25.2.3 TEST_CASE() [3/12]

```
TEST_CASE (
    "Konstruktoriaus kopijavimas" )
```

9.25.2.4 TEST_CASE() [4/12]

```
TEST_CASE (
    "Kopijavimo priskyrimo operatorius" )
```

9.25.2.5 TEST_CASE() [5/12]

```
TEST_CASE (
    "MedIrVidSkaciavimas be pazymiu" )
```

9.25.2.6 TEST_CASE() [6/12]

```
TEST_CASE (
    "MedIrVidSkaciavimas su lyginiais pazymiais bet kokia tvarka" )
```

9.25.2.7 TEST_CASE() [7/12]

```
TEST_CASE (
    "MedIrVidSkaciavimas su nelyginiais pazymiais bet kokia tvarka" )
```

9.25.2.8 TEST_CASE() [8/12]

```
TEST_CASE (
    "MedIrVidSkaciavimas sy lyginiais pazymiais" )
```

9.25.2.9 TEST_CASE() [9/12]

```
TEST_CASE (
    "MedIrVidSkaciavimas sy nelyginiais pazymiais" )
```

9.25.2.10 TEST_CASE() [10/12]

```
TEST_CASE (
    "Perkelimo konstruktorius" )
```

9.25.2.11 TEST_CASE() [11/12]

```
TEST_CASE (
    "Perkelimo priskyrimo operatorius" )
```

9.25.2.12 TEST_CASE() [12/12]

```
TEST_CASE (
    "Setteriai ir getteriai" )
```